

Python for Quant Traders

จากไม่เคยเขียนโค้ด สู่ระบบเทรดอัตโนมัติที่รันจริง

7 Parts · 35 บท · โค้ดทุกบล็อกรันได้จริง และผลลัพธ์ที่แสดงในเล่มมาจากการรันจริง ไม่ใช่พิมพ์ตามที่คุณคิดว่าน่าจะเป็น

📖 เล่มนี้ต่างจากคอร์ส Python ทั่วไปตรงไหน

คอร์สทั่วไปสอนไวยากรณ์แล้วให้ทำโจทย์ที่ไม่มีใครทำจริง · เล่มนี้ทุกตัวอย่างคือปัญหาจริงของคนเทรด — คำนวณ return, หา hedge ratio, ทดสอบ cointegration, เขียน backtest, ต่อ WebSocket รับราคาสด และเล่มนี้ให้ความสำคัญกับสิ่งที่คอร์สอื่นข้าม: วิธีอ่านโค้ดที่คนอื่น (หรือ AI) เขียน · วิธีจับบั๊กที่ไม่ทำให้โปรแกรม error แต่ทำให้ผลลัพธ์ผิด ซึ่งเป็นบั๊กที่แพงที่สุดในสายนี้

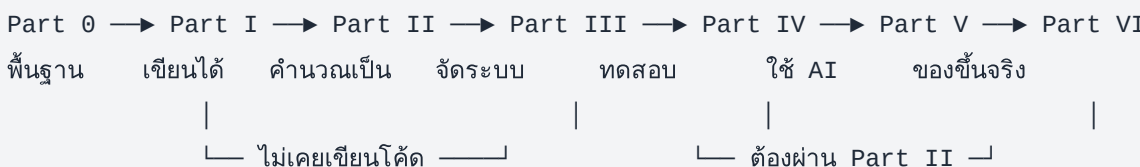
🔍 อ่านยังไงดี — เลือกเส้นทางของคุณ

🟢 ไม่เคยเขียนโค้ดเลย — เริ่ม Part 0 → เรียงไปตามลำดับ ห้ามข้าม · ในแต่ละบท ให้หากล่อง 📌 อ่านว่า: (แปลโค้ดเป็นภาษาคน) และ 🧠 แกะทีละชั้น ก่อนพยายามเข้าใจโค้ดเอง · เจอบล็อกโค้ดยาว ๆ ให้มองหากล่อง 🧱 โครงก่อนดูโค้ด ที่อยู่เหนือนั้น

🟡 เขียนโค้ดเป็นแล้ว มาหาเรื่อง quant — กวาด Part 0-I ผ่านได้ เริ่มจริงที่ Part II · ของสำหรับคุณคือกล่อง

🔵 [รอบสอง] และ ⚠️ กับดัก ที่กระจายทั่วเล่ม — เป็นเรื่องที่คนเขียนโค้ดเป็นแล้วยังพลาดกันประจำ

🟠 อยากได้เฉพาะเรื่อง — แต่ละ Part อ่านแยกได้ ยกเว้น Part IV-VI ที่ควรผ่าน Part II มาก่อน



Part 0-I — จากศูนย์ถึงเขียนโค้ดเป็น

Part 0 รากฐาน — ก่อนเขียนโค้ดบรรทัดแรก

บทที่ 1-5 · คณิตศาสตร์ที่ต้องรู้จริง ๆ · สถิติสำหรับมนุษย์ · ตลาดการเงิน 101 · คิดแบบ Quant · Roadmap

ยังไม่ต้องเขียนโค้ด

mean/ σ /correlation

EV

return vs log return

Part I Python Basics — เขียนโค้ดตัวแรก

บทที่ 6-10 · Setup + **virtual environment** · Control Flow · Data Structures + `with` · NumPy/Pandas ·

Visualization (เส้น · scatter · histogram + bell curve · box plot)

venv

อ่านโค้ดออกเสียง

DataFrame

`.shift(1)`

fat tail

Part II-III — คำนวณเป็น แล้วจัดระบบ

Part II Math Tools — เครื่องมือคำนวณของ Quant

บทที่ 11-17 · Statistics เชิงลึก · Time Series · OLS Regression · **Cointegration & Z-score** · Optimization · Linear Programming · Signals & Filters

VaR/CVaR

ADF · Johansen

hedge ratio

half-life

Kelly

Part III OOP — สร้างระบบที่ขยายได้

บทที่ 18-21 · Classes & Objects · Inheritance + Abstract Base Class · Design Patterns · Mini-Project ระบบเทรดครบวงจร

@property

ABC

Strategy/Observer/Factory

fit แยกจาก signal

Part IV-VI — ทดสอบ · ใช้ AI · ขึ้นจริง

Part IV Backtesting — พิสูจน์ว่ากลยุทธ์ใช้ได้จริงไหม

บทที่ 22-25 · Vectorized · Event-Driven · **Walk-Forward Validation** · Slippage, Fees & Reality

lookahead bias

overfitting

Sharpe/Calmar

ต้นทุนฆ่ากลยุทธ์

Part V AI-Assisted Coding — ใช้ AI โดยไม่โดนมันหลอก

บทที่ 26–30 · Prompting · Code Generation Workflow · **Auditing AI Code** · AI + Quant Libraries · Full Strategy with AI

5 red flags

embargo test

หาบั๊กที่ไม่ error

Part VI Async & Live Trading — ของขึ้นจริง

บทที่ 31–35 · Async Fundamentals · asyncio Patterns · WebSocket & Live Data · Order Execution · Live System Integration

async/await

reconnect + backoff

rate limit

kill switch

เครื่องมือประกอบ

Appendix คำศัพท์ Quant Trading

6 หมวด · สถิติพื้นฐาน · การวัดผลกลยุทธ์ · แนวคิดตลาด · กลยุทธ์ · เครื่องมือสถิติ · Python และเทคนิค — เปิดค้างไว้ข้างๆ ตอนอ่าน

อ่านก่อนเริ่ม — เรื่องเงินจริง

- ทดสอบบน Testnet เสมอ — อย่างนำโค้ดจากเล่มนี้ (หรือจากที่ไหนก็ตาม) ไปรันบนบัญชีจริงโดยตรง
- ห้าม **hardcode API Key** — ใช้ environment variable หรือ secrets manager เท่านั้น
- ตัวเลขในเล่มนี้มาจากข้อมูลจำลอง ที่ออกแบบมาเพื่อสอนแต่ละบท *ไม่ใช่* ผลตอบแทนที่คาดหวังได้จริง — บางบทจงใจใช้ข้อมูลที่ไม่มี edge เลยเพื่อให้เห็นว่า overfitting หน้าตาเป็นอย่างไร

มาตรฐานของเล่มนี้ — ทำไมถึงเชื่อโค้ดในเล่มได้

โค้ดทุกบล็อกถูกสกัดออกมารันจริงบน Python 3.11 + pandas / numpy / scipy / statsmodels รุ่นล่าสุด แบบไล่ทีละบล็อกจากต้นบทเหมือนที่ผู้อ่านทำ · และผลลัพธ์ในกล่อง ► OUTPUT ถูกเทียบกับผลรันจริงทุกบรรทัด

เหตุผลที่ต้องเข้มขนาดนี้: ถ้าผู้อ่านรันตามแล้วได้ตัวเลขไม่ตรงกับหนังสือ **เขาจะคิดว่าตัวเองทำพลาด** แล้วเสียเวลาหาสิ่งที่ไม่ได้ผิด — ซึ่งเป็นประสบการณ์ที่ทำลายความมั่นใจของมือใหม่มากที่สุด

เล่มอื่นในชุดเดียวกัน

เล่มนี้เน้น "เขียนโค้ดเป็น" · ถ้าอยากลงลึกฝั่งทฤษฎีและคณิตศาสตร์ มีเล่มอื่นในโฟลเดอร์เดียวกัน:

- คณิตศาสตร์สำหรับ Options Trading — payoff chart, Black-Scholes, Greeks, Monte Carlo
- ทฤษฎีของ Quant — แผนี่ทฤษฎีทั้งสนาม 1900 → ปัจจุบัน (Random Walk, Portfolio Theory, EMH, Options Pricing, Time Series)

PYTHON FOR QUANT TRADERS · PART 0

รากฐาน — ก่อนเขียนโค้ดบรรทัดแรก

คณิตศาสตร์ · สถิติ · ตลาดการเงิน · Quant Mindset · Roadmap

ออกแบบสำหรับทุกคน — ไม่ว่าจะจบสายอะไร

🚫 อ่าน Part นี้ยังไ

● **ไม่เคยเขียนโค้ด / ไม่ได้จบสายคำนวณ** — Part นี้ไม่มีโค้ดให้เขียนเลย อ่านสบาย ๆ ได้ · เป้าหมายคือให้คุ้นกับ 4 คำที่จะเจอตลอดเล่ม: **mean · σ (ความผันผวน) · correlation · return** · ถ้าเจอสูตรที่ยังไม่เข้าใจให้ข้ามไปก่อน แล้วกลับมาอ่านใหม่ตอนเจอมันในโค้ดจริง — คราวนั้นจะเข้าใจง่ายขึ้นมาก

● **มีพื้นสถิติ/การเงินอยู่แล้ว** — กวาดผ่านได้ทั้ง Part · จุดเดียวที่ควรหยุดอ่านคือ **บทที่ 3.3 (return vs log return)** เพราะเล่มนี้ใช้ทั้งสองแบบสลับกันตามบริบท และ **บทที่ 4 (คิดแบบ Quant)** ที่วางกรอบว่าทำไมทั้งเล่มถึงหมกมุ่นกับ "ระแวงตัวเลขของตัวเอง"

วิสัยทัศน์

โลกเปลี่ยนแล้ว — ไม่ต้องจบสายการเงินหรือ CS ก็เขียนโค้ดวิเคราะห์ตลาดได้

หมอที่อ่านหนังสือเล่มนี้จบจะเห็นว่า p -value ในงานวิจัยยาก กับ p -value ใน regression ของราคาหุ้น คือสิ่งเดียวกัน ทนายที่อ่านจบจะเห็นว่า LP constraint $x_1 + x_2 \leq 1$ คือ "เงื่อนไขสัญญา" ในภาษาคณิตศาสตร์

Part 0 ปูพื้น 5 เรื่องที่จำเป็น — ทั้งหมดเชื่อมตรงกับโค้ดที่จะเขียนใน Part I ถึง VI

แผนที่หนังสือ — 35 บท 7 ส่วน

Part 0

รากฐาน
บท 1–5

← **อยู่**ที่นี่

Part I

Python Basics
บท 6–10

Part II

Math Tools
บท 11–17

Part III

OOP
บท 18–21

Part IV

Backtesting
บท 22–25

Part V

AI-Assisted
บท 26–30

Part VI

Live Trading
บท 31–35

บทที่ 1: คณิตศาสตร์ที่ต้องรู้จริงๆ

แก่นของบทนี้

คณิตศาสตร์ทั้งหมดใน Quant สรุปได้เป็น 4 ความคิด: ฟังก์ชัน (input → output), ผลรวม Σ (บวกหลายค่า), เวกเตอร์ (รายการเลข), เมทริกซ์ (ตารางเลข) — ทั้งหมดแปลเป็น Python ได้ตรงตัว

1.1 ฟังก์ชัน — เครื่องแปลงค่า

ฟังก์ชันคือกฎที่บอกว่า "ใส่อะไรเข้าไป ได้อะไรออกมา" เหมือนสูตรอาหาร

$$f(x) = 2x + 3$$

อ่านว่า: "ฟังก์ชัน f ของ x เท่ากับ สองเท่าของ x บวกสาม"

ตัวอย่าง: $f(5) = 2(5) + 3 = 13$ | $f(10) = 2(10) + 3 = 23$

Running Example: ฟังก์ชันใน Quant

ผลตอบแทนรายวัน (Return) เป็นฟังก์ชันของราคา:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

อ่านว่า: " r ณ เวลา t เท่ากับ ราคาวันนี้ ลบ ราคาเมื่อวาน หารด้วย ราคาเมื่อวาน"

ตัวอย่าง: $P_{t-1} = 100, P_t = 103 \rightarrow r_t = \frac{103 - 100}{100} = 3\%$

ใน Python: `r = (price_today - price_yesterday) / price_yesterday` — เราจะเขียนสิ่งนี้ใน Part I

ฟังก์ชันหลายตัวแปร

ฟังก์ชันรับ input ได้มากกว่า 1 ตัว เช่น ความดันโลหิตเป็นฟังก์ชันของหลายปัจจัย:

$$BP = \alpha + \beta_1 \cdot \text{น้ำหนัก} + \beta_2 \cdot \text{อายุ} + \beta_3 \cdot \text{ออกกำลังกาย}$$

ใน Quant เราทำสิ่งเดียวกัน: ผลตอบแทนหุ้น = ฟังก์ชันของ (ตลาดรวม, อุตสาหกรรม, ข่าว, ...)

1.2 ผลรวม Σ — บวกหลายค่าพร้อมกัน

$$S = \sum_{i=1}^n x_i = x_1 + x_2 + x_3 + \dots + x_n$$

อ่านว่า: "ซิกมาของ x_i ตั้งแต่ $i=1$ ถึง n " = "บวก x ทุกตัวเข้าด้วยกัน"

ตัวอย่าง: ผลกำไรรายสัปดาห์

กำไร 5 วัน: $x = [+200, -150, +300, -80, +120]$ บาท

$$\sum_{i=1}^5 x_i = 200 + (-150) + 300 + (-80) + 120 = +390 \text{ บาท}$$

ใน Python: `sum([200, -150, 300, -80, 120])` คืนค่า 390

► แบบฝึกหัด: คำนวณกำไรรวม

1.3 เวกเตอร์ — รายการตัวเลขที่เรียงลำดับ

เวกเตอร์คือ รายการตัวเลข ไม่มีอะไรน่ากลัวกว่านั้น ใน Python คือ list หรือ numpy array

$$P = [100, 105, 98, 110, 107]$$

แปลว่า: ราคาหุ้น 5 วัน (เวกเตอร์ขนาด 5 มิติ)

คอลัมน์ Python ในตารางนี้แค่ให้เห็นภาพล่วงหน้า — ยังไม่ต้องเข้าใจ เราจะเรียนเขียนจริงใน Part I

Operation พื้นฐานบนเวกเตอร์

Operation	ความหมาย	Python
$x + y$	บวกทีละ element	<code>x + y</code> (numpy)
$c \cdot x$	คูณทุก element ด้วย c	<code>c * x</code>
$x \cdot y$	dot product: $\sum x_i y_i$	<code>np.dot(x, y)</code>
$\ x\ $	ความยาวเวกเตอร์: $\sqrt{\sum x_i^2}$	<code>np.linalg.norm(x)</code>

1.4 เมทริกซ์ — ตาราง Excel ที่มีตัวเลขทุกช่อง

เมทริกซ์คือตาราง $m \times n$ (m แถว, n คอลัมน์) ใน Python คือ pandas DataFrame หรือ numpy 2D array

$$A = \begin{pmatrix} 100 & 200 & 150 \\ 105 & 195 & 148 \\ 98 & 210 & 155 \end{pmatrix}$$

แถว = วัน | คอลัมน์ = หุ้น (AAPL, MSFT, NVDA)

$A[0][0] = 100$ หมายถึง AAPL วันที่ 1 ราคา 100

Matrix ที่สำคัญที่สุดใน Quant: Covariance Matrix

เมทริกซ์ความแปรปรวนร่วมบอกว่าแต่ละคู่หุ้นเดินสัมพันธ์กันแค่ไหน:

$$\Sigma = \begin{pmatrix} \sigma_A^2 & \sigma_{AB} \\ \sigma_{AB} & \sigma_B^2 \end{pmatrix}$$

ตัวเลขในแนวทแยง = ความแปรปรวนของแต่ละหุ้น | นอกแนวทแยง = ความแปรปรวนร่วมของคู่หุ้น
ใช้ใน Part II สำหรับ portfolio optimization

สรุปความเชื่อมโยงกับ Python

คณิตศาสตร์	ใน Python	เริ่มเรียนที่
ฟังก์ชัน $f(x)$	<code>def f(x): return ...</code>	Part I บทที่ 7
ผลรวม Σ	<code>sum() / np.sum()</code>	Part I บทที่ 9
เวกเตอร์	<code>list / np.array</code>	Part I บทที่ 9
เมทริกซ์	<code>pd.DataFrame</code>	Part I บทที่ 9

บทที่ 2: สถิติสำหรับมนุษย์

แก่นของบทนี้

สถิติคือการ "สรุปข้อมูลจำนวนมากให้เป็นตัวเลขไม่กี่ตัว" 5 concept ที่ต้องรู้: ค่าเฉลี่ย, ส่วนเบี่ยงเบน, การกระจายปกติ, correlation, และ regression — ทั้งหมด Python คำนวณได้ใน 1-2 บรรทัด แต่ต้องรู้ *ความหมาย* ก่อนจึงจะตีความผลลัพธ์ได้ถูก

2.1 ค่าเฉลี่ยและส่วนเบี่ยงเบน — อยู่ตรงไหน กระจายแค่ไหน?

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n} \qquad \sigma = \frac{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}}{n}$$

อ่านว่า: "x (x-bar) = ค่าเฉลี่ย" | "σ (sigma) = ส่วนเบี่ยงเบนมาตรฐาน = ค่าเฉลี่ยของ ระยะห่างจากค่าเฉลี่ย"

Running Example: ผลตอบแทนรายวัน

ผลตอบแทน 5 วัน: $r = [+1\%, -0.5\%, +2\%, -1\%, +0.5\%]$

$$r = \frac{1 + (-0.5) + 2 + (-1) + 0.5}{5} = \frac{2}{5} = 0.4\%/\text{วัน}$$

$\sigma \approx 1.08\%/\text{วัน}$

แปลว่า: วันปกติผลตอบแทนอยู่ระหว่างประมาณ -0.68% ถึง +1.48% ($\pm 1\sigma$ จาก r)

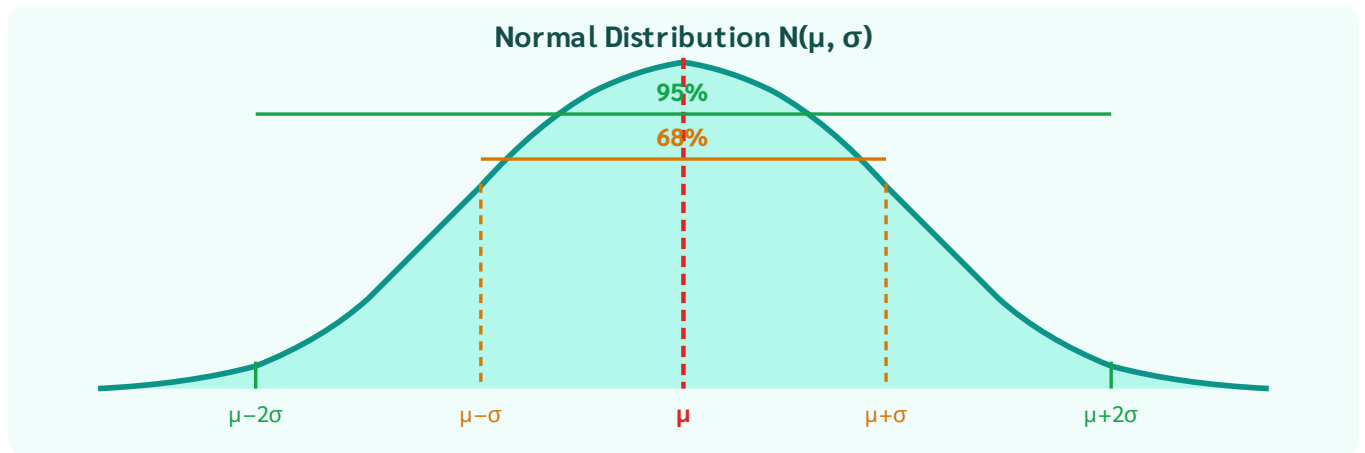
ใน Python: `np.mean(r)` และ `np.std(r)`

ความหมายของ σ ในการเทรด

สินทรัพย์	σ รายวัน	แปลว่า
US Treasury Bond	~0.05%	นิ่งมาก, ผลตอบแทนต่ำ
หุ้นขนาดใหญ่ (S&P500)	~1%	ปกติสำหรับหุ้น
BTC	~3-5%	ผันผวนสูง, โอกาส/ความเสี่ยงมาก
Altcoin ขนาดเล็ก	~10%+	เก็งกำไรสูงมาก

2.2 Normal Distribution — รูประฆัง

ข้อมูลหลายอย่างในธรรมชาติกระจายเป็น "รูประฆัง" (Bell Curve) รอบค่าเฉลี่ย μ ด้วยความกว้าง σ



กฎ 68–95–99.7: ข้อมูล 68% อยู่ใน $\mu \pm \sigma$, 95% อยู่ใน $\mu \pm 2\sigma$, 99.7% อยู่ใน $\mu \pm 3\sigma$

ตัวอย่าง: กฎ 68–95–99.7 กับหุ้น

BTC มีผลตอบแทนรายวัน $\mu = 0\%$, $\sigma = 3\%$

- วันส่วนใหญ่ (68%) BTC เคลื่อนที่ระหว่าง -3% ถึง $+3\%$
- วันที่ตก $> 6\%$ (เกิน 2σ) เกิดได้ประมาณ 5% ของวันทั้งหมด = ประมาณ 12–13 วัน/ปี
- วันที่ตก $> 9\%$ (เกิน 3σ) เกิดได้ประมาณ 0.3% = ประมาณ 1 วัน/ปี

2.3 Correlation — สองตัวแปรเดินไปด้วยกันไหม?

$$r_{XY} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}} \in [-1, +1]$$

$r = +1$: เดินทิศเดียวกันสมบูรณ์ | $r = 0$: ไม่สัมพันธ์ | $r = -1$: เดินสวนทางสมบูรณ์

Correlation $\neq$ Causation (ความสัมพันธ์ $\neq$ เป็นสาเหตุ)

ไอศกรีมขาย correlate กับอัตราการจมน้ำ ($r \approx +0.9$) — แต่ไม่ใช่เพราะกินไอศกรีมแล้วจมน้ำ แต่เพราะ "อากาศร้อน" เป็นตัวแปรที่ซ่อนอยู่ (confounding variable)

ใน Quant: สองหุ้นอาจ correlate กันโดย *ไม่* cointegrate กัน — ต่างกันมาก เราจะเรียนใน Part II

2.4 p-value — มั่นใจแค่ไหนว่าไม่ใช่ความบังเอิญ?

p-value ตอบคำถามว่า: "ถ้าสมมติว่าไม่มีผลจริง (null hypothesis) โอกาสที่จะเห็นข้อมูลแบบนี้โดยบังเอิญคือเท่าไร?"

Running Example: ทดสอบกลยุทธ์

กลยุทธ์ทำกำไรเฉลี่ย +0.1%/วัน ใน 252 วัน (1 ปี)

p-value = 0.03 หมายความว่า:

→ "ถ้ากลยุทธ์ไม่มีประสิทธิภาพจริง (random) โอกาสที่จะเห็นผลดีขนาดนี้โดยบังเอิญ = 3%"

→ ต่ำกว่า 5% = **มีนัยสำคัญทางสถิติ** (statistically significant)

⚠ คำว่า "มีนัยสำคัญ" ตรงนี้แปลว่า "ไม่น่าจะบังเอิญ" เท่านั้น — ไม่ได้แปลว่า "ทำเงินได้" . สองอย่างนี้เป็นคนละคำถามที่ต้องตอบด้วยเครื่องมือคนละชิ้น และการเอามาปนกันคือความผิดพลาดที่แพงที่สุดในสายงานนี้ . เก็บความสงสัยไว้ก่อน — หัวข้อ 25.4b ใน Part IV จะแสดงกลยุทธ์ที่ผ่านการทดสอบทางสถิติอย่างขาดลอยแต่ขาดทุนทุกเดือน

ใน Python ใช้ library ชื่อ statsmodels (จะเขียนโค้ดนี้จริงใน Part II บทที่ 13)

2.5 Regression — หาเส้นที่พอดีที่สุด

สูตรหลัก — OLS REGRESSION

$$y = \alpha + \beta x + \varepsilon$$

α (alpha) = intercept = ค่า y เมื่อ $x = 0$

β (beta) = slope = ถ้า x เพิ่ม 1 หน่วย y เปลี่ยนไป β หน่วย

ε (epsilon) = error = ส่วนที่ model ทำนายไม่ได้

OLS (Ordinary Least Squares) หาค่า α, β ที่ทำให้ $\sum \varepsilon_i^2$ น้อยที่สุด — เรียนลึกใน Part II บทที่ 13

ตัวอย่าง: Hedge Ratio ระหว่างสองหุ้น

เราต้องการหาว่า AAPL เคลื่อนที่ตาม MSFT แค่ไหน:

$$r_{AAPL} = \alpha + \beta \cdot r_{MSFT} + \varepsilon$$

ผล OLS: $\beta = 0.85$ หมายความว่า

→ ทุกครั้งที่ MSFT ขึ้น 1% AAPL มักขึ้น 0.85%

→ ถ้าต้องการ hedge AAPL \$100K ด้วย MSFT → ต้อง short MSFT \$85K

```
# Python (Part II)
import statsmodels.api as sm
X = sm.add_constant(r_msft)
model = sm.OLS(r_aapl, X).fit()
beta = model.params['r_msft'] # = 0.85
```

บทที่ 3: ตลาดการเงิน 101

แก่นของบทนี้

ไม่ต้องรู้จักหุ้นทุกตัว แค่รู้ 4 concept: **สินทรัพย์** (มีอะไรซื้อขายได้บ้าง), **Return** (วัดผลกำไร/ขาดทุนอย่างไร), **Risk = Volatility** (วัดความไม่แน่นอน), **Arbitrage** (หลักการหากำไรไร้ความเสี่ยง) — 4 concept นี้รองรับโค้ดที่จะเขียนตลอดเล่ม

3.1 ประเภทสินทรัพย์

ประเภท	ความหมายง่ายๆ	ตัวอย่าง
หุ้น (Stocks)	ส่วนแบ่งความเป็นเจ้าของบริษัท — กำไรขาดทุนตามบริษัท	AAPL, PTT, ADVANC
พันธบัตร (Bonds)	กู้ยืมเงิน ได้ดอกเบี้ยแน่นอน — ความเสี่ยงต่ำกว่าหุ้น	US Treasury, พันธบัตรรัฐบาลไทย
FX / Crypto	แลกเปลี่ยนระหว่างสกุลเงิน — ราคาขึ้นลงตามอุปสงค์อุปทาน	USD/THB, BTC/USDT
Derivatives	สัญญาที่ราคาขึ้นกับสินทรัพย์อื่น — leverage สูง, ซับซ้อน	Futures, Options, Perpetuals

3.2 Price vs Return

นักวิเคราะห์ทำงานกับ Return มากกว่า Price เพราะ Return เปรียบเทียบข้ามสินทรัพย์ได้

$$r_t^{\text{simple}} = \frac{P_t - P_{t-1}}{P_{t-1}} \quad r_t^{\text{log}} = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Simple vs Log Return — เลือกอันไหน?

	Simple Return	Log Return
สูตร	$(P_t - P_{t-1})/P_{t-1}$	$\ln(P_t/P_{t-1})$
บวกข้ามเวลา	ไม่ได้โดยตรง	ได้ (additive)
กระจายเป็น Normal	ใกล้เคียง	ดีกว่า
ใช้เมื่อ	ผลตอบแทนระยะสั้น (ต่ำกว่า 5%)	วิเคราะห์เชิงสถิติ, cointegration

สำหรับค่าน้อยๆ: $\log \text{ return} \approx \text{simple return}$ (ต่างกันน้อยมาก)

3.3 Risk = Volatility

ใน Finance **Risk = ความไม่แน่นอน = Standard Deviation ของ Return** เรียกว่า Volatility (σ)

ตัวอย่าง: Annualized Volatility

BTC มี $\sigma_{\text{daily}} = 3\%$ ต่อวัน

แปลงเป็นรายปี: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{T}$ โดย $T =$ จำนวนวันซื้อขายต่อปี

$$\sigma_{\text{annual}} = 3\% \times \sqrt{252} \approx 47.6\%/\text{ปี}$$

แปลว่า: ใน 1 ปี ราคา BTC มักเคลื่อนที่ไปมา $\pm 47.6\%$ จากค่าเฉลี่ย

หมายเหตุ: หุ่นไทย/สหรัฐใช้ $\sqrt{252}$ (252 วันทำการ) — แต่ Crypto เปิด 24/7 ใช้ $\sqrt{365}$

BTC: $3\% \times \sqrt{365} \approx 57.3\%/\text{ปี}$ — ตัวเลขต่างกันไม่ใช่ผิด แต่ต้องใช้ให้สม่ำเสมอ

3.4 Arbitrage — ซื้อถูก ขายแพง พร้อมกัน

Arbitrage คือการทำกำไรโดย **ไม่มีความเสี่ยง** จากราคาที่ไม่สมดุลกันระหว่างตลาด



ทำไม Pure Arbitrage ถึงหายากขึ้นเรื่อยๆ?

เมื่อมีคนเห็น Arb → รีบซื้อตลาด A + ขายตลาด B → ราคา A สูงขึ้น, ราคา B ลง → ส่วนต่างหด → หายภายใน ms-วินาที

โอกาสที่เหลือต้องการ: **ความเร็ว** (automated bot), **ข้อมูลที่คนอื่นไม่มี**, หรือ **capital ขนาดใหญ่** สำหรับโอกาสที่ margin น้อย

3.5 Portfolio และ Diversification

$$R_p = \sum_{i=1}^n w_i R_i \quad \sum_{i=1}^n w_i = 1$$

อ่านว่า: "ผลตอบแทน portfolio เท่ากับ ผลรวมถ่วงน้ำหนักของผลตอบแทนแต่ละสินทรัพย์"

Diversification: กระจายเพื่อลด Risk โดยไม่ลด Return

ถ้าหุ้น A และ B มี $r = -1$ (สวนทางสมบูรณ์) → ถือ 50%/50% → portfolio ไม่ขึ้นไม่ลง

ถ้า $r = 0$ (ไม่สัมพันธ์) → $\sigma_p = \sigma/\sqrt{2}$ = ลด risk ได้ 30% โดยไม่ลด return

กฎแฉะ: หา assets ที่ correlation ต่ำ เพื่อกระจาย risk ได้จริง

บทที่ 4: คิดแบบ Quant

แก่นของบทนี้

ทักษะ coding และ math เป็นแค่เครื่องมือ สิ่งที่แยก Quant ออกจากคนทั่วไปคือ **วิธีคิด** — ตัดสินใจด้วยตัวเลข ไม่ใช่ความรู้สึก, รู้ว่าโมเดลมีขีดจำกัด, คิดเรื่อง expected value ก่อนเสมอ

4.1 ทุกอย่างเป็น Data

สิ่งที่เห็น	Quant มองว่า	ใช้ทำอะไร
ราคาหุ้น	Time series ของ returns	หา pattern, signal
ข่าว	Sentiment score [-1, +1]	NLP model ทำนายทิศ
ปริมาณซื้อขาย	Volume series	ยืนยัน signal, หา liquidity
Fear ตลาด	VIX index	ปรับ position size
คำพูด Fed	Hawkish/Dovish score	ปรับ model สำหรับ macro regime

Running Example: หมอกับ Quant คิดคล้ายกัน

หมอไม่ "รู้สึก" ว่าคนไข้เป็นอย่างไร — วัด: BP, HR, SpO₂, GCS, Temp
Quant ไม่ "รู้สึก" ว่าตลาดจะขึ้นหรือลง — ดู data: price, volume, σ , correlation, macro
ทั้งคู่ ตัดสินใจจาก signal ที่วัดได้ ไม่ใช่ intuition — แต่ทั้งคู่ก็รู้ว่า signal ไม่สมบูรณ์เสมอ

4.2 Model = แผนที่ $\neq$ ดินแดน

"All models are wrong, but some are useful" — George E.P. Box

แผนที่กรุงเทพฯ ไม่ใช่กรุงเทพฯ จริง — ไม่มีต้นไม้ไม่มีเสียงไม่มีกลิ่น
แต่แผนที่ช่วยนำทางได้ — และนั่นคือสิ่งที่เราต้องการ

โมเดล Quant เช่นกัน: ไม่ได้อธิบายตลาดครบทุกมิติ แต่ช่วยตัดสินใจได้ดีกว่าไม่มีโมเดล
อันตรายคือการลืมนึกว่าโมเดลคือแผนที่ แล้วเชื่อมั่นมันคือความจริงสมบูรณ์

4.3 Expected Value — ตัดสินใจด้วยค่าคาดหวัง

สูตรหลัก

$$E[X] = \sum_i p_i \cdot x_i$$

p_i = ความน่าจะเป็นที่ผลลัพธ์ i จะเกิด (ทุก p_i รวมกัน = 1)

x_i = มูลค่าของผลลัพธ์ i (บวก = กำไร, ลบ = ขาดทุน)

Quant ดูที่ $E[X]$ ก่อนเสมอ ไม่ใช่แค่ "น่าจะชนะ"

ตัวอย่าง: เปรียบเทียบสองกลยุทธ์ด้วย Expected Value

กลยุทธ์ A

ชนะ 70% ได้ +฿100 | แพ้ 30% เสีย -฿300

ชนะ 70%

+฿100 → +฿70

แพ้ 30%

-฿300 → -฿90

$E[A] = +70 - 90 = -฿20$ ❌

กลยุทธ์ B

ชนะ 40% ได้ +฿800 | แพ้ 60% เสีย -฿200

ชนะ 40%

+฿800 → +฿320

แพ้ 60%

-฿200 → -฿120

$E[B] = +320 - 120 = +฿200$ ✓

กลยุทธ์ A ชนะบ่อยกว่า (70%) แต่ $E[X]$ ติดลบ — กลยุทธ์ B แพ้บ่อยกว่า (60%) แต่ $E[X]$ เป็นบวกมาก

4.4 Backtesting — ทดสอบก่อนใช้เงินจริง

Backtesting คือการนำกลยุทธ์ไป "เล่นซ้ำ" กับข้อมูลราคาในอดีตเพื่อดูว่าถ้าใช้ในอดีตจะได้ผลลัพธ์อย่างไร

อันตราย: **Overfitting** = การเรียน "คำตอบสอบ" ไม่ใช่ "วิชา"

ถ้าปรับ parameter มากพอ กลยุทธ์ใดๆ ก็ดูดีในข้อมูลเก่าได้เสมอ

เหมือนจำคำตอบข้อสอบ — ผ่านข้อสอบเก่าได้ แต่ข้อสอบจริง (live market) ไม่ได้

วิธีป้องกัน: ทดสอบกับข้อมูลที่โมเดลไม่เคยเห็น (*out-of-sample*) — เราจะเรียนใน Part IV

► แบบฝึกหัด: คิดแบบ Quant

บทที่ 5: Roadmap & วิธีอ่านหนังสือเล่มนี้

แก่นของบทนี้

หนังสือเล่มนี้ออกแบบให้ **อ่านตามลำดับ** — แต่ละ Part สร้างบนของเดิม ถ้าติดในบท X ให้อ่านต่อไปก่อน เพราะบางครั้ง context ข้างหน้าช่วยให้เข้าใจย้อนกลับมาได้ดีกว่า

5.1 AI ช่วยได้ที่ไหนบ้าง?

หนังสือเล่มนี้ไม่ได้สอนให้ "จำ syntax" — สอนให้ **รู้ว่าการทำอะไร และตรวจสอบ AI ได้**

AI ทำได้ดี	คุณต้องรู้เอง
เขียน boilerplate / template code	ว่า algorithm ที่ต้องการคืออะไร
แปลง logic เป็น Python syntax	ว่า logic นั้นถูกต้องทาง math ไหม
Debug error message	ว่า root cause จริงคืออะไร
เขียน OLS / optimization ให้	ว่า β หรือ output ที่ได้มีความหมายอะไร
หา library และ function ที่เหมาะสม	ว่าผลลัพธ์สมเหตุสมผลไหม

Running Example: Prompt ที่ดีสำหรับ Quant

```
# Prompt ที่ไม่ดี:
"เขียน trading bot ให้หน่อย"

# Prompt ที่ดี:
"เขียน Python function ที่รับ DataFrame ของราคา 2 assets
คืนค่า hedge ratio beta จาก OLS regression (log prices)
และ z-score ของ spread: log(P1) - beta * log(P2)
ใช้ statsmodels.OLS, normalize z-score ด้วย rolling window 60 วัน"
```

Prompt ที่ดีต้องระบุ: input, output, method, library — เราจะเรียนใน Part V

5.2 Learning Path ตามอาชีพ

พื้นฐานที่มี	Part ที่ต้องใส่ใจเป็นพิเศษ	ข้อได้เปรียบ
หมอ / นักวิทยาศาสตร์	Part II (statistics ลึก)	p-value, hypothesis testing ค่อนข้างเคย
วิศวกร / IT	Part III, VI (OOP, async)	programming concepts ค่อนข้างเคย
นักการเงิน / Trader	Part I (Python basics)	ตลาด, risk management ค่อนข้างเคย
ทนาย / นักบัญชี	Part II (LP constraints)	คิด constraint, rule-based ได้ดี
ไม่มีพื้นฐานใดเลย	Part 0 อ่านซ้ำๆ ทำแบบฝึกหัดทุกข้อ	ไม่มี bad habit จากสาขาอื่น

5.3 Environment ที่ต้องติดตั้งก่อน Part I

- 1. **Python 3.10+** — ภาษาหลัก ดาวน์โหลดจาก python.org
- 2. **VS Code** — โปรแกรมเขียนโค้ด (ฟรี) + Python extension
- 3. **Conda หรือ pip** — จัดการ library
- 4. **AI Assistant** (Claude, ChatGPT, Gemini) — คู่หูตลอดการเรียนรู้

บทที่ 6 สอนขั้นตอนการติดตั้งที่ละเอียดพร้อม screenshot — ไม่ต้องทำตอนนี้

5.4 Checklist ก่อนเริ่ม Part I

► ตรวจสอบความพร้อม — [คลิกเพื่อดู](#)

จบ Part 0 — คุณพร้อมแล้ว

ตอนนี้คุณมีเครื่องมือทางความคิดครบ: **Math** (ฟังก์ชัน, Σ , เวกเตอร์, เมทริกซ์) + **สถิติ** (mean, σ , correlation, regression) + **Finance** (assets, return, risk, arb) + **Quant mindset** (EV, model = map, backtest)

ใน Part I เราจะเริ่มแปลง concept เหล่านี้เป็น Python code ที่รันได้จริง — บรรทัดแรกของคุณรอคุณอยู่ในบทที่ 6

PYTHON FOR QUANT TRADERS · PART I

Python Basics — เริ่มเขียนโค้ดบรรทัดแรก

บทที่ 6–10: Setup · Variables · Control Flow · Data Structures · NumPy/Pandas · Visualization

จบ Part นี้ → โหลดข้อมูลตลาด วิเคราะห์ และ plot กราฟได้จริง

🚫 อ่าน Part นี้ยังไ

● **เขียนโค้ดครั้งแรก** — อ่านช้า ๆ ตามลำดับ ห้ามข้าม · บทที่ 6.3 "อ่านโค้ดให้ออกเสียง" คือหัวใจของทั้ง Part — ถ้าอ่านโค้ดเป็นประโยคภาษาคนได้ ที่เหลือจะง่ายขึ้นทั้งหมด · ในทุกบทให้หากล่อง 📦 **อ่านว่า:** ก่อนพยายามแกะโค้ดเอง · และพิมพ์ตามจริง ๆ อย่าแค่อ่าน — นิวจำได้เร็วกว่าตาจำ

● **เขียน Python เป็นแล้ว** — ข้ามบทที่ 6-8 ได้ แต่อย่าข้าม 6.1 (virtual environment) ถ้ายังไม่เคยใช้ · เริ่มจริงที่บทที่ 9 (NumPy/Pandas) และ 10 (Visualization) · ของสำหรับคุณ: กล่อง 📦 **แกะทีละชั้น** เรื่อง list comprehension, กับดัก pandas alignment ในบทที่ 9, และหัวข้อ 10.4 ที่ทำให้ fat tail กลายเป็นตัวเลขที่เอียงไม่ได้

📌 สิ่งที่จะทำได้เมื่อจบ PART I

- โหลดข้อมูลราคาหุ้น/crypto จาก CSV เข้า Python
 - คำนวณ daily return, rolling mean, volatility ได้ใน 3-5 บรรทัด
 - กรองข้อมูล หาวันที่ผลตอบแทนเกิน threshold ได้
 - plot กราฟราคาและ return ได้ พร้อม label และ title
- ทุกอย่างนี้เป็นพื้นฐานของ quant analysis ทั้งหมดที่จะตามมา

บทที่ 6: Setup & Python Basics

แก่นของบทนี้

Python คือภาษาที่ "อ่านออกเหมือนภาษาอังกฤษ" — ไม่ต้องประกาศ type, ไม่ต้อง compile, รันได้ทันที
บทนี้ตั้ง environment และสร้างความคุ้นเคยกับ variable, type, และ operator ซึ่งเป็นอริฐกัอนแรกของ
ทุก program

6.1 ติดตั้ง Environment

ขั้นตอน 5 ขั้น (ทำครั้งเดียวต่อโปรเจกต์)

1. **Python 3.10+** — ดาวน์โหลดจาก python.org เลือก "Add to PATH" ตอนติดตั้ง
2. **VS Code** — ดาวน์โหลดจาก code.visualstudio.com → ติดตั้ง Extension "Python" ของ Microsoft
3. **สร้างโฟลเดอร์โปรเจกต์ + virtual environment** — สำคัญมาก อธิบายด้านล่าง
4. **ติดตั้งไลบรารี** ลงใน environment นั้น
5. **ทดสอบ** — สร้างไฟล์ test.py พิมพ์ `print("Hello Quant")` แล้วรัน (F5 หรือ `python test.py`)

ขั้นที่ 3–4: virtual environment — ขั้นที่คนข้ามแล้วเสียเวลาที่หลังมากที่สุด

คำสั่ง `pip install pandas` เฉย ๆ จะติดตั้งลง **Python กลางของเครื่อง** ซึ่งใช้ร่วมกันทุกโปรเจกต์ · ตอนมีโปรเจกต์
เดียวก็ดูไม่มีปัญหา แต่พอมีโปรเจกต์ที่สองที่ต้องการ pandas คนละเวอร์ชัน คุณจะติดกับดักที่แก้ยากมาก

```
# — macOS / Linux —
mkdir quant-project && cd quant-project
python3 -m venv .venv          # สร้างกล่องแยกชื่อ .venv
source .venv/bin/activate      # เข้าไปอยู่ในกล่อง

# — Windows (PowerShell) —
mkdir quant-project; cd quant-project
python -m venv .venv
.venv\Scripts\Activate.ps1

# หลัง activate: หน้า prompt จะมี (.venv) นำหน้า
(.venv) $ pip install numpy pandas matplotlib scipy statsmodels

# จดรายการไลบรารีลงไฟล์ เพื่อให้เครื่องอื่น (หรือตัวเราในอนาคต) ทำตามได้
(.venv) $ pip freeze > requirements.txt
```

```
# เครื่องใหม่ / เพื่อนร่วมทีม ใช้แค่บรรทัดเดียว
(.venv) $ pip install -r requirements.txt

# ออกจากกล่องเมื่อเลิกทำงาน
(.venv) $ deactivate
```

📌**อ่านว่า:** อ่าน `python3 -m venv .venv` ว่า: "ใช้ Python สร้างกล่องเปล่าชื่อ `.venv` ไว้ในโฟลเดอร์นี้" — กล่องนี้มี Python กับที่เก็บไลบรารีเป็นของตัวเอง แยกขาดจากเครื่อง · `activate` คือ "เดินเข้าไปอยู่ในกล่อง" ตั้งแต่นั้น `pip install` ทุกครั้งจะลงในกล่องนี้เท่านั้น ไม่ไปยุ่งกับโปรเจกต์อื่น

❌ อาการที่เกิดถ้าข้ามขั้นนี้ — และทำไมมันเจ็บ

- โปรเจกต์เก่าพังเพราะติดตั้งของใหม่ — โปรเจกต์ A ใช้ `pandas 1.5` ได้ดี วันหนึ่งคุณ `pip install` ไลบรารีใหม่ให้โปรเจกต์ B แล้วมันอัปเดต `pandas` เป็น 3.0 → โค้ดโปรเจกต์ A พังทันทีโดยที่คุณไม่ได้แตะมันเลย
- "เครื่องผมรันได้ แต่เครื่องคุณไม่ได้" — ไม่มีรายการว่าต้องใช้อะไรบ้าง เวอร์ชันไหน
- ลบของไม่ได้ — เมื่อทุกอย่างกองรวมกัน คุณจะไม่กล้าถอนอะไรเลยเพราะไม่รู้ว่ามีใครใช้อยู่

🔗กฎ: หนึ่งโปรเจกต์ = หนึ่ง `.venv` · ถ้าเห็น prompt ไม่มี `(.venv)` นำหน้า แปลว่าคุณกำลังติดตั้งลงเครื่องกลางอยู่ — ให้หยุดแล้ว `activate` ก่อน

🔵 [รอบสอง] เกร็ดที่ทำให้เจ็บน้อยลงอีก

- `pip freeze` จดทุกอย่างรวมของที่ติดมาเอง — ได้ไฟล์ยาวเหยียดที่อ่านไม่รู้เรื่อง · ทีมส่วนใหญ่จึงเขียน `requirements.txt` เองเฉพาะที่ใช้ตรง ๆ แล้วปล่อยให้ `pip` จัดการ dependency ที่เหลือ (หรือย้ายไปใช้ `uv` / `poetry` ที่แยก "ที่ฉันขอ" กับ "ที่ตามมา" ให้อัตโนมัติ)
- อย่า `commit` โฟลเดอร์ `.venv` ขึ้น `git` — มันใหญ่หลายร้อย MB และผูกกับ OS ของคุณ · ใส่ `.venv/` ใน `.gitignore` · สิ่งที่คุณ `commit` คือ `requirements.txt` เท่านั้น
- VS Code ต้องรู้ว่าใช้ตัวไหน — กด `Ctrl+Shift+P` → "Python: Select Interpreter" → เลือกตัวที่อยู่ใน `.venv` · ถ้าลืมขั้นนี้ คุณจะเจออาการงงที่สุด: รันใน terminal ได้ แต่กด F5 แล้ว `ModuleNotFoundError`
- ตรวจสอบว่าอยู่ในกล่องจริงไหม — `python -c "import sys; print(sys.prefix)"` ต้องขึ้น path ที่มี `.venv`

Running Example: ตลอด Part I — วิเคราะห์ราคา BTC

ทุกบทใน Part I จะต่อยอดจากตัวอย่างเดียวกัน: โหลดราคา BTC 1 ปี → คำนวณ return → หา signal → plot กราฟ

บท 6 เริ่มจาก variable พื้นฐาน → บท 9 จะเป็น DataFrame จริง → บท 10 จะ plot ออกมาให้เห็น

6.2 Variables และ Types

ใน Python ไม่ต้องประกาศ type — Python รู้เองว่าตัวแปรเก็บอะไร

```
# กำหนดค่าตัวแปร
price = 65000          # int (จำนวนเต็ม)
return_pct = 0.032     # float (ทศนิยม)
symbol = "BTCUSDT"    # str (ข้อความ)
is_long = True         # bool (True/False)

# ดู type
print(type(price))     # <class 'int'>
print(type(return_pct))# <class 'float'>
```

Type	ตัวอย่าง	ใช้กับ
int	65000 , -3 , 0	จำนวนหุ้น, วัน, index
float	0.032 , 65000.5	ราคา, ผลตอบแทน, volatility
str	"AAPL" , "2024-01-01"	ชื่อหุ้น, วันที่, label
bool	True , False	signal on/off, position open/close

ชื่อตัวแปรที่ดีใน Quant Code

- ใช้ snake_case: `daily_return` ไม่ใช่ `dailyReturn`
- ชื่อบอกความหมาย: `btc_price_usd` ดีกว่า `p`
- หลีกเลี่ยง `l` , `0` , `I` (สับสนกับ 1, 0)
- ค่า constant ใช้ตัวใหญ่: `RISK_FREE_RATE = 0.05`

6.3 อ่านโค้ดให้ออกเสียง — ทักษะแรกก่อนหัดเขียน

คนอ่านหนังสือออกเพราะรู้ว่าตัวอักษรแต่ละตัว "ออกเสียง" อย่างไร โค้ดก็เหมือนกัน — ทุกสัญลักษณ์มีเสียงอ่านในหัว ถ้ารู้เสียงอ่าน บรรทัดโค้ดจะกลายเป็นประโยคภาษาไทยธรรมดา และนี่คือทักษะที่สำคัญที่สุดในยุค AI: ต่อให้ไม่เคยเขียนเอง คุณก็อ่านโค้ดที่ AI เขียนให้ออกได้ เหมือนเด็กที่อ่านหนังสือออกก่อนจะเขียนเรียงความ


```
a = "hello world"
```

🔴**อ่านว่า:** เอา "hello world" มาใส่ใน a — เครื่องหมาย = ทำงานจากขวาไปซ้ายเสมอ: ค่าทางขวา ถูกใส่เข้า "กล่อง" ทางซ้าย

```
price = price + 10
```

🔴**อ่านว่า:** เอาค่า price เดิม บวก 10 แล้วใส่กลับเข้า price — บรรทัดนี้**ไม่ใช่**สมการคณิตศาสตร์ (ในคณิต price = price + 10 เป็นไปไม่ได้!) แต่คือคำสั่ง "อัปเดตค่า"

ตารางเสียงอ่าน — ใช้ตลอดทั้งเล่ม

สัญลักษณ์	อ่านว่า	ตัวอย่าง	อ่านทั้งบรรทัดว่า
=	"เอา...มาใส่ใน..."	<code>z = 2.5</code>	เอา 2.5 มาใส่ใน z
==	"ถามว่า...เท่ากับ...ไหม"	<code>z == 2.5</code>	ถามว่า z เท่ากับ 2.5 ไหม (ได้คำตอบ True/False)
!=	"ถามว่า...ไม่เท่ากับ...ไหม"	<code>signal != 0</code>	ถามว่า signal ไม่เท่ากับ 0 ใช่ไหม
>=	"ถามว่า...มากกว่าหรือเท่ากับ...ไหม"	<code>z >= 2.0</code>	ถามว่า z มากกว่าหรือเท่ากับ 2.0 ไหม
.	"ของ" (อ่านย้อนจากขวา)	<code>df.mean()</code>	ค่าเฉลี่ยของ df
[]	"หยิบตัวที่..."	<code>prices[0]</code>	หยิบตัวแรกจาก prices
() หลังชื่อ	"สั่งให้ทำงาน"	<code>calc_return()</code>	สั่งให้ calc_return ทำงาน
: ท้ายบรรทัด	"แล้วทำต่อไปนี่"	<code>if z > 2:</code>	ถ้า z มากกว่า 2 แล้วทำต่อไปนี่

หมายเหตุ: วงเล็บ () มีสองบทบาท — ถ้าอยู่หลังชื่อ (เช่น `print(...)`) แปลว่า "สั่งให้ทำงาน" แต่ถ้าครอบนิพจน์คณิตศาสตร์ (เช่น `(a - b) / b`) แปลว่า "ทำส่วนนี้ก่อน"

⚠️ **กับดักอันดับ 1 ของมือใหม่: = กับ ==**

```
if signal = 1: # ❌ SyntaxError! ใช้ "ใส่ค่า" ในที่ที่ต้อง "ถาม"
```

```
if signal == 1:    # ✓ "ถามว่า signal เท่ากับ 1 ไหม"
```

ง่าย ๆ: หนึ่งขีด = สิ่ง (ใส่ค่า), สองขีด = ถาม (เปรียบเทียบ) — โชคดีที่ Python ป้องกัน error ที่ถ้าใช้ผิดใน if แต่ภาษาอื่นบางภาษาลดผ่านแล้ว bug จะตามหลอกหลอน

วิธีฝึกอ่านโค้ด

ทุกครั้งที่เจอโค้ดในเล่มนี้ ลองอ่านออกเสียงเป็นภาษาไทยที่ละบรรทัดก่อนดูคำอธิบาย ถ้าอ่านแล้วเป็นประโยคที่เข้าใจได้ แปลว่าคุณเข้าใจโค้ดบรรทัดนั้นแล้วจริง ๆ — จากบทนี้ไป จะมีกล่อง 📖 "อ่านว่า" แทรกไว้โค้ดสำคัญ เพื่อให้เทียบเสียงอ่านของตัวเองได้

6.4 Operators และ Expressions

```
# Arithmetic operators
price_today = 65000
price_yesterday = 63000

simple_return = (price_today - price_yesterday) / price_yesterday
print(f"Return: {simple_return:.2%}") # Return: 3.17%

# Operators ที่ใช้บ่อย
profit = 1000
tax_rate = 0.15
net = profit * (1 - tax_rate)    # = 850.0

days = 252
annual_vol = 0.03 * (days ** 0.5) # ** = ยกกำลัง (ทำไมไม่ใช่ ^ ดูกล่องด้านล่าง)

shares = 1000
remainder = shares % 100          # % = เศษจากหาร: 1000 % 100 = 0
lots = shares // 100              # // = หารปัดทิ้งทศนิยม: 1000 // 100 = 10
```

คำถามที่พบบ่อย: ทำไม `\*\*` ไม่ใช่ `^` แบบคณิต?

ใน Python เครื่องหมาย `^` ถูกสงวนไว้สำหรับ "XOR" (bitwise operation ที่ใช้ใน cryptography) จึงต้องใช้ `**` แทนการยกกำลัง — ไม่ผิดปกติ แค่ convention ของภาษา

สำหรับ `//` และ `%`: ถ้า `1000 / 100 = 10.0` (float) แล้ว `1000 // 100 = 10` (int, ปัดทิ้งทศนิยม) และ `1000 % 100 = 0` (เศษ) — สามสิ่งนี้เป็นชุดเดียวกัน: ผลหาร, ผลหารปัดลง, และเศษที่เหลือ

`range(1, 5)` ให้ค่า 1, 2, 3, 4 (ไม่รวมตัวสุดท้าย 5) — rule เดียวกับ slicing `[1:5]` นี่เป็น convention ของ Python เพื่อให้ `len(range(1, 5)) == 5 - 1 == 4` นับออกง่าย

6.5 f-string — แสดงผลแบบมีรูปแบบ

f-string คือวิธี "ประกอบข้อความ" สำหรับแสดงผล — ในเล่มนี้เราใช้ f-string เฉพาะใน `print()` ตอนแสดงผลสุดท้ายเท่านั้น การคำนวณจริงจะแยกเป็นตัวแปรชื่อชัด ๆ เสมอ เพื่อให้อ่านง่าย (AI ชอบยึดการคำนวณเข้าไปใน f-string เลย — เดี่ยวบทหลังจะฝึกอ่านแบบนั้น)

```
symbol = "BTC"
price = 65432.789
ret = 0.0317

# f-string: ใส่ตัวแปรใน { } ได้เลย
print(f"{symbol} ราคา: ${price:,.2f}")      # BTC ราคา: $65,432.79
print(f"Return: {ret:.2%}")                  # Return: 3.17%
print(f"Annualized: {ret * 252:.1f}%")        # Annualized: 8.0%
```

Format codes ที่ใช้บ่อยใน Quant

Code	ความหมาย	ตัวอย่าง	ผลลัพธ์
<code>:.2f</code>	ทศนิยม 2 ตำแหน่ง	<code>f"{65432.789:.2f}"</code>	<code>65432.79</code>
<code>:.0f</code>	ไม่มีทศนิยม	<code>f"{65432:.0f}"</code>	<code>65432</code>
<code>:,</code>	คั่นหลักพัน	<code>f"{65432:,}"</code>	<code>65,432</code>
<code>:.2%</code>	แปลงเป็น % ทศนิยม 2	<code>f"{0.0317:.2%}"</code>	<code>3.17%</code>
<code>:.4f</code>	ทศนิยม 4 ตำแหน่ง	<code>f"{0.0317:.4f}"</code>	<code>0.0317</code>

ตัวอย่าง: คำนวณและแสดงผล Trade Summary

```
entry_price = 63000.0
exit_price  = 65450.0
position_size = 0.5 # BTC

pnl = (exit_price - entry_price) * position_size
ret = (exit_price - entry_price) / entry_price

print(f"Entry:  ${entry_price:,.2f}")
print(f"Exit:   ${exit_price:,.2f}")
print(f"P&L:    ${pnl:,.2f}")
print(f"Return:  {ret:.2%}")
```

► OUTPUT Entry: \$63,000.00 Exit: \$65,450.00 P&L: \$1,225.00 Return: 3.89%

► แบบฝึกหัด: คำนวณ Annualized Return

บทที่ 7: Control Flow & Functions

แก่นของบทนี้

Control flow คือการบอก program ว่า "ทำอะไรเมื่อเงื่อนไขแบบไหน" และ "ทำซ้ำกี่ครั้ง" — ใน Quant ใช้สร้าง signal logic, loop ผ่านข้อมูล, และ function ที่ reuse ได้ทั้ง codebase

7.1 เงื่อนไขคืออะไร — คำถามที่ตอบได้แค่ True/False

ก่อนรู้จัก if ต้องเข้าใจก่อนว่า "เงื่อนไข" คืออะไร: เงื่อนไขคือคำถามที่คำตอบมีแค่ 2 แบบ — True (จริง) หรือ False (เท็จ)

```
z = 2.3

z > 2.0      # คำถาม: z มากกว่า 2.0 ไหม? → คำตอบ: True
z == 2.0     # คำถาม: z เท่ากับ 2.0 ไหม? → คำตอบ: False
z < -2.0     # คำถาม: z น้อยกว่า -2.0 ไหม? → คำตอบ: False
```

คำตอบ True/False เป็น "ค่า" จริง ๆ ใน Python — เอามาใส่ตัวแปรได้เหมือนตัวเลข:

```
is_overbought = z > 2.0      # เอาคำตอบ (True) มาใส่ใน is_overbought
print(is_overbought)         # True
```

📌อ่านว่า: ถามว่า z มากกว่า 2.0 ไหม แล้วเอาคำตอบมาใส่ใน is_overbought — สังเกตว่า = (ใส่ค่า) กับ > (ถาม) ทำงานคนละหน้าที่ในบรรทัดเดียวกัน: ฝั่งขวาก่อนได้คำตอบแล้วค่อยใส่

Comparison Operators — เครื่องมือสร้างคำถาม

Operator	อ่านว่า	ตัวอย่าง
==	เปรียบเทียบกับ...เท่ากับ... (ได้ True/False)	signal == "LONG"
!=	ไม่เท่ากับ...ใช่ไหม	position != 0
> , <	มากกว่า/น้อยกว่า...ไหม	z > 2.0
>= , <=	อย่างน้อย / ไม่เกิน...ไหม	drawdown <= -0.1

7.2 if / elif / else — เครื่องจักรถามคำถามตามลำดับ

if/elif/else คือเครื่องจักรที่ถามคำถามจากบนลงล่างทีละข้อ — เจอข้อไหนตอบ "ใช่" ทำงานของข้อนั้นแล้วออกจากเครื่องทันที ข้อที่เหลือไม่ถูกถามอีกเลย

```
z = 2.3

if z > 2.0:          # ถามข้อ 1: z มากกว่า 2.0 ไหม?
    signal = "SHORT" #   ใช่ → ทำบรรทัดนี้ แล้วออกเลย ไม่ถามต่อ
elif z < -2.0:       # ถามข้อ 2 (เฉพาะเมื่อข้อ 1 ตอบ "ไม่")
    signal = "LONG"  #   ใช่ → ทำบรรทัดนี้ แล้วออก
elif abs(z) < 0.5:   # ถามข้อ 3 (เฉพาะเมื่อข้อ 1 และ 2 ตอบ "ไม่")
    signal = "EXIT"  #   ใช่ → ทำบรรทัดนี้ แล้วออก
else:                # ทุกข้อข้างบนตอบ "ไม่" ทั้งหมด
    signal = "HOLD"  #   ทำบรรทัดนี้ (else ไม่มีคำถาม = "ที่เหลือทั้งหมด")

print(signal)
```

► OUTPUT **SHORT**

🔴**อ่านว่า:** ถ้า z มากกว่า 2.0 แล้วเอา "SHORT" ใส่ใน signal — ไม่งั้นถ้า z น้อยกว่า -2.0 แล้วเอา "LONG" ใส่ใน signal — ไม่งั้นถ้าขนาดของ z น้อยกว่า 0.5 แล้วเอา "EXIT" ใส่ — ไม่งั้น (ที่เหลือทั้งหมด) เอา "HOLD" ใส่

เส้นทางของ $z = 2.3$ ในเครื่องจักรนี้:

ข้อ 1: $z > 2.0$? ($2.3 > 2.0$)

| ตอบ "ใช่" ↓

signal = "SHORT" → ออกจากเครื่องทันที

ข้อ 2, ข้อ 3, else ไม่ถูกถามเลย แม้ว่า $\text{abs}(2.3) < 0.5$ จะเป็น False อยู่แล้วก็ตาม

⚠ elif ไม่เท่ากับ if สองอัน — ตัวอย่างที่ผลต่างกันจริง

```
z = 3.0
# — แบบ elif: ถามต่อเมื่อข้อบนตอบ "ไม่" —
if z > 2.0:
    action = "SHORT" # เข้าข้อนี้ แล้วออกเลย
elif z > 1.0:
    action = "WARN"  # ไม่ถูกถาม
# ผล: action = "SHORT" ✓

# — แบบ if สองอัน: ถามทุกข้อเสมอ —
if z > 2.0:
```

```

    action = "SHORT"      # เข้าซื้อ
if z > 1.0:
    action = "WARN"      # ถูกถามอีก! 3.0 > 1.0 จริง → ทับค่าเดิม
# ผล: action = "WARN" ❌ SHORT หายไปเฉย ๆ

```

ถ้าเงื่อนไขเป็น "ทางเลือกที่เกิดได้ทีละอย่าง" (signal มีได้ค่าเดียว) ต้องใช้ `elif` เสมอ — bug ชนิดนี้ Python ไม่ฟ้อง error โปรแกรมรันได้ปกติแต่ให้ signal ผิด ซึ่งใน trading คือเสียเงินจริง

ลำดับเงื่อนไขสำคัญ — เช็คข้อแคบก่อนข้อกว้าง

```

# ❌ ผิด: ข้อกว้าง (z > 1) อยู่บน → "กิน" ทุกกรณี z > 2 ไปด้วย
if z > 1.0:
    level = "WARN"      # z = 3.0 ก็เข้าซื้อ! ไม่มีวันถึง EXTREME
elif z > 2.0:
    level = "EXTREME"   # โค้ดตายซาก — ไม่มีทางถูกเรียก

# ✓ ถูก: ข้อแคบ/รุนแรงกว่า ขึ้นก่อน
if z > 2.0:
    level = "EXTREME"
elif z > 1.0:
    level = "WARN"

```

เชื่อมหลายเงื่อนไข: and / or / not

ตัวเชื่อม	อ่านว่า	ผลเป็น True เมื่อ
and	"และ"	จริงทั้งคู่
or	"หรือ"	จริงอย่างน้อยหนึ่งข้อ
not	"ไม่/กลับด้าน"	ของเดิมเป็น False

```

# เข้า trade เมื่อ: z สูงพอ "และ" ยังไม่มี position
if z > 2.0 and position == 0:
    position = -1

# ออกจาก trade เมื่อ: z กลับใกล้ 0 "หรือ" ขาดทุนเกิน limit
if abs(z) < 0.5 or loss > max_loss:
    position = 0

```

📌 **อ่านว่า:** ถ้า z มากกว่า 2.0 และ position เท่ากับ 0 (ทั้งสองข้อต้องจริง) แล้วเอา -1 ใส่ใน position

⚠ กับดักการเขียนเงื่อนไข 2 จุด

```
# 1) เงื่อนไขช่วง — ต้องเขียนตัวแปรซ้ำทั้งสองฝั่ง
if z > 1.0 and < 3.0:      # ❌ SyntaxError
if z > 1.0 and z < 3.0:    # ✓
if 1.0 < z < 3.0:          # ✓ Python ยอมให้เขียนแบบคณิตได้ด้วย

# 2) indent (ย่อหน้า) กำหนดว่าบรรทัดไหนอยู่ "ใน" if
if z > 2.0:
    signal = -1
    log_trade()             # อยู่ใน if — ทำเฉพาะเมื่อ z > 2.0

if z > 2.0:
    signal = -1
log_trade()                 # อยู่นอก if — ทำทุกครั้ง! ต่างแค่ย่อหน้าเดียว
```

7.3 for loop — วนซ้ำ

for loop คือคำสั่ง "หยิบของจากรายการมาทีละตัว แล้วทำชุดคำสั่งเดิมซ้ำกับทุกตัว"

```
prices = [63000, 65000, 64500]

for p in prices:
    print(p)                # หยิบสมาชิกจาก prices มาใส่ p ทีละตัว
                           # ทำบรรทัดนี้กับ p ทุกรอบ
```

📌 **อ่านว่า:** สำหรับแต่ละตัวใน prices — เอาตัวนั้นมาใส่ใน p แล้วทำต่อไปนี่: print ค่า p

ดูการทำงานทีละรอบ — เทคนิคสำคัญ: เวลา loop ให้ "เดินเครื่องด้วยมือ" ตามตาราง — จดทั้ง *ค่าตัวแปร loop (i)* และ *state ของ list ที่กำลังสะสม (returns)*:

รอบที่	i	prices[i]	returns (list สะสม)
1	1	65000	[0.0317]
2	2	64500	[0.0317, -0.0077]
3	3	66000	[0.0317, -0.0077, 0.0233]
4	4	65800	[0.0317, -0.0077, 0.0233, -0.0030] → loop จบ

ตัวอย่างจริง: คำนวณ return รายวัน — ต้องใช้ "ราควันนี้" คู่กับ "ราคาเมื่อวาน" จึง loop ด้วยตำแหน่ง (index) แทน:


```

prices = [63000, 65000, 64500, 66000, 65800]

returns = [] # เตรียม list เปล่าไว้เก็บผล
for i in range(1, len(prices)): # i ไหลจาก 1, 2, 3, 4 (เริ่มวันที่สอง)
    today = prices[i] # หยิบราคาวันนี้
    yesterday = prices[i-1] # หยิบราคาเมื่อวาน (ตัวก่อนหน้า)
    r = (today - yesterday) / yesterday # สูตร simple return
    returns.append(r) # เก็บผลต่อท้าย list

print(returns) # [0.0317, -0.0077, 0.0233, -0.0030]

```

```

# enumerate: ได้ทั้งตำแหน่งและค่า พร้อมกัน
for i, price in enumerate(prices):
    print(f"วันที่ {i+1}: ${price:,}")

# zip: loop สองรายการพร้อมกัน (จับคู่ตัวต่อตัว)
symbols = ["BTC", "ETH", "SOL"]
weights = [0.5, 0.3, 0.2]
for sym, w in zip(symbols, weights):
    print(f"{sym}: {w:.0%}")

```

while loop — วนจนกว่าเงื่อนไขจะเป็นเท็จ

for loop กับ while loop แตกต่างกันที่ "รู้ล่วงหน้าไหมว่าจะวนกี่รอบ":

	for loop	while loop
ใช้เมื่อ	รู้จำนวนรอบล่วงหน้า	ไม่รู้จำนวนรอบ — วนจนเงื่อนไขเปลี่ยน
หยุดเมื่อ	หมด list / range	เงื่อนไขกลายเป็น False
ตัวอย่าง Quant	คำนวณ return 252 วัน (รู้ว่า 252 วัน)	เทรดต่อจนทุนหมด (ไม่รู้วันที่วัน)

```

equity = 100000 # เงินเริ่มต้น 100,000 บาท
max_loss = 90000 # ถ้าเงินเหลือน้อยกว่า 90,000 หยุดเทรด
daily_return = -0.004 # สมมติขาดทุนวันละ 0.4% (จงใจให้ติดลบ เพื่อให้ loop มีวันจบ — จริง ๆ จะได้จาก
day = 0 # ตัวนับรอบ

while equity > max_loss: # ก่อนทุกรอบ ถาม: equity ยังมากกว่า max_loss ไหม?
    day = day + 1 # นับรอบที่เทรด
    profit = equity * daily_return # คำนวณกำไร/ขาดทุนวันนี้ (ติดลบ = ขาดทุน)
    equity = equity + profit # อัปเดตเงินทุน (ลดลงเรื่อย ๆ)
    print(f"รอบ {day}: equity = {equity:.2f}") # ไม่ → ออกจาก loop (หยุดเทรด)

print("หยุดเทรด — ขาดทุนถึง limit")

```

```
► OUTPUT รอบ 1: equity = 99600.00 รอบ 2: equity = 99201.60 ... รอบ 27: equity = 89743.32 -
ต่ำกว่า 90,000 แล้ว loop จึงหยุด หยุดเทรด - ขาดทุนถึง limit
```

📖 **อ่านว่า:** ตราบใดที่ equity ยังมากกว่า max_loss ก็ทำต่อไปนี้: เทรดหนึ่งวันแล้วอัปเดต equity (ขาดทุนวันละ 0.4% จึงลดลงทุกรอบ) — เครื่องจะกลับขึ้นไปถามคำถามเดิมซ้ำก่อนเริ่มรอบใหม่ทุกครั้ง พอ equity ตกลงต่ำกว่า 90,000 ในรอบที่ 27 เงื่อนไขเป็น False loop จึงจบ

⚠ Infinite loop — while ที่ไม่มีวันจบ

ถ้าในตัว loop ไม่มีอะไรทำให้เงื่อนไขเปลี่ยนเป็น False ได้เลย โปรแกรมจะวนตลอดกาล (ค้าง) เช่น ลืมอัปเดต equity ในตัวอย่างบน — เช็คเสมอว่า "ตัวแปรในเงื่อนไข ถูกเปลี่ยนค่าข้างใน loop หรือยัง" ถ้าโปรแกรมค้าง กด Ctrl+C เพื่อหยุด

7.4 List Comprehension — Loop ในบรรทัดเดียว (ฝึกอ่านโค้ดแบบ AI ครั้งแรก)

ทุกอย่างที่ทำได้ด้วย loop ธรรมดา เขียนย่อในบรรทัดเดียวได้ด้วย "list comprehension" — คุณไม่จำเป็นต้องเขียนเป็น แต่ต้องอ่านออก เพราะ AI เขียนแบบนี้ตลอด

```
prices = [63000, 65000, 64500, 66000, 65800]

# แบบที่เราเขียน (อ่านออก 100%)
returns = []
for i in range(1, len(prices)):
    today = prices[i]
    yesterday = prices[i-1]
    returns.append((today - yesterday) / yesterday)
```

🤖 AI เขียนแบบนี้ — ฝึกอ่านกัน

```
returns = [(prices[i]-prices[i-1])/prices[i-1] for i in range(1, len(prices))]
```

สูตรอ่าน list comprehension: อ่านครึ่งหลังก่อน แล้วย้อนกลับมาครึ่งแรก

1. `for i in range(1, len(prices))` — "สำหรับแต่ละ `i` ตั้งแต่ 1 ถึงท้ายรายการ" (นี่คือ loop เดิมนั่นเอง)
2. `(prices[i]-prices[i-1])/prices[i-1]` — "คำนวณค่านี้ในแต่ละรอบ" (คือ 3 บรรทัดในตัว loop ยุบเหลือนิพจน์เดียว)
3. `[ ... ]` ครอบทั้งหมด — "เก็บผลทุกรอบลง list" (คือ `returns = []` + `.append()`)

ทั้งบรรทัดอ่านว่า: "สร้าง list ของ return โดยคำนวณจากราคาคู่วันติดกัน ทุกวันตั้งแต่วันที่สอง" — ความหมายเหมือนโค้ด 5 บรรทัดข้างบนเป๊ะ แคย่อ

```
# comprehension + if = กรองของ
loss_days = [r for r in returns if r < 0]
print(f"วันขาดทุน: {len(loss_days)} วัน")
```

📌 **อ่านว่า:** สร้าง list จาก r แต่ละตัวใน returns เฉพาะตัวที่ r น้อยกว่า 0 — แล้วเอามาใส่ใน loss_days

🧠 เมื่อ AI ใช้ if ใน comprehension + ternary — 2 รูปแบบที่ต่างกัน

```
# รูปแบบที่ 1: กรอง (filter) — if อยู่หลัง for
loss_days = [r for r in returns if r < 0]
# รูปแบบที่ 2: เลือกค่า (ternary) — if อยู่หน้า for
labels = ["loss" if r < 0 else "gain" for r in returns]
```

ทั้งสองรูปแบบมี if แต่ตำแหน่งต่างกัน → ความหมายต่างกันสิ้นเชิง:

1. **if หลัง for** = กรองของ: "เอาเฉพาะ r ที่ $r < 0$ " → list สั้นลง
2. **if...else หน้า for** = เลือกค่า: "ถ้า $r < 0$ ให้เป็น 'loss' ไม่งั้น 'gain'" → list ยาวเท่าเดิม แค่เปลี่ยนค่า

จำง่าย: ก่อน for = เลือกว่าแต่ละตัวจะเป็นอะไร, หลัง for = กรองว่าตัวไหนเข้าหรือออก

7.5 Functions — def

```
# นิยาม function
def calc_return(price_now, price_prev):
    """คำนวณ simple return ระหว่างสองราคา"""
    return (price_now - price_prev) / price_prev

# เรียกใช้
r = calc_return(65000, 63000)
print(f"Return: {r:.2%}") # Return: 3.17%

# Default arguments
def annualize(daily_vol, periods=252):
    """แปลง daily volatility เป็น annualized"""
    return daily_vol * (periods ** 0.5)

print(annualize(0.03)) # 0.4762 (252 วัน = หุ่น)
print(annualize(0.03, 365)) # 0.5733 (365 วัน = crypto)
```

Docstring — เขียนทุก function ใน Quant code

บรรทัดแรกใน function (ในเครื่องหมาย `"""`) คือ docstring อธิบาย function นั้น
ช่วยเมื่อ copy code ไปให้ AI ตรวจ — AI จะเข้าใจ intent ได้ถูกต้อง และช่วย debug ได้แม่นยำขึ้น

ตัวอย่าง: Z-score Function แบบ Reusable

```
def zscore(series, window=60):
    """
    คำนวณ rolling z-score ของ series
    Args:
        series: list หรือ numpy array ของราคา/spread
        window: rolling window (default 60 วัน)
    Returns:
        list ของ z-score
    """
    results = [] # เตรียม list เก็บผลลัพธ์
    for i in range(len(series)): # เดินผ่านข้อมูลทีละวัน
        if i < window: # วันแรก ๆ ข้อมูลย้อนหลังยังไม่ครบ window
            results.append(None) # → ใส่ None แทน (คำนวณไม่ได้)
            continue # ข้ามไปวันถัดไปเลย
        window_data = series[i-window:i] # หยิบข้อมูล window วันล่าสุด
        mean = sum(window_data) / window # ค่าเฉลี่ยของหน้าต่างนั้น
        variance = sum((x - mean)**2 for x in window_data) / window # ความแปรปรวน
        std = variance ** 0.5 # ส่วนเบี่ยงเบนมาตรฐาน = รากของ variance
        if std == 0: # กันหารด้วยศูนย์ (ราคานิ่งสนิท)
            results.append(0)
        else:
            z = (series[i] - mean) / std # สูตร z-score: ห่างจากค่าเฉลี่ยกี่ std
            results.append(z)
    return results

# ทดสอบ
spread = [100, 102, 98, 105, 99, 103, 110, 97, 101, 104]
z = zscore(spread, window=5)
print(z[-3:]) # z-score 3 วันล่าสุด
```

AI เขียนฟังก์ชันเดียวกันนี้ใน 1 บรรทัด — ฝึกอ่านกัน

```
z = (s - s.rolling(60).mean()) / s.rolling(60).std()
```

กล่องนี้คือ "ตัวอย่างล่องหน้า" — `.rolling()` จะเรียนละเอียดในบทที่ 9 ตอนนี้แค่ฝึกอ่านโครงสร้างให้ออก

(`s` คือ pandas Series — จะเรียนบทที่ 9) อ่านทีละชิ้นจากในออกนอก:

1. `s.rolling(60)` — "หน้าต่างเลื่อน 60 วัน ของ s" (จุด = "ของ")
2. `s.rolling(60).mean()` — "ค่าเฉลี่ย ของ หน้าต่าง 60 วัน" = ตัวแปร `mean` ในฟังก์ชันเรา
3. `s.rolling(60).std()` — "std ของหน้าต่าง 60 วัน" = ตัวแปร `std` ของเรา
4. `(s - mean) / std` — สูตร z-score เดิมเป๊ะ แค่ทำกับทุกวันพร้อมกัน (ไม่ต้อง loop เอง)

โค้ด 15 บรรทัดของเรากับบรรทัดเดียวนี่ *ทำสิ่งเดียวกัน* — pandas ซ่อน loop ไว้ข้างใน (และเร็วกว่า ~100 เท่า) เราเขียนแบบยาวก่อนเพื่อให้รู้ว่า "ข้างในมันทำอะไร" พอไปบทที่ 9 จะใช้แบบสั้นได้อย่างมั่นใจ

7.6 lambda — Function ย่อ

```
# lambda: function 1 บรรทัด ไม่มีชื่อ
calc_ret = lambda p_now, p_prev: (p_now - p_prev) / p_prev

# ใช้บ่อยกับ sorted() และ map()
trades = [("BTC", 500), ("ETH", -200), ("SOL", 300)]
by_pnl = sorted(trades, key=lambda t: t[1], reverse=True)
print(by_pnl) # [('BTC', 500), ('SOL', 300), ('ETH', -200)]
```

► แบบฝึกหัด: เขียน function คำนวณ Sharpe Ratio

7.7 yield — ฟังก์ชันที่ส่งของที่ละชิ้น

`return` ที่เพิ่งเรียนไปมีลักษณะเด็ดขาด: **ส่งของออกไปแล้วฟังก์ชันจบทันที** . มีคำสั่งอีกตัวที่ทำเกือบเหมือนกันแต่ต่างกันตรงจุดนี้จุดเดียว

ต่างกันแค่ประโยคเดียว

```
return = "นี่ของ แล้วฉันจบแล้ว"
yield = "นี่ของชิ้นหนึ่ง ฉันหยุดตรงนี้ ถ้าอยากได้อีกค่อยเรียก"
```

ฟังก์ชันที่มี `yield` อยู่ข้างในเรียกว่า **generator** — มันจำได้ว่าค้างอยู่บรรทัดไหน แล้ววิ่งต่อจากตรงนั้นเมื่อถูกขอขึ้นถัดไป

☛ **อ่านว่า:** เปรียบกับการสั่งอาหาร: `return` คือยกมาทั้งหมดวางบนโต๊ะ — ได้ครบแน่นอน แต่ต้องมีที่วางทั้งหมด .

`yield` คือตักให้ทีละจาน — โต๊ะเล็กก็กินได้ และถ้าอิมก่อนก็ไม่ต้องตักที่เหลือ . ในงานข้อมูลราคาที่มีเป็นล้านแถว ความต่างนี้คือความต่างระหว่าง "รันได้" กับ "แรมเต็ม"

```
import sys
```

```
def prices_list(n):
    """แบบ return — สร้างลิสต์ทั้งก้อนแล้วค่อยส่งกลับ"""
    out = []
    for i in range(n):
        out.append(65000 + i * 0.5)
    return out

def prices_gen(n):
    """แบบ yield — ส่งทีละตัว ไม่เก็บอะไรไว้เลย"""
    for i in range(n):
        yield 65000 + i * 0.5

N = 1_000_000
lst = prices_list(N)
gen = prices_gen(N)

print(f"list      : กินหน่วยความจำ {sys.getsizeof(lst) / 1024**2:>7.1f} MB")
print(f"generator : กินหน่วยความจำ {sys.getsizeof(gen) / 1024**2:>7.4f} MB")

# ใช้งานเหมือนกันทุกอย่าง — วนด้วย for ก็ได้ ส่งเข้า sum() ก็ได้
print(f"sum 5 ตัวแรก · list={sum(prices_list(5)):.1f} "
      f"· gen={sum(prices_gen(5)):.1f}")
```

► OUTPUT list : กินหน่วยความจำ 8.1 MB generator : กินหน่วยความจำ 0.0002 MB sum 5 ตัวแรก · list=325,005.0 · gen=325,005.0

📌อ่านว่า: ตัวเลข **8.1 MB** เทียบกับ **0.0002 MB** ไม่ได้แปลว่า generator "บีบอัดข้อมูล" — มันไม่ได้เก็บข้อมูลไว้เลยสักตัว · สิ่งที่มีเก็บคือ "สูตรว่าจะสร้างตัวถัดไปยังไง" กับ "ตอนนี้ค้างอยู่ตรงไหน" เท่านั้น · ราคาที่จ่ายคือถ้าอยากได้ตัวที่ 500,000 ต้องเดินผ่าน 499,999 ตัวแรกก่อน — เข้าถึงแบบสุ่ม (gen[500000]) ทำไม่ได้

⚠ generator ใช้ได้ครั้งเดียว — กับดักที่ไม่มี error เตือน

```
g = prices_gen(5)
print(f"รอบแรก sum(g) = {sum(g):.1f}")
print(f"รอบสอง sum(g) = {sum(g):.1f}") # ← generator หมดแล้ว
```

► OUTPUT รอบแรก sum(g) = 325,005.0 รอบสอง sum(g) = 0.0

รอบสองได้ **0.0** ไม่ใช่ error · เพราะ generator เดินไปจนสุดแล้ว การขอต่อจึงได้ "ไม่มีอะไรเลย" ซึ่ง sum() ของความว่างเปล่าคือ 0

ในงานจริงกับดักนี้หน้าตาแบบนี้:

— ตัวอย่างประกอบ —

```
returns = (r for r in load_prices() if r > 0) # generator expression
n       = sum(1 for _ in returns)           # นับ → กินหมดแล้ว
mean    = sum(returns) / n                 # ❌ ได้ 0 · แล้วหารต่อได้ 0.0
```

ไม่มี exception · ไม่มี warning · ได้ค่าเฉลี่ยเป็น 0.0 แล้วไหลเข้าไปในรายงานอย่างสงบ

🚫 **วิธีกัน:** ถ้าต้องใช้ข้อมูลชุดเดิมมากกว่าหนึ่งรอบ ให้แปลงเป็น list ก่อนด้วย `list(gen)` แล้วยอมเสียหน่วยความจำไป · generator เหมาะกับงานที่ไหลผ่านครั้งเดียวจบ เท่านั้น

● [รอบสอง] ที่ไหนในสาย quant ที่ generator เป็นคำตอบจริง ๆ

- **ไฟล์ที่ใหญ่กว่าแรม** — tick data ปีเดียวเป็นสิบลบ GB · `for line in open(f)` คือ generator อยู่แล้วโดยธรรมชาติ อ่านทีละบรรทัดไม่เคยโหลดทั้งไฟล์ · เขียน `open(f).readlines()` เมื่อไรคือโหลดทั้งไฟล์เข้าแรมทันที
- **ข้อมูลที่ยังไม่จบ** — ราคาสดจาก WebSocket ไม่มี "ตัวสุดท้าย" ให้ใส่ list · generator เป็นรูปแบบที่ตรงกับธรรมชาติของมัน (Part VI จะเจอญาติของมันคือ `async for`)
- **ต่อท่อเป็นทอด ๆ** — อ่านไฟล์ → กรอง → แปลง → รวมยอด ถ้าแต่ละขั้นคืน list จะสร้างสำเนาข้อมูลใหม่ทุกขั้น · ถ้าเป็น generator ทั้งสาย ข้อมูล 1 แถวจะไหลผ่านทุกขั้นแล้วถูกทิ้งไป ก่อนแถวถัดไปจะเริ่ม

และที่ไหนที่ไม่ควรใช้: ถ้าข้อมูลใส่แรมได้สบายและต้องวนหลายรอบ — `list` หรือ `pd.Series` ชนะขาดทั้งความง่ายและความเร็ว · **อย่าใช้ generator เพราะมันดูโปร** ใช้เพราะข้อมูลใหญ่เกินหรือไหลไม่จบเท่านั้น

เกร็ดที่โยกกลับไปหัวข้อที่แล้ว: เขียน `[x for x in ...]` ได้ list · เปลี่ยนวงเล็บเหลี่ยมเป็นวงเล็บโค้ง (`x for x in ...`) ได้ **generator expression** — ต่างกันแค่วงเล็บ แต่ตัวหนึ่งกินแรมทั้งก้อน อีกตัวไม่กินเลย

บทที่ 8: Data Structures

แก่นของบทนี้

Python มี 4 data structure หลัก — **list** (รายการเรียงลำดับ), **dict** (key-value), **tuple** (ค่าคงที่), **set** (ค่าไม่ซ้ำ) แต่ละอย่างมีจุดแข็งต่างกัน รู้จักเลือกใช้ถูกตัว code จะสั้นและเร็วขึ้นมาก

8.1 list — รายการที่เรียงลำดับ

```
prices = [63000, 65000, 64500, 66000, 65800]

# Indexing (เริ่มจาก 0)
print(prices[0])    # 63000   (ตัวแรก)
print(prices[-1])   # 65800   (ตัวสุดท้าย - เลขติดลบ = นับจากท้าย)

# Slicing [start:stop:step]
print(prices[1:4])   # [65000, 64500, 66000]
print(prices[:3])    # [63000, 65000, 64500] (3 ตัวแรก)
print(prices[::2])   # [63000, 64500, 65800] (ทุก 2 ตัว)

# เพิ่ม/ลบ
prices.append(67000)    # เพิ่มต่อท้าย
prices.insert(0, 62000)  # แทรกที่ตำแหน่ง 0
last = prices.pop()     # ดึงตัวสุดท้ายออก

# ข้อมูลพื้นฐาน
print(len(prices))      # จำนวนสมาชิก
print(min(prices), max(prices)) # ต่ำสุด สูงสุด
print(sum(prices) / len(prices)) # ค่าเฉลี่ย
```

8.2 dict — Key-Value Store

dict คือโครงสร้างที่ใช้มากที่สุดในการเก็บข้อมูลตลาด — key คือชื่อ/symbol, value คือข้อมูล

```
# OHLCV ของวันหนึ่ง
candle = {
    "open": 64000.0,
    "high": 66500.0,
    "low": 63800.0,
    "close": 65450.0,
```



```

    "volume": 28500.0
}

# เข้าถึงค่า
print(candle["close"])          # 65450.0
print(candle.get("vwap", None)) # None (ไม่มี key นั้น - ไม่ error)

# เพิ่ม/อัปเดต
candle["body"] = candle["close"] - candle["open"] # = 1450.0

# loop ผ่าน dict
for key, value in candle.items():
    print(f"{key:10s}: {value:,.1f}")

# Portfolio เป็น dict ของ dict
portfolio = {
    "BTC": {"size": 0.5, "entry": 63000, "side": "long"},
    "ETH": {"size": 2.0, "entry": 3200, "side": "long"},
    "SOL": {"size": 10.0, "entry": 145, "side": "short"},
}

```

8.3 tuple — ค่าคงที่

```

# tuple เหมือน list แต่ immutable (เปลี่ยนไม่ได้)
config = (2.0, 0.5, 60) # (entry_z, exit_z, window)
entry_z, exit_z, window = config # unpacking

# ใช้เป็น key ของ dict ได้ (list ทำไม่ได้)
pair_params = {
    ("BTC", "ETH"): {"beta": 0.85, "half_life": 4.2},
    ("BTC", "SOL"): {"beta": 1.20, "half_life": 6.8},
}

print(pair_params[("BTC", "ETH")]["beta"]) # 0.85

```

8.4 set — ค่าไม่ซ้ำ

```

# set เก็บเฉพาะค่าที่ไม่ซ้ำ
signals_day1 = {"AAPL", "MSFT", "NVDA"}
signals_day2 = {"MSFT", "NVDA", "TSLA"}

both_days = signals_day1 & signals_day2 # intersection = {"MSFT", "NVDA"}
either_day = signals_day1 | signals_day2 # union
only_day1 = signals_day1 - signals_day2 # difference = {"AAPL"}

```

```
# ลบ duplicate ออกจาก list
trades = ["BTC", "ETH", "BTC", "SOL", "ETH", "BTC"]
unique_symbols = list(set(trades)) # ["BTC", "ETH", "SOL"]
```

เลือก Data Structure ไหน?

ถ้าต้องการ	ใช้	เหตุผล
เก็บ series ราคาเรียงเวลา	list	เรียงลำดับ, append ง่าย
เก็บ OHLCV ของ 1 candle	dict	access ด้วยชื่อ field ได้
ค่า config ที่ไม่เปลี่ยน	tuple	immutable, ใช้เป็น dict key ได้
รายการ symbol ที่เคยเทรด	set	ลบ duplicate อัตโนมัติ
ข้อมูลตลาดหลายวัน	pd.DataFrame	Part 1 บทที่ 9

ตัวอย่าง: สร้าง Return Map ของหลายเหรียญ

```
close_prices = {
    "BTC": [63000, 65000, 64500],
    "ETH": [3100, 3200, 3150],
    "SOL": [140, 148, 145],
}

# แบบที่เราเขียน: loop ผ่านทุกเหรียญ คำนวณ return ล่าสุด
latest_returns = {} # เตรียม dict เปล่า
for sym, prices in close_prices.items(): # หยิบ (ชื่อเหรียญ, list ราคา) ทีละคู่
    last = prices[-1] # ราคาล่าสุด (ตัวสุดท้าย)
    prev = prices[-2] # ราคาก่อนหน้า (ตัวรองสุดท้าย)
    latest_returns[sym] = (last - prev) / prev # เก็บ return โดยใช้ชื่อเหรียญเป็น key

for sym, r in latest_returns.items():
    print(f"{sym}: {r:.2%}")
```

► OUTPUT BTC: -0.77% ETH: -1.56% SOL: -2.03%

```
latest_returns = {
    sym: (prices[-1] - prices[-2]) / prices[-2]
    for sym, prices in close_prices.items()
}
```

สูตรเดียวกับ list comprehension: อ่านบรรทัด **for** ก่อน แล้วย้อนขึ้นไปดูว่าแต่ละรอบสร้างอะไร

1. `for sym, prices in close_prices.items()` — loop เดิม: หยิบ (ชื่อเหรียญ, ราคา) ทีละคู่
2. `sym: (prices[-1] - prices[-2]) / prices[-2]` — แต่ละรอบสร้างคู่ `key: value` = ชื่อเหรียญ : return ล่าสุด
3. `{ ... }` ปีกกาครอบ — เก็บทุกคู่ลง dict (ปีกกา = dict, วงเล็บเหลี่ยม = list)

ความหมายเหมือนโค้ด 5 บรรทัดข้างบนทุกประการ

► แบบฝึกหัด: สร้าง Portfolio Summary

8.5 `with` — เปิดแล้วปิดให้เองไม่ต้องจำ

บทหน้าจะเริ่มอ่านไฟล์จริง · ก่อนไปถึงตรงนั้นมีไวยากรณ์หนึ่งตัวที่ต้องรู้ก่อน เพราะจะเจอไปตลอดทั้งเล่ม — และจะกลับมาในรูปแบบ `async with` ที่ Part VI

```
# ❌ แบบที่ลืมนง่าย — ต้องปิดเอง
f = open("trades.csv", "w")
f.write("symbol,pnl\n")
f.write("BTC,1250\n")
f.close() # ถ้าลืมบรรทัดนี้ ข้อมูลอาจไม่ถูกเขียนลงดิสก์จริง

# ❌ แย่กว่านั้น — ถ้าเกิด error ก่อนถึง close() ไฟล์จะค้างเปิดไว้
# (คอมเมนต์ไว้เพราะรันแล้วพังจริง — ลองเอา # ออกดูได้)
# f = open("trades.csv", "w")
# f.write(str(1 / 0)) # ZeroDivisionError -> ไม่มีใครไปถึง f.close()
# f.close()

# ✅ แบบที่ควรใช้ — with ปิดให้เสมอ ไม่ว่าจะจบแบบไหน
with open("trades.csv", "w") as f:
    f.write("symbol,pnl\n")
    f.write("BTC,1250\n")
# พ้นย่อหน้านั้นเมื่อไหร่ = ไฟล์ถูกปิดแล้วเรียบร้อย แม้ข้างในจะ error ก็ตาม

with open("trades.csv") as f:
    print(f.read())
```

► OUTPUT `symbol,pnl BTC,1250`

📖 **อ่านว่า:** อ่าน `with open("trades.csv", "w") as f:` ว่า: "ยืมไฟล์นี้มาใช้ เรียกมันว่า `f` — พอออกจากย่อหน้านี้ให้คืนของอัตโนมัติ". จุดสำคัญคือคำว่า *อัตโนมัติ*: จบปกติก็ปิด, เกิด error กลางทางก็ปิด, `return` ออกไปกลางคันก็ปิด — ไม่มีเส้นทางไหนที่ไฟล์จะค้างเปิด

ทำไมเรื่อง "ลืมปิด" ถึงสำคัญกว่าที่คิด

- **ไฟล์:** ข้อมูลที่ `write()` ไปอาจยังค้างใน buffer ไม่ลงดิสก์จริงจนกว่าจะปิด — โปรแกรมจบแล้วไฟล์ว่างเปล่า
- **จำนวนไฟล์ที่เปิดพร้อมกันมีเพดาน:** ลูปที่เปิดไฟล์ทีละอันโดยไม่ปิด พอวนหลายพันรอบจะได้ `OSError: Too many open files`
- **ของอย่างอื่นก็ใช้ `with` ได้:** การเชื่อมต่อฐานข้อมูล, network socket, lock — อะไรก็ตามที่ "เปิดแล้วต้องปิด". ใน Part VI เราจะใช้ `async with` กับการเชื่อมต่อ WebSocket ด้วยเหตุผลเดียวกันนะ

🔵 [รอบสอง] ในงาน quant จริงคุณ sẽ เห็น `with` ที่ไหนบ้าง

ส่วนใหญ่ pandas จัดการไฟล์ให้แล้ว (`pd.read_csv("x.csv")` เปิดและปิดให้เอง) — ที่ยังต้องเขียน `with` เองคือ ① การเชื่อมต่อฐานข้อมูล ② การตั้งค่าชั่วคราว ③ การเขียน Excel หลาย sheet (`pd.ExcelWriter`) · สองข้อแรกลองได้เลย:

```
import sqlite3
import pandas as pd

# ① เปิด/ปิด connection ฐานข้อมูล — ":memory:" คือ DB ชั่วคราวในแรม
with sqlite3.connect(":memory:") as conn:
    pd.DataFrame({"symbol": ["BTC", "ETH"],
                  "close": [65000, 3200]}).to_sql("ohlcv", conn, index=False)
    df = pd.read_sql("SELECT * FROM ohlcv WHERE close > 10000", conn)
print(df)

# ② ตั้งค่าชั่วคราวแล้วคืนค่าเดิมอัตโนมัติ
big_df = pd.DataFrame({"x": range(100)})
print("นอก with :", pd.get_option("display.max_rows"))
with pd.option_context("display.max_rows", 5):
    print("ใน with :", pd.get_option("display.max_rows"))
print("พ้น with :", pd.get_option("display.max_rows"), "<- คืนค่าเดิมให้เอง")
```

► OUTPUT `symbol close 0 BTC 65000` นอก with : 60 ใน with : 5 พ้น with : 60 <- คืนค่าเดิมให้เอง

💡 สร้างของแบบนี้เองก็ได้ด้วย `contextlib.contextmanager` — ใช้บ่อยตอนทำตัวจับเวลา หรือตอนต้องเปิด/ปิดการเชื่อมต่อ exchange ในโค้ด live · หลักคิดเดียวกันทั้งหมด: จับคู่ "เปิด" กับ "ปิด" ให้อยู่ในโครงสร้าง

เดียว แทนที่จะฟังความจำของคนเขียน

บทที่ 9: NumPy & Pandas

แก่นของบทนี้

NumPy และ Pandas เป็น "engine" ของ Quant Python — NumPy ทำ math บน array เร็วกว่า loop ธรรมดา 100x+, Pandas จัดการ table ข้อมูลพร้อม label วันที่ ทั้งสองรวมกันทำให้วิเคราะห์ข้อมูลตลาดหลักหมื่นแถวได้ใน milliseconds

9.1 NumPy — คณิตศาสตร์บน Array

```
import numpy as np

# สร้าง array
prices = np.array([63000, 65000, 64500, 66000, 65800])

# Operations ทำทั้ง array พร้อมกัน (vectorized)
returns = np.diff(prices) / prices[:-1] # return ทุกวัน
print(returns)
# [0.0317 -0.0077  0.0233 -0.0030]

# Statistics
print(f"Mean:    {np.mean(returns):.4f}")
print(f"Std:     {np.std(returns):.4f}")
print(f"Max:      {np.max(returns):.4f}")
print(f"Min:      {np.min(returns):.4f}")

# Annualized Vol
annual_vol = np.std(returns) * np.sqrt(252) # ใช้ 252 เพื่อความง่าย — crypto เทรด 24/7 จริงๆ ใน
print(f"Annual Vol: {annual_vol:.1%}")
```

ทำไม NumPy เร็วกว่า Python loop?

Python loop: อ่านแต่ละ element → แปลง type → คำนวณ → เก็บ → ไปต่อ (overhead สูง)

NumPy: ส่ง array ทั้งก้อนให้ C code → คำนวณ parallel → ได้ผล (overhead ต่ำมาก)

ผลลัพธ์: array ขนาด 1,000,000 element → NumPy เร็วกว่า loop ประมาณ 100–1,000x

9.2 ทำไม "ตารางธรรมดา" ถึงไม่พอ

ก่อนตอบว่า DataFrame คืออะไร ลองดูก่อนว่าถ้าไม่มีมัน ชีวิตเป็นยังไง — เก็บราคา 2 เหรียญด้วยเครื่องมือที่เรามีจากบทที่ 8 (list ซ้อน list = "ตารางทำมือ"):

```
# ตารางธรรมดา: list ซ้อน list
table = [
    # [วันที่,      BTC,      ETH]
    ["2024-01-01", 63000, 3100],
    ["2024-01-02", 65000, 3250],
    ["2024-01-03", 64500, 3200],
]

# คำถามง่าย ๆ: "ราคา BTC วันที่ 2024-01-02 คือเท่าไร?"
# → ต้องเขียน loop ค้นเอง:
for row in table:
    if row[0] == "2024-01-02":      # ช่องที่ 0 คือวันที่ (ต้องจำเอง)
        print(row[1])              # ช่องที่ 1 คือ BTC (ต้องจำเอง)
```

ใช้งานได้ แต่มีปัญหา 3 ข้อที่จะกักรบกวนเราแน่นอนเมื่อข้อมูลโตขึ้น:

ปัญหาของตารางทำมือ

1. **ต้องจำตำแหน่งเอง** — "ช่องที่ 1 คือ BTC" ไม่มีเขียนไว้ในโค้ด ถ้าสลับคอลัมน์ในไฟล์ข้อมูลวันไหน โค้ดจะหยิบเลขผิดช่องโดยไม่มี error
2. **ค้นหาต้อง loop ทุกครั้ง** — ข้อมูล 100,000 แถว อยากรู้ราคาวันเดียว ต้องไล่ดูแสนแถว
3. **ข้อมูลหายแล้วแถวเลื่อน** — สมมติ ETH ไม่มีข้อมูลวันที่ 2 ถ้าเก็บแยกสอง list แล้วเอามาคู่กัน ตำแหน่งที่ 1 ของ BTC จะจับคู่กับ ETH ผิดวัน → คำนวณ correlation ผิดทั้งชุด แบบเงิบ ๆ

pandas ถูกสร้างมาแก้ 3 ปัญหานี้ตรง ๆ — และกุญแจของมันคือสิ่งที่เรียกว่า **index**

9.3 Index คืออะไร — ป้ายชื่อประจำแถว

list ธรรมดาเรียกข้อมูลด้วย "ตำแหน่ง": ตัวที่ 0, ตัวที่ 1, ตัวที่ 2 — เหมือนล็อกเกอร์ที่มีแต่เลขตัว

Series ของ pandas คือ list ที่ทุกค่ามีป้ายชื่อติดอยู่ — ป้ายชื่อเหล่านั้นรวมกันเรียกว่า index:

```
import pandas as pd

prices = pd.Series(
    [63000, 65000, 64500],          # ค่า (values)
    index=["2024-01-01", "2024-01-02", "2024-01-03"], # ป้ายชื่อ (index)
    name="BTC"
)

print(prices)
```

```
► OUTPUT 2024-01-01 63000 2024-01-02 65000 2024-01-03 64500 Name: BTC, dtype: int64
```

สังเกตว่า pandas พิมพ์ป้ายชื่อ (ซ้าย) คู่กับค่า (ขวา) เสมอ — ทีนี้คำถาม "ราคาวันที่ 2 ม.ค.?" ตอบได้บรรทัดเดียว ไม่ต้อง loop:

```
print(prices["2024-01-02"]) # 65000 — ถามด้วยป้ายชื่อตรง ๆ
print(prices.iloc[1])      # 65000 — หรือถามด้วยตำแหน่งแบบเดิมก็ทำได้ (iloc)
```

📌 **อ่านว่า:** หยิบค่าที่ป้ายชื่อ "2024-01-02" จาก prices — index ทำให้เราคู่กับข้อมูลด้วย "ภาษาของข้อมูล" (วันที่) แทนตำแหน่ง

และพลังที่สำคัญที่สุดของ index — **จับคู่แถวให้อัตโนมัติ (alignment)** ซึ่งแก้ปัญหาคำข้อ 3 ได้อย่างถาวร:

```
# ETH ไม่มีข้อมูลวันที่ 2 (ตลาดล่ม)
btc = pd.Series([63000, 65000, 64500],
                index=["2024-01-01", "2024-01-02", "2024-01-03"])
eth = pd.Series([3100, 3200],
                index=["2024-01-01", "2024-01-03"]) # ขาดวันที่ 2!

ratio = btc / eth # pandas จับคู่ด้วย "ป้ายชื่อ" ไม่ใช่ตำแหน่ง
print(ratio)
```

```
► OUTPUT 2024-01-01 20.322581 2024-01-02 NaN 2024-01-03 20.156250 dtype: float64
```

วันที่ 2 ได้ `NaN` (Not a Number = "ไม่มีข้อมูล") — pandas บอกตรง ๆ ว่าวันนี้คำนวณไม่ได้ แทนที่จะจับคู่ผิดแถวเจิบ ๆ แบบ list ถ้าเป็นตารางทำมือ ราคา ETH วันที่ 3 จะถูกจับคู่กับ BTC วันที่ 2 — ผิดโดยไม่มีใครรู้

9.4 DataFrame — หลาย Series ที่ใช้ป้ายชื่อชุดเดียวกัน

พอเข้าใจ Series แล้ว DataFrame ตรงไปตรงมามาก: **DataFrame = ตารางที่เกิดจากหลาย Series (หลายคอลัมน์) มาแชร์ index ชุดเดียวกัน** — หน้าตาเหมือน Excel sheet แต่อยู่ใน Python:

```
data = {
    "BTC": [63000, 65000, 64500, 66000, 65800],
    "ETH": [3100, 3250, 3200, 3300, 3280],
    "SOL": [140, 148, 145, 152, 149],
}
dates = pd.date_range("2024-01-01", periods=5) # สร้างป้ายวันที่ 5 วัน
df = pd.DataFrame(data, index=dates)

print(df)
```



```
► OUTPUT BTC ETH SOL 2024-01-01 63000 3100 140 2024-01-02 65000 3250 148 2024-01-03 64500
3200 145 2024-01-04 66000 3300 152 2024-01-05 65800 3280 149
```

กายวิภาคของ DataFrame มี 3 ส่วน — รู้ 3 ची่นี่แล้วอ่านเอกสาร pandas ออกทั้งหมด:

ส่วน	คืออะไร	ในตัวอย่างบน
index	ป้ายชื่อแถว (แนวตั้งซ้ายสุด)	วันที่ 2024-01-01 ถึง 05
columns	ป้ายชื่อคอลัมน์ (แนวนอนบนสุด)	BTC, ETH, SOL
values	ตัวข้อมูลข้างใน (NumPy array)	ตัวเลขราคาทั้งหมด

แล้วใช้ Excel แทนได้ไหม?

ได้ — สำหรับ "คู่มือข้อมูล" Excel ดีมาก แต่สำหรับงาน Quant มี 3 จุดที่ DataFrame ชนะขาด:

1. ทำซ้ำได้: โค้ด 10 บรรทัดรันกับข้อมูลใหม่ทุกวันอัตโนมัติ — Excel ต้องลากสูตรใหม่ทุกครั้ง

2. ขนาด: ข้อมูล 1 นาทีย้อนหลัง 3 ปี ≈ 1.5 ล้านแถว — Excel เริ่มค้าง pandas ทำงานเป็นวินาที

3. ตรวจสอบย้อนหลังได้: ทุกขั้นตอนคำนวณเขียนเป็นโค้ดอ่านได้ ส่งให้คนอื่น (หรือ AI) ตรวจสอบได้ — สูตรที่ฝังใน cell ของ Excel ตรวจสอบยากมาก

คิดง่าย ๆ: Excel = ทำกับข้าวด้วยมือ, DataFrame = โรงงานอาหาร — ทำกินเองมือเดียวใช้มือได้ แต่ถ้าต้องทำทุกวัน วันละหมื่นจาน ต้องใช้โรงงาน

"แล้วมันทำอะไรได้บ้าง?" — ตัวอย่างงานที่ใช้บ่อยที่สุดใน Quant แต่ละงานใช้โค้ดบรรทัดเดียว (ทุกอันได้ผลถูกต้องแม้ข้อมูลขาดบางวัน เพราะ index จับคู่ให้):

อยากได้	โค้ด
return รายวันของทุกเหรียญ	<code>df.pct_change()</code>
ค่าเฉลี่ยเคลื่อนที่ 20 วันของ BTC	<code>df["BTC"].rolling(20).mean()</code>
เฉพาะวันที่ BTC ตกเกิน 2%	<code>df[df["BTC"].pct_change() < -0.02]</code>
correlation ระหว่างทุกคู่เหรียญ	<code>df.pct_change().corr()</code>
กราฟราคาทุกเหรียญ	<code>df.plot()</code>

9.5 Operations ที่ใช้บ่อยใน Quant

```
# --- ดูข้อมูลเบื้องต้น ---
df.head(3)          # 3 แถวแรก
df.tail(3)          # 3 แถวสุดท้าย
df.describe()       # mean, std, min, max, quartile
df.shape            # (5, 3) = 5 แถว 3 คอลัมน์
df.dtypes           # type ของแต่ละคอลัมน์

# --- เลือกข้อมูล ---
df["BTC"]           # คอลัมน์ BTC (Series)
df[["BTC", "ETH"]]  # สองคอลัมน์ (DataFrame)
df.loc["2024-01-03"] # .loc: เลือกด้วย "ป้ายชื่อ" (label) = วันที่
df.iloc[0]          # .iloc: เลือกด้วย "ตำแหน่ง" (position) = แถวที่ 0 คือแถวแรก
df.iloc[-1]         # .iloc[-1] = แถวสุดท้าย (เลขลบนับจากท้าย เหมือน list)

# --- คำนวณ Return ---
returns = df.pct_change() # daily return ทุก column (แถวแรกเป็น NaN เพราะไม่มีวันก่อน)
log_returns = np.log(df/df.shift(1)) # log return: shift(1) เลื่อนข้อมูลลงหนึ่งแถว (= ราคาเมื่อวาน)

# --- Rolling Statistics ---
rolling_mean = df["BTC"].rolling(window=3).mean() # ค่าเฉลี่ย 3 วันล่าสุด (2 แถวแรกเป็น NaN)
rolling_std = df["BTC"].rolling(window=3).std() # std 3 วันล่าสุด
zscore = (df["BTC"] - rolling_mean) / rolling_std # z-score: ห่างจากค่าเฉลี่ยกี่ std
```

.loc vs .iloc — เลือกตัวไหน?

	<code>.loc["2024-01-03"]</code>	<code>.iloc[2]</code>
เลือกด้วย	ป้ายชื่อ (วันที่, ชื่อหุ้น)	ตำแหน่ง (0, 1, 2, -1)
ข้อดี	ชัดเจน ไม่ผิดพลาดแม้เรียงลำดับใหม่	เร็ว ใช้ได้แม้ไม่รู้ชื่อ index
ใช้เมื่อ	รู้วันที่/label ที่ต้องการ	ต้องการ "ตัวแรก" หรือ "ตัวสุดท้าย"

NaN คืออะไร? "Not a Number" = "ไม่มีข้อมูล ณ จุดนี้" — pandas ใส่อัตโนมัติเมื่อคำนวณไม่ได้ (เช่น แถวแรกของ `pct_change()` หรือหน้าตาแรกของ `rolling()`) — `dropna()` ลบแถวที่มี NaN ออก, `fillna(0)` แทน NaN ด้วยศูนย์ — pandas ส่วนใหญ่ข้าม NaN ให้อัตโนมัติ (`mean()` , `std()`) แต่บางงาน (เช่น `corr()`) ต้องการ `dropna()` ก่อน

```
# --- Filter: boolean indexing — "กรองแถวด้วยเงื่อนไข" ---
btc_returns = df["BTC"].pct_change()

# แทนที่จะ loop แล้ว if เองทีละแถว pandas ทำให้ในบรรทัดเดียว:
crash_days = df[btc_returns < -0.02] # เลือกเฉพาะแถวที่ return < -2%
print(f"BTC crash days (<-2%): {len(crash_days)}")
```

```
# เงื่อนไขสองข้อ: ต้องใช้ & และ ครอบแต่ละเงื่อนไขด้วย ()
both_up = df[(btc_returns > 0) & (df["ETH"].pct_change() > 0)]
```

ไฟล์ btc_daily.csv มาจากไหน?

มีสองทาง — ทางที่ 1 (แนะนำสำหรับตอนนี้: ไม่ต้องดาวน์โหลดอะไรเลย รันบล็อกด้านล่างเพื่อสร้างไฟล์จำลองขึ้นมาเอง แล้วโค้ดทั้งหัวข้อนี้อันจะรันได้ทันที

ทางที่ 2: ใช้ข้อมูลจริง — `pip install yfinance` แล้ว

```
import yfinance as yf
raw = yf.download("BTC-USD", start="2023-01-01", auto_adjust=True)
raw.to_csv("btc_daily.csv")
```

⚠ แต่ไฟล์ที่ได้จะยังใช้กับโค้ดด้านล่างไม่ได้ทันที เพราะแหล่งข้อมูลจริงตั้งชื่อคอลัมน์เป็นตัวใหญ่ (Close ไม่ใช่ close) และตั้งชื่อ index ว่า Date — ถ้าเจอ `KeyError: 'close'` คือเรื่องนี้ไม่ใช่คุณพิมพ์ผิด นี่คือนักพัฒนาของงานข้อมูลจริง จึงต้องมีขั้น "จัดบ้าน" ก่อนใช้เสมอ:

```
df = pd.read_csv("btc_daily.csv", index_col=0, parse_dates=True)

# บางเวอร์ชันของ yfinance ให้หัวตารางสองชั้น (MultiIndex) — ยับให้เหลือชั้นเดียว
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)

df.columns = df.columns.str.lower() # Close -> close
df.index.name = "date"              # Date -> date
print(df.columns.tolist())          # เช็คก่อนใช้เสมอ
```

🔗 **นิสัยที่ควรติดตัว:** เปิดไฟล์ข้อมูลใหม่ทุกครั้งให้ `print(df.columns.tolist())` และ `df.head()` ก่อนเขียนโค้ดต่อ — ประหยัดเวลาดี๊ดีได้มหาศาล

Running Example: โหลดข้อมูลจาก CSV และคำนวณ Metrics

```
# — สร้างไฟล์จำลองก่อน เพื่อให้รันได้ทันทีโดยไม่ต้องดาวน์โหลด —
# (ถ้ามี btc_daily.csv จริงแล้ว ข้าม 8 บรรทัดนี้ไปได้เลย)
np.random.seed(42)
dates_sim = pd.date_range("2023-01-01", periods=300, freq="D")
close_sim = 30000 * np.exp(np.cumsum(np.random.normal(0.001, 0.02, 300)))
pd.DataFrame({
    "date": dates_sim, "open": close_sim * 0.999, "high": close_sim * 1.01,
    "low": close_sim * 0.99, "close": close_sim,
```

```

    "volume": np.random.uniform(1e4, 5e4, 300),
}).to_csv("btc_daily.csv", index=False)

# — ตั้งแต่บรรทัดนี้คือโค้ดที่ใช้กับไฟล์จริงได้เหมือนกัน —
# ไฟล์ btc_daily.csv มีคอลัมน์ date, open, high, low, close, volume
df = pd.read_csv("btc_daily.csv", index_col="date", parse_dates=True)
df = df.sort_index()          # เรียงวันที่จากเก่าไปใหม่

close = df["close"]          # หยิบคอลัมน์ราคาปิดมาตั้งชื่อสั้น ๆ

# 1) return รายวัน
df["return"] = close.pct_change() # เปอร์เซ็นต์เปลี่ยนแปลงจากวันก่อน

# 2) log return
prev_close = close.shift(1)     # ราคาเมื่อวาน (เลื่อนลง 1 แถว)
df["log_ret"] = np.log(close / prev_close)

# 3) annualized volatility จากหน้าต่าง 20 วัน
daily_vol_20 = df["return"].rolling(20).std() # std ของ return 20 วันล่าสุด
df["vol_20"] = daily_vol_20 * np.sqrt(252)    # ใช้ 252 เพื่อความง่าย — crypto เทรด 24

# 4) moving average 50 วัน
df["ma_50"] = close.rolling(50).mean()

# 5) z-score: ราคาวันนี้ห่างค่าเฉลี่ย 60 วัน กี่ std
mean_60 = close.rolling(60).mean() # ค่าเฉลี่ยเคลื่อนที่ 60 วัน
std_60 = close.rolling(60).std()   # std เคลื่อนที่ 60 วัน
df["zscore"] = (close - mean_60) / std_60 # สูตร z-score เดิมจากบทที่ 7

print(df.tail()) # ดูผลลัพธ์ 5 วันล่าสุด
print(f"\nAnnualized Vol (20d): {df['vol_20'].iloc[-1]:.1%}")
print(f"Current Z-score: {df['zscore'].iloc[-1]:.2f}")

```

AI จะยุบ 5 ขั้นนี้เหลือแบบนี้ — ฝึกอ่าน method chain

```
df["vol_20"] = df["close"].pct_change().rolling(20).std() * np.sqrt(252)
```

เทคนิคอ่าน chain: อ่านจากซ้ายไปขวา เหมือนสายพานโรงงาน — ข้อมูลผ่านเครื่องจักรทีละตัว จุดคือ "ส่งต่อให้"

1. `df["close"]` — เริ่มจากราคาปิด
2. `.pct_change()` — ส่งเข้าเครื่องคำนวณ return รายวัน
3. `.rolling(20)` — ส่งต่อเข้าหน้าต่างเลื่อน 20 วัน
4. `.std()` — คำนวณ std ของแต่ละหน้าต่าง

5. * `np.sqrt(252)` — สุดท้ายคุณ $\sqrt{252}$ แปลงเป็นรายปี

คือขั้นที่ 1 + 3 ของเรารวมกันในบรรทัดเดียว — chain สั้น ๆ (2–3 จุด) อ่านสบาย แต่ถ้ายาวกว่านั้นแล้วเริ่มงง แยกเป็น
ตัวแปรกลางแบบที่เราเขียนได้เสมอ ผลเหมือนกันปะ และ debug ง่ายกว่าเพราะ print ดูค่ากลางทางได้

► แบบฝึกหัด: คำนวณ Correlation Matrix

บทที่ 10: Visualization

แก่นของบทนี้

กราฟที่ดีบอกเรื่องราวได้ภายใน 3 วินาที — ใน Quant ใช้กราฟเพื่อ debug signal, ตรวจสอบ backtest, และนำเสนอผลลัพธ์ บทนี้ใช้ matplotlib เป็นหลัก (standard, ทำได้ทุกอย่าง) และแนะนำ plotly สำหรับ interactive charts

10.1 matplotlib พื้นฐาน

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# สร้างข้อมูลตัวอย่าง
dates = pd.date_range("2024-01-01", periods=60)
prices = 65000 * np.cumprod(1 + np.random.normal(0.001, 0.02, 60))

# Plot พื้นฐาน
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(dates, prices, color="#0d9488", linewidth=1.5, label="BTC Price")
ax.set_title("BTC/USD Daily Close", fontsize=14, fontweight="bold")
ax.set_xlabel("Date")
ax.set_ylabel("Price (USD)")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("btc_price.png", dpi=150)
plt.show()
```

10.2 Multiple Subplots — Price + Return + Z-score

```
# 3 panels: ราคา, return, z-score
fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)

# สมมติ df มีคอลัมน์ close, return, zscore

# Panel 1: ราคา + MA50
axes[0].plot(df.index, df["close"], label="Close", color="#0d9488")
axes[0].plot(df.index, df["ma_50"], label="MA50", color="#f59e0b", linestyle="--")
```

```

axes[0].set_ylabel("Price ($)")
axes[0].legend()
axes[0].set_title("BTC Analysis Dashboard")

# Panel 2: Daily Return — เลือกสีแท่งทีละวัน: เขียวถ้าบวก แดงถ้าลบ
bar_colors = []
for r in df["return"]:
    if r > 0:
        bar_colors.append("#16a34a")    # เขียว
    else:
        bar_colors.append("#dc2626")    # แดง

axes[1].bar(df.index, df["return"], color=bar_colors, alpha=0.7, width=0.8)
axes[1].axhline(0, color="black", linewidth=0.5)
axes[1].set_ylabel("Daily Return")

# Panel 3: Z-score + threshold lines
axes[2].plot(df.index, df["zscore"], color="#7c3aed", linewidth=1)
axes[2].axhline(2.0, color="#dc2626", linestyle="--", alpha=0.7, label="Entry ( $\pm 2\sigma$ )")
axes[2].axhline(-2.0, color="#dc2626", linestyle="--", alpha=0.7)
axes[2].axhline(0, color="black", linewidth=0.5)
axes[2].set_ylabel("Z-score")
axes[2].legend()

plt.tight_layout()
plt.savefig("btc_dashboard.png", dpi=150)
plt.show()

```

AI จะยัด loop เลือกสีเข้าไปในวงเล็บเลย — ฟีกอ่าน

```

axes[1].bar(df.index, df["return"],
            color=["#16a34a" if r > 0 else "#dc2626" for r in df["return"]])

```

ตรง `color=[...]` คือ comprehension ที่มีเงื่อนไขเลือกค่า (A if เงื่อนไข else B) ฝังอยู่:

1. `for r in df["return"]` — หยิบ return ทีละวัน (loop เดิม)
2. `"#16a34a" if r > 0 else "#dc2626"` — อ่านว่า "เอาสีเขียว ถ้า r บวก ไม่งั้นเอาสีแดง" (รูปแบบนี้เรียก ternary — ค่าที่ได้อยู่หน้า if)
3. ผลคือ list ของสี ส่งให้ `color=` โดยตรง ไม่ต้องตั้งชื่อตัวแปร

ความหมายเหมือน loop 6 บรรทัดของเรา — ถ้าเจอแบบนี้ในโค้ด AI แล้วงง ให้ลากมันออกมาตั้งเป็นตัวแปรข้างนอกก่อน แล้วค่อยอ่าน

10.3 Scatter Plot — Correlation

```
# สร้าง DataFrame ที่มีราคา BTC และ ETH (จำลองข้อมูล 100 วัน)
import numpy as np, pandas as pd
np.random.seed(42)
dates = pd.date_range("2024-01-01", periods=100)
df_multi = pd.DataFrame({
    "BTC": 65000 * np.cumprod(1 + np.random.normal(0.001, 0.02, 100)),
    "ETH": 3200 * np.cumprod(1 + np.random.normal(0.001, 0.025, 100)),
}, index=dates)

# วิเคราะห์ correlation ระหว่าง BTC และ ETH returns
btc_ret = df_multi["BTC"].pct_change().dropna() # return รายวัน ลบแถวแรกที่เป็น NaN
eth_ret = df_multi["ETH"].pct_change().dropna()

fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(btc_ret, eth_ret, alpha=0.4, color="#0d9488", s=20)

# เพิ่ม regression line
m, b = np.polyfit(btc_ret, eth_ret, 1) # slope, intercept
x_line = np.linspace(btc_ret.min(), btc_ret.max(), 100)
ax.plot(x_line, m * x_line + b, color="#dc2626", linewidth=2,
        label=f" $\beta = {m:.2f}$ ")

ax.set_xlabel("BTC Daily Return")
ax.set_ylabel("ETH Daily Return")
ax.set_title(f"BTC vs ETH Returns (r={btc_ret.corr(eth_ret):.2f})")
ax.legend()
plt.tight_layout()
plt.show()
```

Matplotlib Cheat Sheet — สำหรับ Quant

Function	ใช้ทำ
<code>ax.plot(x, y)</code>	เส้นราคา, equity curve
<code>ax.bar(x, y)</code>	return รายวัน, volume
<code>ax.scatter(x, y)</code>	correlation, scatter plot
<code>ax.fill_between(x, y1, y2)</code>	Bollinger Bands, confidence interval
<code>ax.axhline(y)</code>	เส้น threshold (entry/exit level)
<code>ax.axvspan(x1, x2)</code>	ไฮไลต์ช่วงเวลา (crisis, drawdown)

ตัวอย่าง: Equity Curve + Drawdown

```
import matplotlib.pyplot as plt
import numpy as np

# จำลอง equity curve
np.random.seed(42)
daily_ret = np.random.normal(0.0005, 0.015, 252)
equity = 100000 * np.cumprod(1 + daily_ret)

# คำนวณ drawdown
running_max = np.maximum.accumulate(equity)
drawdown = (equity - running_max) / running_max

# Plot
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

ax1.plot(equity, color="#0d9488", linewidth=1.5)
ax1.set_ylabel("Portfolio Value ($)")
ax1.set_title("Strategy Equity Curve")
ax1.grid(True, alpha=0.3)

ax2.fill_between(range(len(drawdown)), drawdown, 0,
                 color="#dc2626", alpha=0.5)
ax2.set_ylabel("Drawdown")
ax2.set_xlabel("Trading Day")
ax2.grid(True, alpha=0.3)

max_dd = drawdown.min()
final = equity[-1]
print(f"Final Equity:   ${final:,.0f}")
print(f"Total Return:   {(final/100000-1):.1%}")
print(f"Max Drawdown:   {max_dd:.1%}")

plt.tight_layout()
plt.show()
```

ทำไม่ชื่อกราฟในเล่มนี้เป็นภาษาอังกฤษทั้งหมด

ถ้าใส่ข้อความไทยใน `set_title()` / `set_xlabel()` / `label=` ตรงๆ matplotlib จะพิมพ์ออกมาเป็น **สี่เหลี่ยมเปล่า** `□□□` พร้อมคำเตือน `Glyph missing from font(s) DejaVu Sans` — เพราะฟอนต์ติดตั้งของ matplotlib ไม่มีตัวอักษรไทย

แก้ได้ถ้าต้องการไทยจริงๆ (ต้องมีไฟล์ฟอนต์ในเครื่อง):

```
import matplotlib
matplotlib.rcParams["font.family"] = "Sarabun" # หรือ "TH Sarabun New", "Noto Sans
matplotlib.rcParams["axes.unicode_minus"] = False # กันเครื่องหมายลบกลายเป็นสี่เหลี่ยม
```

เล่มนี้เลือกใช้**อังกฤษในกราฟไทยในคำอธิบาย** เพื่อให้โค้ดทุกบล็อกกันได้เหมือนกันทุกเครื่องโดยไม่ต้องติดตั้งฟอนต์ก่อน

10.4 Histogram + Bell Curve — รูปร่างของการกระจาย

สามหัวข้อที่ผ่านมาดู**"ราคาเดินทางไปทางไหน"** (เวลาอยู่แกน x) · หัวข้อนี้เปลี่ยนคำถามเป็น**"ผลตอบแทนกระจายตัวอย่างไร"** — ทิ้งเวลาไปเลย เอาแต่ค่ามากองรวมกันแล้วดูรูปร่าง

แก่นของหัวข้อนี้

Histogram ตอบคำถามที่กราฟเส้นตอบไม่ได้: **"วันที่เหวี่ยงแรง ๆ เกิดบ่อยแค่ไหน?"** · และเมื่อวาดเส้นระฆัง (Normal) ทับลงไป คุณจะเห็นด้วยตาว่าผลตอบแทนจริงไม่ใช่ระฆัง — หางหนากว่ามาก · นี่คือสาเหตุที่โมเดลความเสี่ยงซึ่งสมมติ Normal ประเมินความเสี่ยงต่ำเกินไปเสมอ

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# จำลอง return รายวัน 3 ปี แบบที่คล้ายของจริง:
# ส่วนใหญ่เป็นวันธรรมดา + 3% ของวันเป็นวันข่าวแรง (jump)
np.random.seed(3)
n = 750
btc_ret = pd.Series(
    np.random.normal(0.0005, 0.018, n)
    + np.random.choice([0, 1], n, p=[0.97, 0.03]) * np.random.normal(0, 0.07, n),
    name="BTC")
eth_ret = pd.Series(
    np.random.normal(0.0004, 0.026, n)
    + np.random.choice([0, 1], n, p=[0.96, 0.04]) * np.random.normal(0, 0.09, n),
    name="ETH")

mu, sd = btc_ret.mean(), btc_ret.std()

fig, ax = plt.subplots(figsize=(10, 5))

# ① Histogram — density=True เพื่อให้สเกลเทียบกับเส้นโค้งได้
ax.hist(btc_ret, bins=60, density=True, alpha=0.55,
```

```

        color="#0d9488", edgecolor="white", label="BTC returns (actual)")

# ② เส้นระฆัง Normal ที่มี mean/std เดียวกันปะ
x = np.linspace(btc_ret.min(), btc_ret.max(), 400)
ax.plot(x, stats.norm.pdf(x, mu, sd), color="#dc2626", lw=2.5,
        label=f"Normal( $\mu$ = $\{\mu:.4f\}$ ,  $\sigma$ = $\{\sigma:.4f\}$ )")

# ③ ขีดเส้น  $\pm 3\sigma$  ให้เห็นว่า "ขอบของโลก Normal" อยู่ไหน
for k in (-3, 3):
    ax.axvline(mu + k*sd, color="#7c3aed", ls="--", lw=1.2)
ax.text(mu + 3*sd, ax.get_ylim()[1]*0.7, " +3 $\sigma$ ", color="#7c3aed", fontsize=9)

ax.set_title("BTC Daily Returns vs Normal (look at both tails)")
ax.set_xlabel("Daily Return")
ax.set_ylabel("Density")
ax.legend()
ax.grid(True, alpha=0.3)

# ④ ตัวเลขที่ยืนยันสิ่งที่ตาเห็น
print(f"Skewness: {stats.skew(btc_ret):+.2f}      "
      f"Excess kurtosis: {stats.kurtosis(btc_ret):+.2f}      (Normal = 0.00, 0.00)")
print("\nวันที่เหวี่ยงเกิน k เท่าของ  $\sigma$  – จริง vs ที่ Normal ทำนาย")
for k in (1, 2, 3, 4, 5):
    actual = int((btc_ret.abs() > k*sd).sum())
    expected = 2 * stats.norm.sf(k) * n
    ratio = f"{actual/expected:>7.1f}x" if expected >= 0.05 else "    >100x"
    print(f" |r| > {k} $\sigma$  : {actual:4d} วัน · Normal {expected:7.1f} วัน · {ratio}")

plt.tight_layout()
plt.show()

```

► OUTPUT Skewness: +0.55 Excess kurtosis: +10.89 (Normal = 0.00, 0.00) วันที่เหวี่ยงเกิน k เท่าของ σ – จริง vs ที่ Normal ทำนาย |r| > 1σ : 158 วัน · Normal 238.0 วัน · 0.7x |r| > 2σ : 24 วัน · Normal 34.1 วัน · 0.7x |r| > 3σ : 10 วัน · Normal 2.0 วัน · 4.9x |r| > 4σ : 7 วัน · Normal 0.0 วัน · >100x |r| > 5σ : 4 วัน · Normal 0.0 วัน · >100x

📌 **อ่านว่า:** density=True คือกฎของบอล็อกนี้ — ปกติ hist นับ "จำนวนวัน" ในแต่ละแท่ง แต่ density=True เปลี่ยนเป็นสัดส่วนที่ทำให้พื้นที่รวมทั้งกราฟ = 1 · จำเป็นเพราะ norm.pdf ก็มีพื้นที่รวม 1 เหมือนกัน — ถ้าไม่ใส่เส้นแดงจะแบนติดพื้นจนมองไม่เห็นอะไรเลย · สองสิ่งจะเทียบกันได้ ต้องอยู่สเกลเดียวกันก่อน

อ่านตารางนี้ให้ออก — นี่คือ "fat tail" ที่ทุกคนพูดถึง

สังเกตว่ามันแปลกสองทางพร้อมกัน:

- ช่วง 1σ – 2σ เกิดน้อยกว่า Normal (0.7 เท่า) — วันที่ "เหวี่ยงพอประมาณ" มีน้อยกว่าที่ทฤษฎีบอก

- ช่วงเกิน 3σ เกิดมากกว่ามหาศาล — 3σ เกิด 4.9 เท่า · 4σ กับ 5σ เกินร้อยเท่า (Normal ทำนายว่าแทบไม่ควรถูกพบใน 750 วัน)

แปลว่าผลตอบแทนจริงไม่ได้กระจายกว้างกว่า Normal ทั้งกระดาน — มันเอามวลจากช่วงกลาง ๆ ไปกองไว้ที่ *ตรงกลางสุด* (วันเจียบ) และ *ปลายหาง* (วันหายน่ะ) · ตลาดจึงดู "นิ่งกว่าที่คิด" เป็นเดือน ๆ แล้วขยับทีเดียวจนพอร์ตพัง

🔴 **ผลทางปฏิบัติ:** โมเดลที่สมมติ Normal จะบอกว่าการเหวี่ยง -4σ "เกิดทุก 40 ปี" แต่ตารางข้างบนบอกว่ามันเกิด **7 ครั้งใน 3 ปี** · ทุกครั้งที่เห็นคำว่า "เหตุการณ์ 1 ในล้านปี" ในรายงานความเสี่ยงให้ถามก่อนว่าเขาสมมติการกระจายแบบไหน

10.5 Box Plot — เปรียบการกระจายหลายสินทรัพย์

Histogram ดูได้ที่ละตัว · เปรียบหลายตัวพร้อมกันใช้ **box plot** ซึ่งเปรียบเทียบการกระจายทั้งก้อนให้เหลือกล่องเดียว

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))

# — ซ้าย: box plot เปรียบ BTC กับ ETH —
#หมายเหตุ: ไม่ใส่ labels= ใน boxplot() เพราะ matplotlib รุ่นใหม่
# เปลี่ยนชื่อเป็น tick_labels — ตั้งชื่อแกนแยกจึงเข้ากันได้ทุกเวอร์ชัน
bp = ax1.boxplot([btc_ret, eth_ret],
                  patch_artist=True, showfliers=True,
                  medianprops=dict(color="#111827", lw=2))
for patch, color in zip(bp["boxes"], ["#0d9488", "#7c3aed"]):
    patch.set_facecolor(color)
    patch.set_alpha(0.45)

ax1.set_xticks([1, 2])
ax1.set_xticklabels(["BTC", "ETH"])
ax1.axhline(0, color="#9ca3af", lw=1)
ax1.set_title("Daily Return Distribution")
ax1.set_ylabel("Daily Return")
ax1.grid(True, alpha=0.3, axis="y")

# — ขวา: histogram ซ้อนกัน เพื่อเทียบกับ box plot —
ax2.hist(btc_ret, bins=50, alpha=0.5, density=True,
         color="#0d9488", label="BTC")
ax2.hist(eth_ret, bins=50, alpha=0.5, density=True,
         color="#7c3aed", label="ETH")
ax2.set_title("Same data as histogram")
ax2.set_xlabel("Daily Return")
ax2.legend()
ax2.grid(True, alpha=0.3)

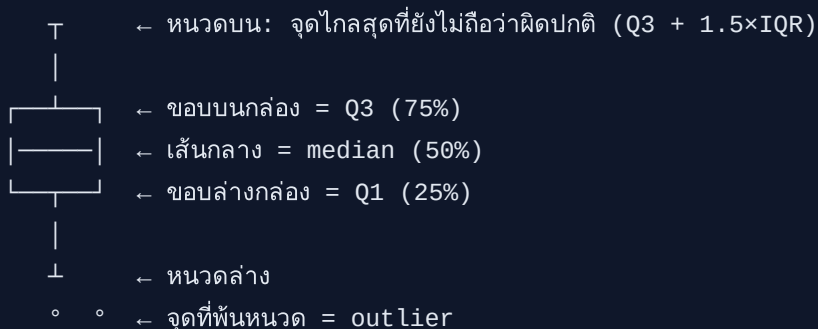
# ตัวเลขที่ box plot กำลังวาด
```

```
for s in (btc_ret, eth_ret):
    q1, med, q3 = s.quantile([0.25, 0.5, 0.75])
    iqr = q3 - q1
    n_out = int(((s < q1 - 1.5*iqr) | (s > q3 + 1.5*iqr)).sum())
    print(f"{s.name}: median={med:+.4f} IQR={iqr:.4f} "
          f"outlier={n_out} วัน ({n_out/len(s):.1%})")

plt.tight_layout()
plt.show()
```

► OUTPUT BTC: median=+0.0008 IQR=0.0255 outlier=17 วัน (2.3%) ETH: median=+0.0009 IQR=0.0321 outlier=20 วัน (2.7%)

📊 box plot บอกอะไร — 5 ตัวเลขในกล่องเดียว



1. กล่อง = ช่วงที่ข้อมูล **50% กลาง** อยู่ · ความสูงของกล่องคือ $IQR = Q3 - Q1$ · BTC IQR 0.0255 vs ETH 0.0321 → **ETH เหวี่ยงกว่าในวันธรรมดา ~26%**

2. เส้นกลาง = median ไม่ใช่ mean — **outlier ดึงไม่ได้** จึงบอก "วันปกติ" ได้ตรงกว่าค่าเฉลี่ย

3. หนด ยาวถึง $1.5 \times IQR$ จากขอบกล่อง (เกณฑ์มาตรฐานไม่ใช่กฎธรรมชาติ)

4. จุดที่พ้นหนด = สิ่งที่ box plot เรียกว่า outlier — BTC มี 44 วัน (5.9%) · ⚠️ ในโลกการเงินอย่าเรียกมันว่า "ข้อมูลผิดพลาด" เพราะวันเหล่านั้นคือวันที่กำไรและขาดทุนจริงเกิดขึ้น

เทียบสองภาพซ้าย-ขวา: box plot บอก "ตรงกลางอยู่ไหน กว้างแคไหน" ได้เร็วและเทียบหลายตัวได้ · histogram บอก "รูปร่างเป็นอย่างไร มีสองยอดไหม เบื้องไหน" ซึ่ง box plot มองไม่เห็น — **ใช้คู่กัน ไม่ใช่เลือกอย่างเดียว**

🔵 [รอบสอง] histogram หลอกตาได้ และเครื่องมือที่แม่นยำกว่าสำหรับดูหาง

จำนวน bins เปลี่ยนข้อสรุปได้ — ข้อมูลชุดเดิมปะะ ๆ ถ้าใช้ `bins=10` จะดูเป็นระฆังสวย ๆ แต่ `bins=200` จะดูขาดวันเป็นหนาม · ไม่มีค่าที่ "ถูก" · ลองสองสามค่าเสมอก่อนสรุปอะไรจากรูปร่าง (หรือใช้ `bins="fd"` ให้ matplotlib เลือกด้วยกฎ Freedman-Diaconis)

```
# ดูผลของ bins ต่อข้อสรุป — ข้อมูลชุดเดียวกันทั้งสามภาพ
fig, axes = plt.subplots(1, 3, figsize=(12, 3.2))
for ax, b in zip(axes, [10, 60, 200]):
```

```
ax.hist(btc_ret, bins=b, density=True, color="#0d9488", alpha=0.6)
ax.set_title(f"bins={b}")
ax.set_xlim(-0.15, 0.15)
plt.tight_layout(); plt.show()
```

สำหรับดูทางโดยเฉพาะ histogram เป็นเครื่องมือที่แย เพราะทางมีข้อมูลน้อย แท่งจึงเตี้ยจนมองไม่ออกว่าต่างจาก Normal แค่ไหน เครื่องมือที่ตรงงานคือ **QQ-plot** ซึ่งเอา quantile ของข้อมูลจริงไปพล็อตกับ quantile ของ Normal — ถ้าตรงกันจะได้เส้นตรง 45° ส่วนที่ *โก่งขึ้นปลายทั้งสองข้าง* คือทางที่หนากว่า Normal:

```
fig, ax = plt.subplots(figsize=(5.5, 5))
stats.probplot(btc_ret, dist="norm", plot=ax)
ax.set_title("QQ-plot: bends at both ends = fat tail")
ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

📖 **โยงไปข้างหน้า:** ตัวเลข excess kurtosis +10.89 ที่เห็นในหัวข้อนี้จะกลับมาเป็นตัวละครหลักใน บทที่ 11 ตอนคำนวณ VaR — ที่นั่นเราจะเห็นว่าใช้ Normal คำนวณ VaR ทำให้ประเมินความเสี่ยงต่ำไปประมาณ 1.4 เท่า และทำไมต้องเปลี่ยนไปใช้ Student-t

10.6 plotly — Interactive Charts (แนะนำ)

```
# pip install plotly
import plotly.graph_objects as go

fig = go.Figure()

# Candlestick chart
fig.add_trace(go.Candlestick(
    x=df.index,
    open=df["open"],
    high=df["high"],
    low=df["low"],
    close=df["close"],
    name="BTC/USDT"
))

# เพิ่ม MA
fig.add_trace(go.Scatter(
    x=df.index, y=df["ma_50"],
    line=dict(color="#f59e0b", width=1.5),
    name="MA50"
))
```

```
fig.update_layout(
    title="BTC/USDT Candlestick",
    yaxis_title="Price (USD)",
    xaxis_rangeslider_visible=False,
    template="plotly_dark"
)
fig.show() # เปิดใน browser — zoom, pan ได้
```

► แบบฝึกหัด: Plot Spread Z-score พร้อม Entry/Exit Signals

จบ Part I — ทักษะที่ได้

ตอนนี้คุณสามารถ:

- อ่านโค้ดออกเสียงเป็นภาษาไทยได้ที่ละบรรทัด — รวมถึง comprehension และ method chain ที่ AI ชอบเขียน ✓
- เขียน Python function, loop, และ conditional logic สำหรับ signal generation ✓
- เก็บข้อมูลด้วย list, dict, tuple, set เลือกถูกตามสถานการณ์ ✓
- โหลดข้อมูลตลาดเข้า pandas DataFrame คำนวณ return, rolling vol, z-score ✓
- plot equity curve, drawdown, scatter plot, candlestick ด้วย matplotlib/plotly ✓

ใน **Part II** เราจะนำ Python ที่รู้มาต่อเข้ากับ math จาก Part 0: OLS regression, optimization, linear programming, cointegration

⚠ ก่อนไป Part II — สิ่งสำคัญที่ต้องจำ

1. **Lookahead bias:** signal ที่คำนวณจากข้อมูลวัน t ต้องถูก `.shift(1)` ก่อน execute วัน $t+1$ — Part IV จะอธิบายละเอียด แต่จำหลักนี้ไว้ตั้งแต่ตอนนี้
2. **Code ใน Part I ยังไม่ optimize:** loop แบบ manual ที่เขียนเร็วพอสำหรับเรียน แต่ช้ากว่า pandas vectorized ใน Part II ประมาณ 10–100x สำหรับข้อมูลหลายหมื่นแถว
3. ตัวเลข "ดูดี" ยังไม่น่าเชื่อ: Sharpe หรือ Return ที่คำนวณในตัวอย่าง Part I–II เป็น in-sample ทั้งหมด Part IV จะสอน Walk-Forward เพื่อยืนยันว่าดีจริงหรือ overfit

PYTHON FOR QUANT TRADERS · PART II

Math Tools — เครื่องมือคณิตศาสตร์สำหรับ Quant

บทที่ 11–17: Statistics · Time Series · OLS Regression · Cointegration · Optimization · Linear
Programming · Signals

จบ Part นี้ → สร้าง pair trading strategy ตั้งแต่ต้นจนได้ signal พร้อม trade

🚫 อ่าน Part นี้ยังไ้

🟢 **เพ็ญจบ Part I** — Part นี้คือจุดที่ได้เริ่มยากขึ้นชัดเจน · กลยุทธ์ที่ได้ผล: อ่าน **กล่อง** 💡 **แก่นของบทนี้** ให้เข้าใจก่อน แล้วค่อยลงโค้ด · เจอฟังก์ชันสถิติที่ไม่รู้จัก (`adfuller` , `coint` , `minimize`) **ไม่ต้องเข้าใจคณิตศาสตร์ข้างในทั้งหมด** — เข้าใจว่า "ใส่อะไรเข้าไป ได้อะไรออกมา แล้วตีความยังไง" ก็พอสำหรับรอบแรก

🟢 **มีพื้น stat/econometrics** — เนื้อทฤษฎีคุ้นอยู่แล้ว เข้าได้ · ที่ควรอ่านคือ **กล่อง** ⚠️ **กับดัก** ทั้งหมด โดยเฉพาะ **บทที่ 14 เรื่อง null hypothesis กลับหัว** (p เล็ก = ชั่วดี) ที่คนอ่านผิดกันบ่อยมากจนเลือกคู่ที่ไม่ cointegrate มาเทรดทั้งพอร์ต · และ **บทที่ 13 เรื่องข้อมูลล้นมาจาก pandas ไม่ใช่ statsmodels**

📌 สิ่งที่จะทำได้เมื่อจบ PART II

- หาคู่หุ้นที่ cointegrate กัน ด้วย ADF test และ Johansen test
 - ประมาณค่า (estimate) hedge ratio β ด้วย OLS regression (statsmodels)
 - สร้าง spread และคำนวณ z-score สำหรับ entry/exit signal
 - Maximize Sharpe ratio ด้วย `scipy.optimize`
 - กำหนด portfolio weight constraints ด้วย Linear Programming
- ทุกอย่างนี้คือ **core engine** ของ Statistical Arbitrage

🔗 Running Example ตลอด Part II — Pair Trading: BTC Perp Bybit ↔ BTC Perp Binance

เราจะสร้าง pair trading strategy ที่ละขั้นตอนตลอด 7 บท:

บท 11 → ตรวจสอบ distribution ของ spread · บท 12 → ทดสอบ stationarity · บท 13 → ประมาณ β ด้วย OLS

บท 14 → ยืนยัน cointegration · บท 15 → optimize position size · บท 16 → ใส่ capital constraint · บท 17 → สร้าง signal pipeline

บทที่ 11: Statistics & Probability เชิงลึก

แก่นของบทนี้

สถิติใน Part 0 เป็นแนวคิด — บทนี้แปลงเป็น Python จริง ด้วย `scipy.stats` และ `numpy` เน้น 3 เรื่องที่ Quant ใช้จริง: hypothesis testing (กลยุทธ์นี้ดีจริงหรือแค่โชค?), VaR/CVaR (ความเสี่ยงสูงสุดคือเท่าไร?), และ fat tail (ทำไมต้องระวัง Black Swan?)

11.1 Hypothesis Testing — กลยุทธ์ดีจริงหรือแค่โชค?

สมมติฐาน: H_0 (null) = "กลยุทธ์ไม่ดีกว่าสุ่ม" vs H_1 = "กลยุทธ์ดีกว่าสุ่มจริงๆ"

```
from scipy import stats
import numpy as np

# ผลตอบแทนรายวัน 252 วัน จากกลยุทธ์
np.random.seed(42)
strategy_returns = np.random.normal(loc=0.0008, scale=0.015, size=252)

# t-test: H0 = mean return = 0 (ไม่ดีกว่าสุ่ม)
t_stat, p_value = stats.ttest_1samp(strategy_returns, popmean=0)

print(f"Mean Return: {strategy_returns.mean():.4f} ({strategy_returns.mean()*252:.1%}/year)")
print(f"t-statistic: {t_stat:.3f}")
print(f"p-value: {p_value:.4f}")

if p_value < 0.05:
    print("✓ มีนัยสำคัญ — กลยุทธ์น่าจะดีกว่าสุ่ม (p < 5%)")
else:
    print("✗ ไม่มีนัยสำคัญ — อาจเป็นแค่โชค")
```

► OUTPUT

```
Mean Return: 0.0007 (18.7%/year)
t-statistic: 0.814
p-value: 0.4167
✗ ไม่มีนัยสำคัญ — อาจเป็นแค่โชค
```

ทำไม Return ดูดีแต่ p-value ยังสูง?

20% ต่อปีดูน่าสนใจ แต่ถ้า σ สูง (1.5%/วัน) — noise มากกว่า signal มาก
ต้องมีข้อมูลมากพอ (หลาย 100 วัน) ถึงจะ "แยก" signal ออกจาก noise ได้

$$t = \frac{\bar{r}}{\sigma/\sqrt{n}} \rightarrow \text{ยิ่ง } n \text{ มาก } t \text{ ยิ่งใหญ่} \rightarrow p\text{-value} \text{ ยิ่งเล็ก}$$

⚠️ แล้วถ้าผ่านล่ะ — $p < 0.05$ แปลว่า "ทำเงินได้" หรือเปล่า

ไม่ · และนี่คือความเข้าใจผิดที่แพงที่สุดในบรรดาความเข้าใจผิดทั้งหมดในหนังสือเล่มนี้

สังเกตบรรทัดที่เขียนว่า `popmean=0` ให้ดี — เรากำลังถามว่า "ผลตอบแทนต่างจากศูนย์ไหม" · แต่คำถามที่เราสนใจจริงๆ คือ "ผลตอบแทนชนะค่าธรรมเนียมไหม" · สองคำถามนี้ไม่เหมือนกัน และสถิติตอบให้แค่คำถามที่เราถาม

สูตร $t = \bar{r} / (\sigma/\sqrt{n})$ ในกล่องข้างบนบอกความจริงอีกด้านที่ต้องเห็นให้ครบ: ตัวหารเล็กลงเรื่อย ๆ ตาม $\sqrt{n}$ · แปลว่าถ้าเก็บข้อมูลมากพอ edge เล็กแค่ไหนก็ผลักให้ $p < 0.05$ ได้ทั้งนั้น — รวมถึง edge ที่เล็กกว่าค่าธรรมเนียมหลายเท่า · $p\text{-value}$ ไม่เคยรู้ว่าคุณจ่ายค่าธรรมเนียมเท่าไร เพราะไม่มีใครบอกมัน

📖 หัวข้อ 25.4b ใน Part IV แสดงตัวอย่างที่วัดเป็นตัวเลขได้: กลยุทธ์ที่ $t = +6.62$ · $p < 0.0001$ (มี edge จริงแน่นอน) แต่ขาดทุนปีละ 27,000 บาท เพราะ edge เล็กกว่าต้นทุน · พร้อมวิธีแก้ที่ใช้เวลาแก้แค่บรรทัดเดียว

11.2 VaR และ CVaR — ความเสี่ยงสูงสุดคือเท่าไร?

$$\text{VaR}_{95\%} = \text{percentile}_{5\%}(r) \quad \text{CVaR}_{95\%} = E[r \mid r \leq \text{VaR}_{95\%}]$$

VaR = ขาดทุนสูงสุดใน 95% ของวันทั้งหมด

CVaR = ขาดทุนเฉลี่ยในวันที่แย่ที่สุด 5%

`returns = strategy_returns` # จากข้างบน

`capital = 1_000_000` # ทุน 1 ล้านบาท

`var_95 = np.percentile(returns, 5)`

`cvar_95 = returns[returns <= var_95].mean()`

`print(f"VaR 95%: {var_95:.2%} → ขาดทุนไม่เกิน {abs(var_95)*capital:,.0f} บาท/วัน (95% ของเวลา)`

`print(f"CVaR 95%: {cvar_95:.2%} → เฉลี่ยขาดทุน {abs(cvar_95)*capital:,.0f} บาท ในวันที่แย่ที่สุด 5%`

หมายเหตุ CVaR: ต้องการ sample เพียงพอ

CVaR 95% ใช้เฉพาะ return ที่อยู่ใน worst 5% — ถ้ามีข้อมูล 100 วัน จะใช้แค่ 5 วัน

ถ้าข้อมูลน้อยกว่า 60 วัน CVaR อาจไม่น่าเชื่อถือ ควรใช้ `if len(tail) >= 5: cvar = tail.mean()`

หรือเพิ่ม `n_tail = max(1, int(len(returns) * 0.05))` เพื่อรับประกัน minimum sample

11.3 Fat Tail — ทำไม Normal Distribution ถึงอันตราย?

```
from scipy import stats

# เปรียบ Normal กับ t-distribution (fat tail)
normal_99 = stats.norm.ppf(0.01) # -2.33σ
t5_99      = stats.t.ppf(0.01, df=5) # -3.36σ (df=5 = fat tail)

print(f"Normal 1% VaR: {normal_99:.2f}σ")
print(f"t(df=5) 1% VaR: {t5_99:.2f}σ")
print(f"ใช้ Normal จะ underestimate ความเสี่ยง {t5_99/normal_99:.1f}x")
```

อย่าเชื่อ Normal Distribution สำหรับ crypto และช่วง crisis

BTC crash 50%+ เกิดหลายครั้ง — ถ้าใช้ Normal Distribution จะบอกว่า "แทบเป็นไปไม่ได้" (เกิน 10σ — เหตุการณ์ที่สูตร Normal บอกว่าแทบเป็นไปไม่ได้เลยในช่วงชีวิตเรา)

ในความจริง: ราคาสินทรัพย์มี "fat tail" = เหตุการณ์สุดขั้วเกิดบ่อยกว่า Normal คาด

ใช้ t-distribution, Cornish-Fisher, หรือ historical simulation แทน Normal สำหรับ risk management

ตัวอย่าง: สรุป Return Statistics แบบมืออาชีพ

```
def return_stats(returns, name="Strategy", capital=1_000_000):
    """สรุป statistics สำหรับ return series"""
    r = np.array(returns)
    ann = 252

    mean_daily = r.mean()                # return เฉลี่ยต่อวัน
    std_daily = r.std()                   # ความผันผวนต่อวัน
    sharpe = mean_daily / std_daily * np.sqrt(ann)  # Sharpe รายปี (คูณ  $\sqrt{252}$ )
    skew = stats.skew(r)                  # ความเบ้ (ลบ = หางซ้ายยาว = อันตราย)
    kurt = stats.kurtosis(r)              # excess kurtosis (บวก = fat tail)
    var95 = np.percentile(r, 5)           # จุดตัด worst 5% ของวัน
    cvar95 = r[r <= var95].mean()         # ค่าเฉลี่ยของวันที่แย่กว่า VaR

    equity = capital * np.cumprod(1 + r)   # เส้นเงินทุนสะสมรายวัน
    rolling_max = np.maximum.accumulate(equity)  # จุดสูงสุดที่เคยไปถึง ณ แต่ละวัน
    max_dd = ((equity - rolling_max) / rolling_max).min()  # % ร่วงจากยอดที่ลึกที่สุด

    print(f"=== {name} ===")
    print(f"Annual Return: {mean_daily*ann:.1%}   Sharpe: {sharpe:.2f}")
    print(f"Annual Vol: {std_daily*np.sqrt(ann):.1%}   Max DD: {max_dd:.1%}")
    print(f"Skewness: {skew:.2f}   Kurtosis: {kurt:.2f}   (Normal: 0, 0)")
    print(f"VaR 95%: {var95:.2%}   CVaR 95%: {cvar95:.2%}")

return_stats(strategy_returns)
```

► แบบฝึกหัด: เปรียบ Sharpe ของสองกลยุทธ์

บทที่ 12: Time Series Basics

แก่นของบทนี้

ข้อมูลราคาเรียงตามเวลา มีคุณสมบัติพิเศษที่ข้อมูลทั่วไปไม่มี: แต่ละจุดสัมพันธ์กับจุดก่อนหน้า (autocorrelation) และอาจ "ล่องลอย" ไปเรื่อยๆ โดยไม่กลับ (non-stationary) การเข้าใจ 2 เรื่องนี้คือ กุญแจของ Statistical Arbitrage ทั้งหมด

12.1 Stationarity — ล่องลอยหรือวนกลับ?

Stationary series: mean และ variance คงที่ตลอดเวลา — "วนกลับหาค่ากลาง"

Non-stationary series: mean เปลี่ยนตามเวลา — "ล่องลอยไม่มีทิศทาง" (random walk)

```
import numpy as np

# Non-stationary: Random Walk (ราคา BTC)
np.random.seed(0)
random_walk = np.cumsum(np.random.normal(0, 1, 300))

# Stationary: Spread ระหว่างสองตลาด
spread = np.random.normal(0, 1, 300) # OU-like (mean-reverting)

# สังเกต: random_walk drift ออกไปเรื่อยๆ
#         spread วนอยู่รอบ 0
print(f"Random Walk range: {random_walk.min():.1f} to {random_walk.max():.1f}")
print(f"Spread range:      {spread.min():.1f} to {spread.max():.1f}")
```

ทำไม Stationarity ถึงสำคัญสำหรับ Pair Trading?

ถ้า spread ไม่stationary แปลว่าไม่มีแรงดึงกลับ — z-score ที่สูงอยู่อาจสูงต่อไปเรื่อยๆ โดยไม่กลับมา เท่ากับเปิด trade แล้วอาจขาดทุนไม่หยุด

ถ้า spread stationary → มีแรงดึงกลับ → z-score สูงแล้ว "ต้องลง" ในที่สุด → เป็น basis ของ arb

เราต้องพิสูจน์ก่อนเสมอว่า spread เป็น stationary จริง — ไม่ใช่แค่ดูกราฟ

12.2 ADF Test — ทดสอบ Stationarity

Augmented Dickey-Fuller (ADF) test: H_0 = non-stationary (random walk) vs H_1 = stationary

```

from statsmodels.tsa.stattools import adfuller

def test_stationarity(series, name="Series"):
    """รัน ADF test และแสดงผลแบบเข้าใจง่าย"""
    result = adfuller(series, autolag="AIC")
    adf_stat = result[0]
    p_value = result[1]
    crit_vals = result[4]

    print(f"\n--- ADF Test: {name} ---")
    print(f"ADF Statistic: {adf_stat:.4f}")
    print(f"p-value: {p_value:.4f}")
    print(f"Critical (1%): {crit_vals['1%']:.4f}")
    print(f"Critical (5%): {crit_vals['5%']:.4f}")

    if p_value < 0.05:
        print(f"✓ STATIONARY (p={p_value:.3f} < 0.05) – กลับหาค่ากลาง")
    else:
        print(f"✗ NON-STATIONARY (p={p_value:.3f} > 0.05) – random walk")
    return p_value < 0.05

# ทดสอบกับข้อมูลจาก pair
test_stationarity(random_walk, "BTC Price (Random Walk)")
test_stationarity(spread, "BTC Spread (Mean-Reverting)")

```

12.3 Autocorrelation — ราคาวันนี้สัมพันธ์กับเมื่อวานแค่ไหน?

```

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# ACF/PACF ของ returns (stationary)
plot_acf(spread, lags=30, ax=axes[0][0], title="ACF – Spread (Stationary)")
plot_pacf(spread, lags=30, ax=axes[0][1], title="PACF – Spread")

# ACF/PACF ของ raw price (non-stationary)
plot_acf(random_walk, lags=30, ax=axes[1][0], title="ACF – Price (Non-stationary)")
plot_pacf(random_walk, lags=30, ax=axes[1][1], title="PACF – Price")

plt.tight_layout()
plt.savefig("acf_comparison.png", dpi=150)
plt.show()

```


อ่าน ACF plot อย่างไร?

Pattern ที่เห็น	แปลว่า	ทำอะไรต่อ
ค่าลด exponentially ใน ACF	AR process (mean-reverting) ✓	ใช้เป็น spread ได้
ค่าสูงทุก lag ไม่ลด	Non-stationary (random walk)	ต้อง difference ก่อน
ค่าตัดที่ lag 1 ใน ACF	MA(1) process	ต้องใส่ MA term ใน model

► แบบฝึกหัด: ทดสอบ stationarity ของ spread จริง

บทที่ 13: OLS Regression

แก่นของบทนี้

OLS (Ordinary Least Squares) เป็นเครื่องมือหลักสำหรับประมาณ hedge ratio β — อัตราส่วนที่บอกว่าต้อง long/short ในสัดส่วนเท่าไรเพื่อสร้าง spread ที่ stationary — คำว่า 'ประมาณ' ในที่นี้หมายถึงการคำนวณหาค่าที่เหมาะสมที่สุดจากข้อมูล ไม่ใช้การเดาคร่าว ๆ สูตร matrix form:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

13.1 OLS ด้วย statsmodels

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# จำลองราคา BTC สองตลาด — เหรียญเดียวกัน คนละตลาด
np.random.seed(42)
n = 252
btc_bybit = 65000 + np.cumsum(np.random.normal(0, 100, n))

# ส่วนต่างราคาสองตลาด (spread) ไม่ได้สุ่มลอย ๆ — มันดึงกลับเข้าค่ากลาง
# ใช้ OU process ตัวเดียวกับบทที่ 12: กำไรของ pair trade มาจากแรงดึงนี้
kappa, sigma_ou = 0.15, 25.0          # kappa สูง = ดึงกลับเร็ว
spread_ou = np.zeros(n)
for t in range(1, n):
    spread_ou[t] = (spread_ou[t-1]
                    + kappa * (0 - spread_ou[t-1])
                    + np.random.normal(0, sigma_ou))

# Binance = ราคา Bybit + basis 0.03% + spread ที่ดึงกลับได้
btc_binance = btc_bybit * 1.0003 + spread_ou

# OLS: log(P_binance) = alpha + beta * log(P_bybit) + epsilon
log_bybit = np.log(btc_bybit)
log_binance = np.log(btc_binance)

# ใส่ชื่อคอลัมน์ด้วย DataFrame — เพื่อเรียก params ด้วย "ชื่อ" ได้ทีหลัง
X = sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))
y = log_binance
```

```

model = sm.OLS(y, X).fit()

alpha = model.params['const']
beta = model.params['log_bybit']
resid = model.resid          # epsilon (spread)
r2 = model.rsquared
p_beta = model.pvalues['log_bybit']

print(f"alpha (intercept): {alpha:.6f}")
print(f"beta (hedge ratio): {beta:.6f}")
print(f"R²: {r2:.4f}")
print(f"p-value (beta): {p_beta:.6f}")
print(f"\nSpread (residuals) stats:")
print(f"  Mean: {resid.mean():.6f}")
print(f"  Std: {resid.std():.6f}")

```

► OUTPUT

```

alpha (intercept): 0.207854
beta (hedge ratio): 0.981260
R²: 0.9903
p-value (beta): 0.000000

Spread (residuals) stats:
  Mean: 0.000000
  Std: 0.000695

```

🔴**อ่านว่า:** บรรทัดที่ต้องสังเกต: `sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))` — ทำไมต้องห่อด้วย DataFrame? เพราะถ้าส่ง **numpy array** เข้าไปตรง ๆ ผลลัพธ์ `model.params` จะเป็น array ที่ไม่มีชื่อเรียกได้แค่ตำแหน่ง (`params[0]` , `params[1]`) → บรรทัด `params['const']` จะพังทันที (`IndexError`) .พอห่อเป็น DataFrame ที่มีชื่อคอลัมน์ statsmodels จะคืน **Series** ที่มี **label** ให้ → เรียกด้วยชื่อได้ อ่านโค้ดรู้เรื่องกว่า และไม่พลาดสลับตำแหน่ง α กับ β

⚠ **กับดักที่เจอบ่อย — "ชื่อ" มาจาก pandas ไม่ใช่ statsmodels**

นี่คือความสับสน numpy ↔ pandas ที่คลาสสิกที่สุดในงาน quant: **statsmodels** ไม่ได้ตั้งชื่อคอลัมน์ให้คุณ — มันแค่ส่งต่อชื่อที่มาจาก pandas ถ้าต้นทางไม่มีชื่อ ปลายทางก็ไม่มี

วิธีเช็คเร็ว ๆ ก่อนเรียกด้วยชื่อ: `print(type(model.params))` — ถ้าได้ `ndarray` ให้ใช้ตำแหน่ง ถ้าได้ `Series` ใช้ชื่อได้

13.2 ดีความผล OLS

Output	ความหมาย	ค่าที่ดี
β	Hedge ratio — Long 1 บาท Bybit ต้อง Short β บาท Binance	ใกล้ 1.0 สำหรับ same asset
α	Constant spread ระหว่างสองตลาด	ขึ้นอยู่กับ pair
R^2	สัดส่วน variance ที่ X อธิบายได้	> 0.85 สำหรับ good pair
p-value (β)	มั่นใจแค่ไหนว่า $\beta \neq 0$	< 0.001 สำหรับ cointegrated pair
Residuals	Spread = ส่วนที่ไม่สัมพันธ์กับ X — นี่คือนี่ที่เราจะ trade	Mean ≈ 0 , Stationary

13.3 Rolling OLS — β ที่ Adaptive

β ไม่คงที่ตลอดเวลา ต้องอัปเดตสม่ำเสมอ

```

from statsmodels.regression.rolling import RollingOLS

# Rolling OLS window 60 วัน
window = 60
# X ตัวเดิมจาก 13.1 — เป็น DataFrame ที่มีชื่อคอลัมน์อยู่แล้ว
X = sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))

rolling_model = RollingOLS(log_binance, X, window=window).fit()
rolling_beta = rolling_model.params['log_bybit']

# สร้าง rolling spread
rolling_alpha = rolling_model.params['const']
spread = log_binance - (rolling_alpha + rolling_beta * log_bybit)

print(f"Beta range: {rolling_beta.dropna().min():.4f} to {rolling_beta.dropna().max():.4f}")
print(f"Spread Std: {spread.std():.6f}")

```

► OUTPUT

```

Beta range: 0.9071 to 1.1820
Spread Std: 0.000718

```

📌 **อ่านว่า:** อ่าน `rolling_model.params['log_bybit']` ว่า: "หีบค่า β ของทุกวัน ออกมา" — ต่างจาก 13.1 ที่ได้ β ตัวเดียว ตรงนี้ได้เป็น **Series ยาวเท่าจำนวนวัน** (60 วันแรกเป็น NaN เพราะยังนับไม่ครบหน้าต่าง) · ดูช่วง 0.907–1.182 = β ขยับได้ถึง $\pm 18\%$ รอบ 1.0 นี่แหละความหมายของ " **β ไม่คงที่**" ที่หัวข้อนี้จะบอก — ถ้า hedge ด้วย β ตัวเดียว ค้างไว้ตลอดปี คุณจะ hedge ผิดสัดส่วนเป็นระยะ ๆ

window เลือกอย่างไร?

- สั้นเกินไป (เช่น 10 วัน): β volatile มาก, noise สูง $\rightarrow$ signal ผิดพลาดบ่อย

- ยาวเกินไป (เช่น 500 วัน): β ตอบสนองช้ามาก → พลาด regime change
- ดี: window $\approx 3-5$ เท่าของ half-life ของ spread — เรียนวิธีหา half-life ในบทที่ 14

ตัวอย่าง: สร้าง Z-score จาก OLS Residuals

```
def build_spread(price_a, price_b, window=60):
    """
    สร้าง spread และ z-score จาก OLS rolling regression
    ทำ 3 ขั้นตอน: (1) หา beta → (2) สร้าง spread → (3) คำนวณ z-score
    Returns: DataFrame ของ spread, beta, zscore
    """
    log_a = np.log(price_a)          # แปลงเป็น log price ทั้งคู่
    log_b = np.log(price_b)

    # — ขั้น 1: หา hedge ratio (beta) ด้วย rolling OLS —
    X = sm.add_constant(log_b)       # เพิ่มคอลัมน์ค่าคงที่ (intercept) ให้ regression
    roll = RollingOLS(log_a, X, window=window).fit()
    beta = roll.params.iloc[:, 1]     # คอลัมน์ 0 คือ const (จาก add_constant), คอลัมน์ 1 คือ beta
    alpha = roll.params['const']      # intercept

    # — ขั้น 2: spread = ส่วนที่ "เหลือ" หลังหักความสัมพันธ์ออก —
    predicted = alpha + beta * log_b  # ราคา A ที่ "ควรจะเป็น" ตามราคา B
    spread = log_a - predicted        # ของจริง - ที่ควรเป็น = ความเพี้ยน

    # — ขั้น 3: z-score ของ spread (สูตรเดิมจาก Part I) —
    roll_mean = spread.rolling(window).mean()
    roll_std = spread.rolling(window).std()
    zscore = (spread - roll_mean) / roll_std

    return pd.DataFrame({
        "spread": spread,
        "beta": beta,
        "zscore": zscore
    })

df_pair = build_spread(
    pd.Series(btc_bybit),
    pd.Series(btc_binance),
    window=60
)
print(df_pair.tail())
print(f"\nCurrent Z-score: {df_pair['zscore'].iloc[-1]:.2f}")
```

► OUTPUT

	spread	beta	zscore
247	-0.000126	0.933411	-0.497962
248	0.000514	0.933770	0.348072
249	0.000118	0.932216	-0.160151
250	-0.000725	0.929067	-1.226366

251 -0.000047 0.929990 -0.320480

Current Z-score: -0.32

🔗**อ่านว่า:** แกนของฟังก์ชันนี้อยู่ที่สองบรรทัด: `predicted = alpha + beta * log_b` แล้ว `spread = log_a - predicted` — อ่านว่า "ราคา A ที่ควรจะเป็นตามราคา B" แล้วเอา "ของจริง ลบ ที่ควรเป็น". ส่วนที่เหลือคือความเพี้ยน และความเพี้ยนนี้แหละคือสิ่งที่เราเทรด ไม่ใช่ตัวราคา

⚠️ สังเกตสองบรรทัดที่หิบบค่าคนละแบบ — ไม่ใช่ความมั่งง่าย

ในฟังก์ชันเดียวกันมีทั้ง `roll.params['const']` (ใช้ชื่อ) และ `roll.params.iloc[:, 1]` (ใช้ตำแหน่ง) — เพราะ `sm.add_constant()` ตั้งชื่อให้เฉพาะคอลัมน์ที่มันเติมเข้าไปเองคือ `const`. ส่วนคอลัมน์ข้อมูลเดิมจะได้ชื่อก็ต่อเมื่อ Series ที่ส่งเข้ามามีชื่ออยู่แล้ว — ที่นี้ส่ง `pd.Series(btc_binance)` ซึ่งไม่มีชื่อ จึงต้องอ้างด้วยตำแหน่ง

🔗**อยากให้เรียกด้วยชื่อได้ทั้งคู่** ให้ตั้งชื่อ Series ตั้งแต่ต้น: `pd.Series(btc_binance, name="log_b")` — หลักการเดียวกับที่เจอในบทที่ 13

► แบบฝึกหัด: interpret OLS output

บทที่ 14: Cointegration & Z-score Signal

แก่นของบทนี้

Correlation บอกว่าสองสิ่งเดินไปทิศเดียวกัน — Cointegration บอกว่าสองสิ่ง *ผูกกันระยะยาว* แม้ระยะสั้นจะแยกออกจากกันได้ Cointegration = ข้อพิสูจน์ว่า spread มีแรงดึงดูดกลับจริง = basis ของ Statistical Arbitrage ทั้งหมด

14.1 Correlation vs Cointegration

	Correlation สูง	Cointegrated
หมายถึง	เดินทิศเดียวกันในระยะสั้น	ผูกกันในระยะยาว มี "แรงดึง"
Spread	อาจ drift ออกไปเรื่อยๆ	กลับหาค่ากลางเสมอ
ตัวอย่าง	หุ้นสอง sector เดียวกัน	BTC สองตลาด, ADR vs local
Trade ได้ไหม?	เสี่ยง — spread อาจไม่กลับ	ใช่ — spread ต้องกลับ (ในทฤษฎี)

14.2 Engle-Granger Test — 2 ขั้นตอน

วิธี Engle-Granger: (1) OLS หา residuals → (2) ADF test บน residuals

```
from statsmodels.tsa.stattools import adfuller, coint

# วิธีที่ 1: Engle-Granger ด้วย coint() โดยตรง
score, pvalue, crit_vals = coint(np.log(btc_bybit), np.log(btc_binance))

print(f"Cointegration test:")
print(f"  Test statistic: {score:.4f}")
print(f"  p-value:       {pvalue:.4f}")
print(f"  Critical (5%): {crit_vals[1]:.4f}")

if pvalue < 0.05:
    print(f"✓ COINTEGRATED — spread มีแรงดึงดูดกลับจริง")
else:
    print(f"✗ NOT COINTEGRATED — อย่า trade pair นี้!")

# วิธีที่ 2: ADF บน residuals (เข้าใจกระบวนการมากขึ้น)
log_a = np.log(btc_bybit)
```



```
log_b = np.log(btc_binance)
X = sm.add_constant(log_b)
residuals = sm.OLS(log_a, X).fit().resid
adf_resid = adfuller(residuals)
print(f"\nADF on residuals p-value: {adf_resid[1]:.4f}")
```

► OUTPUT

Cointegration test:

Test statistic: -4.7762

p-value: 0.0004

Critical (5%): -3.3606

✓ COINTEGRATED — spread มีแรงดึงกลับจริง

ADF on residuals p-value: 0.0001

📌 **อ่านว่า:** p-value = 0.0004 แล้วสรุปว่า "cointegrated" — ทิศทางนี้กลับหัวจากที่คนส่วนใหญ่คุ้น · เพราะสมมติฐานตั้งต้น (null) ของ `coint()` คือ "คู่นี้ *ไม่* cointegrate" · p-value เล็ก = หลักฐานแรงพอจะล้มสมมติฐานนั้น = คู่นี้ cointegrate จริง · จำสั้น ๆ: **p เล็ก = ขาวดี**

❌ **ข้อผิดพลาดที่พบบ่อยที่สุดในเรื่องนี้ — อ่าน p-value กลับด้าน**

ถ้าแปลอ่านแบบ " $p > 0.05 = \text{ผ่าน}$ " เหมือนการทดสอบทั่วไป คุณจะเลือกคู่นี้ *ไม่* cointegrate มาเทรดทั้งหมด และทิ้งคู่ที่ดีที่สุด — ผลคือพอร์ตที่ spread ไม่มีแรงดึงกลับ ซึ่งคือการเดาทิศทางสั้น ๆ โดยที่คุณคิดว่ากำลังทำ market-neutral อยู่

สาเหตุที่พลาด: การทดสอบตระกูลนี้ (ADF, coint, KPSS บางตัว) ตั้ง null เป็น "สิ่งที่ *ไม่*อยากได้" — ต่างจาก t-test ที่ null คือ "ไม่มีผล" · **เปิดคู่มือดู null ทุกครั้งก่อนตีความ** อย่าเดาจากความเคยชิน

14.3 Johansen Test — สำหรับ 3+ Pairs

```
from statsmodels.tsa.vector_ar.vecm import coint_johansen

# Johansen ทดสอบ cointegration ระหว่าง 3 series พร้อมกัน
btc_okx = 65000 + np.cumsum(np.random.normal(0, 100, n)) # ตลาดที่ 3

data = np.column_stack([np.log(btc_bybit), np.log(btc_binance), np.log(btc_okx)])
result = coint_johansen(data, det_order=0, k_ar_diff=1)

print("Johansen Trace Statistics:")
for i in range(3):
    trace_stat = result.lr1[i]
    crit_5pct = result.cvt[i, 1]
    print(f" r≤{i}: trace={trace_stat:.2f}, crit(5%)={crit_5pct:.2f} "
          f"{'✓' if trace_stat > crit_5pct else 'x'}")
```

```
# r=0: ทดสอบว่ามีอย่างน้อย 1 cointegrating vector
# r≤1: ทดสอบว่ามีอย่างน้อย 2
```

► OUTPUT

Johansen Trace Statistics:

```
r≤0: trace=35.43, crit(5%)=29.80 ✓
r≤1: trace=10.34, crit(5%)=15.49 ✗
r≤2: trace=0.79, crit(5%)=3.84 ✗
```

 อ่านผล Johansen — มันคือ "บันได" ไม่ใช่คำตอบเดียว

```
r≤0: trace=35.43, crit=29.80 ✓ ← ล้มได้
r≤1: trace=10.34, crit=15.49 ✗ ← ล้มไม่ได้ → หยุดตรงนี้
r≤2: trace= 0.79, crit= 3.84 ✗
```

วิธีอ่าน: ไล่จากบนลงล่าง หยุดที่แถวแรกที่ล้มไม่ได้ แล้วเลขแถวนั้นคือคำตอบ

1. $r \leq 0$ ถามว่า "มี cointegrating vector **0 อัน** ใช่ไหม" · trace 35.43 > 29.80 → **ล้มข้อนี้ได้** แปลว่ามีอย่างน้อย 1 อัน → ไต่ขึ้นแถวถัดไป
2. $r \leq 1$ ถามว่า "มีไม่เกิน **1 อัน** ใช่ไหม" · trace 10.34 < 15.49 → **ล้มไม่ได้** → ยอมรับว่าจริง
3. สรุป: **มี cointegrating vector 1 อันพอดี** — คือ 3 ตลาดนี้มีความสัมพันธ์ระยะยาวชุดเดียว (ตามที่ควรเป็น เพราะเราสร้าง Bybit กับ Binance ให้เกาะกัน ส่วน OKX เป็นเส้นอิสระ)

 ✓ ในผลลัพธ์แปลว่า "ล้ม null ได้" ไม่ได้แปลว่า "ดี" — ✓ ที่แถวสุดท้ายทุกแถวจะแปลว่าไม่มีความสัมพันธ์ระยะยาวเลย

🔵 [รอบสอง] ทำไมต้อง Johansen ทั้งที่มี Engle-Granger แล้ว

Engle-Granger ต้องเลือกก่อนว่าใครเป็นตัวแปรตาม (`coint(a, b)` กับ `coint(b, a)` ให้ p-value ไม่เท่ากัน!) — พอมี 3 สินทรัพย์ขึ้นไป การเลือกนี้กลายเป็นการตัดสินใจตามอำเภอใจที่กระทบผลลัพธ์ · Johansen ทดสอบทุกทิศพร้อมกันโดยไม่ต้องเลือก และบอกได้ด้วยว่ามีความสัมพันธ์กี่ชุดไม่ใช่แค่ "มี/ไม่มี"

ในทางปฏิบัติ: 2 ขา → Engle-Granger พอ (อ่านง่ายกว่า) · 3 ขาขึ้นไป (เช่น basket ข้ามตลาด, curve trade) → Johansen เท่านั้น · และ `k_ar_diff` ที่ตั้งเป็น 1 นั้นควรเลือกด้วย information criterion จริง ๆ ไม่ใช่ค่าคงที่ — ตั้งผิดทำให้ผลพลิกได้

14.4 Half-Life — Spread กลับเร็วแค่ไหน?

Half-life คือเวลาที่ spread ใช้กลับมากครึ่งทางจากค่าสุดขีด — ใช้เลือก window และ holding period

$$\text{Half-life} = \frac{-\ln(2)}{\ln(\hat{\phi})} \approx \frac{-\ln(2)}{\hat{\phi} - 1}$$

ซ้าย = สูตรแม่นยำ (โค้ดด้านล่างใช้สูตรนี้) · ขวา = ค่าประมาณ (ใช้ได้เมื่อ $\hat{\phi}$ ใกล้ 1)

```
def calc_half-life(spread):
    """คำนวณ half-life ของ mean-reverting spread"""
    spread_lag = pd.Series(spread).shift(1)
    delta      = pd.Series(spread) - spread_lag
    df_hl = pd.DataFrame({"delta": delta, "lag": spread_lag}).dropna()

    X = sm.add_constant(df_hl["lag"])
    phi = sm.OLS(df_hl["delta"], X).fit().params["lag"]

    if phi >= 0:
        return np.inf # ไม่ mean-reverting (random walk หรือ explosive)
    if phi <= -1:
        return np.inf # Oscillating — ไม่ใช่ mean-reverting จริง
    half-life = -np.log(2) / np.log(1 + phi) # สูตรแม่นยำ: phi จาก regression = φ-1 ดังนั้น 1
    return half-life

hl = calc_half-life(resid)
print(f"Half-life: {hl:.1f} วัน")
print(f"แนะนำ: rolling window ≈ {hl*3:.0f} วัน, hold ไม่เกิน {hl*2:.0f} วัน")
```

► OUTPUT

Half-life: 3.8 วัน

แนะนำ: rolling window ≈ 11 วัน, hold ไม่เกิน 8 วัน

🔴**อ่านว่า:** อ่านผลนี้ว่า: "spread ใช้เวลา **3.8 วัน** เพื่อเดินทางกลับมาจากจุดที่เหวี่ยงออกไป" — ตัวเลขนี้ตั้งค่าให้คุณสองอย่างทันที: **window** ต้องยาวพอเห็นรอบการดึงกลับ (≈3 เท่าของ half-life) และ **holding period** ต้องไม่ยาวเกินจนถือค้างหลังกำไรวิ่งมาแล้ว (≈2 เท่า) · ⚠️ ถ้าได้ half-life สั้นมาก (<1 วัน) แปลว่า spread เป็น noise ไม่ใช่ mean-reversion จริง — ค่าคำแนะนำจะกลายเป็น 0 วันซึ่งเทรดไม่ได้ นั่นคือสัญญาณว่า pair นี้ไม่ควรเทรด ไม่ใช่ว่าโค้ดพัง

ตัวอย่าง: Signal Generation Pipeline ครบวงจร

```
def pair_signal(price_a, price_b, window=60,
               entry_z=2.0, exit_z=0.5):
    """
    สร้าง trading signal สำหรับ pair trading
    Returns: DataFrame พร้อม signal column
    """
    df = build_spread(price_a, price_b, window)
    z = df["zscore"]

    signals = pd.Series("HOLD", index=z.index)
    signals[z > entry_z] = "SHORT_A_LONG_B" # spread แพงเกินไป
    signals[z < -entry_z] = "LONG_A_SHORT_B" # spread ถูกเกินไป
    signals[abs(z) < exit_z] = "EXIT"

    df["signal"] = signals
    # △ สำคัญมาก: signal ที่ได้จาก df.index[t] ใช้ข้อมูลถึงวัน t
    # ต้อง .shift(1) ใน backtest ก่อน multiply ด้วย return เสมอ
    return df

result = pair_signal(
    pd.Series(btc_bybit),
    pd.Series(btc_binance),
    window=60, entry_z=2.0, exit_z=0.5
)
print(result[["spread", "beta", "zscore", "signal"]].tail(10))
```

บทที่ 15: Optimization

แก่นของบทนี้

Optimization คือการหาค่าตัวแปร (เช่น portfolio weights, position size) ที่ทำให้ objective function (เช่น Sharpe ratio) ดีที่สุด ภายใต้ constraints (เช่น weights รวม = 1) Python ทำได้ด้วย `scipy.optimize.minimize`

$$\max_{\mathbf{w}} \text{Sharpe}(\mathbf{w}) = \frac{\mathbf{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}} \quad \text{s.t.} \quad \sum w_i = 1, w_i \geq 0$$

15.1 scipy.optimize.minimize — API

```
from scipy.optimize import minimize
import numpy as np

# สมมติว่าต้องการหา x ที่ทำให้ f(x) = (x-3)^2 น้อยที่สุด
def f(x):
    return (x[0] - 3)**2

result = minimize(f, x0=[0], method="SLSQP")
print(f"Optimal x: {result.x[0]:.4f} (คาดหวัง: 3.0)")
print(f"Min f(x): {result.fun:.6f}")
print(f"Success: {result.success}")
```

► OUTPUT

```
Optimal x: 3.0000 (คาดหวัง: 3.0)
Min f(x): 0.000000
Success: True
```

📖 **อ่านว่า:** อ่าน `minimize(f, x0=[0])` ว่า: "หาค่า x ที่ทำให้ f น้อยที่สุด โดยเริ่มเดาที่ x=0" — สิ่งที่ต้องเข้าใจคือ optimizer **ไม่รู้จักรัสตรของคุณ** มันแค่ลองค่า ดูว่าผลออกมาเท่าไร แล้วขยับไปทางที่ผลลดลง ทำซ้ำจนขยับแล้วไม่ดีขึ้น. `x0` จึงสำคัญ — จุดเริ่มต่างกันอาจได้คำตอบต่างกันถ้าฟังก์ชันมีหลุมหลายหลุม

⚠ **result.success** ต้องเช็คทุกครั้ง — optimizer โกหกแบบเงิบ ๆ ได้

`minimize()` คืนค่ากลับมาเสมอไม่ว่าจะหาเจอหรือไม่·ถ้ามันไม่converge ค่าใน `result.x` จะเป็นตำแหน่งที่มันยอมแพ้ไม่ใช่คำตอบ — และถ้าคุณไม่เช็ค `result.success` คุณจะเอา "น้ำหนักพอร์ตที่ไม่ใช่คำตอบ" ไปลงเงินจริง โดยไม่มีerrorใดๆเตือน

15.2 Sharpe Ratio Maximization

```
def sharpe_maximize(returns_df, risk_free=0.0, periods=252):
    """
    หา portfolio weights ที่ maximize Sharpe ratio
    returns_df: DataFrame ของ daily returns (cols = assets)
    """
    n = len(returns_df.columns)
    mu = returns_df.mean().values * periods          # annualized mean
    sigma = returns_df.cov().values * periods         # annualized cov

    def neg_sharpe(weights):
        """คืนค่า negative Sharpe (minimize = maximize Sharpe)"""
        port_ret = weights @ mu
        port_var = weights @ sigma @ weights
        return -(port_ret - risk_free) / np.sqrt(port_var)

    # Constraints
    constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}] # sum = 1
    bounds = [(0, 1)] * n # no short
    x0 = np.ones(n) / n # equally weighted

    result = minimize(neg_sharpe, x0,
                      method="SLSQP",
                      bounds=bounds,
                      constraints=constraints)

    if not result.success:
        print("Warning: optimizer did not converge")

    weights = result.x
    port_ret = weights @ mu
    port_vol = np.sqrt(weights @ sigma @ weights)

    print("=== Max Sharpe Portfolio ===")
    for name, w in zip(returns_df.columns, weights):
        print(f" {name}: {w:.1%}")
    print(f" Expected Return: {port_ret:.1%}/yr")
    print(f" Volatility: {port_vol:.1%}/yr")
    print(f" Sharpe Ratio: {(port_ret-risk_free)/port_vol:.2f}")
    return weights
```

```
# ทดสอบ
np.random.seed(0)
rets = pd.DataFrame(np.random.multivariate_normal(
    [0.001, 0.0007, 0.0012],
    [[0.0004, 0.0002, 0.0001],
     [0.0002, 0.0003, 0.0001],
     [0.0001, 0.0001, 0.0005]], 252),
    columns=["BTC", "ETH", "SOL"])

optimal_w = sharpe_maximize(rets)
```

🧑‍🎓 ฝึกอ่าน 2 ท่อนที่ดูสลับที่สุดในโค้ดข้างบน

```
# ท่อนที่ 1
port_ret = weights @ mu
# ท่อนที่ 2
constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}]
```

ท่อนที่ 1: เครื่องหมาย @ คือ "คูณแบบเวกเตอร์/เมทริกซ์" — `weights @ mu` อ่านว่า "เอาน้ำหนักแต่ละตัว คูณ return คาดหวังของ asset นั้น แล้วบวกรวมกัน" เท่ากับเขียน loop เองว่า:

```
port_ret = 0
for w, m in zip(weights, mu):
    port_ret = port_ret + w * m
```

ท่อนที่ 2: อ่านจากในออกนอก:

1. `lambda w: np.sum(w) - 1` — ฟังก์ชันจิ๋ว: "รับ w มา แล้วคืนค่า ผลรวมของ w ลบ 1"
2. `"type": "eq"` — บอก optimizer ว่าค่าที่ฟังก์ชันนี้คืน **ต้องเท่ากับ 0** → แปลว่า $\text{sum}(w) - 1 = 0 \rightarrow \text{sum}(w) = 1$ = "ลงทุนเต็มพอดิไม่ขาดไม่เกิน"
3. `[ { ... } ]` — เป็น list ของ dict เพราะใส่ได้หลายเงื่อนไข (scipy กำหนดรูปแบบนี้)

นี่คือ "ภาษาเฉพาะ" ของ scipy — เจอครั้งแรกทุกคนงง พออ่านออกแล้วจะเห็นว่ามันแค่บอกว่า *"เงื่อนไข: น้ำหนักรวมกัน ต้องเป็น 1"*

15.3 Kelly Criterion — Optimal Position Size

$$f^* = \frac{\mu - r_f}{\sigma^2} \quad (\text{continuous Kelly})$$

```
def kelly_fraction(mean_ret, std_ret, risk_free=0.0, fraction=0.25):
    """
    คำนวณ Kelly fraction สำหรับ continuous distribution
    fraction: ใช้ 0.25 (Quarter-Kelly) เพื่อความปลอดภัย
```

```

"""
kelly_full = (mean_ret - risk_free) / (std_ret ** 2)
kelly_frac = kelly_full * fraction

print(f"Full Kelly:      {kelly_full:.2f}x")
print(f"Quarter Kelly: {kelly_frac:.2f}x ← แนะนำ")
return kelly_frac

# ตัวอย่าง: กลยุทธ์ที่มี Sharpe = 1.5
daily_mean = 0.001      # 0.1% ต่อวัน
daily_std  = 0.015      # 1.5% ต่อวัน
f = kelly_fraction(daily_mean, daily_std)

```

Kelly ที่ใช้ที่นี้ = Continuous Kelly (สำหรับ return กระจาย Normal)

สูตร $f^* = \frac{\mu - r_f}{\sigma^2}$ เหมาะกับ asset ที่ return กระจายแบบ Normal distribution ต่อเนื่อง
 สำหรับ **discrete bet** เช่น Binary outcome (win ได้ b เท่า, lose 1): $f^* = \frac{pb - (1-p)}{b}$

ใน Quant Trading ส่วนใหญ่ใช้ continuous Kelly หรือบ่อยกว่านั้นใช้ **Half Kelly ($f^*/2$)** หรือ **Quarter Kelly ($f^*/4$)** เพราะ:

- Return จริงไม่ Normal (fat tail)
- Estimate μ และ σ มี error เสมอ
- Full Kelly มี drawdown รุนแรงมากในช่วง streak ของ losing trades

อย่าใช้ Full Kelly ในทางปฏิบัติ

Full Kelly maximize geometric growth ในทฤษฎี แต่ในทางปฏิบัติ:

- ประมาณ parameters จากข้อมูลมีความผิดพลาด → Full Kelly อาจ oversize มาก
- Drawdown จาก Full Kelly ใหญ่มาก (50%+ เป็นปกติ) → ทนไม่ไหว

ใช้ **Quarter-Kelly ($f^*/4$)** เป็นจุดเริ่มต้น — Part IV จะพาไปลึกขึ้นในเรื่องนี้

ตัวอย่าง: Mean-Variance Optimization (Efficient Frontier)

```
def efficient_frontier(returns_df, n_points=100):  
    """  
    สร้าง efficient frontier:  
    สำหรับ return เป้าหมายแต่ละระดับ → หาพอร์ตที่ vol ต่ำสุด  
    """  
    n = len(returns_df.columns)  
    mu = returns_df.mean().values * 252 # expected return รายปี  
    sigma = returns_df.cov().values * 252 # covariance รายปี  
  
    # ไล่ return เป้าหมาย จากต่ำสุดถึงสูงสุด เป็น n_points จุด  
    target_returns = np.linspace(mu.min(), mu.max(), n_points)  
  
    def portfolio_variance(w):  
        """สิ่งที่ต้องการ minimize: ความแปรปรวนของพอร์ต"""  
        return w @ sigma @ w  
  
    frontier_vols = [] # เก็บ vol ต่ำสุดของแต่ละเป้า  
    for target in target_returns:  
        # เงื่อนไข 1: น้ำหนักรวม = 1 (ลงทุนเต็มพอดี)  
        sum_to_one = {"type": "eq", "fun": lambda w: np.sum(w) - 1}  
  
        # เงื่อนไข 2: return ของพอร์ต = target ของรอบนี้  
        # (t=target จำเป็น! ดูล่องกับดักด้านล่าง)  
        hit_target = {"type": "eq", "fun": lambda w, t=target: w @ mu - t}  
  
        result = minimize(portfolio_variance,  
                           x0=np.ones(n) / n, # เริ่มจากน้ำหนักเท่ากันทุกตัว  
                           method="SLSQP",  
                           bounds=[(0, 1)] * n, # ห้าม short ( $0 \leq w \leq 1$ )  
                           constraints=[sum_to_one, hit_target])  
  
        if result.success:  
            min_vol = np.sqrt(result.fun) # ได้ variance ต่ำสุด → ถอดรากเป็น vol  
        else:  
            min_vol = np.nan # แก้มิได้ (เป้าสูง/ต่ำเกิน) → เว้นจุดนี้  
        frontier_vols.append(min_vol)  
  
    return target_returns, np.array(frontier_vols)  
  
rets_arr, vols_arr = efficient_frontier(rets)  
  
import matplotlib.pyplot as plt  
fig, ax = plt.subplots(figsize=(8, 5))  
ax.plot(vols_arr, rets_arr, color="#0d9488", linewidth=2, label="Efficient Frontier")  
ax.set_xlabel("Annual Volatility")
```

```
ax.set_ylabel("Annual Return")
ax.set_title("Efficient Frontier (BTC, ETH, SOL)")
ax.legend()
plt.tight_layout()
plt.show()
```

⚠ กับดัก lambda ใน loop — ทำไมต้องเขียน `lambda w, t=target`

ถ้าเขียน `lambda w: w @ mu - target` เฉย ๆ — lambda จะ "จำชื่อตัวแปร" ไม่ใช่ "จำค่า" พอ loop จบ `target` ชี้ไปที่ค่าสุดท้ายแล้ว ทุก lambda ที่สร้างไว้จะใช้ค่าเดียวกันหมด → frontier ผิดทั้งเส้นแบบเงียบ ๆ ไม่มี error

ทำ `t=target` คือการ "ถ่ายสำเนาค่า ณ รอบนั้น" เก็บไว้ใน `t` ของ lambda ตัวนั้นเอง — นี่เป็นกับดัก Python คลาสสิกที่แม้แต่ AI ยังเขียนพลาดเป็นครั้งคราว เจอ `lambda` ใน `loop` เมื่อไหร่ ให้มองหา = แบบนี้เสมอ

► แบบฝึกหัด: Minimize Portfolio Variance (Min-Vol Portfolio)

บทที่ 16: Linear Programming

แก่นของบทนี้

Linear Programming (LP) คือ optimization ที่ทั้ง objective function และ constraints เป็น linear — เร็วกว่า nonlinear optimization มาก ใช้ใน Quant สำหรับ: กำหนด portfolio weight ภายใต้หลาย constraint พร้อมกัน, minimize transaction cost, sizing หลาย position พร้อมกัน

$$\min_{\mathbf{w}} \mathbf{c}^\top \mathbf{w} \quad \text{s.t. } \mathbf{A}\mathbf{w} \leq \mathbf{b}, \mathbf{A}_{eq}\mathbf{w} = \mathbf{b}_{eq}, \mathbf{lb} \leq \mathbf{w} \leq \mathbf{ub}$$

16.1 scipy.linprog — API

```
from scipy.optimize import linprog

# ตัวอย่างง่าย: minimize cost ของการซื้อ 3 assets
# minimize: 2*w1 + 3*w2 + 1.5*w3
# subject to: w1+w2+w3 = 1 (fully invested)
#             w1 >= 0.1 (min 10% BTC)
#             w2 >= 0.1 (min 10% ETH)

c = [2.0, 3.0, 1.5]                # objective: cost coefficients
A_eq = [[1, 1, 1]]                # equality: sum = 1
b_eq = [1]
bounds = [(0.1, 0.6), (0.1, 0.5), (0, 0.8)] # min/max ของแต่ละ w

result = linprog(c, A_eq=A_eq, b_eq=b_eq,
                 bounds=bounds, method="highs")

print(f"Optimal weights: {result.x.round(3)}")
print(f"Minimum cost: {result.fun:.4f}")
print(f"Success: {result.success}")
```

16.2 Portfolio LP — ใส่ Constraints หลายอย่างพร้อมกัน

```
from scipy.optimize import linprog
import numpy as np

def lp_portfolio(expected_returns, min_return=0.10,
                 max_weights=None, min_weights=None,
```

```

        max_sector=None, sectors=None):
    """
    Minimize portfolio variance (approximated as linear)
    ด้วย LP – เหมาะสำหรับ constraint-heavy problem

    expected_returns: array ของ expected annual return แต่ละ asset
    min_return: minimum portfolio return ที่ต้องการ
    """
    n = len(expected_returns)

    # Objective: minimize negative return (= maximize return)
    # Linear approximation – ใช้สำหรับ constraint-heavy problems
    c = -np.array(expected_returns)

    # Equality: sum(w) = 1
    A_eq = [np.ones(n)]
    b_eq = [1.0]

    # Inequality: min return constraint
    # w @ returns >= min_return → -w @ returns <= -min_return
    A_ub = [-np.array(expected_returns)]
    b_ub = [-min_return]

    # Bounds: default 0-1 (long only)
    if min_weights is None: min_weights = [0.0] * n
    if max_weights is None: max_weights = [1.0] * n
    bounds = list(zip(min_weights, max_weights))

    result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                     A_eq=A_eq, b_eq=b_eq,
                     bounds=bounds, method="highs")

    return result

# ตัวอย่าง: 5 assets
assets = ["BTC", "ETH", "SOL", "AAPL", "GOLD"]
returns = [0.35, 0.25, 0.40, 0.12, 0.08]

result = lp_portfolio(
    returns,
    min_return = 0.20,
    min_weights = [0.05]*5,      # min 5% ทุก asset
    max_weights = [0.40]*5,      # max 40% ทุก asset
)

print("LP Portfolio Allocation:")
for asset, w in zip(assets, result.x):

```

```
print(f" {asset:5s}: {w:.1%}")
print(f"Expected Return: {np.dot(result.x, returns):.1%}")
```

16.3 PuLP — สำหรับ Integer Programming

```
# pip install pulp
from pulp import *

# Integer LP: เลือก k หุ้นที่ดีที่สุดจาก n ตัว
scores = {"BTC": 0.85, "ETH": 0.70, "SOL": 0.60,
          "AAPL": 0.55, "GOLD": 0.40}
costs = {"BTC": 5000, "ETH": 2000, "SOL": 500,
         "AAPL": 1000, "GOLD": 800}
budget = 8000 # งบ 8000 บาท

prob = LpProblem("SelectAssets", LpMaximize)

# ตัวแปร binary: 1=เลือก, 0=ไม่เลือก
x = {a: LpVariable(f"x_{a}", cat="Binary") for a in scores}

# Objective: maximize total score
prob += lpSum(scores[a] * x[a] for a in scores)

# Constraints
prob += lpSum(costs[a] * x[a] for a in scores) <= budget # budget
prob += lpSum(x[a] for a in scores) <= 3 # เลือกได้ max 3 ตัว

prob.solve(PULP_CBC_CMD(msg=0))

print("Selected assets:")
for a in scores:
    if x[a].value() == 1:
        print(f" {a}: score={scores[a]}, cost={costs[a]}")
```

ตัวอย่าง: Position Sizing LP สำหรับ Pair Trading

```
# กำหนด: มี 3 pairs พร้อม trade, capital จำกัด
# objective: maximize expected PnL
# constraints: total capital, max drawdown per pair, correlation limit

pairs      = ["BTC-Bybit/Binance", "ETH-OKX/Bybit", "SOL-Binance/OKX"]
exp_pnl    = [0.15, 0.12, 0.20] # expected annual return
max_dd     = [0.05, 0.08, 0.10] # max expected drawdown
capital    = 1_000_000           # total capital บาท

# Minimize negative PnL = maximize PnL
c          = [-e for e in exp_pnl] # negate for minimization

# sum(allocation) = 1.0 (fully allocated)
A_eq = [[1, 1, 1]]
b_eq = [1.0]

# เงื่อนไข: drawdown ที่คาดหวังของแต่ละ pair ≤ 5% ของ capital
# allocation_i × max_dd_i ≤ 0.05 — แยกกันทีละ pair = matrix แนวทาง
A_ub = np.diag(max_dd) # [[0.05, 0, 0],
                        # [0, 0.08, 0],
                        # [0, 0, 0.10]]
b_ub = [0.05] * 3

bounds = [(0.0, 0.5)] * 3 # max 50% ต่อ pair

result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                 A_eq=A_eq, b_eq=b_eq,
                 bounds=bounds, method="highs")

print("Optimal Position Sizing:")
for pair, alloc in zip(pairs, result.x):
    print(f" {pair}: {alloc:.1%} = ฿{alloc*capital:.0f}")
print(f"Expected Portfolio Return: {-result.fun:.1%}")
```

AI มักสร้าง matrix แนวทางด้วย nested comprehension — ฟังก์ชัน

```
A_ub = [[max_dd[i] if j==i else 0 for j in range(3)] for i in range(3)]
```

comprehension ซ้อนสองชั้น — อ่านจาก **for** ขวาสุดก่อน: `for i` สร้างทีละแถว แล้วค่อยอ่าน `for j` ที่อยู่ถัดมาทางซ้าย เพื่อสร้างทีละช่องในแถวนั้น:

1. `for i in range(3)` (ชั้นนอก, ขวาสุด) — สร้าง 3 แถว แถวละหนึ่ง `i`
2. `for j in range(3)` (ชั้นใน) — แต่ละแถวมี 3 ช่อง ช่องละหนึ่ง `j`
3. `max_dd[i] if j==i else 0` — แต่ละช่อง: "เอาค่า `max_dd` ถ้าอยู่บนเส้นทแยง (แถว=คอลัมน์) **ไม่**งั้นใส่ 0"

ผลคือ matrix เดียวกับ `np.diag(max_dd)` เป๊ะ — เวอร์ชัน `np.diag` สั้นกว่า อ่านง่ายกว่า และผิดยากกว่า ถ้า AI เขียนแบบ nested มา ขอให้มันแก้เป็น `np.diag` ได้เลย

► แบบฝึกหัด: LP Constraint สำหรับ Sector Limits

บทที่ 17: Signals & Filters

แก่นของบทนี้

Signal คือ "เมื่อไหร่ควร trade" — บทนี้รวม 3 เครื่องมือ: **Bollinger Bands** (signal จาก volatility), **Kalman Filter** (track β แบบ adaptive), และ **Signal Pipeline** (รวมหลาย signal เข้าด้วยกัน) ซึ่งเป็น boilerplate ที่ใช้ได้กับทุก strategy

17.1 Bollinger Bands — Signal จาก Volatility

```
def bollinger_bands(series, window=20, n_std=2):
    """คำนวณ Bollinger Bands"""
    s = pd.Series(series)
    mean = s.rolling(window).mean()
    std = s.rolling(window).std()
    upper = mean + n_std * std
    lower = mean - n_std * std
    return pd.DataFrame({"mid": mean, "upper": upper, "lower": lower,
                        "pct_b": (s - lower) / (upper - lower)})

# pct_b: 0 = ที่ lower band, 1 = ที่ upper band
# signal: SHORT เมื่อ pct_b > 1, LONG เมื่อ pct_b < 0
bb = bollinger_bands(spread, window=20, n_std=2)
signal = pd.Series("HOLD", index=bb.index)
signal[bb["pct_b"] > 1.0] = "SHORT"
signal[bb["pct_b"] < 0.0] = "LONG"
```

17.2 Kalman Filter — Track β แบบ Adaptive

Kalman Filter ประมาณ β แบบ real-time โดยอัปเดตทุกครั้งที่มีข้อมูลใหม่ — ดีกว่า rolling OLS ตรงที่ไม่มี window cutoff และ smooth กว่า

Kalman Filter ทำงานสองจังหวะ — "เดา แล้วแก้เดา" ซ้ำไปเรื่อย ๆ

ลองนึกถึงการเดาน้ำหนักเพื่อนทุกครั้งที่เราเจอ:

จังหวะ 1 — เดา (Predict): "วันนี้น่าจะหนักเท่าเดิมจากครั้งก่อน แต่เวลาผ่านไปแล้ว ความมั่นใจของเราลดลงนิดหน่อย"

จังหวะ 2 — แก้เดา (Update): เห็นข้อมูลใหม่ (เพื่อนตัวจริง) → เทียบกับที่เดา → ถ้าต่างกัน ขยับค่าเดาเข้าหาความจริง ตามสัดส่วนความมั่นใจ — มั่นใจค่าเดาเดิมมากก็ขยับนิดเดียว ไม่มั่นใจก็ขยับเยอะ

ตัวเลข "สัดส่วนการขยับ" นี้คือ **Kalman Gain (K)** — หัวใจของทั้ง algorithm มีแค่นี้

```
def kalman_hedge_ratio(price_a, price_b,
                       delta=1e-4, vt=1e-3):
    """
    Track hedge ratio beta ด้วย Kalman Filter
    delta: process noise (ใหญ่ = ยอมให้ beta เปลี่ยนเร็ว)
    vt: observation noise (ใหญ่ = ราคา มี noise มาก เชื่อข้อมูลใหม่น้อยลง)
    Returns: array ของ beta ณ แต่ละเวลา
    """
    n = len(price_a)
    beta = np.zeros(n) # เตรียมที่เก็บ beta ของทุกวัน
    P = np.ones(n) # P = "ความไม่มั่นใจ" ในค่า beta (variance ของค่าเดา)
    beta[0] = 1.0 # วันแรก: เดาไว้ก่อนว่า beta = 1

    for t in range(1, n):
        # — จังหวะ 1: เดา (Predict) —
        # beta วันนี้น่าจะเท่าเมื่อวาน แต่ความไม่มั่นใจเพิ่มขึ้นตาม delta
        P_pred = P[t-1] + delta

        # — จังหวะ 2: แก้เดา (Update) ด้วยข้อมูลใหม่ —
        x = price_b[t] # ราคาตลาด B วันนี้ (ตัวพยากรณ์)

        y_pred = beta[t-1] * x # ราคา A "ที่ควรเป็น" ตามค่าเดาเดิม
        innov = price_a[t] - y_pred # ของจริง - ที่เดา = "เดาพลาดเท่าไร"

        S = x**2 * P_pred + vt # ความแปรปรวนรวมของความพลาด
                                # (จากความไม่มั่นใจใน beta + noise ของราคา)
        K = P_pred * x / S # Kalman Gain: "ควรขยับค่าเดากี่ %"
                                # ไม่มั่นใจมาก (P สูง) → K ใหญ่ → ขยับเยอะ
                                # ราคา noise มาก (vt สูง) → K เล็ก → ขยับนิดเดียว

        beta[t] = beta[t-1] + K * innov # ขยับ beta เข้าหาความจริง ตามสัดส่วน K
        P[t] = (1 - K * x) * P_pred # แก้เดาแล้ว → ความมั่นใจเพิ่มขึ้น (P ลดลง)

    return beta

# ทดสอบ
log_a = np.log(btc_bybit)
log_b = np.log(btc_binance)
kf_beta = kalman_hedge_ratio(log_a, log_b)

print(f"Kalman Beta (latest 5): {kf_beta[-5:].round(6)}")
```

17.3 Signal Pipeline — รวมทุกอย่างเข้าด้วยกัน

```
class PairTradingSignal:
    """
    Signal pipeline สำหรับ Pair Trading
    รวม: OLS hedge ratio, z-score, Bollinger filter
    """
    def __init__(self, window=60, entry_z=2.0, exit_z=0.5):
        self.window = window
        self.entry_z = entry_z
        self.exit_z = exit_z

    def fit(self, price_a: pd.Series, price_b: pd.Series):
        """ประมาณ parameters จากข้อมูลประวัติ"""
        self.price_a = price_a
        self.price_b = price_b

        # OLS spread
        self.df = build_spread(price_a, price_b, self.window)

        # Half-life
        self.halflife = calc_halflife(self.df["spread"].dropna())
        print(f"Half-life: {self.halflife:.1f} วัน")

        # ADF test
        adf_p = adfuller(self.df["spread"].dropna())[1]
        self.is_stationary = adf_p < 0.05
        print(f"Stationary: {self.is_stationary} (p={adf_p:.3f})")
        return self

    def signal(self) -> pd.Series:
        """สร้าง trading signal"""
        z = self.df["zscore"]
        sig = pd.Series("HOLD", index=z.index)
        sig[z > self.entry_z] = "SHORT_A"
        sig[z < -self.entry_z] = "LONG_A"
        sig[abs(z) < self.exit_z] = "EXIT"
        return sig

    def summary(self):
        """แสดง summary ของ current state"""
        current_z = self.df["zscore"].iloc[-1]
        current_b = self.df["beta"].iloc[-1]
        sig = self.signal().iloc[-1]
        print(f"=== Pair Signal Summary ===")
        print(f"Current Z-score: {current_z:.3f}")
        print(f"Current Beta: {current_b:.6f}")
```

```

print(f"Signal:          {sig}")
print(f"Is Stationary:    {self.is_stationary}")

# ใช้งาน
pipeline = PairTradingSignal(window=60, entry_z=2.0, exit_z=0.5)
pipeline.fit(pd.Series(btc_bybit), pd.Series(btc_binance))
pipeline.summary()

```

จบ Part II — สิ่งที่ได้แล้ว

- ทดสอบ stationarity ด้วย ADF test ✓
- ยืนยัน cointegration ด้วย Engle-Granger และ Johansen ✓
- ประมวล β ด้วย OLS และ Kalman Filter ✓
- สร้าง z-score signal สำหรับ entry/exit ✓
- Maximize Sharpe ratio ด้วย `scipy.optimize` ✓
- กำหนด portfolio constraint ด้วย Linear Programming ✓
- รวมทุกอย่างเป็น `PairTradingSignal` class ✓

ใน **Part III** เราจะ refactor code นี้ให้เป็น OOP ที่ maintainable และ extensible

⚠ คำเตือน: ตัวเลขใน Part II คือ In-Sample — อย่าเชื่อ 100%

ยินดีด้วย — มาถึงตรงนี้แสดงว่าคุณสร้าง strategy แรกได้แล้ว แต่มีเรื่องสำคัญหนึ่งอย่างที่ต้องรู้ก่อนใช้เงินจริง: Sharpe 2.0+ และ Return 15%+ ที่เห็นใน Part II ล้วน **คำนวณจากข้อมูลชุดเดียวกับที่ optimize parameter** (in-sample)

ในชีวิตจริง (out-of-sample) ผลลัพธ์มักต่ำกว่า **50–80%** เพราะ:

- ตลาดเปลี่ยน — pattern ในอดีตไม่ซ้ำแบบในอนาคต
- Parameter ที่ดีในอดีต = overfit กับ noise ในอดีต
- Transaction cost ที่ลืมใส่ กินไป 20–50% ของ gross PnL

Part IV จะสอน Walk-Forward Validation — วิธีเดียวที่ยืนยันได้ว่า strategy มี edge จริง ไม่ใช่แค่ overfit อย่าใช้เงินจริงจนกว่าจะผ่านการทดสอบใน Part IV

Object-Oriented Programming — สร้างระบบที่ ขยายได้

บทที่ 18–21: Classes · Inheritance · Design Patterns · OOP Mini-Project

จบ Part นี้ → เปลี่ยน script ที่รันครั้งเดียวเป็น trading system ที่ใช้ได้จริง

🚫 อ่าน Part นี้ยังไ้

● เขียนฟังก์ชันเป็นแล้ว แต่ไม่เคยเขียน class — บทที่ 18.1 เล่าเหตุผลก่อนสอนไวยากรณ์ให้อ่านตรงนั้นให้เข้าใจว่า "ทำไมโค้ดแบบ Part II เริ่มไม่พอ" · แล้วยอมรับไปก่อนว่า self ดูแปลก — พอเขียนไป 2-3 คลาสจะชินเอง · เจอบล็อกโค้ดยาว ๆ ให้ดูตารางหน้าที่ของแต่ละเมธอดก่อนอ่านรายละเอียด

● เขียน OOP ภาษาอื่นมาแล้ว — ไวยากรณ์คุ้นอยู่แล้ว · ของสำหรับคุณคือ ● [รอบสอง] เรื่องกับดักที่ ABC จับไม่ได้ (บังคับได้แค่ "ต้องมีเมธอดชื่อนี้" ไม่รู้ว่าข้างในทำอะไร) และกล่องกับดัก cache กับ @property ที่ทำให้ค่าค้างโดยไม่ error · สองเรื่องนี้คือบักที่เคยอยู่ในโค้ดของเล่มนี้เองจริง ๆ

ทำไมต้องเรียน OOP?

โค้ดจาก Part II ทำงานได้ดีกับ pair เดียว แต่พอเพิ่มกลยุทธ์หรือสินทรัพย์ใหม่จะเริ่มยุ่งยาก: ต้องแก้โค้ดหลายที่พร้อมกัน, copy-paste แล้วแก้ทีละจุด, และตัวแปรกระจายไปทั่วโปรเจกต์

OOP แก้ปัญหาเหล่านี้ด้วยการ จัดกลุ่มข้อมูลและ behavior เข้าด้วยกันเป็น class — เปลี่ยนทีเดียวกระทบทุกที่

Running Example ตลอด Part III — Refactor PairTradingSignal จาก Part II

เราจะนำ PairTradingSignal จาก Part II มาออกแบบใหม่ให้เป็น OOP ที่ดี:

บท 18 → เข้าใจ class พื้นฐาน · บท 19 → Abstract base class + inheritance

บท 20 → ใส่ design patterns · บท 21 → Mini-project: ระบบครบวงจร

บทที่ 18: Classes & Objects

แก่นของบทนี้

Class คือ "blueprint" — นิยามว่า object ประเภทนี้เก็บข้อมูลอะไร (attributes) และทำอะไรได้บ้าง (methods) Object คือ "ชิ้นงานจริง" ที่สร้างจาก blueprint นั้น เหมือนแบบบ้าน (class) กับบ้านจริง (object)

18.1 ทำไมต้องมี Class — เรื่องเล่าจาก Pair ที่สอง

โค้ดสไตล์ Part I/II (ตัวแปร + ฟังก์ชัน) ทำงานได้ดีกับ pair เดียว:

```
# Part II style: ตัวแปรวางไว้ระดับบนสุดของไฟล์
window = 60
entry_z = 2.0
df = build_spread(btc_bybit, btc_binance, window)
hl = calc_half-life(df["spread"])
# สมมติว่า make_signal คือฟังก์ชันสร้างสัญญาณที่เราเขียนไว้แล้วใน Part II
signal = make_signal(df, entry_z)
```

ปัญหาเริ่มเมื่อเทรด **pair ที่สอง** ซึ่งใช้ parameter คนละชุด:

```
# เพิ่ม pair ที่สอง... ตัวแปรเริ่มแตกหน่อ
window_2 = 90 # pair นี้ mean-revert ช้ากว่า ใช้ window ยาวกว่า
entry_z_2 = 1.5
df_2 = build_spread(eth_okx, eth_bybit, window_2)
hl_2 = calc_half-life(df_2["spread"])

signal = make_signal(df, entry_z)
signal_2 = make_signal(df_2, entry_z) # ← bug! ลืมเปลี่ยนเป็น entry_z_2
# Python ไม่ฟ้องอะไรเลย — signal ผิดเจียบ ๆ
```

สังเกตว่า bug นี้เกิดเพราะข้อมูลกับ parameter ของแต่ละ pair ไม่ได้ผูกกัน — มันแค่ "วางอยู่ใกล้กัน" ในไฟล์ แล้วเราต้องจำเองว่าอะไรคู่กับอะไร ลองนึกต่อ: ถ้ามี 10 pairs จะมีตัวแปร 40+ ตัวลอยอยู่ และทุกการเรียกฟังก์ชันคือโอกาสหยิบผิดคู่

Class คือคำตอบ: กล่องที่มัด "ข้อมูล + parameter + ฟังก์ชันที่ทำงานกับมัน" ไว้ด้วยกันเป็นชั้นเดียว แล้วสร้างกล่องก็ได้:

```
# แต่ละ pair คือ object หนึ่งชิ้น – ถือของของตัวเองครบ
pair1 = PairStrategy("BTC-Bybit", "BTC-Binance", window=60, entry_z=2.0)
pair2 = PairStrategy("ETH-OKX", "ETH-Bybit", window=90, entry_z=1.5)

signal1 = pair1.make_signal() # ใช้ window=60, entry_z=2.0 ของตัวเองเสมอ
signal2 = pair2.make_signal() # ใช้ของตัวเอง – หยิบสลับกันไม่ได้โดยโครงสร้าง
```

📌**อ่านว่า:** สั่งให้ make_signal ของ pair1 ทำงาน – จุด (.) ยังอ่านว่า "ของ" เหมือนเดิมจาก Part I แค่นี้ "ของ" หมายถึงทั้งข้อมูลและฟังก์ชันที่อยู่ในกล่องเดียวกัน

CLASS ไม่ใช่ความรู้ใหม่

Class = dict (เก็บข้อมูล) + function (ทำงาน) เอามาผูกติดกัน – สองอย่างที่ใช้คล่องแล้วจาก Part I

ความใหม่มีแค่ 2 สิ่ง:

1. self = "กล่องใบนี้" – เมื่อเราพิมพ์ pair1.make_signal() Python จะแปลงเป็น

PairStrategy.make_signal(pair1) ให้อัตโนมัติ

พูดง่าย ๆ คือ self ก็คือ pair1 ที่ถูกส่งเข้ามาเป็น argument แรกเสมอ – เราไม่ต้องส่งเอง Python จัดการให้

2. self.window = "หยิบ window จาก dict ของกล่องใบนี้" – ภายในเป็นแค่ dict ธรรมดา แค่ syntax ต่างกัน: cfg["window"] กับ self.window คือสิ่งเดียวกัน

18.2 Class พื้นฐาน

```
# แบบ procedural (Part I/II style) – ข้อมูลกระจาย
symbol_a = "BTCUSDT-Bybit"
symbol_b = "BTCUSDT-Binance"
window = 60
entry_z = 2.0

# แบบ OOP – ข้อมูลอยู่ในที่เดียว
class PairConfig:
    """เก็บ configuration ของ pair trading"""

    def __init__(self, symbol_a, symbol_b,
                  window=60, entry_z=2.0, exit_z=0.5):
        self.symbol_a = symbol_a
        self.symbol_b = symbol_b
        self.window = window
        self.entry_z = entry_z
        self.exit_z = exit_z
```



```

def __repr__(self):
    return (f"PairConfig({self.symbol_a} / {self.symbol_b}, "
            f"window={self.window}, entry_z=±{self.entry_z})")

def is_valid(self):
    """ตรวจสอบว่า config สมเหตุสมผล"""
    return (self.window >= 20 and
            self.entry_z > self.exit_z > 0)

# สร้าง object
cfg = PairConfig("BTCUSDT-Bybit", "BTCUSDT-Binance", window=60)
print(cfg)
print(f"Valid: {cfg.is_valid()}")

```

► OUTPUT

```

PairConfig(BTCUSDT-Bybit / BTCUSDT-Binance, window=60, entry_z=±2.0)
Valid: True

```

18.3 ส่วนประกอบของ Class

ส่วนประกอบ	ใช้งาน	ตัวอย่าง
<code>__init__</code>	Constructor — รันเมื่อ <code>obj = Class(...)</code>	เก็บ parameters, สร้าง initial state
<code>__repr__</code>	String representation สำหรับ debug	<code>print(obj)</code> แสดงข้อมูลสำคัญ
<code>self</code>	ชี้ไปยัง object ปัจจุบัน	<code>self.window</code> คือ attribute ของ object นี้
Instance method	Function ที่รับ <code>self</code> เป็น argument แรก	<code>def is_valid(self):</code>
Class method	Method ที่แชร์ระหว่างทุก object	<code>@classmethod def from_dict(cls, d):</code>
Static method	Function ใน class ที่ไม่ใช่ <code>self</code>	<code>@staticmethod def annualize(daily):</code>

`self` กับ `cls` ต่างกันอย่างไร?

	<code>self</code>	<code>cls</code>
ชี้ไปที่	object ตัวนั้น (กล่องใบนี้)	class เอง (แบบพิมพ์เขียว)
ใช้ใน	method ทั่วไป	<code>@classmethod</code>
ตัวอย่าง	<code>self.window</code> = window ของ object นี้	<code>cls(rets, name)</code> = สร้าง object ใหม่จาก class

@classmethod ใช้บ่อยสำหรับ "alternative constructor" เช่น

TradingMetrics.from_prices(prices) = "สร้าง TradingMetrics จาก prices แทนที่จะส่ง returns โดยตรง"

```
# โค้ดตลอด Part III ใช้สองตัวนี้ – import ไว้ครั้งเดียวใช้ได้ทั้ง Part
import numpy as np
import pandas as pd

class TradingMetrics:
    """คำนวณและเก็บ performance metrics"""

    TRADING_DAYS = 252 # class variable – แชร์ทุก instance

    def __init__(self, returns, name="Strategy"):
        self.returns = np.array(returns)
        self.name = name
        self._cache = {} # _ prefix = "ควรใช้ภายใน class" (convention)

    @property
    def sharpe(self):
        """คำนวณ Sharpe Ratio (cached)"""
        if "sharpe" not in self._cache:
            m = self.returns.mean()
            s = self.returns.std()
            self._cache["sharpe"] = m / s * np.sqrt(self.TRADING_DAYS)
        return self._cache["sharpe"]

    @property
    def annual_return(self):
        return self.returns.mean() * self.TRADING_DAYS

    @property
    def annual_vol(self):
        return self.returns.std() * np.sqrt(self.TRADING_DAYS)

    @property
    def max_drawdown(self):
        equity = np.cumprod(1 + self.returns)
        peak = np.maximum.accumulate(equity)
        return ((equity - peak) / peak).min()

    @classmethod
    def from_prices(cls, prices, name="Strategy"):
        """สร้าง TradingMetrics จาก price series แทน returns"""
        rets = np.diff(prices) / prices[:-1]
        return cls(rets, name)
```

```

@staticmethod
def annualize(daily_val, periods=252):
    """Utility: annualize daily metric"""
    return daily_val * np.sqrt(periods)

def __repr__(self):
    return (f"TradingMetrics({self.name}: "
            f"Sharpe={self.sharpe:.2f}, "
            f"Return={self.annual_return:.1%}, "
            f"DD={self.max_drawdown:.1%})")

# ใช้งาน
import numpy as np
np.random.seed(42)
rets = np.random.normal(0.0005, 0.012, 252)

m = TradingMetrics(rets, "BTC Pair")
print(m)
print(f"Annual Vol: {m.annual_vol:.1%}")

```

► OUTPUT

```

TradingMetrics(BTC Pair: Sharpe=0.62, Return=11.5%, DD=-16.4%)
Annual Vol: 18.4%

```

🤖 3 เครื่องหมาย ที่โผล่พร้อมกัน — ต่างกันที่ "ใครถูกส่งเข้าไปเป็นตัวแรก"

```

@property                                # ① ได้ self
def sharpe(self): ...

@classmethod                             # ② ได้ cls (ตัวคลาสเอง)
def from_prices(cls, prices): ...

@staticmethod                             # ③ ไม่ได้อะไรเลย
def annualize(daily_val): ...

```

ทั้งสามตัวคือ **decorator** — ป้ายที่แปะบนฟังก์ชันเพื่อเปลี่ยนวิธีเรียกใช้ · ง่ายที่สุดโดยดูว่า **parameter ตัวแรก** คืออะไร:

1. `@property + self` — "อ่านได้เหมือนข้อมูล แต่จริง ๆ คำนวณสด" · เรียก `m.sharpe` **ไม่ใส่วงเล็บ** · ใช้เมื่อค่า นั้น "เป็นคุณสมบัติของชิ้นงานนี้" เช่น Sharpe ของพอร์ตนี้
2. `@classmethod + cls` — "ผลิตชิ้นงานใหม่ด้วยวิธีอื่น" · `cls` คือตัวคลาส จึงเขียน `return cls(...)` เพื่อสร้าง object ได้ · ที่นี้ `from_prices()` รับราคาแล้วแปลงเป็น *return* ให้ก่อน — เรียกว่า **alternative constructor**: `TradingMetrics.from_prices(p)` ใช้ได้โดยไม่ต้องมี object ก่อน

3. `@staticmethod` — "ฟังก์ชันช่วยที่บังเอิญเก็บไว้ในคลาสนี้". ไม่แตะ `self` ไม่แตะ `cls` เลย.

`annualize()` แปลงค่ารายวันเป็นรายปี — ใครส่งเลขอะไรมาก็คำนวณให้ไม่เกี่ยวกับพอร์ตโฟลิโอ. เก็บไว้ในคลาสนี้ เพราะอยู่เป็นหมวดเดียวกันเท่านั้น

เทียบให้เห็นภาพ: `@property = "คุณสมบัติของชิ้นงานนี้"`. `@classmethod = "โรงงานผลิตชิ้นงาน"`.

`@staticmethod = "เครื่องมือที่วางไว้ในลิ้นชักเดียวกัน"`

📖 **อ่านว่า:** อีกจุดที่ต้องสังเกตใน `sharpe`: `if "sharpe" not in self._cache:` — คำนวณครั้งแรกครั้งเดียว แล้วเก็บคำตอบไว้ เรียกซ้ำก็หยิบของเก่ามาให้. เรียกว่า **caching** ช่วยเมื่อการคำนวณหนักและถูกเรียกหลายรอบ. ⚠️ แต่มันมีราคา — ดูก่อนถึงได้ไป

⚠️ **cache กับ `@property` = คู่ที่ขัดกันเอง ถ้าข้อมูลเปลี่ยนได้**

คอมเมนต์ใต้โค้ดบอกว่า "ถ้า `returns` เปลี่ยน → ผลลัพธ์เปลี่ยนอัตโนมัติ" — จริงสำหรับ `annual_vol` (ไม่ cache) แต่ **ไม่จริงสำหรับ `sharpe`** เพราะมัน cache ไว้แล้ว

```
m = TradingMetrics(rets)
print(f"sharpe รอบแรก : {m.sharpe:.4f}")          # คำนวณ แล้วเก็บลง cache
print(f"vol รอบแรก : {m.annual_vol:.4f}")

# เปลี่ยนข้อมูลเป็นชุดที่ผันผวนกว่าเดิมมาก
np.random.seed(7)
m.returns = np.random.normal(0.002, 0.030, 252)

print(f"sharpe รอบสอง : {m.sharpe:.4f} <-- ❌ ค่าเดิม! หยิบจาก cache")
print(f"vol รอบสอง : {m.annual_vol:.4f} <-- ✅ ค่าใหม่ เพราะไม่ได้ cache")
```

► OUTPUT

```
sharpe รอบแรก : 0.6233
vol รอบแรก : 0.1839
sharpe รอบสอง : 0.6233 <-- ❌ ค่าเดิม! หยิบจาก cache
vol รอบสอง : 0.4552 <-- ✅ ค่าใหม่ เพราะไม่ได้ cache
```

🚫 **กฎ:** cache ได้เมื่อข้อมูลต้นทางไม่เปลี่ยนหลังสร้าง object (immutable). ถ้าเปลี่ยนได้ ต้องล้าง cache ตอนเปลี่ยน — วิธีมาตรฐานคือใช้ `@property` คู่กับ `setter` ที่สั่ง `self._cache.clear()`. บั๊กจากการลืมล้าง cache หายากมากเพราะโค้ดไม่ error แค่ให้คำตอบเก่า

18.3b @ คืออะไร — decorator ใน 1 บรรทัด

หัวข้อถัดไปจะใช้ `@property` และหลังจากนั้นทั้งเล่มจะเจอ `@abstractmethod` · `@dataclass` ·

`@pytest.fixture` · ก่อนจะไปถึง ขอปักหมุดว่าเครื่องหมาย `@` ทำอะไร เพราะมันมีกฎข้อเดียวจริงๆ

กฎข้อเดียวของ DECORATOR

@something ที่วางไว้เหนือ def f แปลว่า:

```
f = something(f)
```

แค่นั้น . ไม่มีเวทมนตร์อะไรเพิ่ม . @ เป็นแค่ทางลัดในการเขียนประโยคข้างบนให้อ่านง่ายขึ้น

📌**อ่านว่า:** อ่านย้อนกลับจะเข้าใจเร็วกว่า: "เอาฟังก์ชันที่เพิ่งเขียนเสร็จ ส่งเข้าไปในฟังก์ชันอีกตัว แล้วเอาผลลัพธ์มาแทนที่ชื่อเดิม". ดังนั้น **decorator** คือฟังก์ชันที่รับฟังก์ชันเข้าไป แล้วคืนฟังก์ชันออกมา . เหตุผลที่มันมีประโยชน์คือ ฟังก์ชันที่คืนออกมามักเป็นวัตถุที่ "ห่อ" ของเดิมไว้ แล้วแอบทำอะไรเพิ่มก่อน/หลัง โดยที่โค้ดที่เรียกใช้ไม่ต้องรู้เลย

ลองเขียนเองสักตัว — @timed ที่จับเวลาให้ทุกฟังก์ชันที่แปะมัน . เป็นของที่ใช้จริงตอนหา bottleneck ใน backtest:

```
import functools
import time

TIMINGS = {} # เก็บเวลาที่วัดได้ไว้ดูทีหลัง

def timed(func):
    """รับฟังก์ชัน → คืนฟังก์ชันตัวใหม่ที่ห่อของเดิมไว้พร้อมจับเวลา"""

    @functools.wraps(func) # ← อธิบายข้างล่าง
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs) # เรียกของเดิมตามปกติ
        TIMINGS[func.__name__] = time.perf_counter() - t0
        return result # คืนผลเดิมไม่แตะต้อง

    return wrapper # ← คืน "ฟังก์ชัน" ไม่ใช่ผลลัพธ์

@timed
def sum_squares_loop(prices):
    """บวกกำลังสองด้วยลูป Python"""
    total = 0.0
    for p in prices:
        total += p * p
    return total

@timed
def sum_squares_numpy(prices):
    """บวกกำลังสองด้วย NumPy"""
```

```

return float((prices ** 2).sum())

prices_big = np.random.default_rng(0).normal(65000, 500, 2_000_000)
a = sum_squares_loop(prices_big)
b = sum_squares_numpy(prices_big)

print(f"ผลลัพธ์ตรงกันไหม: {abs(a - b) / a < 1e-9}")

# decorator เก็บเวลาให้ทั้งสองตัวโดยที่ตัวฟังก์ชันไม่รู้เรื่องเลย
print(f"decorator เก็บเวลาไว้ให้: {sorted(TIMINGS)}")
print(f"NumPy เร็วกว่า loop Python: "
      f"{TIMINGS['sum_squares_numpy'] < TIMINGS['sum_squares_loop']}")

# ⚠ ไม่พิมพ์ "เร็วกว่าที่เท่า" เพราะเวลาของ NumPy เล็กมากจนสัญญาณรบกวน
#   กลับ — วัดจริงได้ตั้งแต่ 15 ถึง 50 เท่าในเครื่องเดียวกัน

print(f"ชื่อฟังก์ชันยังเป็น: {sum_squares_loop.__name__}")
print(f"docstring ยังอยู่: {sum_squares_loop.__doc__}")

```

► OUTPUT

```

ผลลัพธ์ตรงกันไหม: True
decorator เก็บเวลาไว้ให้: ['sum_squares_loop', 'sum_squares_numpy']
NumPy เร็วกว่า loop Python: True
ชื่อฟังก์ชันยังเป็น: sum_squares_loop
docstring ยังอยู่: บวกกำลังสองด้วย loop Python

```

⚠ ทำไมบล็อกนี้ไม่พิมพ์ว่า "เร็วกว่าที่เท่า"

ตอนแรกเขียนให้พิมพ์อัตราส่วนออกมาตรง ๆ · วัดซ้ำในเครื่องเดียวกัน 3 รอบติดได้ **19 · 22 · 49 เท่า** — ตัวเลขเดียวกันไม่เคยออกมาซ้ำ

สาเหตุคือตัวหารเล็กเกินไป: NumPy ใช้เวลาราว 0.005–0.012 วินาที ซึ่งอยู่ในระดับเดียวกับความแปรปรวนของตัวจับเวลาและภาระเครื่อง · หารด้วยตัวเลขที่กว้าง 2 เท่า ผลหารก็กว้าง 2 เท่าตาม

🚫 **บทเรียนที่ใช้ได้กับทุกการวัด:** ก่อนรายงานอัตราส่วนให้ดูก่อนว่าตัวหารใหญ่พอเทียบกับความแปรปรวนของตัวเองหรือยัง · ถ้าไม่ให้รายงานสิ่งที่ทนต่อสัญญาณรบกวนแทน — ที่นี่คือ "อันไหนเร็วกว่า" (True ทุกครั้ง) · หลักการเดียวกับที่บทที่ 25 บอกว่า Sharpe จากไม่กี่เทรดอ่านไม่ได้

👤 ทำไมต้องมีฟังก์ชันซ้อนฟังก์ชัน

```

def timed(func):
    ① รับ "ฟังก์ชันเดิม" เข้ามา
    def wrapper(*args, **kwargs):
        ② สร้างฟังก์ชันตัวใหม่
        ...จับเวลา...
        result = func(*args, **kwargs)
        ③ ข้างในเรียกของเดิม

```

```
return result
return wrapper
```

④ คืน "ฟังก์ชันตัวใหม่" ออกไป

```
# @timed แปลงเป็นภาษาคน:
# sum_squares_loop = timed(sum_squares_loop)
# → ชื่อเดิมชี้ไปที่ wrapper แล้ว · ของเดิมถูกเก็บไว้ในตัวแปร func
```

1. ต้องมี 2 ชั้นเพราะมี 2 จังหวะที่ต่างกัน · ชั้นนอก (`timed`) ทำงานครั้งเดียวตอน Python อ่านเจอ `@timed` · ชั้นใน (`wrapper`) ทำงานทุกครั้งที่มีคนเรียกฟังก์ชัน · เขียนชั้นเดียวจะแยกสองจังหวะนี้ไม่ได้
2. `*args, **kwargs` — แปลว่า "รับอะไรมาก็ได้ แล้วส่งต่อทั้งหมด" · จำเป็นเพราะ `timed` ต้องใช้ได้กับฟังก์ชันทุกแบบ โดยไม่รู้ล่วงหน้าว่ามันรับพารามิเตอร์อะไร · ถ้าเขียน `def wrapper(prices)` ตายตัว มันจะห่อได้แค่ฟังก์ชันที่รับ `prices` ตัวเดียว
3. `return wrapper` ไม่มีวงเล็บ — คืนตัวฟังก์ชันไม่ใช่ผลลัพธ์ของการเรียกมัน · ใส่วงเล็บเป็น `return wrapper()` เมื่อไร decorator จะพังทันทีเพราะชื่อฟังก์ชันจะไปชี้ที่ค่าที่คำนวณเสร็จแล้ว แทนที่จะชี้ที่ฟังก์ชัน
4. `@functools.wraps(func)` — ถ้าไม่ใช่ `sum_squares_loop.__name__` จะกลายเป็น "wrapper" และ docstring จะหายไป เพราะชื่อเดิมชี้ไปที่ `wrapper` แล้วจริง ๆ · `wraps` คือการก๊อปปี้ docstring/signature ของเดิมมาแปะไว้บน `wrapper` · ผลเสียของการลืมใส่ไม่ใช่แค่ความสวยงาม — debugger, traceback, และ `help()` จะรายงานว่าทุกฟังก์ชันในโปรเจกต์ชื่อ `wrapper` เหมือนกันหมด

✅ ที่นี่ `@property` ก็ไม่ใช่เรื่องลึกลับอีกต่อไป

`property` เป็นฟังก์ชันที่ Python มีให้อยู่แล้ว · เขียน

```
# — ตัวอย่างประกอบ —
@property
def pnl(self): ...
```

ก็คือการเขียน `pnl = property(pnl)` เป๊ะ ๆ ตามกฎข้อเดียวข้างบน · สิ่งที่ `property()` คืนออกมาไม่ใช่ฟังก์ชันธรรมดา แต่เป็นวัตถุชนิดพิเศษที่ Python รู้จัก แล้วเรียกให้เองตอนมีคนอ่าน `obj.pnl` — จึงไม่ต้องใส่วงเล็บ

ตัวอื่นที่จะเจอต่อไปก็กฎเดียวกันหมด: `@abstractmethod` แปะป้ายไว้ให้ ABC ตรวจ · `@staticmethod` บอกว่าไม่ต้องส่ง `self` · `@dataclass` รับคลาสเข้าไปแล้วคืนคลาสที่มี `__init__` / `__repr__` เติมให้ (ใช้กับคลาสได้ด้วย ไม่ใช่แค่ฟังก์ชัน)

🔵 [รอบสอง] decorator ที่รับพารามิเตอร์ — และกับดักเรื่องจังหวะเวลา

อยากเขียน `@retry(3)` ให้ลองใหม่ 3 ครั้ง? สังเกตว่ามันมีวงเล็บ ซึ่งแปลว่า `retry(3)` ถูกเรียกก่อน แล้วผลลัพธ์ของมันต่างหากที่ทำหน้าที่เป็น decorator · ดังนั้นต้องมี 3 ชั้น: ชั้นนอกรับพารามิเตอร์ → คืน decorator → คืน wrapper

```
# — ตัวอย่างประกอบ —
def retry(n_times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*a, **kw):
            ...
        return wrapper
    return decorator
```

กับดักที่ทำให้คนตกบ่อยที่สุด: โค้ดในชั้นนอกทำงานตอน **import** ไฟล์ ไม่ใช่ตอนเรียกฟังก์ชัน. ถ้าเผลอเขียนอะไรที่มีผลข้างเคียงไว้ตรงนั้น (อ่าน config · เปิด connection · อ่านเวลาปัจจุบัน) มันจะทำงานทันทีที่ไฟล์ถูก import และทำครั้งเดียวตลอดอายุโปรแกรม

ในระบบเทรตนี้เป็นเรื่องใหญ่: decorator ที่อ่าน `datetime.now()` ไว้ในชั้นนอก จะค้างอยู่ที่เวลา import ตลอดไป — bot ที่รันข้ามวันจะยังคิดว่าเป็นวันที่เริ่มรัน · กฎ: ชั้นนอกให้มีแค่การรับค่าและ **return** เท่านั้น อะไรที่ต้องสดต้องอยู่ในชั้นในสุด

18.4 @property — Attribute ที่คำนวณแบบ Lazy

`@property` ทำให้ method เรียกใช้ได้เหมือน attribute — ไม่ต้องใส่วงเล็บ และคำนวณเมื่อถูกเรียกเท่านั้น. ตามกฎในหัวข้อที่แล้ว นั่นคือ `pnl = property(pnl)` ไม่มีอะไรมากกว่านั้น

```
# — เทียบให้ดู (สองแบบนี้อยู่ด้วยกันไม่ได้ — เลือกได้แบบเดียว) —
#
# ถ้า annual_vol เป็น method ธรรมดา:
#     print(m.annual_vol())    ← ต้องใส่ () ทั้งที่มันคือ "ค่า" ไม่ใช่ "คำสั่ง"
#
# ถ้าใส่ @property ทับไว้:
print(f"{m.annual_vol:.4f}")  # ← อ่านเหมือน attribute · ชัดกว่า

# ที่คลาสนี้เลือกแบบหลัง (ดูบรรทัด @property ในโค้ดข้างบน)
# ลองเติม () ดูจะได้ TypeError: 'numpy.float64' object is not callable
# — เพราะ m.annual_vol กลายเป็น "ตัวเลข" ไปแล้ว ไม่ใช่ฟังก์ชัน
```

► OUTPUT

0.4552

อ่าน error นั้นให้ออก — มันบอกความจริงของ `@property` ทั้งหมด

`TypeError: 'numpy.float64' object is not callable` แปลตรงตัวว่า "ของชิ้นนี้เป็นเลขทศนิยม เอาไปเรียกแบบฟังก์ชันไม่ได้". นั่นคือหลักฐานว่า `@property` ทำอะไร: พอแปะเข้าไป `m.annual_vol` ก็ไม่ใช่ฟังก์ชันอีกต่อไป มันคือผลลัพธ์ที่คำนวณเสร็จแล้ว

เก็บวิธีอ่านนี้ไว้ใช้ต่อ — เวลาเจอ 'X' object is not callable ให้แปลว่า "คุณใส่ () ต่อท้ายของที่ไม่ใช่ฟังก์ชัน" · สาเหตุที่พบบ่อยที่สุดสองข้อคือ **เมโลใส่ () ให้ property กับ ตั้งชื่อตัวแปรทับชื่อฟังก์ชันไปแล้ว** (เช่น `sum = 0` แล้วเรียก `sum(...)` ที่หลัง)

📌**อ่านว่า:** คลาสข้างล่างมี 3 `@property` · วิธีอ่านที่ตรงที่สุดคือ "ช่องข้อมูลที่คิดสดใหม่ทุกครั้งที่มีการถาม" — เก็บไว้เฉพาะของที่บอกกันมา (`entry_price`, `size`, `exit_price`) ส่วนของที่คำนวณต่อได้ (`pnl`, `return_pct`, `is_open`) ไม่เก็บ แต่คิดตอนถูกเรียก · ข้อดีที่ได้ฟรีคือไม่มีวันไม่ตรงกัน — ถ้าเก็บ `self.pnl` ไว้เป็นตัวแปรแล้วมีคนแก้ไข `exit_price` ที่หลัง ตัวเลขเก่าจะค้างอยู่โดยไม่มีใครรู้

ตัวอย่าง: Position class พร้อม computed properties

```
class Position:
    """แทนตำแหน่งการเทรดเดี่ยว"""

    def __init__(self, symbol, side, size, entry_price,
                  fee_rate: float = 0.001):
        self.symbol      = symbol
        self.side        = side          # "long" หรือ "short"
        self.size        = size          # จำนวน unit
        self.entry_price = entry_price
        self.exit_price  = None
        self.fee_rate    = fee_rate

    @property
    def is_open(self):
        return self.exit_price is None

    @property
    def pnl(self):
        if self.exit_price is None:
            raise ValueError("Position ยังไม่ปิด")
        raw = (self.exit_price - self.entry_price) * self.size
        return raw if self.side == "long" else -raw

    @property
    def return_pct(self):
        return self.pnl / (self.entry_price * self.size)

    def close(self, exit_price):
        self.exit_price = exit_price
        return self

    def __repr__(self):
        status = "OPEN" if self.is_open else f"CLOSED PnL={self.pnl:+,.0f}"
        return f"Position({self.side} {self.size} {self.symbol} @ {self.entry_price})"

# ทดสอบ
pos = Position("BTCUSDT", "long", 0.5, 63000)
print(pos)
pos.close(65500)
print(pos)
print(f"Return: {pos.return_pct:.2%}")
```

► OUTPUT

```
Position(long 0.5 BTCUSDT @ 63,000 [OPEN])
```

Position(long 0.5 BTCUSD @ 63,000 [CLOSED PnL=+1,250])

Return: 3.97%

📺 4 ท่าที่คลาสนี้ใช้ และจะเจอซ้ำในทุกคลาสหลังจากนี้

```
@property
def pnl(self):
    if self.exit_price is None:
        raise ValueError("Position ยังไม่ปิด") ① property ที่ปฏิเสธได้
    raw = (self.exit_price - self.entry_price) * self.size
    return raw if self.side == "long" else -raw ② พลิกเครื่องหมายให้ short

def close(self, exit_price):
    self.exit_price = exit_price
    return self ③ คืนตัวเอง = ต่อจุดได้

def __repr__(self): ④ หน้าตาตอนถูก print
    ...
```

1. **property ที่ raise ได้** — ถ้า pnl ของ position ที่ยังไม่ปิด แล้วโปรแกรมหยุดทันทีพร้อมบอกเหตุผล · ดิกว่า คืน 0 หรือ None มากเพราะ 0 จะไหลเข้าไปรวมใน P&L ของพอร์ตเจียบ ๆ แล้วคุณก็จะเห็นกำไรรวมที่ผิดโดยไม่มีอะไรเตือน
2. `raw if self.side == "long" else -raw` — **conditional expression** อ่านเรียงตามทีเขียน: "เอา raw ถ้าเป็น long ไม่งั้นเอา -raw" · เหตุผลทางการเงินคือ short กำไรตอนราคาลง ซึ่ง `exit - entry` จะติดลบพอดี จึงพลิกเครื่องหมายที่เดียวจบ แทนที่จะเขียนสูตรแยกสองชุด
3. `return self` ทำย close() — ทำให้เขียนต่อกันเป็นสายได้ เช่น `Position(...).close(65500).pnl` · เรียกว่า **method chaining** · ต้นทุนคือ `close()` ทำสองหน้าที่ (แก้ของ + คืนของ) ซึ่งบางทีก็ถือว่าไม่ควรทำ — รู้ไว้ว่ามีสองสำนัก
4. `__repr__` — กำหนดว่า `print(pos)` จะแสดงอะไร · ถ้าไม่เขียนจะได้ `<__main__.Position object at 0x7f3c...>` ซึ่งไร้ประโยชน์ตอนไล่ปัญหา · กฎที่ใช้ได้เสมอ: `__repr__` ควรใส่ค่าที่ทำให้คุณระบุตัวมันได้ ไม่ใช่คำบรรยายสวย ๆ

⚠️ `fee_rate` ในคลาสนี้ถูกเก็บไว้เฉย ๆ — ไม่มีใครใช้

`__init__` รับ `fee_rate` เข้ามาและเก็บใส่ `self.fee_rate` แต่ทั้ง `pnl` และ `return_pct` ไม่ได้แตะมันเลย · ตัวเลข `PnL=+1,250` กับ `Return: 3.97%` ที่เห็นจึงเป็นผลแบบ **ไม่มีค่าธรรมเนียม** ทั้งที่หน้าตาของคลาสบอกไว้ชัดให้แล้ว (แบบฝึกหัดข้างล่างคือการเติมส่วนที่ขาดนี้)

ยกมาให้ดูเพราะนี่คือรูปแบบของบั๊กที่หาเจอยากที่สุดใน OOP — พารามิเตอร์ที่ถูกรับและถูกเก็บ แต่ไม่เคยถูกใช้ · โค้ดรันผ่าน ไม่มี warning ไม่มี error และชื่อตัวแปรทำหน้าที่โกหกแทน · คนอ่านเห็น `fee_rate` ในลายเซ็นก็เชื่อ

ไปแล้วว่าหักให้

🚫 **วิธีจับ:** ทุกครั้งที่เขียน `self.x = x` ใน `__init__` ให้ค้นหา `self.x` ในคลาสทันที. ถ้าเจอที่เดียวคือ บรรทัดที่เพิ่งเขียน แปลว่ามันยังไม่ได้ทำงาน — ไม่ก็ยังไม่เสร็จ ไม่ก็ควรลบทิ้ง

🔵 [รอบสอง] ราคาที่ต้องจ่ายของ property ที่ raise ได้

`pnl` โยน `ValueError` ตอน `position` ยังเปิดอยู่ ซึ่งถูกต้องในแง่ตรรกะ แต่มีผลข้างเคียงที่ควรรู้ล่วงหน้า: การ "แค่ดูค่า" กลายเป็นการกระทำที่ล้มเหลวได้

- ใน debugger หรือ notebook ที่ไล่แสดง attribute ทุกตัวของ object ให้อัตโนมัติ การมองก็ทำให้เกิด exception ได้
- เครื่องมือที่แปลง object เป็น dict/JSON แบบวนทุก property จะพังทันทีที่เจอ `position` ที่ยังเปิดอยู่
- `__repr__` ในคลาสนี้รอดมาได้เพราะเช็ก `is_open` ก่อนแตะ `pnl` — ถ้าเขียน `__repr__` ให้พิมพ์ `pnl` ตรงๆ การ `print` object ที่ยังไม่ปิดจะระเบิด และการ `print` ที่ระเบิดคือฝืนร้ายตอนไล่บั๊ก เพราะเครื่องมือที่ใช้หาปัญหากลายเป็นตัวสร้างปัญหาเสียเอง

ทางเลือกที่ทีมใหญ่มักใช้: ให้ property คืน `None` เมื่อยังไม่มีคำตอบ แล้วบังคับให้คนเรียกตัดสินใจอย่างชัดเจนว่าจะทำอะไรกับ `None` . หรือแยกเป็นสองชื่อ — `pnl` (raise) กับ `pnl_or_none` . ไม่มีคำตอบถูกข้อเดียว แต่ต้องเลือกให้เหมือนกันทั้งโปรเจกต์ ไม่งั้นคนใช้จะเดาไม่ถูกว่าต้องกัน exception ตรงไหนบ้าง

▶ แบบฝึกหัด: เพิ่ม fee calculation ให้ Position

อย่าเพิ่งข้ามแบบฝึกหัดข้างบน — โค้ดที่เหลือทั้ง Part ใช้เวอร์ชันนั้น

เฉลยข้างบนไม่ใช่ของเสริม. มันคือ `Position` ตัวที่หนังสือใช้ต่อไปจนจบ Part III เพราะเป็นตัวที่มี `net_pnl` (กำไรหลังหักค่าธรรมเนียมทั้งขาเข้าและขาออก). ตั้งแต่บทที่ 19 เป็นต้นไปจะเห็น `position.net_pnl` ไล่ตลอด — ทั้งใน `BaseStrategy.log_trade` และ `Portfolio.metrics`

ถ้ารันตามในไฟล์เดียวแล้วข้ามบล็อกนั้นไป จะเจอ:

```
AttributeError: 'Position' object has no attribute 'net_pnl'
```

ซึ่งอ่านยากมากถ้าไม่รู้ที่มา เพราะ error จะไล่ที่บทที่ 19 หรือ 21 — ห่างจากต้นเหตุจริงหลายสิบบรรทัด. ทางเฉลยแล้วรันบล็อกนั้นก่อน แล้วค่อยอ่านต่อ

สรุปสัญญาของสองเวอร์ชันไว้เทียบ:

	Position ในเนื้อหาหลัก (18.4)	Position ในเฉลย (ใช้ต่อจากนี้)
เก็บ fee_rate	เก็บ แต่ไม่ได้ใช้	เก็บ และใช้จริง
pnl	กำไรก่อนหักค่าธรรมเนียม	—
gross_pnl / net_pnl	ไม่มี	มีทั้งคู่ · net หัก fee สองขา

บทที่ 19: Inheritance & Abstract Base Class

แก่นของบทนี้

Inheritance ให้ class ลูกรับ attribute และ method จาก class แม่ — ป้องกัน code ซ้ำ Abstract Base Class (ABC) กำหนด "สัญญา" ว่า class ลูกต้อง implement method ใดบ้าง เหมือน interface ใน Java/C++

19.1 Inheritance พื้นฐาน

```
class BaseStrategy:
    """Base class สำหรับทุก trading strategy"""

    def __init__(self, name, capital=100_000):
        self.name = name
        self.capital = capital
        self.trades = []          # history ของทุก trade
        self.equity = [capital]   # equity curve

    def log_trade(self, position: Position):
        """บันทึก trade ที่ปิดแล้ว"""
        if position.is_open:
            raise ValueError("position นี้ยังไม่ถูกปิด – ต้อง close() ก่อน")
        self.trades.append(position)
        self.capital += position.net_pnl
        self.equity.append(self.capital)

    def summary(self):
        if not self.trades:
            print(f"{self.name}: ยังไม่มี trade")
            return
        pnls = [t.net_pnl for t in self.trades]
        winners = [p for p in pnls if p > 0]
        losers = [p for p in pnls if p <= 0]
        print(f"=== {self.name} ===")
        print(f"Trades: {len(self.trades)}")
        print(f"Win Rate: {len(winners)/len(pnls):.1%}")
        print(f"Avg Win: {sum(winners)/len(winners) if winners else 0:+,.0f}")
        print(f"Avg Loss: {sum(losers)/len(losers) if losers else 0:+,.0f}")
        print(f"Total PnL: {sum(pnls):+,.0f}")
        print(f"Final Capital: {self.capital:+,.0f}")
```

```

def __repr__(self):
    return f"{self.__class__.__name__}({self.name}, capital={self.capital:,.0f})"

class PairTradingStrategy(BaseStrategy):
    """Pair trading ต่อยอดจาก BaseStrategy"""

    def __init__(self, name, pair_config: PairConfig, capital=100_000):
        super().__init__(name, capital) # เรียก __init__ ของแม่
        self.config = pair_config
        self.signal_engine = None

    def fit(self, price_a, price_b):
        """เทรน signal engine"""
        # PairTradingSignal ถูก import สมมติจาก Part II - ในโปรเจกต์จริงให้วางในไฟล์เดียวกัน
        self.signal_engine = PairTradingSignal(
            window = self.config.window,
            entry_z = self.config.entry_z,
            exit_z = self.config.exit_z
        ).fit(price_a, price_b)
        return self

    def get_signal(self):
        if self.signal_engine is None:
            raise RuntimeError("ต้อง fit() ก่อน")
        return self.signal_engine.signal().iloc[-1]

# ทดสอบ inheritance
cfg = PairConfig("BTC-Bybit", "BTC-Binance")
strategy = PairTradingStrategy("BTC Pair v1", cfg, capital=500_000)
print(strategy)
print(f"เป็น BaseStrategy ไหม? {isinstance(strategy, BaseStrategy)}")

```

► OUTPUT

```

PairTradingStrategy(BTC Pair v1, capital=500,000)
เป็น BaseStrategy ไหม? True

```

 **ฝึกอ่าน:** `super().__init__(name, capital)`

```

class PairTradingStrategy(BaseStrategy):
    def __init__(self, name, pair_config, capital=100_000):
        super().__init__(name, capital) # ← บรรทัดนี้

```

อ่านจากในออกนอก:

1. `super()` — "คลาสแม่ของฉัน" = `BaseStrategy` (ที่อยู่ใน parentheses ของ class definition)

2. `super().__init__` — เรียก `__init__` ของ คลาสแม่

3. `super().__init__(name, capital)` — ส่ง `name` กับ `capital` ให้แม่ไปตั้งค่า `self.name`, `self.capital`, `self.trades`, `self.equity` เหมือนที่แม่ทำ

ถ้าไม่เรียก `super().__init__()` → `self.name` และ `self.capital` จะไม่ถูกสร้าง → เรียกทีหลังจะ

`AttributeError` — ลูกต้องเรียกแม่ให้ทำงานของแม่ก่อน แล้วค่อยทำงานของตัวเอง

19.2 Abstract Base Class — บังคับให้ implement

```
from abc import ABC, abstractmethod

class Strategy(ABC):
    """Abstract base class – blueprint สำหรับทุก strategy"""

    def __init__(self, name, capital=100_000):
        self.name = name
        self.capital = capital
        self.trades = []

    @abstractmethod
    def fit(self, data):
        """ต้อง implement – train model"""
        pass

    @abstractmethod
    def signal(self, data) -> str:
        """ต้อง implement – คืน 'LONG', 'SHORT', 'EXIT', 'HOLD'"""
        pass

    @abstractmethod
    def position_size(self, signal: str) -> float:
        """ต้อง implement – คืน size เป็น fraction ของ capital"""
        pass

    def summary(self): # concrete method – ไม่ต้อง override
        pnls = [t.net_pnl for t in self.trades]
        total = sum(pnls) if pnls else 0
        print(f"{self.name}: {len(pnls)} trades, PnL={total:+,.0f}")

    def __repr__(self): # concrete เหมือนกัน – ลูกทุกตัวได้ไปใช้ฟรี
        return (f"{type(self).__name__}"
                f"({self.name}, capital={self.capital:,.0f})")

# ถ้าลืม implement abstractmethod จะ error ทันที
class BadStrategy(Strategy):
    pass # ไม่ implement fit, signal, position_size
```



```
# s = BadStrategy("test") ← TypeError: Can't instantiate abstract class
```

concrete method คือเมธอดที่ไม่มี `@abstractmethod` — class แม่เขียน implementation ไว้ครบแล้ว class ลูกใช้ได้เลยโดยไม่ต้องเขียนซ้ำ (ตรงข้ามกับ abstract method ที่ลูกต้องเขียนเอง)

📖 **อ่านว่า:** ดู `__repr__` ในคลาสแม่ให้ดี — มันใช้ `type(self).__name__` ไม่ใช่เขียน "Strategy" ตรงๆ. `self` ตอนรันคือลูก ไม่ใช่แม่ ดังนั้น `PairStrategy` จะพิมพ์ว่า `PairStrategy(...)` และ `MomentumStrategy` จะพิมพ์ว่า `MomentumStrategy(...)` โดยที่เขียนโค้ดแค่ครั้งเดียวในแม่ — นี่คือประโยชน์ของ inheritance ที่เห็นชัดที่สุดในไฟล์นี้

📖 **ฝึกอ่าน:** `@abstractmethod` และ `ABC`

```
from abc import ABC, abstractmethod

class Strategy(ABC):
    # ABC = Abstract Base Class
    @abstractmethod
    def signal(self, data) -> str:
        pass
```

อ่านทีละส่วน:

1. `ABC` ใน `(ABC)` — บอก Python ว่า "class นี้เป็น abstract — ห้ามสร้าง object โดยตรง"
2. `@abstractmethod` — decorator ที่ "ติดป้าย" ว่า method นี้ **ต้องถูก override โดยลูก** — ถ้าลูกไม่ทำ สร้าง object ไม่ได้เลย
3. `pass` ใน method body — บอกว่า "แม่ไม่ได้ implement จริงๆ แค่กำหนดชื่อ/signature ไว้"

เปรียบ `ABC` กับ interface ของหมอ: "ทุกคนที่จบแพทย์ ต้องมี license — ถ้าไม่มีจะทำงานเป็นหมอไม่ได้" — `ABC` คือการบังคับแบบเดียวกัน: ทุก strategy ต้อง implement ทั้ง `fit()`, `signal()` และ `position_size()`

ประโยชน์ของ `ABC` ใน trading system

- บังคับว่าทุก strategy ต้องมี `fit()`, `signal()`, `position_size()`
- Backtester ทำงานกับ `Strategy` ประเภทไหนก็ได้ — ไม่ต้องรู้ว่าเป็น pair, momentum, หรือ vol
- เพิ่ม strategy ใหม่ → implement 3 methods → ทำงานกับ backtester ทันที

19.3 Encapsulation — ซ่อน implementation detail

```
class RiskManager:
    """จัดการ risk — ซ่อน implementation ภายใน"""
```

```

def __init__(self, max_position_pct=0.20, max_drawdown_pct=0.10):
    self._max_pos = max_position_pct    # _ = "private by convention"
    self._max_dd = max_drawdown_pct
    self.__peak_equity = None           # __ = name-mangled (private จริงๆ)

def check(self, signal, capital, current_equity):
    """คืน True ถ้า trade ผ่าน risk check"""
    if signal == "HOLD":
        return False

    # อัปเดต peak
    if self.__peak_equity is None or current_equity > self.__peak_equity:
        self.__peak_equity = current_equity

    # Drawdown check
    drawdown = (current_equity - self.__peak_equity) / self.__peak_equity
    if drawdown < -self._max_dd:
        print(f"RISK BLOCK: Drawdown {drawdown:.1%} เกิน limit {-self._max_dd:.1%}")
        return False

    return True

def size(self, signal, capital, volatility):
    """คำนวณ position size ที่ปลอดภัย"""
    if volatility <= 0:
        return 0

    # Vol-adjusted sizing: target 1% daily vol contribution
    target_vol = 0.01
    raw_size = target_vol / volatility
    return min(raw_size, self._max_pos) # ไม่เกิน max position

@property
def max_position(self):
    return self._max_pos

# max_position อ่านได้ แต่เปลี่ยนตรงๆ ไม่ควร
rm = RiskManager(max_position_pct=0.15, max_drawdown_pct=0.10)
print(f"Max position: {rm.max_position:.0%}")
print(f"Approved: {rm.check('LONG', 500000, 490000)}")

```

📌อ่านว่า: คลาสข้างล่างยาว 55 บรรทัด แต่โครงมีแค่ 2 ช่วงเวลา — จับให้ได้แล้วอ่านไพล่วดเดียว: `fit()` คือ "เรียนจากอดีต ทำครั้งเดียว" · `signal()` คือ "ตัดสินใจกับแท่งล่าสุด ทำทุกแท่ง" · ทุกบรรทัดใน 55 บรรทัดนั้นอยู่ในช่วงใดช่วงหนึ่งเสมอ · และเส้นแบ่งนี้เองคือสิ่งที่กำหนดการมองอนาคต — ถ้าเผลอเอาการคำนวณที่ต้องใช้ข้อมูลทั้งชุดไปไว้ใน `signal()` เมื่อไร กลยุทธ์จะเริ่มโง่งันที่โดยไม่มี error ใด ๆ

ตัวอย่าง: PairStrategy ที่ implement ABC ครบ

```
# ต้อง import เพิ่ม: import numpy as np · import pandas as pd
#                      import statsmodels.api as sm

class PairStrategy(Strategy):
    """Concrete pair trading strategy – implement ครบ 3 abstract methods"""

    def __init__(self, name, symbol_a, symbol_b,
                 window=60, entry_z=2.0, capital=100_000):
        super().__init__(name, capital)
        self.symbol_a = symbol_a
        self.symbol_b = symbol_b
        self.window    = window
        self.entry_z    = entry_z
        self._alpha    = None
        self._beta     = None
        self._mu       = None
        self._sd       = None

    def fit(self, data: pd.DataFrame):
        """เรียนความสัมพันธ์จากข้อมูลอดีต – ทำครั้งเดียว (หรือนาน ๆ ครั้ง)

        เก็บ 4 ค่า: alpha, beta (ความสัมพันธ์) + mu, sd (ระดับปกติของ spread)
        """
        log_a = np.log(data[self.symbol_a])
        log_b = np.log(data[self.symbol_b])

        X = sm.add_constant(pd.DataFrame({"log_b": log_b}))
        res = sm.OLS(log_a, X).fit()
        self._alpha = float(res.params["const"])
        self._beta  = float(res.params["log_b"])

        resid = log_a - (self._alpha + self._beta * log_b)
        self._mu = float(resid.mean())
        self._sd = float(resid.std())
        if self._sd == 0 or np.isnan(self._sd):
            raise ValueError("spread ไม่มีความผันผวน – ข้อมูล train ผิดปกติ")
        return self

    def signal(self, data=None) -> str:
        """ให้คะแนนแท่งล่าสุด – เรียกได้ทุก timestep โดยไม่ต้อง fit ใหม่"""
        if self._beta is None:
            raise RuntimeError("ต้องเรียก fit() ก่อน signal()")
        if data is None or len(data) == 0:
            return "HOLD"

        # ใช้เฉพาะราคา "แท่งล่าสุด" ที่ส่งเข้ามา – ไม่แตะข้อมูลอนาคต
```

```

log_a = np.log(data[self.symbol_a].iloc[-1])
log_b = np.log(data[self.symbol_b].iloc[-1])
spread = log_a - (self._alpha + self._beta * log_b)
z = (spread - self._mu) / self._sd

if z > self.entry_z: return "SHORT"
if z < -self.entry_z: return "LONG"
if abs(z) < 0.5:      return "EXIT"
return "HOLD"

def position_size(self, signal: str) -> float:
    if signal == "HOLD": return 0.0
    if signal == "EXIT": return 0.0
    return 0.10 # 10% ของ capital (จะ optimize ใน Part IV)

# ตรวจสอบว่า implement ครบ
strat = PairStrategy("BTC Pair", "Bybit", "Binance")
print(isinstance(strat, Strategy)) # True
print(strat)

```

🤖 แพตเทิร์น "วัตถุลองเฟส" — ยังไม่พร้อม แล้วค่อยพร้อม

```

def __init__(self, ...):
    super().__init__(name, capital)    ① ให้คลาสแม่ตั้งของของมันก่อน
    self._alpha = None                 ② จอที่ไว้ — ยังไม่มีค่า
    self._beta = None
    self._mu = None
    self._sd = None

def fit(self, data):                  ③ เฟสเรียน: เต็มค่าทั้ง 4
    ...
    return self

def signal(self, data=None):
    if self._beta is None:             ④ ยาม: กันใช้ก่อนพร้อม
        raise RuntimeError("ต้องเรียก fit() ก่อน signal()")

```

1. `super().__init__(name, capital)` — เรียก `__init__` ของ **คลาสแม่** ให้ตั้งค่าส่วนที่เป็นของกลาง (ชื่อ-เงินทุน) ก่อน แล้วเราค่อยตั้งของเฉพาะตัว. **ลิมบรทัดนี้ = ของที่คลาสแม่ต้องตั้งจะไม่มีเลย** แล้วจะไปพังทีหลัง เป็น `AttributeError` ที่ดูไม่ออกว่าเกี่ยวข้องกับ `__init__`
2. ตั้งเป็น `None` ไว้ก่อน ทั้งที่ยังไม่มีค่า — ไม่ใช่ความสิ้นเปลือง แต่เป็นการประกาศว่าวัตถุนี้นี้มีช่องอะไรบ้าง. คนอ่าน `__init__` อย่างเดียวก็เห็นครบว่าคลาสนี้จำอะไรไว้. ถ้าไม่ประกาศแล้วไปสร้างเอาใน `fit()` คนอ่านต้องไล่ทั้งคลาสถึงจะรู้

3. `_` นำหน้าชื่อ (`_alpha`, `_beta`) — เป็นข้อตกลงระหว่างคน ว่า "ของภายในอย่าแตะจากข้างนอก" · Python ไม่ได้บังคับอะไรเลย แต่ได้จริง · แต่พอเห็น `_` ก็รู้ทันทีว่าถ้าไปแก้เอง อาจพังโดยที่คนเขียนไม่ได้รับประกันอะไรให้

4. ยามที่บรรทัดแรกของ `signal()` — `if self._beta is None: raise` . นี่คือการที่ทำให้แพดเทิร์นสองเฟสปลอดภัย · ถ้าไม่มียามตัวนี้การเรียก `signal()` ก่อน `fit()` จะไม่ error แต่จะไปคำนวณด้วย `None` แล้วได้ `TypeError` ลึก ๆ กลางสูตร ซึ่งอ่านไม่ออกว่าต้นเหตุคือลืม `fit()`

`fit()` จบด้วย `return self` เหมือน `Position.close()` — เขียน `PairStrategy(...).fit(train).signal(latest)` ต่อกันได้แบบบรรทัดเดียว

● [รอบสอง] `mu` กับ `sd` ถูกแก้ไขไว้ตั้งแต่วันที่ `fit` — และนั่นคือจุดที่กลยุทธ์คู่ตายจริง

ดูให้ดีกว่า `signal()` เอา spread ของแท่งล่าสุดไปเทียบกับ `self._mu` และ `self._sd` ที่คำนวณไว้ตั้งแต่ตอน `fit()` · トラバิดที่ยังไม่เรียก `fit()` ใหม่ ตัวเลขสองตัวนี้จะไม่ขยับอีกเลย

ผลที่ตามมาเมื่อความสัมพันธ์ของคู่เหรียญเปลี่ยนไปจริง ๆ (ซึ่งเกิดเสมอ — เปลี่ยนค่าธรรมเนียม เปลี่ยนสภาพคล่อง มีคนถอนสภาพคล่องออก):

- spread จริงเลื่อนไปอยู่ระดับใหม่ แต่ `mu` ยังชี้ไปที่ระดับเก่า
- `z` จึงค้างอยู่ฝั่งเดียวถาวร → กลยุทธ์เปิดไม้เดิมซ้ำ ๆ แล้วไม่เคยได้สัญญาณ EXIT เพราะ `abs(z) < 0.5` ไม่มีวันเป็นจริงอีก
- อาการที่เห็นจากภายนอกคือ "กลยุทธ์เคยดีแล้วจู่ ๆ ก็ค้างสถานะยาว" — ไม่มี error ไม่มี log ผิดปกติ

ทางแก้มี 2 แนว และเลือกได้ตามว่าคุณเชื่ออะไร:

1. **fit ใหม่เป็นระยะ** (เช่น ทุกสัปดาห์ ด้วยข้อมูลย้อนหลังหน้าต่างคงที่) — ง่ายสุด และยังคงได้ว่า "เรียนจากอดีตเท่านั้น" อยู่
2. **ใช้ค่าที่ขยับตามเวลา** — `rolling mean/std` หรือ `RollingOLS` สำหรับ `beta` · แม่นกว่าแต่ต้องระวังมองอนาคตมากเกินไป เพราะการคำนวณย้ายเข้าไปอยู่ในเส้นทางของ `signal()` แล้ว

และไม่่ว่าจะเลือกทางไหน สิ่งที่ต้องมีคู่กันเสมอคือ ตัวเฝ้าดูว่า spread ยังอยู่ในระดับที่เคยเรียนมาไหม — ถ้าหลุดกรอบไปนานผิดปกติ ให้หยุดเทรดคู่ นั้น ไม่ใช่ปล่อยให้มันถล่นต่อไปเรื่อย ๆ

► แบบฝึกหัด: สร้าง `MomentumStrategy` ที่ implement ABC

บทที่ 20: Design Patterns

แก่นของบทนี้

Design Pattern คือ "สูตรสำเร็จ" ที่นักพัฒนาทั่วโลกใช้แก้ปัญหาเดิม ๆ มาหลายสิบปี — ไม่ต้องคิดใหม่ตั้งแต่ต้น ใน Quant system ใช้บ่อย 3 patterns: **Strategy Pattern** (สลับ algorithm ได้โดยไม่แก้ backtester), **Observer Pattern** (แจ้งเตือนเมื่อมี event เช่น signal เกิด), **Factory Pattern** (สร้าง object จาก config โดยไม่ hard-code ชื่อ class)

20.1 Strategy Pattern — สลับ Algorithm

Backtester ทำงานกับ *ทุก* strategy โดยไม่รู้ว่าเป็นประเภทไหน — ขอแค่ implement `Strategy ABC`

```
class Backtester:
    """
    Generic backtester – ทำงานกับ Strategy ใดก็ได้
    Strategy Pattern: algorithm (strategy) แยกจาก runner (backtester)
    """

    def __init__(self, strategy: Strategy, risk_manager: RiskManager = None):
        self.strategy = strategy
        self.rm = risk_manager or RiskManager()
        self.results = []

    def run(self, data: pd.DataFrame, price_col: str) -> pd.DataFrame:
        """
        รัน backtest บน data
        data: DataFrame ที่มีราคา
        price_col: ชื่อ column ของราคาที่ใช้ execute trade
        """
        prices = data[price_col].values
        capital = self.strategy.capital
        equity = [capital]
        pos = None # position ปัจจุบัน

        self.strategy.fit(data.iloc[:60]) # train บน 60 วันแรก

        for i in range(60, len(data)):
            window_data = data.iloc[max(0, i-60):i]
            sig = self.strategy.signal(window_data)
            price = prices[i]
```

```

        # Exit logic
        if pos and (sig == "EXIT" or
                    (sig == "LONG" and pos.side == "short") or
                    (sig == "SHORT" and pos.side == "long")):
            pos.close(price)
            capital += pos.net_pnl
            self.strategy.trades.append(pos)
            pos = None

        # Entry logic
        if pos is None and sig in ("LONG", "SHORT"):
            if self.rm.check(sig, capital, capital):
                frac = self.strategy.position_size(sig)
                size = (capital * frac) / price
                side = "long" if sig == "LONG" else "short"
                pos = Position(price_col, side, size, price)

        equity.append(capital)

    data = data.copy()
    data["equity"] = equity[:len(data)]
    return data

# ใช้งาน Strategy Pattern:
# สลับระหว่าง pair กับ momentum โดยไม่แก้ Backtester เลย
pair_bt = Backtester(PairStrategy("BTC Pair", "Bybit", "Binance"))
mom_bt = Backtester(MomentumStrategy("BTC Mom", "Bybit"))

```

20.2 Observer Pattern — Event System

Observer ให้ object "สมัครรับ" event จาก object อื่น — เมื่อ event เกิด ทุก observer ถูกแจ้ง

```

from typing import Callable, List

class EventBus:
    """
    Simple event bus — Observer Pattern
    ใช้สำหรับ: signal เกิด → แจ้ง logger, risk manager, order executor
    """

    def __init__(self):
        self._listeners: dict[str, List[Callable]] = {}

    def subscribe(self, event: str, callback: Callable):
        """สมัครรับ event"""
        self._listeners.setdefault(event, []).append(callback)

```

```

def publish(self, event: str, data=None):
    """ส่ง event ให้ทุก subscriber"""
    for cb in self._listeners.get(event, []):
        cb(data)

# ตัวอย่าง: เมื่อ signal เกิด → logger, risk, executor ทำงาน
bus = EventBus()

# Subscribers
def on_signal_logger(data):
    print(f"[LOG] Signal: {data['signal']} @ {data['price']:, .0f}")

def on_signal_risk(data):
    if data['signal'] != "HOLD":
        print(f"[RISK] Checking signal: {data['signal']}")

def on_signal_execute(data):
    if data['signal'] == "LONG":
        print(f"[EXEC] Placing LONG order @ {data['price']:, .0f}")

bus.subscribe("signal", on_signal_logger)
bus.subscribe("signal", on_signal_risk)
bus.subscribe("signal", on_signal_execute)

# Publish event
bus.publish("signal", {"signal": "LONG", "price": 65000})

```

► OUTPUT

```

[LOG] Signal: LONG @ 65,000
[RISK] Checking signal: LONG
[EXEC] Placing LONG order @ 65,000

```

20.3 Factory Pattern — สร้าง Object จาก Config

```

class StrategyFactory:
    """
    Factory Pattern: สร้าง strategy object จาก config dict
    ป้องกัน hard-code ชื่อ class ใน code หลัก
    """

    _registry = {}

    @classmethod
    def register(cls, name: str, strategy_class):
        cls._registry[name] = strategy_class

```



```

@classmethod
def create(cls, config: dict) -> Strategy:
    strat_type = config.get("type")
    if strat_type not in cls._registry:
        raise ValueError(f"Unknown strategy: {strat_type}. "
                           f"Available: {list(cls._registry.keys())}")
    klass = cls._registry[strat_type]
    return klass(**{k: v for k, v in config.items() if k != "type"})

# ลงทะเบียน strategies
StrategyFactory.register("pair", PairStrategy)
StrategyFactory.register("momentum", MomentumStrategy)

# สร้างจาก config (อาจมาจาก JSON file)
config = {
    "type": "pair",
    "name": "BTC Pair v2",
    "symbol_a": "Bybit",
    "symbol_b": "Binance",
    "window": 60,
    "entry_z": 2.0,
    "capital": 500_000
}

strat = StrategyFactory.create(config)
print(strat)
print(type(strat).__name__)

```

► OUTPUT

```

PairStrategy(BTC Pair v2, capital=500,000)
PairStrategy

```

เมื่อไหร่ควรใช้ Design Pattern แต่ละแบบ

Pattern	ใช้เมื่อ	ตัวอย่างใน trading
Strategy	มีหลาย algorithm ที่สลับกันได้	Pair / Momentum / Volatility strategy
Observer	หลาย component ต้องรู้เมื่อ event เกิด	Signal → Log + Risk + Execute
Factory	สร้าง object จาก config โดยไม่ hard-code	Load strategy จาก JSON config

บทที่ 21: OOP Mini-Project — Trading System

แก่นของบทนี้

รวมทุกอย่างจาก Part III เป็น trading system ที่ใช้งานได้จริง: **Portfolio** (เก็บ positions หลายตัว), **RiskManager** (ควบคุม risk), **Backtester** (ทดสอบย้อนหลัง), **StrategyFactory** (สร้างจาก config) — ทั้งหมดเชื่อมกันด้วย EventBus

21.1 Architecture Overview



21.2 Portfolio class

```
class Portfolio:
    """
    จัดการ positions หลายตัวพร้อมกัน
    เก็บ state ทั้งหมดของระบบการเทรด
    """

    def __init__(self, initial_capital=1_000_000):
        self.initial_capital = initial_capital
        self.cash = initial_capital
        self.open_positions = {} # symbol -> Position
        self.closed_positions = []
        self._equity_history = [initial_capital]

    def open(self, symbol, side, size, price, fee_rate=0.001):
        """เปิด position ใหม่"""
        if symbol in self.open_positions:
            raise ValueError(f"มี position {symbol} อยู่แล้ว")

        # long = +1 · short = -1 · ตัวเดี่ยวนั้นคุมทิศทางของเงินทั้งคลาส
        # long ควักเงินไปซื้อของ → cash ลด
        # short ขายของที่ยืมมา ได้เงิน → cash เพิ่ม (แต่ติดหนี้เป็นของ)
        sign = 1 if side == "long" else -1
```

```

notional = size * price
fee       = notional * fee_rate

# ใช้ notional เป็นเกณฑ์ทั้งสองฝั่ง - short ก็ต้องมีเงินวางประกัน
if notional + fee > self.cash:
    raise ValueError(
        f"เงินไม่พอ: ต้องการ {notional + fee:,.0f}, มี {self.cash:,.0f}"
    )

self.cash -= sign * notional + fee
self.open_positions[symbol] = Position(symbol, side, size, price, fee_rate)
return self

def close(self, symbol, price):
    """ปิด position"""
    if symbol not in self.open_positions:
        raise ValueError(f"ไม่มี position {symbol}")
    pos = self.open_positions.pop(symbol)
    pos.close(price)

    # ปิดคือทำกลับด้านของตอนเปิด - เครื่องหมายจึงกลับ แต่ fee ยังจ่ายเสมอ
    sign = 1 if pos.side == "long" else -1
    notional = pos.size * price
    self.cash += sign * notional - notional * pos.fee_rate

    self.closed_positions.append(pos)
    self._equity_history.append(self.total_equity({symbol: price}))
    return pos

def total_equity(self, current_prices: dict) -> float:
    """มูลค่ารวม = cash + มูลค่าของที่ถืออยู่ (short นับเป็นลบ)"""
    unrealized = sum(
        (1 if pos.side == "long" else -1)
        * pos.size * current_prices.get(sym, pos.entry_price)
        for sym, pos in self.open_positions.items()
    )
    return self.cash + unrealized

@property
def metrics(self):
    pnls = [p.net_pnl for p in self.closed_positions]
    if not pnls:
        return {"trades": 0}
    winners = [p for p in pnls if p > 0]
    return {
        "trades": len(pnls),
        "win_rate": len(winners) / len(pnls),
        "total_pnl": sum(pnls),
    }

```

```

        "avg_win": sum(winners)/len(winners) if winners else 0,
        "avg_loss": sum(p for p in pnls if p <= 0) / max(1, len(pnls)-len(winners))
    }

    def __repr__(self):
        return (f"Portfolio(cash={self.cash:,.0f}, "
                f"positions={list(self.open_positions.keys())}, "
                f"trades={len(self.closed_positions)}")

```



sign ตัวเดียวคุมทิศทางของเงินทั้งคลาส

```

sign = 1 if side == "long" else -1

self.cash -= sign * notional + fee      # เปิด
self.cash += sign * notional - notional * fee_rate    # ปิด

unrealized = sum(sign(pos) * pos.size * ราคาปัจจุบัน ...) # ตีมูลค่า

```

1. **ทำไมต้องมี sign** — long กับ short เป็นสิ่งตรงข้ามกันในทุกมิติ: long ควักเงินไปซื้อของ (cash ลด แล้วได้ของมาถือ) · short ขายของที่ยืมมา (cash เพิ่ม แต่ติดหนี้เป็นของที่ต้องซื้อคืน) · แทนที่จะเขียนสูตรแยกสองชุดแล้วเสี่ยงแก่ง ไม่ครบใช้ตัวคูณ ± 1 ตัวเดียวคุมทั้งหมด
2. **fee ไม่คูณ sign** — สังเกตให้ดี · ทิศทางของเงินต้นกลับได้ แต่ค่าธรรมเนียมจ่ายออกเสมอทั้งขาเข้าและขาออก ทั้ง long และ short · ถ้าเผละคูณ sign ไปด้วย จะกลายเป็น "short แล้วได้ค่าธรรมเนียมคืน" ซึ่งไม่มีในโลกจริง
3. **total_equity ต้องคูณ sign ด้วย** — เพราะของที่ถือ short มีมูลค่าติดลบต่อพอร์ต · ราคาขึ้น = หนี้เพิ่ม = equity ลด · ถ้าลืมจุดนี้ พอร์ตจะรายงานว่ากำไรตอนที่ขาดทุนจริง
4. **ด้านตรวจเงินใช้ notional ไม่ใช่กระแสเงินสด** — สำหรับ short กระแสเงินสดเป็นบวก (ได้เงินเข้า) ถ้าเช็คจากตัวนั้นจะเปิด short ขนาดเท่าไรก็ได้แม้เงินในบัญชีจะไม่พอปิดสถานะ · ในโลกจริงต้องวางเงินประกัน จึงเช็คจาก **ขนาดสถานะ** เป็นเกณฑ์ทั้งสองฝั่ง

วิธีตรวจว่าบัญชีถูกจริง คือดูว่า **ตัวเลขจากสองที่ต้องตรงกัน**: กำไรที่ **Position** รายงาน กับเงินสดที่ขยับจริงในพอร์ต · ถ้าไม่ตรงเมื่อไร แปลว่ามีเงินหายหรือกองอยู่ที่ไหนสักแห่ง

```

# ตรวจ short: เปิดที่ 100 แล้วราดาลงไป 90 → short ต้องกำไร
pf = Portfolio(initial_capital=1_000_000)
pf.open("TEST", "short", size=100, price=100.0)

print(f"cash หลังเปิด short : {pf.cash:,.2f}")
print(f"equity ตอนราคา 90 : {pf.total_equity({'TEST': 90.0}):,.2f} ← ต้องกำไร")
print(f"equity ตอนราคา 110 : {pf.total_equity({'TEST': 110.0}):,.2f} ← ต้องขาดทุน")

pos = pf.close("TEST", 90.0)

```

```
cash_moved = pf.cash - 1_000_000

print(f"\nPosition.net_pnl บวก : {pos.net_pnl:+, .2f}")
print(f"cash ขยับจริง      : {cash_moved:+, .2f}")
print("ตรงกัน ✓" if abs(pos.net_pnl - cash_moved) < 1e-9 else "✗ ไม่ตรง - บัญชีรั่ว")
```

► OUTPUT

```
cash หลังเปิด short : 1,009,990.00
equity ตอนราคา 90   : 1,000,990.00 ← ต้องกำไร
equity ตอนราคา 110  : 998,990.00  ← ต้องขาดทุน

Position.net_pnl บวก : +981.00
cash ขยับจริง      : +981.00
ตรงกัน ✓
```

เทคนิคที่ใช้ได้กับทุกระบบบัญชี — "ตรวจจากสองทาง"

บล็อกข้างบนไม่ได้ตรวจว่า "ตัวเลขเท่ากับ 981 ไหม" — ถ้าตรวจแบบนี้ต้องมานั่งแก้ทุกครั้งที่เปลี่ยนค่าธรรมเนียมหรือขนาดไม้. มันตรวจว่า **สองเส้นทางที่คำนวณคนละวิธีให้คำตอบตรงกันไหม**: ทางหนึ่งคิดจากราคาเข้า-ออกของ Position อีกทางคิดจากเงินสดที่ไหลเข้าออกพอร์ตจริง

เงื่อนไขแบบนี้เป็นจริงเสมอไม่ว่าตัวเลขจะเปลี่ยนไปแค่ไหน — เป็นการทดสอบชนิดเดียวกับที่บทที่ 28 เรียกว่า "ตรวจสอบคุณสมบัติ ไม่ใช่ตรวจคำตอบ" . และในระบบเทรดจริงมันคือด้านสุดท้ายที่จับได้ว่า fee คิดขาดไปข้างหนึ่งหรือ short คิดกลับทิศ

🚩 ทุกครั้งที่เขียนโค้ดที่ขยับเงินให้หาตัวเลขเดียวกันที่คำนวณได้จากอีกทางเสมอ แล้วเทียบกัน . ถ้าหาไม่เจอ แปลว่าระบบนั้นยังไม่มีทางรู้เลยว่าตัวเองผิดหรือถูก

21.3 ประกอบทุก Component เป็น TradingSystem

TradingSystem ออกแบบตามแนวคิด **Facade** — เป็น "จุดติดต่อเดียว" ที่ซ่อนความซับซ้อนข้างในไว้เหมือนรีโมททีวี ที่กดปุ่มเดียวแต่ข้างในสั่งงานหลายวงจร ผู้ใช้ไม่ต้องรู้ว่า มี Strategy, Portfolio, RiskManager ทำงานประสานกันอย่างไร

```
class TradingSystem:
    """
    Facade class — จัดการทุกอย่างผ่านจุดเดียว
    ใช้: system = TradingSystem.from_config(config)
    """

    def __init__(self, strategy: Strategy,
                  portfolio: Portfolio,
                  risk_manager: RiskManager):
        self.strategy = strategy
        self.portfolio = portfolio
```

```

self.rm          = risk_manager
self.bus         = EventBus()
self._setup_listeners()

def _setup_listeners(self):
    self.bus.subscribe("signal", self._on_signal)
    self.bus.subscribe("error", self._on_error)

def _on_signal(self, data):
    sig  = data["signal"]
    price = data["price"]
    sym  = data["symbol"]

    if sig == "LONG" and sym not in self.portfolio.open_positions:
        capital = self.portfolio.total_equity({})
        if self.rm.check(sig, capital, capital):
            frac = self.strategy.position_size(sig)
            size = (capital * frac) / price
            self.portfolio.open(sym, "long", size, price)
            print(f"[OPEN] LONG {sym} size={size:.4f} @ {price:,.0f}")

    elif sig in ("EXIT", "SHORT") and sym in self.portfolio.open_positions:
        pos = self.portfolio.close(sym, price)
        print(f"[CLOSE] {sym} PnL={pos.net_pnl:+,.0f}")

def _on_error(self, data):
    print(f"[ERROR] {data}")

@classmethod
def from_config(cls, config: dict):
    """สร้างระบบทั้งหมดจาก config dict"""
    strategy = StrategyFactory.create(config["strategy"])
    portfolio = Portfolio(config.get("capital", 100_000))
    risk_mgr = RiskManager(
        max_position_pct=config.get("max_position", 0.20),
        max_drawdown_pct=config.get("max_drawdown", 0.10)
    )
    return cls(strategy, portfolio, risk_mgr)

def step(self, symbol, price, data):
    """ประมวลผล 1 timestep"""
    sig = self.strategy.signal(data)
    self.bus.publish("signal", {"signal": sig, "price": price, "symbol": symbol})

def report(self):
    m = self.portfolio.metrics
    print(f"\n=== Final Report: {self.strategy.name} ===")
    for k, v in m.items():

```

```

if isinstance(v, float):
    print(f" {k:12s}: {v:.2%}" if "rate" in k else f" {k:12s}: {v:+,.0f}")
else:
    print(f" {k:12s}: {v}")

```

21.4 ทดลองใช้ระบบทั้งหมด

```

# Config ทั้งระบบในที่เดียว
SYSTEM_CONFIG = {
    "capital": 1_000_000,
    "max_position": 0.15,
    "max_drawdown": 0.10,
    "strategy": {
        "type": "pair",
        "name": "BTC Pair Production",
        "symbol_a": "Bybit",
        "symbol_b": "Binance",
        "window": 60,
        "entry_z": 2.0,
        "capital": 1_000_000
    }
}

# สร้างระบบ
system = TradingSystem.from_config(SYSTEM_CONFIG)

# จำลองข้อมูลและวิ่ง
import numpy as np, pandas as pd
import statsmodels.api as sm # PairStrategy ข้างในใช้ OLS
from statsmodels.regression.rolling import RollingOLS

np.random.seed(0)
n = 300 # train 60 วัน + live 240 วัน
dates = pd.date_range("2024-01-01", periods=n)
bybit = 65000 + np.cumsum(np.random.normal(0, 80, n))

# spread ต้องดึงกลับเข้าค่ากลาง (OU) ไม่ใช่ noise ลอย ๆ
# ถ้าเป็น noise กลยุทธ์จะไม่มีอะไรให้จับ = แทบไม่เกิดเทรดเลย
kappa, sigma_ou = 0.15, 45.0
ou = np.zeros(n)
for t in range(1, n):
    ou[t] = ou[t-1] + kappa * (0 - ou[t-1]) + np.random.normal(0, sigma_ou)

binance = bybit + ou # ติดกับ bybit + spread ที่ mean-revert
df = pd.DataFrame({"Bybit": bybit, "Binance": binance}, index=dates)

```

```
# Train ครั้งแรก
system.strategy.fit(df.iloc[:60])

# Live simulation (240 steps)
REFIT_EVERY = 20 # refit ทุก 20 วัน

print(f"=== Simulating {n-60} days ===")
for i in range(60, n):
    window = df.iloc[max(0, i-60):i]

    # ความสัมพันธ์ระหว่างสองตลาดเปลี่ยนไปเรื่อย ๆ — ต้อง refit เป็นรอบ
    if i > 60 and (i - 60) % REFIT_EVERY == 0:
        system.strategy.fit(window)

    price = df["Bybit"].iloc[i]
    system.step("Bybit", price, window)

system.report()
```

► OUTPUT

```
=== Simulating 240 days ===
[OPEN] LONG Bybit size=1.5391 @ 64,971
[CLOSE] Bybit PnL=-200
[OPEN] LONG Bybit size=1.5429 @ 64,800
[CLOSE] Bybit PnL=-337
...
[CLOSE] Bybit PnL=-3

=== Final Report: BTC Pair Production ===
trades      : 6
win_rate    : 0.00%
total_pnl   : -860
avg_win     : 0
avg_loss    : -143
```

🔴**อ่านว่า:** สังเกตบรรทัด `if i > 60 and (i - 60) % REFIT_EVERY == 0:`

`system.strategy.fit(window)` — เราเรียก `fit()` ซ้ำกลางทางได้เลย เพราะออกแบบให้ `fit()` (เรียนความสัมพันธ์) แยกจาก `signal()` (ให้คะแนนแท่งล่าสุด) · ถ้าเขียนแบบเอา z-score ทั้งชุดไป cache ไว้ใน `fit()` การ refit กลางทางจะยุ่งกว่านี้มาก — **นี่คือผลตอบแทนของการแยกหน้าที่ให้ชัด** ที่ Part นี้พยายามสอน

⚠️ **อ่านตัวเลข PnL ของ mini-project นี้อย่างไร — อย่าตีความว่ากลยุทธ์ดีหรือแย่**

ดู log จะเห็นว่าทุกเทรดเป็น LONG Bybit — **ขาดเดียว** · เพราะ `system.step("Bybit", price, window)` ส่งเข้าไปแค่ symbol เดียวกับราคาเดียว ดังนั้น **Portfolio** จึงเปิดสถานะใน Bybit เท่านั้น ไม่ได้ short Binance คู่กัน

ผลคือ กำไร/ขาดทุนที่เห็นมาจากทิศทางราคา Bybit ไม่ใช่จากการบีบตัวของ spread — signal มาจาก spread แต่การ execute เป็น directional bet · ตัวเลข `total_pnl` จึงเป็นสัญญาณรบกวนล้วน ๆ **ไม่ว่าจะออกมาบวกหรือลบก็ไม่ได้บอกอะไรเกี่ยวกับคุณภาพของกลยุทธ์**

🎯 สิ่งที่ mini-project นี้พิสูจน์สำเร็จคือ "โครงสร้าง" ไม่ใช่ "ผลตอบแทน": config → Factory สร้าง strategy ได้ · EventBus ส่งต่อ signal ถึง risk manager และ executor ครบวงจร · Portfolio บันทึกทุกเทรดและสรุป metrics ได้ · เพิ่ม strategy ใหม่ไม่ต้องแก้ระบบ

🔧 โจทย์ต่อยอด (ตรงนี้คือก้าวถัดไปของจริง): ทำให้ Portfolio รับสถานะ สองขา ได้ — `step()` ต้องรับราคาทั้งคู่, การเปิดสถานะต้อง long ขาหนึ่ง short อีกขาด้วยอัตราส่วน `beta` ที่ `fit()` คำนวณไว้ และ PnL ต้องรวมสองขา · ทำได้แล้วตัวเลขในรายงานจะเริ่มมีความหมาย

จบ Part III — สิ่งที่ได้จาก OOP

ก่อน OOP (Part II): functions กระจาย, เพิ่ม strategy ใหม่ = แก้หลายไฟล์

หลัง OOP (Part III): เพิ่ม strategy ใหม่ = สร้าง class ใหม่ที่ implement ABC → ทำงานกับ Backtester และ TradingSystem ทันที

- Strategy ABC บังคับ interface ✓
- Portfolio เก็บ state ครบ ✓
- EventBus แยก concerns ✓
- StrategyFactory + config สร้างระบบจาก JSON ✓
- TradingSystem ประกอบทุกอย่างผ่านจุดเดียว ✓

หมายเหตุเรื่องประสิทธิภาพ: Backtester ใน Part III ใช้ Python for-loop ซึ่งช้ากว่า vectorized ใน Part IV ประมาณ 50–100x ใช้เพื่อเรียนรู้ logic ได้ แต่สำหรับ parameter search ให้ใช้ vectorized ใน Part IV เท่านั้น

ใน Part IV เราจะนำ system นี้ไป backtest อย่างจริงจัง: walk-forward, slippage, commission, overfitting

■ [สารบัญทั้งเล่ม](#) · [คำศัพท์ Quant Trading](#)

Backtesting — ทดสอบก่อนใช้เงินจริง

บทที่ 22–25: Vectorized · Event-Driven · Walk-Forward · Slippage & Fees

จบ Part นี้ → รู้ว่า strategy ที่ดูดีบนกระดาษ จะรอดในชีวิตจริงไหม

🔗 อ่าน Part นี้ยังไ

● **มาถึงตรงนี้ได้แล้ว** — Part นี้สำคัญกว่าที่คิด: มันคือส่วนที่ตัดสินว่ากลยุทธ์ของคุณจริงหรือหลอกตัวเอง · ถ้าเวลาน้อยให้อ่าน **22.1 (lookahead bias)** กับ **บทที่ 25 (ต้นทุน)** ให้เข้าใจก่อน — สองเรื่องนี้ทำให้ backtest ส่วนใหญ่ในอินเทอร์เน็ตใช้ไม่ได้ · และดูตัวเลขติดลบในบทนี้ให้ดี มันตั้งใจให้ติดลบ

● **เคยเขียน backtest มาแล้ว** — ข้ามโครงสร้างพื้นฐานได้ · ที่ควรอ่าน: **22.1b (full-sample statistics)** ซึ่งเป็น lookahead ที่แนบเนียนกว่าการลิม shift(1) มาก · **บทที่ 24** ที่การแบ่ง train/test ผิดทำให้ "out-of-sample" ไม่ใช่ out-of-sample จริง · และตาราง cost sensitivity ในบทที่ 25 ที่ Sharpe ไหลจาก 2.67 → 0.21 ด้วยกลยุทธ์เดียวกันเป๊ะ โดย**ไม่ได้**กลับสักรบรทัด

ทำไม BACKTESTING ถึงสำคัญมาก?

ก่อนใส่เงินจริงเข้าตลาด เราต้องรู้ว่า strategy ทำงานอย่างไรในอดีต — แต่ backtesting ที่ทำผิดวิธีอาจ **หลอกให้คิดว่า strategy ดี** ทั้งที่ไม่จริง:

- **Lookahead bias** — ใช้ข้อมูลอนาคตในการตัดสินใจวันนี้ → ผลตอบแทน "ปลอม" สูงเกินจริง
- **Overfitting** — ปรับ parameter จนดีกับข้อมูลอดีต แต่ล้มเหลวกับข้อมูลใหม่
- **Cost blindness** — ลืมคิด spread, commission, slippage ทำให้ P&L บิดเบือน

Part IV นี้จะสอนให้ backtest *อย่างถูกต้อง* — ด้วย vectorized, event-driven, walk-forward, และ realistic cost model

Running Example ตลอด Part IV — BTC Bybit/Binance Pair Strategy

เราจะต่อยอด PairStrategy และ TradingSystem จาก Part II/III มาทดสอบอย่างเป็นระบบ:

บท 22 → Vectorized backtest เร็ว · บท 23 → Event-driven สมจริง

บท 24 → Walk-forward ป้องกัน overfit · บท 25 → ใส่ต้นทุนจริง และดูว่า edge ยังอยู่ไหม

บทที่ 22: Vectorized Backtest

แก่นของบทนี้

Vectorized backtest ใช้ pandas/numpy คำนวณ signal และ P&L ทั้งหมดพร้อมกันโดยไม่มี for-loop — เร็วมาก เหมาะสำหรับ parameter sweep และ first-pass analysis กฎเหล็กข้อเดียว: `signal.shift(1)` ก่อนคุณ return เสมอ เพื่อหลีกเลี่ยง lookahead bias

แบบทดสอบก่อนเริ่ม — Lookahead Bias คืออะไร?

สมมติว่าคุณเขียนโค้ดแบบนี้:

```
signal = (zscore > 2.0).astype(int)
ret_strategy = signal * daily_return
```

? โค้ดนี้มี lookahead bias หรือเปล่า?

คำตอบ: มี — `signal[t]` คำนวณจาก zscore วัน `t` แต่ `daily_return[t]` = ราคาปิดวัน `t` / ราคาปิดวัน `t-1` ซึ่งคุณยังไม่รู้ตอน `t` เริ่มต้น

แก้ด้วย: `ret_strategy = signal.shift(1) * daily_return`

กฎเหล็ก: signal วัน `t` → execute วัน `t+1` เสมอ

22.1 Lookahead Bias — ศัตรูตัวร้ายของ Backtesting

Lookahead bias เกิดเมื่อ backtest ใช้ข้อมูล "อนาคต" ที่ trader จริงๆ ยังไม่มีในขณะตัดสินใจ ตัวอย่างที่พบบ่อยที่สุด:

```
# ตัวอย่างข้อมูลจำลอง BTC Bybit/Binance
import numpy as np
import pandas as pd

np.random.seed(42)
n = 500
dates = pd.date_range("2023-01-01", periods=n, freq="D")
bybit = 25000 + np.cumsum(np.random.normal(20, 300, n))

# ส่วนต่างสองตลาดต้อง "ติงกลับ" ได้ ไม่ใช่สุ่มลอย ๆ — ใช้ OU เหมือนบทที่ 12
# ถ้าใช้ noise ใดๆ spread จะไม่มีแรงติงกลับ = กลยุทธ์นี้ไม่มี edge ตั้งแต่ต้น
kappa, sigma_ou = 0.15, 140.0
ou = np.zeros(n)
for t in range(1, n):
    ou[t] = ou[t-1] + kappa * (0 - ou[t-1]) + np.random.normal(0, sigma_ou)

binance = bybit + ou # ราคาใกล้เคียง Bybit + spread ที่ mean-revert

df = pd.DataFrame({"bybit": bybit, "binance": binance}, index=dates)

# คำนวณ z-score ของ spread
WINDOW = 60
spread = np.log(df["bybit"]) - np.log(df["binance"])
roll_mean = spread.rolling(WINDOW).mean()
roll_std = spread.rolling(WINDOW).std()
zscore = (spread - roll_mean) / roll_std

ENTRY_Z, EXIT_Z = 2.0, 0.5

# Raw signal (ยังไม่ lag)
raw_signal = pd.Series(0.0, index=df.index)
raw_signal[zscore > ENTRY_Z] = -1.0 # short spread
raw_signal[zscore < -ENTRY_Z] = 1.0 # long spread
# forward-fill: ถือ position ต่อกันว่า signal จะเปลี่ยน
raw_signal = raw_signal.replace(0, np.nan).ffill().fillna(0)

#
```

```
# คำนวณ daily return ของ spread
spread_return = df["bybit"].pct_change() - df["binance"].pct_change()

# ❌ ผิด - Lookahead Bias: signal วัน t คุณ return วัน t
# ราคา close ที่คำนวณ signal กับราคาที่ใช้ execute เป็นวันเดียวกัน
bad_pnl = (raw_signal * spread_return).dropna()

# ✅ ถูก - shift(1): signal วัน t execute ที่ราคาวัน t+1
good_signal = raw_signal.shift(1).fillna(0)
good_pnl = (good_signal * spread_return).dropna()

print(f"Same-bar (ไม่ shift) cumulative return: {(1+bad_pnl).prod()-1:+.2%}")
print(f"Shift(1) (ถูกต้อง) cumulative return: {(1+good_pnl).prod()-1:+.2%}")
print(f"ส่วนต่าง: {(1+bad_pnl).prod()-(1+good_pnl).prod():+.2%}")
```

► OUTPUT

```
Same-bar (ไม่ shift) cumulative return: +7.11%
Shift(1) (ถูกต้อง) cumulative return: +17.65%
ส่วนต่าง: -10.55%
```

😞 เอ๊ะ — ทำไมเวอร์ชัน "ผิด" กลับได้น้อยกว่า?

ผลออกมาสวนความคาดหมาย และนี่คือบทเรียนที่สำคัญกว่าตัวเลข · เหตุผล: signal ของเราเป็น **mean-reversion** — `raw_signal[t]` ถูกจุดชนวนโดยการเคลื่อนไหวที่เพิ่งเกิดขึ้นในวัน t เอง · พอเอาไปคูณ `spread_return[t]` ซึ่งคือการเคลื่อนไหวอันเดียวกันนั้น เราจึงกำลัง "เติมพินสวนทางหลังจากมันวิ่งไปแล้ว" = ขาดทุนที่แท่ง entry ทุกครั้ง

❗ สิ่งที่ต้องจำ: ไม่ใช่ทุก lookahead ที่ทำให้ผลดูดีขึ้น — แต่ทุก lookahead ทำให้ผล **ไม่จริง** · อย่าใช้ "ผลดูดีเกินไป" เป็นเครื่องตรวจจับ lookahead เพราะบางแบบทำให้ผลดูแย่ลง แล้วคุณก็จะถึงกลยุทธ์ที่ตีไปโดยไม่รู้ตัว · เครื่องตรวจจับที่เชื่อถือได้มีอย่างเดียว: **ไล่ดูทุกบรรทัดว่าข้อมูลที่ใช้นั้น เวลา t มีอยู่จริงตอน t หรือยัง**

22.1b Lookahead แบบที่มองด้วยตาแทบไม่เห็น — full-sample statistics

แบบที่ซ่อนตัวเก่งกว่ามาก เพราะมันไม่ได้อยู่ในบรรทัดที่เกี่ยวกับ *เวลา* เลย: ค่า mean/std ของ z-score จาก **ข้อมูลทั้งหมด** แทนที่จะเป็น rolling window · โค้ดสั้นกว่า อ่านสะอาดกว่า และไม่มีอะไรในหน้าตาของมันบอกว่าคุณผิด

```
# ❌ ผิด - ใช้ mean/std ของทั้งชุด: วันแรกของ backtest "รู้" ค่าเฉลี่ยของทั้งปีแล้ว
z_full = (spread - spread.mean()) / spread.std()

# ✅ ถูก - rolling: ใช้เฉพาะข้อมูลที่มีจริง ณ เวลานั้น
z_roll = (spread - spread.rolling(WINDOW).mean()) / spread.rolling(WINDOW).std()

def make_signal(z):
    # NaN = "ไม่มีเหตุการณ์" · ตัวเลข = "มีเหตุการณ์" (เข้า หรือ ออก)
    s = pd.Series(np.nan, index=df.index)
    s[z > ENTRY_Z] = -1.0
    s[z < -ENTRY_Z] = 1.0
    s[z.abs() < EXIT_Z] = 0.0 # exit ก็เป็นเหตุการณ์เหมือนกัน
    return s.ffill().fillna(0).shift(1).fillna(0)

pnl_full = (make_signal(z_full) * spread_return).dropna()
pnl_roll = (make_signal(z_roll) * spread_return).dropna()

print(f"❌ full-sample z: {(1+pnl_full).prod()-1:+.2%}")
print(f"✅ rolling z: {(1+pnl_roll).prod()-1:+.2%}")
print(f"ghost profit: {(1+pnl_full).prod()-(1+pnl_roll).prod():+.2%}")
```

► OUTPUT

```
❌ full-sample z: +19.35%
✅ rolling z: +19.78%
ghost profit: -0.43%
```

💡 **อ่านว่า:** `spread.mean()` ดูไร้พิษภัยมาก — แต่มันคือค่าเฉลี่ยของทั้ง **500 วัน** · การใช้มันตัดสินใจในวันที่ 61 เท่ากับบอกว่า "ตอนนั้นฉันรู้แล้วว่าอีก 439 วันข้างหน้าราคาเฉลี่ยจะเป็นเท่าไร" · ต่างจาก `.rolling(60).mean()` ที่มีย้อนหลังแค่ 60 วันที่ผ่านมาแล้วจริง ๆ

😞 อ้าว — ตัวเลขบอกว่ารู้แล้วได้น้อยลง 0.43%

ถ้าคุณกำลังรอให้ตัวเลข "❌" สูงกว่า "✅" เพื่อจับผิด — มันไม่มาแบบนั้น · และนี่คือเหตุผลที่หัวข้อนี้อยู่

ข้อมูลชุดเดียวบออะไรไม่ได้เลย เพราะส่วนต่างนี้เป็นเรื่องของโชคในชุดข้อมูลนั้น · วิธีเดียวที่จะรู้ว่าการวิ่งส่งผลอย่างไร คือทำซ้ำหลาย ๆ ชุดแล้วดูการกระจาย ไม่ใช่ดูค่าเดียว

ลองสร้างข้อมูลแบบเดียวกัน 300 ชุด (คนละเมล็ดสุ่ม) แล้ววัดส่วนต่างทุกชุด:

```
# ทำซ้ำ 300 ชุด — ดูว่า "การวิ่ง" ส่งผลอย่างไรโดยรวม ไม่ใช่ชุดเดียว
def leak_gap(rng, n=500):
    # สร้างข้อมูล 1 ชุด แล้วคืน (ผลของ z แบบวิ่ง) ลบ (ผลของ z แบบถูก)
    bybit = 25000 + np.cumsum(rng.normal(20, 300, n))
    ou = np.zeros(n)
    for t in range(1, n):
        ou[t] = ou[t-1] + 0.15 * (0 - ou[t-1]) + rng.normal(0, 140.0)
    idx = pd.date_range("2023-01-01", periods=n, freq="D")
    d = pd.DataFrame({"bybit": bybit, "binance": bybit + ou}, index=idx)

    sp = np.log(d["bybit"]) - np.log(d["binance"])
    zf = (sp - sp.mean()) / sp.std() # วิ่ง
    zr = (sp - sp.rolling(WINDOW).mean()) / sp.rolling(WINDOW).std()
    r = d["bybit"].pct_change() - d["binance"].pct_change()

    def sig(z):
        s = pd.Series(np.nan, index=idx)
        s[z > ENTRY_Z] = -1.0
        s[z < -ENTRY_Z] = 1.0
        s[z.abs() < EXIT_Z] = 0.0
        return s.fillna(0).shift(1).fillna(0)

    return ((1 + (sig(zf) * r).dropna()).prod()
            - (1 + (sig(zr) * r).dropna()).prod())

rng = np.random.default_rng(0)
gaps = np.array([leak_gap(rng) for _ in range(300)])

print("ทดลอง 300 ชุดข้อมูล — ส่วนต่าง (z แบบวิ่ง ลบ z แบบถูก)")
print(f"เฉลี่ย : {gaps.mean():+.2%}")
print(f"ส่วนเบี่ยงเบน : {gaps.std():.2%}")
print(f"แบบวิ่งชนะ : {(gaps > 0).mean():.0%} ของชุดข้อมูล")
print(f"ช่วง 5%-95% : {np.percentile(gaps, 5):+.2%} "
      f"ถึง {np.percentile(gaps, 95):+.2%}")
```

► OUTPUT

```
ทดลอง 300 ชุดข้อมูล — ส่วนต่าง (z แบบวิ่ง ลบ z แบบถูก)
เฉลี่ย : +0.73%
ส่วนเบี่ยงเบน : 5.06%
แบบวิ่งชนะ : 55% ของชุดข้อมูล
ช่วง 5%-95% : -7.92% ถึง +9.05%
```

อ่านตัวเลขสีบรรทัดนี้ให้ขาด — มันเปลี่ยนวิธีคิดเรื่อง lookahead ทั้งหมด

ค่าเฉลี่ย +0.73% แทบไม่ใช่ประเด็น · ประเด็นอยู่ที่ ส่วนเบี่ยงเบน 5.06% และช่วง -7.92% ถึง +9.05%

แปลเป็นภาษาคน: การวิ่งแบบนี้ไม่ได้ทำให้ผลดีขึ้นอย่างเป็นระบบ — มันทำให้ผลมั่วอย่างเป็นระบบ · backtest ชุดใดชุดหนึ่งอาจเกินจริงไป 9% หรือต่ำกว่าจริง 8% และคุณไม่มีทางรู้ว่าชุดของคุณอยู่ตรงไหน เพราะคุณมีข้อมูลชุดเดียว

ถ้าจับ lookahead ด้วยวิธี "ดูว่าผลดีเกินจริงไหม" คุณจะพลาดทันที 45% ของกรณี (ชุดที่วิ่งแล้วผลออกมาแย่ลง) · และในชุดที่ผลออกมาดีขึ้น ก็ยังแยกไม่ออกอยู่ดีว่ามาจาก edge จริงหรือจากการวิ่ง

❌ ข้อสรุปที่ใช้ได้ตลอด: อย่าถามว่า "การวิ่งนี้ทำให้ผลดีขึ้นเท่าไร" — ให้ถามว่า "ผลนี้ทำซ้ำในชีวิตจริงได้ไหม" · `spread.mean()` ของทั้งชุดไม่มีอยู่จริงในวันที่ 61 ดังนั้น backtest ที่เชื่อมั่นจึงเป็นตัวเลขที่ไม่มีใครทำซ้ำได้ — จบตรงนั้น ไม่ต้องเถียงกันว่ามันบวกหรือลบ

● [รอบสอง] ทำไมการวิ่งแบบนี้ถึงไม่กำไรพออย่างที่คาด

สัญญาณบอกบอกว่า "รู้อากาศ = ได้เปรียบ" แต่ `spread.mean()` ของทั้งชุดไม่ใช่ข้อมูลอนาคตที่เอาไปทำเงินได้โดยตรง · มันเป็นแค่ค่าคงที่ตัวหนึ่งที่ไม่เปลี่ยนตำแหน่งเส้นแบ่ง entry/exit ทั้งเส้น

ผลคือกลยุทธ์เข้าเทรดคนละจุดกันไปเลยทั้งคู่ — ไม่ใช่ "เข้าจุดเดิมแต่ได้เปรียบขึ้น" . จุดใหม่นั้นบางชุดบังเอิญดีกว่า บางชุดบังเอิญแยกว่า จึงออกมาเป็น 55/45

เทียบกับการรู้ว่าให้กำไรจริง — เช่นเอาราคาพริงมีมาตัดสินใจวันนี้ (shift(-1)) . แบบนั้นกลยุทธ์จะชนะเกือบทุกชุดข้อมูล และ Sharpe จะพุ่งไปในระดับที่เป็นไปได้ . เส้นแบ่งอยู่ที่ว่าสิ่งที่เราออกมาบอก "ทิศทางของราคาข้างหน้า" หรือแค่ "บริบททางสถิติ"

สองอย่างนี้ต้องแก้เหมือนกัน แต่อาการต่างกันคนละขั้ว — และเพราะแบบหลังไม่มีอาการให้เห็น มันจึงรอดสายตาไปถึง production ได้บ่อยกว่ามาก

กฎเหล็ก 2 ข้อของ Vectorized Backtest

① shift(1) เสมอ — signal สร้างจากราคา close วันที่ t → execute ได้เร็วสุดที่ open วันที่ $t + 1$

② ทุกสถิติต้องเป็น rolling หรือ expanding เท่านั้น — เจอ .mean(), .std(), .min(), .max(), .quantile() ที่ไม่มี .rolling() หรือ .expanding() นำหน้า ให้สงสัยไว้ก่อนทุกครั้ง . รวมถึงการ normalize/scale ข้อมูลก่อนแบ่ง train-test ด้วย (เรื่องนี้ลงลึกใน Part V)

22.2 Vectorized Backtest ฉบับสมบูรณ์

📖อ่านว่า: ฟังก์ชันยาว แต่มีหมายเลขกำกับให้แล้วทั้ง 7 ขั้นตอน — และลำดับของมันไม่ใช่เรื่องบังเอิญ มันคือสายพานเดียวที่แปลงราคาเป็นเงิน: ราคา → ① spread/z-score → ② สัญญาณ → ③ หน่วง 1 แท่ง → ④ ผลตอบแทน → ⑤ หักค่าธรรมเนียม → ⑥ equity → ⑦ drawdown . ทุกขั้นเป็นคอลัมน์ทั้งคอลัมน์ ไม่มีลูปสักตัว นั่นคือความหมายของคำว่า vectorized . และเหตุผลที่มันเร็วกว่าการวนทีละแท่งเป็นร้อยเท่า คือ pandas ส่งงานทั้งคอลัมน์ลงไปที่โค้ดภาษา C คิดรวดเดียว

```
def vectorized_backtest(
    price_a: pd.Series,
    price_b: pd.Series,
    window: int = 60,
    entry_z: float = 2.0,
    exit_z: float = 0.5,
    fee_rate: float = 0.001,
) -> pd.DataFrame:
    """
    Vectorized pair trading backtest on price series A and B
    ส่งคืน DataFrame ที่มี: signal, net_return, equity, drawdown
    """
    df = pd.DataFrame({"A": price_a, "B": price_b})

    # — 1. Spread และ z-score —————
    log_spread = np.log(df["A"]) - np.log(df["B"])
    roll_mean = log_spread.rolling(window).mean()
    roll_std = log_spread.rolling(window).std()
    df["zscore"] = (log_spread - roll_mean) / roll_std

    # — 2. Raw signal (ยังไม่ shift) —————
    # NaN = "แท่งนี้ไม่มีอะไรเกิดขึ้น" → ffill จะถือ position เดิมต่อ
    # ตัวเลข = "มีเหตุการณ์" ซึ่งมี 3 แบบ: เข้า short . เข้า long . ออก
    raw = pd.Series(np.nan, index=df.index)
    raw[df["zscore"] > entry_z] = -1.0 # spread กว้างเกิน → short
    raw[df["zscore"] < -entry_z] = 1.0 # spread แคบเกิน → long
    raw[df["zscore"].abs() < exit_z] = 0.0 # กลับสู่ปกติ → ปิด
    # ⚠ ffill ต้องมา "หลัง" ทั้งเข้าและออกถูก mark ครบแล้วเท่านั้น
    raw = raw.ffill().fillna(0)

    # — 3. SHIFT(1) — หัวใจสำคัญ —————
    df["signal"] = raw.shift(1).fillna(0)

    # — 4. Returns —————
    ret_a = df["A"].pct_change()
    ret_b = df["B"].pct_change()
    # long spread = long A, short B → P&L = ret_A - ret_B
    gross_return = df["signal"] * (ret_a - ret_b)

    # — 5. Transaction cost (เมื่อ position เปลี่ยน) —————
    position_change = df["signal"].diff().abs() # 1 เมื่อ flip/exit
    cost_per_bar = position_change * fee_rate * 2 # ทั้ง A และ B
    df["net_return"] = gross_return - cost_per_bar
```

```
# — 6. Equity curve (normalized: เริ่มที่ 1.0) —————
df["equity"] = (1 + df["net_return"]).cumprod()

# — 7. Drawdown —————
peak = df["equity"].cummax()
df["drawdown"] = (df["equity"] - peak) / peak

return df.dropna()

# รัน backtest กับข้อมูล BTC จำลอง
result = vectorized_backtest(
    price_a = pd.Series(bybit, index=dates, name="Bybit"),
    price_b = pd.Series(binance, index=dates, name="Binance"),
    window=60, entry_z=2.0, exit_z=0.5, fee_rate=0.001,
)

print(result[["zscore", "signal", "net_return", "equity", "drawdown"]].tail(6).round(5))
```

► OUTPUT

```
zscore  signal  net_return  equity  drawdown
2024-05-09  0.95021    0.0      0.0  1.13252  -0.002
2024-05-10  1.24011    0.0      0.0  1.13252  -0.002
2024-05-11  1.15657    0.0     -0.0  1.13252  -0.002
2024-05-12  0.11836    0.0     -0.0  1.13252  -0.002
2024-05-13 -0.21545    0.0     -0.0  1.13252  -0.002
2024-05-14  0.07878    0.0      0.0  1.13252  -0.002
```

🧠 3 บรรทัดที่มือใหม่มักเขียนผิด และวิธีอ่านให้ถูก

```
raw = pd.Series(np.nan, index=df.index)  ❶ NaN = "ไม่มีเหตุการณ์"
raw[z > entry_z] = -1.0
raw[z < -entry_z] = 1.0
raw[z.abs() < exit_z] = 0.0              exit ก็เป็นเหตุการณ์
raw = raw.ffill().fillna(0)              ffill ที่เดียวตอนท้าย

position_change = df["signal"].diff().abs()  ❷ นับจังหวะที่ต้องจ่าย fee
cost_per_bar = position_change * fee_rate * 2

df["equity"] = (1 + df["net_return"]).cumprod()  ❸ ทบต้น ไม่ใช่บวกสะสม
```

1. ทำไมต้องเริ่มด้วย `np.nan` ไม่ใช่ `0.0` — เพราะ `ffill` แปลว่า "ช่องว่างให้เอาค่าก่อนหน้ามาเติม" · ถ้าเริ่มด้วย 0 แล้วค่อยแปลง 0 เป็น NaN ที่หลัง จะเกิดปัญหาว่า 0 ที่แปลว่า "ปิดสถานะ" กับ 0 ที่แปลว่า "ไม่มีอะไรเกิดขึ้น" หน้าตาเหมือนกันจนแยกไม่ออก · การเริ่มจาก NaN ทำให้สองอย่างนี้ ต่างกันตั้งแต่แรก: NaN = เจียบ · 0 = สั่งปิด
2. `diff().abs()` คือจำนวน "ขา" ที่ต้องซื้อขาย — จาก 0→1 ได้ 1 · จาก 1→-1 ได้ 2 (ปิดของเดิมแล้วเปิดฝั่งตรงข้าม จ่ายสองต่อ) · คูณ * 2 ท้ายสุด เพราะ pair trade ขยับสองเหรียญพร้อมกันเสมอ · จับให้ได้ว่า `fee_rate * 2` คือ "สองเหรียญ" ส่วน `diff().abs()` คือ "กี่ครั้ง" — คนละมิติกัน
3. `(1 + r).cumprod()` ไม่ใช่ `r.cumsum()` — เงินทบต้นไม่ได้บวกกันตรง ๆ · ได้ 10% แล้วเสีย 10% ไม่ได้กลับมาที่เดิม แต่เหลือ 0.99 · ในบทที่ 30 จะเห็นอีกสำนวนคือ `np.exp(log_ret.cumsum())` ซึ่งให้ผลเดียวกันเมื่อใช้ผลตอบแทนแบบ log — เลือกอันไหนก็ได้ แต่ห้ามผสมกัน

บรรทัด `return df.dropna()` ตัดแถวแรก ๆ ที่ z-score ยังเป็น NaN ทั้ง (60 แถวแรก) · ดูเล็กน้อยแต่สำคัญ — ถ้าไม่ตัด `equity` จะเริ่มนับจากช่วงที่ยังไม่มีสัญญาณ แล้ว `annualized return` จะถูกเจือจางด้วยวันที่ไม่ได้เทรด

22.3 Performance Metrics: Sharpe, Calmar, Max Drawdown

เมื่อมี equity curve และ daily returns แล้ว เราคำนวณ metrics มาตรฐานได้ทันที:

```
def compute_metrics(result: pd.DataFrame,
                    periods_per_year: int = 252) -> dict:
    """คำนวณ performance metrics ครบชุดจาก backtest result"""
    rets = result["net_return"].dropna()
    eq = result["equity"].dropna()
    dd = result["drawdown"].dropna()

    # Annualized return (compound)
    total_return = eq.iloc[-1] - 1.0
```



```

n_years      = len(rets) / periods_per_year
ann_return   = (1 + total_return) ** (1 / max(n_years, 1e-9)) - 1

# Annualized volatility
ann_vol = rets.std() * np.sqrt(periods_per_year)

# Sharpe Ratio (risk-free rate = 0 สำหรับ crypto)
sharpe = ann_return / ann_vol if ann_vol > 0 else 0.0

# Max Drawdown
max_dd = float(dd.min())

# Calmar Ratio = annualized return / |max drawdown|
calmar = ann_return / abs(max_dd) if max_dd != 0 else 0.0

# Win Rate (บน active days เท่านั้น)
active     = rets[rets != 0]
win_rate = float((active > 0).mean()) if len(active) > 0 else 0.0

# Profit Factor = gross profit / gross loss
wins = active[active > 0].sum()
losses = abs(active[active < 0].sum())
profit_factor = wins / losses if losses > 0 else float("inf")

# Average trade duration (consecutive non-zero positions)
pos = result["signal"]
in_trade = (pos != 0).astype(int)
avg_duration = (in_trade.groupby(
    (in_trade != in_trade.shift()).cumsum()
).sum().replace(0, np.nan).dropna().mean())

return {
    "Total Return (%)"      : round(total_return * 100, 2),
    "Ann. Return (%)"      : round(ann_return * 100, 2),
    "Ann. Volatility (%)"  : round(ann_vol * 100, 2),
    "Sharpe Ratio"         : round(sharpe, 3),
    "Max Drawdown (%)"    : round(max_dd * 100, 2),
    "Calmar Ratio"         : round(calmar, 3),
    "Win Rate (%)"         : round(win_rate * 100, 1),
    "Profit Factor"         : round(profit_factor, 3),
    "Avg Trade Duration"   : round(float(avg_duration), 1),
}

m = compute_metrics(result)
print("-" * 44)
for k, v in m.items():
    print(f" {k:24s}: {v}")

```

► OUTPUT

```

=====
Total Return (%)      : 13.25
Ann. Return (%)      : 7.37
Ann. Volatility (%)  : 4.15
Sharpe Ratio         : 1.775
Max Drawdown (%)    : -2.24
Calmar Ratio         : 3.296
Win Rate (%)         : 52.9
Profit Factor         : 1.72
Avg Trade Duration   : 7.6

```

ตีความตัวเลข

Sharpe **1.775** · Calmar **3.296** · Profit Factor **1.72** — ตัวเลขสวยมาก และนั่นคือสิ่งที่ต้องระวังที่สุด

สิ่งที่ตัวเลขชุดนี้พิสูจน์คือ **โค้ด backtest ทำงานถูกต้อง** — ไม่ใช่ว่ากลยุทธ์นี้ใช้ได้จริง · เหตุผลอยู่ที่ข้อมูล: เราสร้าง spread ด้วยกระบวนการ OU ที่มีแรงดึงกลับอยู่ในตัวโดยนิยาม (ดูบล็อกสร้างข้อมูลต้นบท) · เอาจกลยุทธ์ mean-reversion ไปวิ่งบนข้อมูลที่ mean-revert จริง แล้วได้กำไร — นั่นคือ**การตรวจว่าเครื่องมือวัดไม่เสีย**ไม่ใช่การค้นพบ

$$\frac{R_{\text{ann}} - R_f}{\sigma_{\text{ann}}} \approx \text{Calmar} =$$

$$\frac{R_{\text{ann}}}{\text{MaxDD}}$$

คำถามที่ต้องถามต่อจึงไม่ใช่ "Sharpe พอไหม" แต่คือ "ตลาดจริงมีแรงดึงกลับแบบที่เราใส่ลงไปหรือเปล่า และมันอยู่ได้นานแค่ไหน" — บทที่ 24 (Walk-Forward) มีไว้ตอบข้อนี้

● [รอบสอง] Sharpe 1.775 บนข้อมูลจำลอง แปลว่าอะไรกันแน่

ลำดับของเหตุผลตรงนี้สำคัญว่าตัวเลขทุกตัวในบท · เราเป็นคนใส่ edge ลงไปในข้อมูลเอง ด้วยบรรทัด `ou[t] = ou[t-1] + kappa * (0 - ou[t-1]) + ... · kappa = 0.15` คือแรงดึงกลับที่เรากำหนด · ถ้ากลยุทธ์ไม่ทำกำไรบนข้อมูลแบบนี้ แปลว่ามีบีกในโค้ด

ดังนั้นบทบาทที่ถูกต้องของตัวเลขชุดนี้คือ การทดสอบเชิงบวก (positive control) — เหมือนที่ห้องแล็บใส่ตัวอย่างที่รู้ผลอยู่แล้วเข้าเครื่องเพื่อดูว่าเครื่องอ่านค่าได้ · มันตอบว่า "backtest นี้จับ edge ที่มีอยู่จริงได้" ไม่ได้ตอบว่า "มี edge ในตลาดจริง"

✗ **คู่ที่ควรทำเสมอ:** รัน backtest ตัวเดียวกันบนข้อมูลที่ไม่มี edge (เช่น random walk ล้วน ไม่มี OU) ด้วย · ถ้ามันยังกำไรอยู่ แปลว่าโค้ดรั่วที่ไหนสักแห่ง · การมีทั้ง positive control และ negative control คือความต่างระหว่าง "รันแล้วได้เลข" กับ "รู้ตัวเลขนั้นเชื่อได้"

⚠ กับดักที่ตามมาทันที: พอเห็น Sharpe 1.775 สัญชาตญาณคือ "ลองปรับ window กับ entry_z ให้ดีกว่านี้อีก" — ระวังให้ดี · การไล่จนพาราเมตอร์บนชุดข้อมูลเดียวกันจนตัวเลขสวยขึ้นเรื่อย ๆ คือ **overfit** ที่แนบเนียนที่สุด และบนข้อมูลจำลองที่เรารู้คำตอบอยู่แล้ว มันยิ่งไร้ความหมาย · บทที่ 24 มีไว้บอกว่าค่าที่ดูมานั้นจริงหรือแค่ฟลุค

22.4 Equity Curve และ Drawdown

```
def summarize_equity(result: pd.DataFrame) -> None:
    """แสดง equity curve summary แบบ text"""
    eq = result["equity"]
    dd = result["drawdown"] * 100

    # Monthly returns
    monthly = (1 + result["net_return"]).resample("ME").prod() - 1
    print("Monthly Returns (%):")
    print(" " + " " .join(
        f"{d.strftime('%b%y')}: {r*100:+5.1f}%"
        for d, r in monthly.items()
    )[:120])

    print(f"\nEquity: start={eq.iloc[0]:.4f} "
          f"peak={eq.max():.4f} "
          f"end={eq.iloc[-1]:.4f}")
    print(f"Drawdown: worst={dd.min():.2f}% "
          f"avg_when_down={dd[dd<0].mean():.2f}%")

    # Time spent in drawdown
    pct_in_dd = (dd < -1).mean() * 100
    print(f"Time in drawdown (>1%): {pct_in_dd:.1f}% of trading days")

summarize_equity(result)
```

► OUTPUT

Monthly Returns (%):

Mar23: +2.2% Apr23: +1.7% May23: +0.6% Jun23: +1.1% Jul23: +0.0% Aug23: +1.2% Sep23: +0.7% Oct23: +0.8% Nov23: +

Equity: start=1.0000 peak=1.1348 end=1.1325

Drawdown: worst=-2.24% avg_when_down=-0.26%

Time in drawdown (>1%): 1.4% of trading days

ตัวอย่าง: Parameter Sweep แบบ Vectorized

ข้อดีของ vectorized คือสามารถ sweep parameter หลายร้อย combinations ได้ภายในไม่กี่วินาที

```
def parameter_sweep(price_a, price_b,
                    windows, entry_zs, fee_rate=0.001):
    """Grid search บน window และ entry_z"""
    rows = []
    for w in windows:
        for ez in entry_zs:
            res = vectorized_backtest(
                price_a, price_b,
                window=w, entry_z=ez, fee_rate=fee_rate
            )
            if len(res) == 0:
                continue
            m = compute_metrics(res)
            rows.append({
                "window": w,
                "entry_z": ez,
                "sharpe": m["Sharpe Ratio"],
                "calmar": m["Calmar Ratio"],
                "max_dd": m["Max Drawdown (%)"],
                "n_active": int((res["signal"] != 0).sum()),
            })
    df_out = pd.DataFrame(rows)
    return df_out.sort_values("sharpe", ascending=False).reset_index(drop=True)

sweep = parameter_sweep(
    pd.Series(bybit, index=dates),
    pd.Series(binance, index=dates),
    windows = [30, 45, 60, 90, 120],
    entry_zs = [1.5, 2.0, 2.5, 3.0],
)
print(sweep.head(8).to_string(index=False))
```

► OUTPUT

window	entry_z	sharpe	calmar	max_dd	n_active
120	2.0	2.276	7.497	-1.07	71
120	1.5	2.075	4.401	-2.05	112
90	2.0	2.074	3.887	-2.20	94
45	2.5	1.947	3.765	-1.44	51
90	2.5	1.884	6.043	-0.75	23
60	1.5	1.811	3.724	-2.24	144
60	2.0	1.775	3.296	-2.24	107
60	2.5	1.593	2.208	-2.20	35

► แบบฝึกหัด: เพิ่ม Sortino Ratio ใน compute_metrics

บทที่ 23: Event-Driven Backtest

แก่นของบทนี้

Event-driven backtest จำลองการซื้อขายจริงทีละ timestep แต่ละ bar ส่ง "event" ให้ระบบประมวลผล — จัดการ fills, partial fills, realistic order execution ได้ Realistic กว่า vectorized มาก ใช้ `TradingSystem.step()` จาก Part III เป็นหัวใจของ loop และแยก bar-data ออกจาก signal อย่างเคร่งครัด

23.1 ความต่างระหว่าง Vectorized และ Event-Driven

คุณสมบัติ	Vectorized	Event-Driven
ความเร็ว	เร็วมาก (numpy)	ช้ากว่า 10–100x
ความสมจริง	ต่ำ — ไม่มี partial fill	สูง — จำลองได้ละเอียด
Lookahead bias	ต้องระวัง shift(1) เอง	ป้องกันโดยโครงสร้าง loop
Order types	Market order เท่านั้น	Limit, Stop, IOC ได้
Portfolio state	ยุ่งยากถ้า multi-asset	จัดการง่ายกว่า
ใช้เมื่อ	Parameter sweep เบื้องต้น	Final validation ก่อน live

23.2 Data Structures สำหรับ Event System

📖อ่านว่า: บล็อกข้างล่างไม่มีตรรกะให้คิดตามเลยสักบรรทัด — มันคือการประกาศว่า "ในระบบนี้มีของอยู่ 5 ชนิด และแต่ละชนิดมีช่องอะไรบ้าง" · อ่านเหมือนอ่านสารบัญ: `BarData` = ราคา 1 แท่ง · `Signal` = สิ่งที่ถูกกลยุทธ์อยากทำ · `Fill` = สิ่งที่เกิดขึ้นจริงหลังส่งคำสั่ง · `PortfolioState` = สภาพพอร์ต ณ ขณะนั้น · ความต่างระหว่าง `Signal` กับ `Fill` คือหัวใจของทั้งบท — สิ่งที่ยากทำกับสิ่งที่ได้จริงไม่เท่ากัน และช่องว่างตรงนั้นคือที่ที่เงินหายไป

```
from dataclasses import dataclass, field
from typing import Optional, List
from enum import Enum

class OrderSide(Enum):
    LONG = "long"
    SHORT = "short"
    EXIT = "exit"

@dataclass
class BarData:
    """ข้อมูลราคาสำหรับ 1 timestep (1 bar)"""
    timestamp: pd.Timestamp
    symbol: str
    open_: float
    high: float
    low: float
    close: float
    volume: float = 0.0

@dataclass
class Signal:
    """Output จาก strategy.step()"""
    timestamp: pd.Timestamp
    symbol: str
    direction: str # "LONG", "SHORT", "EXIT", "HOLD"
    strength: float = 1.0

@dataclass
class Fill:
    """ผลการ execute order จริง"""
    timestamp: pd.Timestamp
    symbol: str
```

```

side:      str      # "long" หรือ "short"
quantity:  float
fill_price: float
commission: float
slippage:  float

@property
def total_cost(self) -> float:
    return self.commission + self.slippage

@dataclass
class PortfolioState:
    """State ของ portfolio ณ เวลานั้น"""
    cash:      float
    positions: dict = field(default_factory=dict) # symbol -> qty
    avg_prices: dict = field(default_factory=dict) # symbol -> avg cost
    realized_pnl: float = 0.0
    unrealized_pnl: float = 0.0

    def total_equity(self, current_prices: dict) -> float:
        unreal = sum(
            qty * current_prices.get(sym, self.avg_prices.get(sym, 0))
            for sym, qty in self.positions.items()
        )
        return self.cash + unreal

```

เขียนโค้ดให้เราฟรี ๆ ก็บรรทัด

```

@dataclass
class Signal:
    timestamp: pd.Timestamp
    symbol: str
    direction: str
    strength: float = 1.0

# ↑ เท่ากับเขียนเองแบบนี้ทั้งหมด:
# def __init__(self, timestamp, symbol, direction, strength=1.0): ...
# def __repr__(self): ...      - พิมพ์ออกมาแล้วอ่านรู้เรื่อง
# def __eq__(self, other): ... - เทียบ == แล้วดูค่าข้างใน ไม่ใช่ที่อยู่หน่วยความจำ

```

1. สิ่งที่ได้ฟรีที่มีค่าที่สุดคือ `__eq__` — object ธรรมดาสองตัวที่มีค่าเหมือนกันเป๊ะจะยังถือว่า *ไม่เท่ากัน* เพราะ Python เทียบที่อยู่หน่วยความจำ · พอเป็น dataclass มันเทียบค่าข้างในทีละช่อง ซึ่งทำให้เขียน test แบบ `assert got == Signal(...)` ได้ตรง ๆ
2. `open_`: float มีขีดล่างต่อท้ายเพราะ `open` เป็นชื่อฟังก์ชันของ Python เอง · ตั้งชื่อทับได้จริงและจะไม่ error ทันที แต่จะไปฟ้องใครสักคนเรียก `open()` เพื่อเปิดไฟล์ในไฟล์เดียวกัน · ขีดล่างต่อท้ายคือธรรมเนียมมาตรฐานสำหรับกรณีนี้ (เจอบ่อยกับ `class_`, `id_`, `type_`)
3. `field(default_factory=dict)` — จุดนี้สำคัญมาก อ่านให้ดี · เขียน `positions: dict = {}` ตรง ๆ ไม่ได้ เพราะค่าเริ่มต้นใน Python ถูกสร้างครั้งเดียวตอนนิยามคลาส ไม่ใช่ทุกครั้งที่สร้าง object · ผลคือทุก `PortfolioState` ที่สร้างขึ้นจะใช้ `dict` ก้อนเดียวกันร่วมกันทั้งหมด — พอร์ตหนึ่งเปิดสถานะ อีกพอร์ตก็เห็นด้วย · `default_factory=dict` แปลว่า "เรียก `dict()` ใหม่ทุกครั้งที่สร้าง" · dataclass ถึงกับโยน error ทันทีถ้าคุณใส่ `dict/list` เป็นค่าเริ่มต้นตรง ๆ เพราะกับดักนี้ทำคนตกกันมาเยอะเกินไป
4. `@property total_cost` ใน Fill — dataclass ก็ยังใส่ property เองได้ตามปกติ · `commission + slippage` คำนวณสดทุกครั้ง จึงไม่มีวันไม่ตรงกับสองช่องนั้น (แนวคิดเดียวกับ `Position.pnl` ในบทที่ 18)

● เทียบกับ Portfolio ในบทที่ 21 — คนละวิธีเก็บ "ทิศทาง"

สังเกต `PortfolioState.positions` ว่าเป็น `symbol -> qty` เลข ๆ ไม่มีช่อง `side` เลย · ทิศทางถูกเก็บไว้ในเครื่องหมายของจำนวน — long คือบวก short คือลบ

ผลที่ตามมาคือ `total_equity` เขียนได้สั้น ๆ ว่า `qty * price` แล้วถูกต้องทั้งสองฝั่งโดยอัตโนมัติ ไม่ต้องมี `if side == "long"` ที่ไหนเลย · เทียบกับ Portfolio ในบทที่ 21 ที่เก็บ `side` เป็นข้อความแยกต่างหาก แล้วต้องคูณ `sign` เองในทุกมรดกที่ขยับเงิน — สิมที่เดียวก็คิดบัญชี short กลับทิศทันที

🔗 บทเรียนที่ใหญ่กว่าเรื่อง short: ถ้าข้อมูลชิ้นหนึ่งต้องถูกตีความเหมือนกันทุกที่ที่ใช้ให้ออกแบบให้การตีความนั้นฝังอยู่ในตัวข้อมูลเอง อย่าปล่อยให้เป็นหน้าที่ของคนเขียนโค้ดที่ต้องจำให้ครบทุกจุด · ตัวเลขติดลบจำแทนเราได้ แต่ `if` ที่กระจายอยู่ 5 ที่ต้องอาศัยความจำของคน

23.3 Event-Driven Loop ที่ใช้ TradingSystem.step()

🧩 โครงก่อนดูโค้ด — 170 บรรทัด แต่มีวิธีอ่านที่ทำให้เหลือ 4 ก้อน

```
class EventDrivenBacktester:
    def __init__(...)          # ตั้งค่า + จองที่เก็บสถานะ

    def _compute_zscore(...)   1
    def _compute_zscore_from_prices(...) | ผู้ช่วย — เรียกจาก run() เท่านั้น
    def _generate_signal(...)   | (ขีดล่างน้ำหนัก = ของภายใน)
    def _execute_signal(...)    2

    def run(df_a, df_b)        # ... เริ่มอ่านที่นี่ ไม่ใช่บรรทัดแรก
        for แต่ละแท่ง:
            ① รับ MarketEvent      # เห็นราคาใหม่ 1 แท่ง
            ② อัปเดต signal         # คัด z-score → ตัดสินใจ
            ③ Execute signal        # แปลงการตัดสินใจเป็น Fill
            ④ Mark-to-market equity # ตีมูลค่าพอร์ต ณ ราคาปิด
```

- 1.อ่านจาก run() ก่อนเสมอ — อย่าไล่จากบรรทัดแรก · run() คือโครงเรื่อง ส่วนเมธอด _ ทั้งสี่คือรายละเอียดที่มันเรียกใช้ · ถ้าไล่จากบนลงล่าง คุณจะเจอรายละเอียดก่อนที่จะรู้ว่ามันถูกใช้ทำอะไร ซึ่งเป็นวิธีที่ทำให้หลงเร็วที่สุด
- 2.ทั้งบล็อกไม่มีแนวคิดใหม่เลยแม้แต่นิดเดียว — z-score (บทที่ 12) · signal จาก entry/exit (บทที่ 22) · dataclass (หัวข้อที่แล้ว) · คลาสและ self (Part III) · งานของบล็อกนี้คือ "ต่อสายให้ถูก" ไม่ใช่สอนของใหม่ ถ้าตรงไหนอ่านไม่ออก ให้กลับไปทบทวนเรื่อง ไม่ใช่ฝืนอ่านต่อ
- 3.ความต่างจาก vectorized ทั้งหมดอยู่ที่ loop for ตัวเดียว · แบบ vectorized คิดทั้งคอลัมน์รวดเดียว จึงต้องระวัง shift(1) เอง · แบบนี้เดินทีละแท่งและ ณ แท่งที่ t โค้ดมีข้อมูลถึง t เท่านั้นจริง ๆ — การมองอนาคตจึงถูกกันโดยโครงสร้าง ไม่ใช่โดยวินัยของคนเขียน · แลกมาด้วยความเร็วที่ช้ากว่าหลายสิบเท่า

ลำดับ ① → ④ ในลูปห้ามสลับ · โดยเฉพาะ ④ ต้องอยู่หลัง ③ — ตีมูลค่าพอร์ตก่อนที่คำสั่งของแท่งนั้นจะถูก execute เท่ากับรายงาน equity ของสถานะที่ยังไม่มีจริง

```
class EventDrivenBacktester:
    """
    Event-driven backtester ที่ใช้ TradingSystem จาก Part III
    ทุก bar → MarketEvent → Strategy.step() → Signal → Execution → Fill
    """

    def __init__(self,
                  window: int = 60,
                  entry_z: float = 2.0,
                  exit_z: float = 0.5,
                  fee_rate: float = 0.001,
                  slippage_bps: float = 5.0,
                  initial_capital: float = 1_000_000.0):
        self.window = window
        self.entry_z = entry_z
        self.exit_z = exit_z
        self.fee_rate = fee_rate
        self.slippage_bps = slippage_bps # basis points
        self.initial_capital = initial_capital

        # State
        self.cash = initial_capital
        self.position = 0.0 # +1 = long spread, -1 = short spread, 0 = flat
        self.entry_price = None
        self.bar_history: List[BarData] = []

        # Records
        self.fills: List[Fill] = []
        self.equity_curve: List[dict] = []

    def _compute_zscore(self) -> Optional[float]:
        """คำนวณ z-score จาก bar history ปัจจุบัน"""
        if len(self.bar_history) < self.window:
            return None
        recent = self.bar_history[-self.window:]
        spreads = [np.log(b.close / b.volume) if b.volume > 0
                   else 0.0 for b in recent]
```

```

# (volume ที่ใช้แทน price_b เพื่อความง่าย)
arr = np.array(spreads)
std = arr.std()
return float((arr[-1] - arr.mean()) / std) if std > 0 else 0.0

def _compute_zscore_from_prices(self,
                                closes_a: np.ndarray,
                                closes_b: np.ndarray) -> float:
    """คำนวณ z-score จาก price arrays สองตัว"""
    if len(closes_a) < self.window:
        return 0.0
    window_a = closes_a[-self.window:]
    window_b = closes_b[-self.window:]
    spread = np.log(window_a) - np.log(window_b)
    std = spread.std()
    return float((spread[-1] - spread.mean()) / std) if std > 0 else 0.0

def _generate_signal(self, z: float) -> str:
    """แปลง z-score เป็น trading signal"""
    if z > self.entry_z:
        return "SHORT" # spread กว้าง → short
    if z < -self.entry_z:
        return "LONG" # spread แคบ → long
    if abs(z) < self.exit_z and self.position != 0:
        return "EXIT" # mean-reverted → ออก
    return "HOLD"

def _execute_signal(self, sig: str, price_a: float,
                   price_b: float, ts: pd.Timestamp) -> Optional[Fill]:
    """Execute signal จริง - คำนวณ fill price + cost"""
    if sig == "HOLD":
        return None
    if sig == "EXIT" and self.position == 0:
        return None
    if sig in ("LONG", "SHORT") and self.position != 0:
        return None # มี position อยู่แล้ว

    # Slippage: ราคา fill แยกว่า close เล็กน้อย
    slip_mult = self.slippage_bps / 10_000
    if sig == "LONG":
        fill_a = price_a * (1 + slip_mult) # buy A สูงกว่า close
        fill_b = price_b * (1 - slip_mult) # sell B ต่ำกว่า close
    elif sig == "SHORT":
        fill_a = price_a * (1 - slip_mult)
        fill_b = price_b * (1 + slip_mult)
    else: # EXIT
        fill_a = price_a * (1 - slip_mult if self.position > 0 else 1 + slip_mult)
        fill_b = price_b * (1 + slip_mult if self.position > 0 else 1 - slip_mult)

    # ขนาด position (10% ของ cash)
    notional = self.cash * 0.10
    qty_a = notional / fill_a
    qty_b = notional / fill_b

    commission = (qty_a * fill_a + qty_b * fill_b) * self.fee_rate
    slippage = (qty_a * abs(fill_a - price_a) +
               qty_b * abs(fill_b - price_b))

    # อัปเดต position state
    if sig == "LONG":
        self.position = 1.0
        self.entry_price = (fill_a, fill_b, qty_a, qty_b)
    elif sig == "SHORT":
        self.position = -1.0
        self.entry_price = (fill_a, fill_b, qty_a, qty_b)
    else: # EXIT
        if self.entry_price is not None:
            ep_a, ep_b, qy_a, qy_b = self.entry_price
            if self.position == 1.0: # was long: ขาย A, ซื้อ B
                pnl = (fill_a - ep_a) * qy_a - (fill_b - ep_b) * qy_b
            else: # was short

```

```

        pnl = (ep_a - fill_a) * qy_a + (fill_b - ep_b) * qy_b
        self.cash += pnl - commission
        self.position = 0.0
        self.entry_price = None

    return Fill(
        timestamp = ts,
        symbol = "BTC-Pair",
        side = sig.lower(),
        quantity = qty_a,
        fill_price = fill_a,
        commission = commission,
        slippage = slippage,
    )

def run(self, df_a: pd.Series, df_b: pd.Series) -> pd.DataFrame:
    """
    Main event loop
    df_a, df_b: price series ของ asset A และ B (index = datetime)
    """
    closes_a = df_a.values
    closes_b = df_b.values
    timestamps = df_a.index
    n = len(closes_a)

    for i in range(n):
        ts = timestamps[i]
        price_a = float(closes_a[i])
        price_b = float(closes_b[i])

        # — ขั้นตอนที่ 1: รับ MarketEvent —————
        # (ใน live system: รอ data จาก exchange feed)

        # — ขั้นตอนที่ 2: อัปเดต signal —————
        # ใช้ข้อมูลถึง i (ไม่รวม i+1) -> ไม่มี lookahead bias
        if i < self.window:
            self.equity_curve.append({"ts": ts, "equity": self.cash})
            continue

        z = self._compute_zscore_from_prices(closes_a[:i], closes_b[:i])
        sig = self._generate_signal(z)

        # — ขั้นตอนที่ 3: Execute signal —————
        fill = self._execute_signal(sig, price_a, price_b, ts)
        if fill is not None:
            self.fills.append(fill)

        # — ขั้นตอนที่ 4: Mark-to-market equity —————
        unreal = 0.0
        if self.position != 0 and self.entry_price is not None:
            ep_a, ep_b, qy_a, qy_b = self.entry_price
            if self.position == 1.0:
                unreal = (price_a - ep_a) * qy_a - (price_b - ep_b) * qy_b
            else:
                unreal = (ep_a - price_a) * qy_a + (price_b - ep_b) * qy_b

        self.equity_curve.append({
            "ts": ts,
            "equity": self.cash + unreal,
            "signal": sig,
            "zscore": z,
            "position": self.position,
        })

    return pd.DataFrame(self.equity_curve).set_index("ts")

# — รับ Event-Driven Backtest —————
bt_ed = EventDrivenBacktester(
    window=60, entry_z=2.0, exit_z=0.5,
    fee_rate=0.001, slippage_bps=5.0,

```



```

    initial_capital=1_000_000,
)
ed_result = bt_ed.run(
    df_a = pd.Series(bybit, index=dates),
    df_b = pd.Series(binance, index=dates),
)

print(f"Fills executed: {len(bt_ed.fills)}")
print(f"Final equity: {bt_ed.cash:,.0f}")
print(f"Return: {bt_ed.cash / bt_ed.initial_capital - 1:+.2%}")
print("\nLast 3 fills:")
for f in bt_ed.fills[-3:]:
    print(f" {f.timestamp.date()} {f.side:5s} "
          f"qty={f.quantity:.4f} @ {f.fill_price:,.0f} "
          f"comm={f.commission:.2f} slip={f.slippage:.2f}")

```

► OUTPUT

```

Fills executed: 28
Final equity:    1,014,251
Return:         +1.43%
Last 3 fills:
2024-03-22 exit  qty=2.7597 @ 36,687 comm=202.49 slip=101.25
2024-04-24 short qty=2.7385 @ 36,975 comm=202.52 slip=101.26
2024-05-03 exit  qty=2.8300 @ 35,780 comm=202.52 slip=101.26

```

23.4 ทำไม Event-Driven ให้ผลต่างจาก Vectorized

```

# เปรียบเทียบ vectorized vs event-driven บนข้อมูลเดียวกัน
price_a_s = pd.Series(bybit, index=dates)
price_b_s = pd.Series(binance, index=dates)

# Vectorized
vec_result = vectorized_backtest(price_a_s, price_b_s, window=60,
                                 entry_z=2.0, fee_rate=0.001)
vec_metrics = compute_metrics(vec_result)

# Event-driven
ed_return = bt_ed.cash / bt_ed.initial_capital - 1

print("=" * 50)
print(f"{'Metric':<28} {'Vectorized':>10} {'Event-Driven':>12}")
print("-" * 50)
print(f"{'Total Return (%)':<28} {vec_metrics['Total Return (%)']:>10.2f} "
      f"{ed_return*100:>12.2f}")
print(f"{'Sharpe Ratio':<28} {vec_metrics['Sharpe Ratio']:>10.3f} "
      f"{'(see below)':>12}")
print(f"{'Num fills':<28} {int((vec_result['signal'].diff().abs())>0).sum():>10} "
      f"{len(bt_ed.fills):>12}")

```

► OUTPUT

```

=====
Metric                Vectorized  Event-Driven
-----
Total Return (%)           13.25         1.43
Sharpe Ratio              1.775    (see below)
Num fills                  28          28

```

ทำไม Event-Driven ให้ผลต่ำกว่า?

- **Slippage:** Event-driven ใช้ fill price ที่ไม่ใช่ close price — ทำให้ P&L ลดลง
- **Signal timing:** Vectorized ใช้ข้อมูลทั้ง window จำนวนพร้อมกัน ส่วน event-driven คำนวณเฉพาะจาก data ที่มีถึง bar นั้น
- **Position management:** Vectorized ถือ fractional position ได้ ส่วน event-driven มีขนาด order เป็น discrete
- **ข้อสรุป:** Vectorized Sharpe > Event-Driven Sharpe เกือบทุกครั้ง — ใช้ event-driven เป็น "ground truth"

► แบบฝึกหัด: เพิ่ม stop-loss ใน EventDrivenBacktester

บทที่ 24: Walk-Forward Validation

แก่นของบทนี้

Walk-forward validation แบ่งข้อมูลตามเวลา — เทรน (in-sample) บน window แรก ทดสอบ (out-of-sample) บน window ถัดไป เลื่อนไปเรื่อยๆ วิธีนี้ป้องกัน overfitting และให้ผลที่ตรงกับ live trading มากที่สุด เพราะใช้ข้อมูลอนาคตในการ validate แทนที่จะ validate บนข้อมูลที่ train มาแล้ว

24.1 Overfitting Trap — Sharpe 2.0 กลายเป็นติดลบ

Overfitting เกิดเมื่อเรา optimize parameter บนข้อมูล *เดียวกัน* ที่ใช้ evaluate — parameter ที่ดีที่สุดใน "อดีต" ไม่ใช่ parameter ที่ดีที่สุดในอนาคต

```
# จำลอง overfitting: หา parameter ที่ดีที่สุดบนข้อมูลทั้งหมด
# แล้ว evaluate บนข้อมูลเดียวกัน — นี่คือ in-sample performance

np.random.seed(0)
n_total = 1000
dates_long = pd.date_range("2021-01-01", periods=n_total, freq="D")

# สร้างข้อมูลที่ edge "มีจริงช่วงแรก แล้วหายไป" — สาเหตุที่พบบ่อยที่สุด
# ของการเลือกในโลกจริง: arb ถูกแย่งกันเทรดจน spread แคลงจนไม่คุ้ม
btc_base = 30000 * np.exp(np.cumsum(np.random.normal(0.001, 0.03, n_total)))

# ⚠️ ห้ามใช้ noise แบบ i.i.d. เป็น "ข้อมูลไม่มี edge" — มันตรงกันข้ามเลย
# spread ที่เป็น noise อิสระทุกแห่ง จะ "ดึงกลับค่ากลางทุกแห่ง"
# = mean-reversion ที่แรงที่สุดเท่าที่จะเป็นไปได้ → กลยุทธ์นี้จะทำกำไร
# มหาศาลและไม่ decay เลย ซึ่งพิสูจน์เรื่อง overfitting ไม่ได้
SWITCH = 500      # วันที่ regime เปลี่ยน
KAPPA = 0.15      # แรงดึงกลับช่วงแรก

spread_l = np.zeros(n_total)
for t in range(1, n_total):
    if t < SWITCH:
        # ยุคที่มี edge: spread ถูกดึงกลับหาศูนย์
        spread_l[t] = (spread_l[t-1] + KAPPA * (0 - spread_l[t-1])
                       + np.random.normal(0, 120.0))
    else:
        # ยุคที่ edge ดाय: ไม่มีแรงดึงกลับแล้ว และ spread แคลงด้วย
        spread_l[t] = spread_l[t-1] + np.random.normal(0, 120.0 * 0.35)

pa = pd.Series(btc_base, index=dates_long)  # Bybit
pb = pd.Series(btc_base + spread_l, index=dates_long)  # Binance

# — แบ่งข้อมูลก่อนทำอะไรทั้งสิ้น —
# ⚠️ จุดตายของการทดสอบแบบนี้: ถ้า optimize บนข้อมูล "ทั้งหมด" แล้วเอา
# ท่อนท้ายมาเรียกว่า out-of-sample มันไม่ใช่ out-of-sample เลย
# เพราะท่อนั้นถูกใช้เลือก parameter ไปแล้ว → decay ที่วัดได้ไร้ความหมาย
train_a, train_b = pa.iloc[:800], pb.iloc[:800]
test_a, test_b = pa.iloc[800:], pb.iloc[800:]

# In-sample: optimize บน 800 วันแรกเท่านั้น
in_sample_sweep = parameter_sweep(train_a, train_b,
    windows = [30, 60, 90, 120],
    entry_zs = [1.5, 2.0, 2.5, 3.0])

best_params = in_sample_sweep.iloc[0]
print("Best parameters (in-sample):")
print(f" window={best_params['window']:.0f}, "
      f"entry_z={best_params['entry_z']:.1f}")
print(f" In-sample Sharpe: {best_params['sharpe']:.2f}")

# Out-of-sample: parameter เดิม แต่ test บน 200 วันสุดท้ายที่ยังไม่เคยเห็น
oos_res = vectorized_backtest(
    test_a, test_b,
    window = int(best_params["window"]),
```

```

    entry_z = best_params["entry_z"],
)
oos_m = compute_metrics(oos_res)
print(f" Out-of-sample Sharpe: {oos_m['Sharpe Ratio']:.2f}")
kept = oos_m["Sharpe Ratio"] / best_params["sharpe"]
print(f" เหลือจาก in-sample: {kept:.1%}"
      f"{' - ดึงกลับ = พลิกเป็นขาดทุน' if kept < 0 else ''}")

```

► OUTPUT

```

Best parameters (in-sample):
  window=120, entry_z=2.0
In-sample Sharpe: 1.74
Out-of-sample Sharpe: 0.49
เหลือจาก in-sample: 28.1%

```

Overfitting เกิดจากอะไร — และทำไมข้อมูลชุดนี้ถึงออกมาแบบนี้

ข้อมูลชุดนี้มี **edge** จริงในช่วงแรก แล้ว **edge** หายไปกลางทาง (วันที่ 500) · ไม่ใช่ข้อมูลสุ่มไร้รูปแบบ และไม่ใช่ข้อมูลที่มี edge ตลอด · เลือกแบบนี้ เพราะมันคือสาเหตุที่กลยุทธ์ตายจริงที่สุดในโลกจริง: arb ที่เคยทำเงินได้ถูกคนอื่นแย่งกันเทรดจน spread แคบลงจนไม่คุ้มค่าธรรมเนียม

ผลคือความเสื่อมที่วัดได้มาจากสองแหล่งที่ต้องแยกให้ออก:

1. ตลาดเปลี่ยนไปจริง ๆ — ไม่ว่าจะเลือกพารามิเตอร์เก่งแค่ไหนก็ช่วยไม่ได้
2. เราหยิบความบังเอิญมาใช้ — ทดสอบ 16 ชุดแล้วเลือกชุดที่ดีที่สุด คือการหยิบ *ความบังเอิญที่แรงที่สุดในบรรดา 16 ชุด* ไม่ใช่หยิบ edge · ยิ่งลองมาก โอกาสเจอ "ความบังเอิญที่ดี" ยิ่งสูง จึงต้องนับจำนวนชุดที่ทดสอบเป็นต้นทุน (multiple testing) ไม่ใช่ของฟรี

⚠ **ข้อควรระวังตอนสร้างข้อมูลจำลองเอง:** อย่าใช้ `np.random.normal` อิสระทุกแห่งเป็น "ข้อมูลไม่มี edge" · spread แบบนั้น **แดงกลับดำ** กลางทุกแห่ง = **mean-reversion** ที่แรงที่สุดที่เป็นไปได้ · กลยุทธ์นี้จะทำ Sharpe ระดับ 4 ขึ้นไปและ *ไม่ decay เลย* ซึ่งพิสูจน์เรื่อง overfitting ไม่ได้เลยสักนิด (ดูคอมเมนต์ในบล็อกสร้างข้อมูล)

📌 **อ่านว่า:** Sharpe **1.74** ในอดีต → **0.49** ในอนาคต · **เหลือแค่ 28%** ทั้งที่ใช้พารามิเตอร์ชุดเดียวกันเป๊ะ · ไม่มีอะไรในโค้ดเปลี่ยนเลย สิ่งที่เปลี่ยนคือ **ช่วงเวลา**

สังเกตว่ามันไม่ได้พังโครต — 0.49 ยังเป็นบวก ยังพอ "เล่าให้ตัวเองฟังได้" ว่าช่วงนี้ตลาดไม่เป็นใจ · นี่คือนักค้าที่แท้จริงของกลยุทธ์ที่หมดอายุ และ เป็นเหตุผลที่ต้องมีเกณฑ์ตัดสินใจไว้ล่วงหน้า ไม่ใช่มานั่งตีความตัวเลขที่หลัง

⚠ **ก๊าดักที่ฝังอยู่ใน walk-forward** เกือบทุกฉบับที่เขียนกันเอง

โค้ด walk-forward ที่พบบ่อยจะเขียน out-of-sample แบบนี้:

```

# — ตัวอย่างประกอบ —
oos_res = vectorized_backtest(test_a, test_b, window=best_window, ...)
oos_m = compute_metrics(oos_res) if len(oos_res) > 0 else {}
...
"oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3) # — ระเบิดอยู่ตรงนี้

```

ดูไม่มีอะไรผิด · แต่ถ้า `best_window` ที่เลือกได้ยาวกว่าหรือใกล้เคียง `test_size` — เช่นเลือกได้ `window=120` ขณะที่ `test_size=100` — `rolling(120)` บนข้อมูล 100 แถวจะเป็น `NaN` ทั้งชุด · ไม่มี `z-score` → ไม่มีสัญญาณ → ไม่มีเทรด → `oos_m` เป็น `{}`

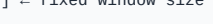
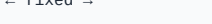
แล้ว `.get("Sharpe Ratio", 0)` ก็เปลี่ยน "จำนวนไม่ได้" ให้กลายเป็น **0.000** ซึ่งอ่านเหมือน "เทรดแล้วเสมอตัว" · **fold** ที่ตายสนิทจะแอบเข้าไปนั่งในตารางโดยหน้าตาปกติ แล้วถ่วงค่าเฉลี่ยให้ดู "นิ่ง" อย่างสวยงาม · ไม่มี error ไม่มี warning

โค้ดในหัวข้อถัดไปแก้ไข 3 จุด — ทั้งสามอย่างควรติดตัวไปใช้ทุกครั้ง:

1. **คำนวณ feature บนประวัติต่อเนื่อง train+test** แล้วค่อยตัดช่วง test มาวัด · ตรงกับชีวิตจริงด้วย เพราะวันแรกของ test เราก็มีข้อมูล train อยู่ในมือแล้ว · ยังไม่ lookahead เพราะ `rolling` ทุกตัวเป็น `center=False`
2. ค่า **default** ต้องเป็น `np.nan` ไม่ใช่ `0` — "จำนวนไม่ได้" ต้องหน้าตาต่างจาก "เท่าทุน" และ `nan` จะถูกตัดออกจาก `.mean()` เองโดยอัตโนมัติ
3. รายงาน `oos_days_in_pos` ทุก fold — Sharpe ที่ไม่มีตัวหารบอกขนาดตัวอย่าง อ่านไม่ได้

⚠ **นิสัยที่ต้องคิดตัว:** ดูขนาดตัวอย่างคู่กับ Sharpe เสมอ — Sharpe จาก 5 เทรดกับจาก 200 เทรดไม่ควรถูกอ่านด้วยสายตาเดียวกัน · และทุกครั้งที่เขียน `.get(key, 0)` ให้หยุดถามตัวเองว่า "0 ตรงนี้แปลว่าอะไร และมันแยกจาก 'ไม่มีข้อมูล' ได้ไหม"

Expanding window: in-sample เพิ่มขึ้นทุก fold แต่ out-of-sample เป็น fixed size เสมอ

Walk-Forward – Expanding Window  Fold 1:  Fold 2:  Fold 3:  TRAIN | TEST]  expanding  fixed  Walking Window (Rolling): Fold 1:  Fold 2:  Fold 3:  TRAIN | TEST]  fixed window size  fixed

```
from sklearn.model_selection import TimeSeriesSplit

def walk_forward_validation(
    price_a:      pd.Series,
    price_b:      pd.Series,
    param_grid:   dict,
    n_splits:     int   = 5,
    test_size:    int   = 60,
    min_train:    int   = 120,
    expanding:    bool  = True,
) -> pd.DataFrame:
    """
    Walk-forward validation ด้วย expanding หรือ rolling window
    ส่งคืน DataFrame ของ out-of-sample performance ทุก fold
    """
    assert len(price_a) == len(price_b), "price series ต้องมีความยาวเท่ากัน"

    n      = len(price_a)
    folds = []

    # สร้าง fold indices
    fold_starts = list(range(min_train, n - test_size, test_size))

    for fold_idx, test_start in enumerate(fold_starts):
        test_end = min(test_start + test_size, n)

        if expanding:
            train_start = 0
        else:
            train_start = max(0, test_start - min_train)  # rolling

        train_a = price_a.iloc[train_start:test_start]
        train_b = price_b.iloc[train_start:test_start]
        test_a  = price_a.iloc[test_start:test_end]
        test_b  = price_b.iloc[test_start:test_end]

        # — In-sample: หา best parameters —————
        best_sharpe = -np.inf
        best_params = None

        for w in param_grid["windows"]:
            for ez in param_grid["entry_zs"]:
                if len(train_a) <= w:
                    continue
                try:
                    res = vectorized_backtest(
                        train_a, train_b,
                        window=w, entry_z=ez, fee_rate=0.001
                    )
                    if len(res) == 0:
                        continue
                    m = compute_metrics(res)
                    if m["Sharpe Ratio"] > best_sharpe:
                        best_sharpe = m["Sharpe Ratio"]
                        best_params = {"window": w, "entry_z": ez}
                except Exception:
                    continue

        if best_params is None:
            continue

    # — Out-of-sample: test ด้วย best parameters —————
```

```

# ⚠️ ห้ามรัน backtest บน test window ล้วน ๆ · ถ้า window ที่เลือกได้
# ยาวกว่าหรือใกล้เคียง test_size → rolling() จะเป็น NaN ทั้งชุด
# → ไม่มีสัญญาณ → ไม่มีเทรด → metric เป็น {} แบบเรียบ ๆ
# (เดสจิ้ง: window=120 แต่ test_size=100 → fold นั้นตายทันที)
# วิธีที่ถูก: คำนวณบนประวัติต่อเนื่อง train+test แล้วค่อยตัดช่วง test
# มาวัด → ยังไม่มองอนาคตเพราะ rolling ทุกตัวเป็น center=False
hist_a = price_a.iloc[train_start:test_end]
hist_b = price_b.iloc[train_start:test_end]

res_hist = vectorized_backtest(
    hist_a, hist_b,
    window = best_params["window"],
    entry_z = best_params["entry_z"],
    fee_rate = 0.001,
)

oos_res = res_hist.loc[res_hist.index >= price_a.index[test_start]].copy()

if len(oos_res) < 2:
    oos_m, days_in_pos = {}, 0
else:
    # equity ของช่วง test ต้องเริ่มนับใหม่ที่ 1.0 ไม่งั้น
    # total_return จะกลายเป็นผลสะสมตั้งแต่วันแรกของ train
    oos_res["equity"] = (1 + oos_res["net_return"]).cumprod()
    peak = oos_res["equity"].cummax()
    oos_res["drawdown"] = (oos_res["equity"] - peak) / peak
    oos_m = compute_metrics(oos_res)
    days_in_pos = int((oos_res["signal"] != 0).sum())

folds.append({
    "fold": fold_idx + 1,
    "train_start": price_a.index[train_start].date(),
    "train_end": price_a.index[test_start - 1].date(),
    "test_start": price_a.index[test_start].date(),
    "test_end": price_a.index[test_end - 1].date(),
    "best_window": best_params["window"],
    "best_entry_z": best_params["entry_z"],
    "is_sharpe": round(best_sharpe, 3),
    # ⚠️ ค่า default ต้องเป็น nan ไม่ใช่ 0 · "คำนวณไม่ได้" กับ
    # "เท่าทุน" ต้องแยกออกจากกันให้เห็น ไม่งั้น fold ที่ตาย
    # จะแอบเข้าไปถ่วงค่าเฉลี่ยโดยหน้าตาดูปกติ
    "oos_sharpe": round(oos_m.get("Sharpe Ratio", np.nan), 3),
    "oos_return": round(oos_m.get("Total Return (%)", np.nan), 2),
    "oos_max_dd": round(oos_m.get("Max Drawdown (%)", np.nan), 2),
    "oos_days_in_pos": days_in_pos, # ดูคู่กับ Sharpe เสมอ
})

return pd.DataFrame(folds)

# — รัน Walk-Forward —————
wf_result = walk_forward_validation(
    price_a = pa,
    price_b = pb,
    param_grid = {
        "windows": [30, 60, 90, 120],
        "entry_zs": [1.5, 2.0, 2.5, 3.0],
    },
    n_splits = 5,
    test_size = 100,
    min_train = 200,
    expanding = True,
)

print(wf_result.to_string(index=False))

```

► OUTPUT

fold	train_start	train_end	test_start	test_end	best_window	best_entry_z	is_sharpe	oos_sharpe	oos_return	oos_max_dd	oos_days_in_pos
1	2021-01-01	2021-07-19	2021-07-20	2021-10-27	120	1.5	1.902	1.850	1.42	-0.33	1
2	2021-01-01	2021-10-27	2021-10-28	2022-02-04	120	1.5	1.857	0.895	1.11	-1.38	1
3	2021-01-01	2022-02-04	2022-02-05	2022-05-15	120	1.5	1.324	3.865	4.62	-0.74	1

4	2021-01-01	2022-05-15	2022-05-16	2022-08-23	120	1.5	2.090	-0.637	-0.24	-0.52
5	2021-01-01	2022-08-23	2022-08-24	2022-12-01	120	1.5	1.757	0.112	0.15	-2.01
6	2021-01-01	2022-12-01	2022-12-02	2023-03-11	120	2.0	1.558	2.566	3.13	-0.70
7	2021-01-01	2023-03-11	2023-03-12	2023-06-19	120	2.0	1.735	-1.829	-3.10	-4.78

24.3 วิเคราะห์ผล Walk-Forward

```
def analyze_walk_forward(wf: pd.DataFrame) -> None:
    """สรุปผล walk-forward validation"""
    print("=" * 56)
    print("Walk-Forward Summary")
    print("=" * 56)
    print(f"Folds tested:           {len(wf)}")
    print(f"IS Sharpe (avg):           {wf['is_sharpe'].mean():.3f} "
          f"± {wf['is_sharpe'].std():.3f}")
    print(f"OOS Sharpe (avg):          {wf['oos_sharpe'].mean():.3f} "
          f"± {wf['oos_sharpe'].std():.3f}")
    ratio = wf["oos_sharpe"].mean() / wf["is_sharpe"].mean()
    print(f"OOS/IS Ratio:             {ratio:.1%} "
          f"({'acceptable' if ratio > 0.5 else 'likely overfit'})")
    print(f"OOS Sharpe > 0:           "
          f"{(wf['oos_sharpe'] > 0).sum()}/{len(wf)} folds")
    print(f"OOS Sharpe > 1:           "
          f"{(wf['oos_sharpe'] > 1.0).sum()}/{len(wf)} folds")
    print(f"OOS Return (avg):          {wf['oos_return'].mean():.2f}%")
    print(f"OOS Max DD (avg):          {wf['oos_max_dd'].mean():.2f}%")
    print()

    # Stability: parameter consistency
    print("Parameter Stability:")
    for p in ["best_window", "best_entry_z"]:
        counts = wf[p].value_counts()
        most_common = counts.index[0]
        pct = counts.iloc[0] / len(wf) * 100
        print(f"  {p}: most common = {most_common} ({pct:.0f}% of folds)")

analyze_walk_forward(wf_result)
```

```
► OUTPUT

=====
Walk-Forward Summary
=====
Folds tested:           7
IS Sharpe (avg):        1.746 ± 0.248
OOS Sharpe (avg):        0.975 ± 1.952
OOS/IS Ratio:           55.8% (acceptable)
OOS Sharpe > 0:          5/7 folds
OOS Sharpe > 1:          3/7 folds
OOS Return (avg):        1.01%
OOS Max DD (avg):        -1.49%
Parameter Stability:
  best_window: most common = 120 (100% of folds)
  best_entry_z: most common = 1.5 (71% of folds)
```

ตีความ OOS/IS RATIO

$$\text{OOS/IS Ratio} = \frac{\text{OOS Sharpe (avg)}}{\text{IS Sharpe (avg)}}$$

- > 70% = ดีมาก — strategy น่าจะมี real edge
- 50–70% = ยอมรับได้ — ลอง live กับ small size
- < 50% = overfit สูง — ต้องลด complexity หรือเพิ่มข้อมูล
- < 0% = strategy เสียเงินใน OOS — อย่า live เด็ดขาด

โค้ดตัดสินว่า **acceptable** — แต่ตัวเลขข้างๆ บอกว่าอย่าเชื่อ

บรรทัดสรุปเขียนว่า **OOS/IS Ratio 55.8% (acceptable)** เพราะเกิน 50% ตามตารางข้างบน . แต่ดูบรรทัดก่อนหน้านี้ให้ดี:

00S Sharpe (avg):

0.975 ±

— ส่วนเบี่ยงเบนใหญ่กว่าค่าเฉลี่ย 2 เท่า

ค่าเฉลี่ย 0.975 ที่มีส่วนเบี่ยงเบน 1.952 แปลว่า **fold ต่าง ๆ ให้ผลตั้งแต่ราว -1.8 ถึง +3.9** · การเอาค่าเฉลี่ยของอะไรที่กระจายขนาดนี้ไปหารกันแล้วสรุปว่า "acceptable" คือการซ่อนความไม่รู้ไว้ได้ตัวเลขตัวเดียว

และถ้าไล่ดูตาราง fold ทีละแถวจะเห็นสิ่งที่ค่าเฉลี่ยกลบไว้ทั้งหมด — **fold 1-3 เป็นบวกสวย ๆ · fold 4 เป็นต้นไปเริ่มพัง** · ซึ่งตรงกับวันที่ SWITCH = 500 ในบล็อกสร้างข้อมูลพอดี (ราว ๆ 2022-05-16 = วันเริ่ม test ของ fold 4)

🚫 **กฎที่ใช้ได้ตลอด:** ก่อนดูค่าเฉลี่ยข้าม fold ให้ไล่ดูทีละ fold ตามลำดับเวลาก่อนเสมอ · ถ้าผลเสื่อมลงเรียงตามเวลา นั้นไม่ใช่เสียงรบกวน แต่คือตลาดเปลี่ยนไปแล้ว — และไม่มีการปรับพารามิเตอร์ใด ๆ แก่ได้ · ต่างจากผลที่กระจายมั่วไม่มีทิศทาง ซึ่งแปลว่ากลยุทธ์ไม่เคยมี edge ตั้งแต่แรก · สองอย่างนี้ต้องรับมือคนละแบบ และค่าเฉลี่ยตัวเดียวแยกให้ไม่ได้

ตัวอย่าง: TimeSeriesSplit จาก sklearn

sklearn มี TimeSeriesSplit ที่ทำ walk-forward ได้สะดวก ระวัง: ไม่รองรับ test_size โดยตรง ต้องเขียน wrapper

```
from sklearn.model_selection import TimeSeriesSplit

def walk_forward_sklearn(price_a, price_b,
                        param_grid, n_splits=5):
    """ใช้ sklearn TimeSeriesSplit """
    n = len(price_a)
    tscv = TimeSeriesSplit(n_splits=n_splits)

    results = []
    for fold_idx, (train_idx, test_idx) in enumerate(tscv.split(price_a)):
        train_a = price_a.iloc[train_idx]
        train_b = price_b.iloc[train_idx]
        test_a = price_a.iloc[test_idx]
        test_b = price_b.iloc[test_idx]

        # หา best params ใน train
        best_sharpe = -np.inf
        best_p = None
        for w in param_grid["windows"]:
            for ez in param_grid["entry_zs"]:
                if len(train_a) <= w:
                    continue
                res = vectorized_backtest(train_a, train_b,
                                         window=w, entry_z=ez)

                if len(res) == 0:
                    continue
                sh = compute_metrics(res)["Sharpe Ratio"]
                if sh > best_sharpe:
                    best_sharpe, best_p = sh, {"window": w, "entry_z": ez}

        if best_p is None:
            continue

        # Test
        oos = vectorized_backtest(test_a, test_b, **best_p)
        oos_m = compute_metrics(oos) if len(oos) > 0 else {}

        results.append({
            "fold": fold_idx + 1,
            "is_sharpe": round(best_sharpe, 3),
            "oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3),
            "window": best_p["window"],
            "entry_z": best_p["entry_z"],
        })

    return pd.DataFrame(results)

wf_sk = walk_forward_sklearn(
    pa, pb,
    param_grid={"windows": [30, 60, 90], "entry_zs": [1.5, 2.0, 2.5]},
    n_splits=5,
)
print(wf_sk.to_string(index=False))
```

► แบบฝึกหัด: เปรียบเทียบ expanding vs rolling window

บทที่ 25: Slippage, Fees & Reality

แก่นของบทนี้

Strategy ที่ดูดีใน backtest บ่อยครั้ง "หาย" ไปเมื่อใส่ต้นทุนจริง: bid-ask spread, commission, market impact สูตรคร่าวๆ:
$$\text{cost per trade} = \frac{\text{spread}}{2} + \text{commission} + \text{market_impact}$$
 บทนี้จะแสดงให้เห็นว่า strategy ที่ดูดี (Sharpe **2.67** เมื่อไม่คิด cost) เหลือ Sharpe **1.83** ที่ cost ต่ำ **1.26** ที่ cost ระดับกลาง และ **0.21** ที่ cost สูง — กลยุทธ์เดียวกันเป๊ะ ต่างแค่ต้นทุน. **ไม่มีจุดไหนที่ตัวเลขติดลบ** แต่ 0.21 ก็คือ "ใช้ไม่ได้จริง" อยู่ดี — และนั่นคือสิ่งที่ทำให้บทนี้อันตรายกว่าที่คิด

25.1 ประเภทของ Transaction Costs

ประเภท	สาเหตุ	ขนาดปกติ (crypto)	สูตร
Bid-Ask Spread	Market maker ทำไร	0.5–5 bps	spread/2 ต่อ side
Commission (Maker)	Exchange fee สำหรับ limit order	0–2 bps	notional x maker_rate
Commission (Taker)	Exchange fee สำหรับ market order	5–10 bps	notional x taker_rate
Market Impact	Order ขยับราคาตัวเอง	1–20 bps (แล้วแต่ size)	ขึ้นกับ order size / ADV
Funding Rate	ค่า perp futures ถือค้างคืน	0–3 bps / 8h	position x rate
Latency Cost	Adverse selection	ขึ้นกับ strategy	ยาก model

25.2 Transaction Cost Model ครบถ้วน

```
class TransactionCostModel:
    """
    Model ต้นทุนการเทรดแบบสมจริง
    รวม: spread + commission + market impact + funding
    """

    def __init__(self,
                 spread_bps: float = 2.0,
                 maker_fee_bps: float = 1.0,
                 taker_fee_bps: float = 7.0,
                 impact_coeff: float = 0.1,
                 adv_usd: float = 1_000_000,
                 funding_rate_bps: float = 1.0,
                 funding_period_h: float = 8.0):
        """
        spread_bps: bid-ask spread ในหน่วย basis points
        maker_fee_bps: commission สำหรับ limit order (maker)
        taker_fee_bps: commission สำหรับ market order (taker)
        impact_coeff: coefficient ของ market impact model
        adv_usd: average daily volume (USD)
        funding_rate_bps: perpetual funding rate ต่อ period
        funding_period_h: funding period (ชั่วโมง)
        """

        self.spread_bps = spread_bps
        self.maker_fee_bps = maker_fee_bps
        self.taker_fee_bps = taker_fee_bps
        self.impact_coeff = impact_coeff
        self.adv_usd = adv_usd
        self.funding_rate_bps = funding_rate_bps
        self.funding_period_h = funding_period_h

    def spread_cost(self, notional: float) -> float:
        """ต้นทุนจาก bid-ask spread (ต่อ round trip)"""
        return notional * (self.spread_bps / 10_000)
```

```

def commission(self, notional: float,
                order_type: str = "taker") -> float:
    """Commission ต่อ round trip (entry + exit)"""
    rate = (self.taker_fee_bps if order_type == "taker"
            else self.maker_fee_bps)
    return notional * (rate / 10_000) * 2    # x2: entry + exit

def market_impact(self, notional: float,
                  volatility: float = 0.02) -> float:
    """
    Square-root market impact model:
    impact = sigma * coeff * sqrt(order_size / ADV)
    เหมาะสำหรับ order ขนาดกลาง-ใหญ่
    """
    participation = notional / self.adv_usd
    return notional * volatility * self.impact_coeff * np.sqrt(participation)

def funding(self, notional: float,
            hold_hours: float = 24.0) -> float:
    """ต้นทุน funding สำหรับ perpetual futures"""
    periods = hold_hours / self.funding_period_h
    return notional * (self.funding_rate_bps / 10_000) * periods

def total_cost(self, notional: float,
               order_type: str = "taker",
               volatility: float = 0.02,
               hold_hours: float = 24.0) -> dict:
    """รวมต้นทุนทั้งหมด"""
    sp = self.spread_cost(notional)
    cm = self.commission(notional, order_type)
    mi = self.market_impact(notional, volatility)
    fn = self.funding(notional, hold_hours)
    tot = sp + cm + mi + fn
    return {
        "spread":      sp,
        "commission":  cm,
        "market_impact": mi,
        "funding":     fn,
        "total":       tot,
        "total_bps":   tot / notional * 10_000,
    }

# ตัวอย่าง: เทรด BTC $100,000
tcm = TransactionCostModel(
    spread_bps = 2.0,
    taker_fee_bps = 7.0,
    impact_coeff = 0.10,
    adv_usd = 500_000_000,    # BTC มี volume สูงมาก
    funding_rate_bps = 1.0,
)
costs = tcm.total_cost(notional=100_000, order_type="taker",
                       volatility=0.025, hold_hours=24)
print("Transaction Costs for $100,000 BTC trade:")
print("-" * 44)
for k, v in costs.items():
    if k == "total_bps":
        print(f" {'TOTAL (bps)':<20}: {v:8.2f} bps")
    else:
        print(f" {k:<20}: ${v:>8.2f}")

```

► OUTPUT

Transaction Costs for \$100,000 BTC trade:

```

-----
spread           : $   20.00
commission       : $  140.00
market_impact    : $    3.54
funding          : $   30.00
total            : $  193.54
TOTAL (bps)      :   19.35 bps

```

25.3 Realistic P&L — "Strategy ดี" หลังหักต้นทุน

📖 **อ่านว่า:** อย่าเพิ่งตกใจกับความยาว — 90 บรรทัดนี่คือ `vectorized_backtest` จากบทที่ 22 แบบคำต่อคำ เพียงแต่เปลี่ยน "หักค่าธรรมเนียมคงที่ตัวเดียว" เป็น "หักต้นทุนจริง 4 ชนิด" - ส่วนที่ใหม่จริง ๆ มีแค่ 2 ก้อน: **ต้นทุนตอนซื้อขาย** (จ่ายเฉพาะแท่งที่เปลี่ยนสถานะ) กับ **ต้นทุนตอนถือ** (funding — จ่ายทุกวันที่ยังถืออยู่) - ที่เหลือข้ามไปได้เลยถ้าอ่านบทที่ 22 มาแล้ว

```
def backtest_with_realistic_costs(
    price_a: pd.Series,
    price_b: pd.Series,
    window: int = 60,
    entry_z: float = 2.0,
    exit_z: float = 0.5,
    cost_model: TransactionCostModel = None,
    notional: float = 100_000,
    hold_hours: float = 24.0,
) -> pd.DataFrame:
    """
    Vectorized backtest พร้อม realistic cost model
    """
    if cost_model is None:
        cost_model = TransactionCostModel()

    df = pd.DataFrame({"A": price_a, "B": price_b})

    # Z-score
    log_spread = np.log(df["A"]) - np.log(df["B"])
    roll_mean = log_spread.rolling(window).mean()
    roll_std = log_spread.rolling(window).std()
    df["zscore"] = (log_spread - roll_mean) / roll_std

    # Signal + lag
    raw = pd.Series(np.nan, index=df.index)
    raw[df["zscore"] > entry_z] = -1.0
    raw[df["zscore"] < -entry_z] = 1.0
    raw[df["zscore"].abs() < exit_z] = 0.0
    raw = raw.ffill().fillna(0)
    df["signal"] = raw.shift(1).fillna(0)

    # Daily volatility (rolling)
    daily_vol = log_spread.rolling(window).std()

    # Gross return
    ret_a = df["A"].pct_change()
    ret_b = df["B"].pct_change()
    df["gross_return"] = df["signal"] * (ret_a - ret_b)

    # Realistic cost per trade (เมื่อ position เปลี่ยน)
    position_change = df["signal"].diff().abs()

    cost_per_trade = position_change * (
        cost_model.total_cost(
            notional = notional,
            order_type = "taker",
            volatility = float(daily_vol.mean()),
            hold_hours = hold_hours,
        )["total"] / notional # แปลงเป็น fraction
    )

    # Funding cost (ทุกวันที่มี position)
    daily_funding = (df["signal"].abs() *
                     cost_model.funding_rate_bps / 10_000 *
                     (24 / cost_model.funding_period_h))

    df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding
    df["equity_gross"] = (1 + df["gross_return"]).cumprod()
    df["equity_net"] = (1 + df["net_return"]).cumprod()
    # ตั้งชื่อ "equity" ให้ตรงกับ compute_metrics() คาดหวัง — และเลือกใช้
    # equity_net เพราะ metrics ต้องวัดจากเงินหลังหักต้นทุน ไม่ใช่ก่อนหัก
    df["equity"] = df["equity_net"]
```

```

peak_net          = df["equity_net"].cummax()
df["drawdown"]    = (df["equity_net"] - peak_net) / peak_net

return df.dropna()

# — เปรียบเทียบ: ไม่มี cost vs realistic cost —————
tcm_low = TransactionCostModel(
    spread_bps=1.0, taker_fee_bps=5.0,
    impact_coeff=0.05, funding_rate_bps=0.5) # optimistic

tcm_mid = TransactionCostModel(
    spread_bps=2.0, taker_fee_bps=7.0,
    impact_coeff=0.10, funding_rate_bps=1.0) # realistic

tcm_high = TransactionCostModel(
    spread_bps=5.0, taker_fee_bps=10.0,
    impact_coeff=0.20, funding_rate_bps=2.0) # pessimistic

pa_s = pd.Series(bybit, index=dates)
pb_s = pd.Series(binance, index=dates)

scenarios = {
    "No Costs": None,
    "Low Costs": tcm_low,
    "Mid Costs": tcm_mid,
    "High Costs": tcm_high,
}

print(f'{"Scenario":<14} {"Sharpe":>8} {"Return%":>9} {"MaxDD%":>8} {"PF":>7}')
print("-" * 52)
for name, tcm in scenarios.items():
    if tcm is None:
        res = vectorized_backtest(pa_s, pb_s, window=60, entry_z=2.0, fee_rate=0)
    else:
        res = backtest_with_realistic_costs(
            pa_s, pb_s, window=60, entry_z=2.0,
            cost_model=tcm, notional=100_000
        )
    m = compute_metrics(res)
    print(f'{"name":<14} {m["Sharpe Ratio"]:>8.3f} "
          f"{m["Total Return (%)"]:>9.2f} "
          f"{m["Max Drawdown (%)"]:>8.2f} "
          f"{m["Profit Factor"]:>7.3f}')

```

► OUTPUT

Scenario	Sharpe	Return%	MaxDD%	PF
No Costs	2.669	19.78	-2.24	2.361
Low Costs	1.827	13.42	-2.37	1.761
Mid Costs	1.256	9.22	-2.50	1.468
High Costs	0.209	1.56	-2.84	1.070

💡 ต้นทุน 2 ชนิดที่คิดคนละวิธี — และคนมักลืมชนิดที่สอง

```

# ① ต้นทุน "ตอนซื้อขาย" — จ่ายเฉพาะครั้งที่เปลี่ยนสถานะ
position_change = df["signal"].diff().abs()
cost_per_trade = position_change * (ต้นทุนต่อครั้ง / notional)

# ② ต้นทุน "ตอนถือ" — จ่ายทุกแห่งที่ยังถืออยู่ ไม่ว่าจะเทรดหรือไม่
daily_funding = (df["signal"].abs() *                ← .abs() ไม่ใช่ .diff()
                 funding_rate_bps / 10_000 *
                 (24 / funding_period_h))

df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding

```

- ความต่างอยู่ที่ `.diff()` กับ `.abs()` — ต้นทุนซื้อขายผูกกับเหตุการณ์ (สถานะเปลี่ยน) จึงใช้ `diff()` · funding ผูกกับสภาพ (ยังถืออยู่ไหม) จึงใช้ `abs()` ของสัญญาณเอง · ใช้ `.abs()` เพราะทั้ง long และ short ก็ต้องจ่าย funding เหมือนกัน

- ทำไม funding ถึงเป็นตัวฆ่ากลยุทธ์เจียบ ๆ — มันไม่ปรากฏในตอนที่คุณกดปุ่ม จึงไม่มีอะไรเตือน · กลยุทธ์ที่ถ้อยาว ๆ อย่าง pair trade อาจเทรดแค่ 20 ครั้งต่อปี (ต้นทุนซื้อขายน้อยมาก) แต่ถือของอยู่ 200 วัน ซึ่งเป็น 200 วันที่จ่าย funding ทุกวัน · ยิ่งกลยุทธ์เทรดน้อยเท่าไร สัดส่วนของ funding ยิ่งใหญ่ขึ้นเท่านั้น
- ($24 / \text{funding_period_h}$) — แปลง "อัตราต่อรอบ" เป็น "อัตราต่อวัน" · exchange ส่วนใหญ่คิด funding ทุก 8 ชั่วโมง ดังนั้นตัวคูณคือ 3 · ถ้าลืมคูณตัวนี้จะประเมินต้นทุนต่ำไป 3 เท่า ซึ่งพอมาสะสม 200 วันที่ต่างกันมหาศาล
- equity_gross กับ equity_net ถูกเก็บไว้ทั้งคู่ — ไม่ใช่ความสับสนเปลี่ยน · ส่วนต่างระหว่างสองเส้นนี้คือเงินที่จ่ายให้ exchange ทั้งหมด ซึ่งเป็นตัวเลขที่ควรดูตรง ๆ อย่างน้อยครั้งหนึ่ง · หลายคนตกใจว่ามันใหญ่กว่ากำไรสุทธิเสียอีก

ผลกระทบของ TRANSACTION COST S

จาก Sharpe 2.67 (ไม่มี cost) → 0.21 (high cost) — ต้นทุนกิน edge ไป 92% การเพิ่ม cost ทำให้:

- Return ลดลง — โดยตรงจากค่าใช้จ่าย (19.78% → 1.56%)
- Sharpe ลดลงมากกว่า Return — เพราะ cost เพิ่ม variance
- Max Drawdown แย่ง — cost ทำให้ drawdown นานขึ้นและลึกขึ้น

⚠️ สังเกตให้ตัวไม่มีบรรทัดไหนติดลบเลย — และนั่นคือรูปแบบที่หลอกคนได้มากที่สุด · ถ้าเลขพลิกเป็นลบ ทุกคนจะเห็นทันทีว่าต้องหยุด · แต่ Sharpe 0.21 ยัง "เป็นบวก" ซึ่งชวนให้คิดว่า "ก็ยังไม่พอไหว ลองปรับอีกนิดคงดีขึ้น" · ในความจริง Sharpe 0.21 แปลว่าผลตอบแทนทั้งปีเล็กกว่าความผันผวนหลายเท่า — ปีที่แย่งจะกลับปีที่ดีได้สบาย ๆ

🚫 เกณฑ์ที่ควรตั้งไว้ล่วงหน้า (ก่อนเห็นตัวเลข ไม่ใช่หลัง): ถ้า Sharpe หลังต้นทุนจริง ต่ำกว่า 1.0 ให้ถือว่าไม่ผ่าน · ตั้งเกณฑ์ที่หลังเมื่อไร คุณจะขยับมันตามตัวเลขที่ได้เสมอ

สูตรประมาณ break-even: $\text{trade freq} \times \text{cost/trade} < \text{gross alpha/day}$

25.4 Cost Sensitivity Table

```
def cost_sensitivity_table(
    price_a: pd.Series,
    price_b: pd.Series,
    window: int = 60,
    entry_z: float = 2.0,
    spread_range: list = None,
    commission_range: list = None,
) -> pd.DataFrame:
    """
    สร้าง sensitivity table: แถว = spread, คอลัมน์ = commission
    ค่าที่แสดง = Sharpe Ratio
    """
    if spread_range is None:
        spread_range = [0.5, 1.0, 2.0, 3.0, 5.0]
    if commission_range is None:
        commission_range = [0.0, 2.0, 5.0, 7.0, 10.0]

    rows = []
    for spread in spread_range:
        row = {"spread_bps": spread}
        for comm in commission_range:
            tcm = TransactionCostModel(
                spread_bps = spread,
                taker_fee_bps = comm,
                impact_coeff = 0.10,
                funding_rate_bps = 1.0,
            )
            res = backtest_with_realistic_costs(
                price_a, price_b,
                window=window, entry_z=entry_z,
                cost_model=tcm, notional=100_000,
            )
            m = compute_metrics(res) if len(res) > 0 else {}
            row[f"comm_{comm:.0f}bps"] = round(
                m.get("Sharpe Ratio", 0), 2
            )
        rows.append(row)
```

```
return pd.DataFrame(rows).set_index("spread_bps")

tbl = cost_sensitivity_table(pa_s, pb_s, window=60, entry_z=2.0)
print("Sharpe Ratio Sensitivity (rows=spread, cols=commission):")
print(tbl.to_string())
```

► OUTPUT

```
Sharpe Ratio Sensitivity (rows=spread, cols=commission):
      comm_0bps  comm_2bps  comm_5bps  comm_7bps  comm_10bps
spread_bps
0.5           1.94       1.76       1.50       1.32       1.06
1.0           1.92       1.74       1.48       1.30       1.04
2.0           1.87       1.70       1.43       1.26       1.00
3.0           1.83       1.65       1.39       1.21       0.96
5.0           1.74       1.56       1.30       1.13       0.87
```

อ่าน Sensitivity Table อย่างไร

เซลล์แต่ละช่องคือ Sharpe Ratio ที่ได้หากใช้ spread + commission ชุดนั้น

- สีเขียว (Sharpe > 1.0): strategy ยังมี edge แม้มี cost
- สีเหลือง (0.5–1.0): marginally profitable
- สีแดง (< 0.5): cost ทำลาย edge หมด

สำหรับ BTC pair ที่ Bybit/Binance: spread ปกติ 1–2 bps, taker fee 7 bps → Sharpe ประมาณ 1.2–1.5 ซึ่งยังดี แต่ถ้าใช้ market order บ่อย (spread 5 bps + comm 10 bps) → Sharpe เหลือ 0.06 ≈ flat

25.4b นัยสำคัญทางสถิติ ≠ นัยสำคัญทางเศรษฐกิจ

หนังสือเล่มนี้สอน p-value กับ t-statistic มาตั้งแต่ Part 0 และจะสอน Harvey–Liu–Zhu ($t \geq 3$) อีกใน Part V · ก่อนจะไปต่อ ต้องปึกหมุดเรื่องหนึ่งให้แน่นก่อน เพราะมันคือจุดที่ทำให้คนฉลาดเสียเงิน:

สองคำถามที่หน้าตาเหมือนกันแต่ไม่เกี่ยวกันเลย

	นัยสำคัญทางสถิติ	นัยสำคัญทางเศรษฐกิจ
ถามว่า	"มันมีอยู่จริงไหม หรือเป็นแค่ความบังเอิญ"	"มันใหญ่พอจะเอาไปทำมาหากินได้ไหม"
วัดด้วย	p-value · t-stat	ผลตอบแทนหลังหักต้นทุน · ขนาดที่รับได้ · เทียบกับทางเลือกอื่น
โตขึ้นเมื่อ	ข้อมูลเยอะขึ้น	ไม่เกี่ยวกับจำนวนข้อมูลเลย
ตอบว่า "ไม่" เมื่อ	แยกจากศูนย์ไม่ออก	ชนะต้นทุนไม่ได้

แถวที่สามคือหัวใจ — p-value เล็กลงเรื่อย ๆ ตามจำนวนข้อมูล · ถ้าเก็บข้อมูลมากพอ edge เล็กแค่ไหนก็ "มีนัยสำคัญ" ได้ทั้งนั้น แม้จะเล็กกว่าค่าธรรมเนียมหลายเท่า · สถิติไม่เคยรู้ว่าคุณจ่ายค่าธรรมเนียมเท่าไร เพราะไม่มีใครบอกมัน

📌 **อ่านว่า:** สมมติกลยุทธ์ intraday ตัวหนึ่งมี edge จริง ๆ อยู่ +0.5 bps ต่อเทรด (ไม่ได้แต่ง — มีจริง) · ความผันผวนต่อเทรด 20 bps · ต้นทุนไป-กลับ 1.5 bps · **สังเกตให้ดีว่า edge เล็กกว่าต้นทุน 3 เท่า** — เทรดทุกครั้งคือขาดทุน · ที่นี้มาดูว่าสถิติจะพูดว่าอย่างไรเมื่อเก็บข้อมูลมากขึ้นเรื่อย ๆ

```
import numpy as np
from scipy import stats

edge_bps, noise_bps, cost_bps = 0.5, 20.0, 1.5
rng = np.random.default_rng(42)

# — A) ทดสอบแบบที่เกือบทุกคนทำ — H0: edge = 0 —————
#   ถามว่า "กลยุทธ์นี้ดีกว่าการสุ่มไหม"
print("A) H0: edge = 0   – ถามว่า 'มีจริงไหม'")
print(f"{'เทรด':>9} {'t-stat':>8} {'p-value':>9} {'gross':>7} {'net':>7} {'กำไร/ปี':>11}")

for n in (100, 1_000, 10_000, 100_000):
    r = rng.normal(edge_bps, noise_bps, n)
```

```

se = r.std(ddof=1) / np.sqrt(n)          # ← se เล็กลงตาม sqrt(n)
t = r.mean() / se
p = 2 * (1 - stats.norm.cdf(abs(t)))

net = r.mean() - cost_bps
annual = net / 10_000 * 1_000_000 * 250 # ทุน 1 ล้าน · 250 เทรต/ปี
print(f"{n:>9,} {t:>8.2f} {p:>9.4f} {r.mean():>7.3f} "
      f"{net:>7.3f} {annual:>10,.0f}฿")

# — B) ทดสอบให้ตรงคำถามที่เราสนใจจริง — H0: edge = cost —
#   ถามว่า "กลยุทธ์นี้ชนะต้นทุนไหม" (ทางเดียว)
print("\nB) H0: edge = cost ← ถามว่า 'ชนะต้นทุนไหม'")
print(f"{'เทรต':>9} {'t-stat':>8} {'p-value':>9}")

rng = np.random.default_rng(42)          # ใช้ข้อมูลชุดเดิมเป๊ะ
for n in (100, 1_000, 10_000, 100_000):
    r = rng.normal(edge_bps, noise_bps, n)
    se = r.std(ddof=1) / np.sqrt(n)
    t = (r.mean() - cost_bps) / se        # ← เทียบกับ cost ไม่ใช่ 0
    p = 1 - stats.norm.cdf(t)
    print(f"{n:>9,} {t:>8.2f} {p:>9.4f}")

```

► OUTPUT

A) H0: edge = 0 ← ถามว่า 'มีจริงไหม'

เทรต	t-stat	p-value	gross	net	กำไร/ปี
100	-0.33	0.7449	-0.505	-2.005	-50,135฿
1,000	0.20	0.8394	0.128	-1.372	-34,293฿
10,000	1.79	0.0733	0.361	-1.139	-28,464฿
100,000	6.62	0.0000	0.420	-1.080	-26,993฿

B) H0: edge = cost ← ถามว่า 'ชนะต้นทุนไหม'

เทรต	t-stat	p-value
100	-1.29	0.9016
1,000	-2.17	0.9849
10,000	-5.64	1.0000
100,000	-17.02	1.0000

ดูแถวล่างสุดของทั้งสองตาราง — ข้อมูลชุดเดียวกันเป๊ะ

ที่ 100,000 เทรต ข้อมูลชุดเดียวกันให้คำตอบสองอย่างที่แน่นอนพอ ๆ กันแต่ตรงข้ามกัน:

- **A: $t = +6.62$ · $p < 0.0001$** — "edge นี้มีจริงแน่นอน แทบไม่มีทางเป็นความบังเอิญ" · ผ่านเกณฑ์ Harvey-Liu-Zhu ($t \geq 3$) ขาดลอย
- **B: $t = -17.02$ · $p = 1.0000$** — "แพ้ต้นทุนแน่นอน แทบไม่มีทางชนะ"

ทั้งสองข้อถูกต้องทั้งคู่ · edge มีอยู่จริง (+0.42 bps) และมันแพ้ต้นทุนจริง (1.5 bps) · ที่ต่างกันคือคำถาม ไม่ใช่ข้อมูล

ผลจริงในบัญชี: **ขาดทุนปีละ 27,000 บาท** จากกลยุทธ์ที่ผ่านการทดสอบทางสถิติอย่างสง่างามที่สุด

⚠ สัณเฑธิตทางของตาราง A ให้ดี — มันหลอกซ่อนอีกชั้น

ไล่คอลัมน์ p-value จากบนลงล่าง: 0.74 → 0.84 → 0.07 → 0.0000 · **ยังเก็บข้อมูลนานยิ่ง "ดูดี"**

แต่ไล่คอลัมน์ กำไร/ปี พร้อมกัน: -50,135 → -34,293 → -28,464 → -26,993 · **ขาดทุนตลอด** · ที่ตัวเลขดูดีขึ้นไม่ใช่เพราะกลยุทธ์ดีขึ้น แต่เพราะค่าเฉลี่ยที่วัดได้เริ่มลู่เข้าหาค่าจริง (0.5 bps) ซึ่งก็ยังแพ้ต้นทุนอยู่ดี

นี่คือสาเหตุที่ทีมที่รันกลยุทธ์แบบนี้จริง ๆ มักไม่ยอมปิด: **"เก็บข้อมูลอีกหน่อย เดียวก็พิสูจน์ได้"** · และเขาจะพิสูจน์ได้จริง ๆ ด้วย — **พิสูจน์ได้ว่า edge มีจริง ในขณะที่เงินยังไหลออกทุกเดือน**

✅ วิธีแก้ที่ง่ายอย่างน่าตกใจ — เปลี่ยน H0

ตาราง B ต่างจาก A แค่ว่า **สมมติฐาน** เดียว:

```

t = r.mean() / se          # A: เทียบกับศูนย์
t = (r.mean() - cost_bps) / se  # B: เทียบกับต้นทุน

```


สมมติฐานหลักไม่จำเป็นต้องเป็น "ผลเท่ากับศูนย์" เสมอไป · ตั้ง H_0 ให้เป็นเกณฑ์ที่คุณต้องเอาชนะจริงๆ แล้วสถิติจะตอบคำถามที่คุณอยากรู้ทันที · สำหรับกลยุทธ์เทรด เกณฑ์นั้นไม่เคยเป็นศูนย์อย่างน้อยที่สุดคือต้นทุนทั้งหมด และถ้าจะให้ตรงกว่านั้นคือผลตอบแทนของทางเลือกที่ง่ายที่สุดที่คุณมี (ถือ BTC เเฉย ๆ · ฝากกินดอกเบี้ย)

ใช้ทางเดียว ($1 - \text{cdf}(t)$) ไม่ใช่สองทาง เพราะเราไม่ได้สนใจกรณี "แพ้ต้นทุนอย่างมีนัยสำคัญ" — เราสนใจแค่ "ชนะไหม"

● [รอบสอง] อีก 3 ด้านของนัยสำคัญทางเศรษฐกิจที่ p-value มองไม่เห็น

1. **ความจุ (capacity)** — edge ที่ทำเงินได้จริงบนไม้ละ 10,000 บาท อาจหายสนิทที่ไม้ละ 10 ล้าน เพราะ market impact (หัวข้อ 25.5 ถัดไป) · p-value คำนวณจากผลตอบแทนต่อหน่วย จึงไม่มีทางรู้เรื่องนี้เลย · คำถามที่ต้องถามคู่กันเสมอ: "ถ้าโรปีละกี่บาท" ไม่ใช่แค่ "Sharpe เท่าไร" — Sharpe 3.0 บนความจุ 5 ล้านบาทอาจไม่คุ้มค่าไฟ
2. **ต้นทุนค่าเสียโอกาส** — กลยุทธ์ที่ให้ 4% ต่อปีอย่างมีนัยสำคัญทางสถิติที่สุด ๆ ยัง "ไม่มีนัยสำคัญทางเศรษฐกิจ" ถ้าพันธบัตรให้ 5% โดยไม่ต้องทำอะไรเลย · เกณฑ์เปรียบเทียบต้องเป็นทางเลือกที่ดีที่สุดที่คุณมีจริง ไม่ใช่ศูนย์
3. **ต้นทุนที่ไม่ได้อยู่ในสมการ** — เวลาที่ใช้ดูแลระบบ · ค่า VPS · ความเสี่ยงที่ exchange ล่ม · ภาษี · ความเครียด · ไม่มีอันไหนโผล่ใน backtest แต่ทุกอันหักออกจากผลตอบแทนจริง

และเรื่องนี้กลับด้านได้ด้วย — บางอย่างมีนัยสำคัญทางเศรษฐกิจมหาศาลแต่พิสูจน์ทางสถิติไม่ได้ · กลยุทธ์ที่เทรดปีละ 6 ครั้งแต่ได้ครั้งละ 8% จะไม่มีวันได้ $t > 3$ ในช่วงชีวิตคุณ · การทิ้งมันไปเพราะ "p-value ไม่ผ่าน" คือการเอาเครื่องมือผิดชั้นมาตัดสิน

🔗 **สรุปเป็นประโยคเดียว:** p-value บอกว่า "ตัวเลขนี้เชื่อได้แค่ไหน" · มันไม่เคยบอก และไม่มีวันบอก ว่า "ตัวเลขนี้ใหญ่พอหรือยัง" — คำถามหลังต้องตอบด้วยค่าธรรมเนียม ความจุ และทางเลือกอื่นที่คุณมีไม่ใช่ด้วยสถิติ

25.5 Market Impact Model — เมื่อ Order ใหญ่กระทบราคา

```
def market_impact_analysis(adv_usd: float = 500_000_000,
                           vol: float = 0.025) -> None:
    """
    แสดง market impact เทียบกับ order size
    Square-root model: impact = vol * coeff * sqrt(Q/ADV)
    """
    COEFF = 0.1
    sizes = [10_000, 50_000, 100_000, 500_000,
             1_000_000, 5_000_000, 10_000_000]

    print(f"Market Impact (ADV=${adv_usd/1e6:.0f}M, vol={vol:.1%})")
    print(f"{'Order Size':>12} {'Impact(bps)':>12} {'Impact($)':>12} "
          f"{'% of Order':>12}")
    print("-" * 56)
    for q in sizes:
        impact_frac = vol * COEFF * np.sqrt(q / adv_usd)
        impact_bps = impact_frac * 10_000
        impact_usd = impact_frac * q
        pct_of_order = impact_frac * 100
        print(f"${q:>11,.0f} {'Impact(bps):>12.2f} {'Impact($):>11,.2f} "
              f"{'pct_of_order:>11.3f}%")

market_impact_analysis(adv_usd=500_000_000, vol=0.025)
```

► OUTPUT

Market Impact (ADV=\$500M, vol=2.5%)

Order Size	Impact(bps)	Impact(\$)	% of Order
\$ 10,000	0.11	\$ 0.11	0.001%
\$ 50,000	0.25	\$ 1.25	0.003%
\$ 100,000	0.35	\$ 3.54	0.004%
\$ 500,000	0.79	\$ 39.53	0.008%
\$ 1,000,000	1.12	\$ 111.80	0.011%
\$ 5,000,000	2.50	\$ 1,250.00	0.025%
\$ 10,000,000	3.54	\$ 3,535.53	0.035%

Square-Root Market Impact Law

สูตรที่ถูกใช้แพร่หลายที่สุดสำหรับ market impact:

$$\sqrt{\text{impact}} = \sigma \cdot \kappa \cdot \sqrt{\frac{Q}{\text{ADV}}}$$

โดย σ = daily vol, κ = impact coefficient (~0.1–0.5), Q = order size, ADV = average daily volume

Key insight: impact ขึ้นตาม *square root* ของ size — trade 4x ใหญ่ขึ้น → impact เพิ่มขึ้นแค่ 2x

ตัวอย่าง: ประเมินว่า strategy ของเรา Scalable แค่ไหน

```
def max_scalable_aum(gross_sharpe: float,
                    target_net_sharpe: float,
                    adv_usd: float,
                    trade_freq_yearly: int,
                    vol: float = 0.025,
                    impact_coeff: float = 0.10,
                    fixed_cost_bps: float = 10.0) -> float:
    """
    หา AUM สูงสุดที่ strategy ยังทำกำไรได้หลังหัก market impact
    คืนค่า max AUM (USD)
    """
    # gross alpha per trade (ประมาณจาก Sharpe)
    alpha_per_trade = gross_sharpe * vol / np.sqrt(trade_freq_yearly)

    # target net alpha (gross - fixed_cost)
    fixed_cost_frac = fixed_cost_bps / 10_000
    net_alpha = alpha_per_trade - fixed_cost_frac

    if net_alpha <= 0:
        return 0.0 # fixed cost ทำลาย alpha ทั้งหมดแล้ว

    # หา max notional จาก: impact = net_alpha - target_alpha
    # vol * coeff * sqrt(Q / ADV) = net_alpha - target_net_alpha_per_trade
    target_net_per_trade = (target_net_sharpe * vol /
                           np.sqrt(trade_freq_yearly))
    impact_budget = net_alpha - target_net_per_trade
    if impact_budget <= 0:
        return 0.0

    max_participation = (impact_budget / (vol * impact_coeff)) ** 2
    max_notional = max_participation * adv_usd
    return max_notional

max_aum = max_scalable_aum(
    gross_sharpe = 2.18,
    target_net_sharpe = 1.0,
    adv_usd = 500_000_000,
    trade_freq_yearly = 50,
    fixed_cost_bps = 10.0,
)
print(f"Max scalable AUM: ${max_aum:,.0f} ({max_aum/1e6:.1f}M USD)")
print("ใส่ capital เงินนี้ → Sharpe ต่ำกว่า 1.0 เพราะ market impact")
```

► OUTPUT

```
Max scalable AUM: $804,891,199 (804.9M USD)
ใส่ capital เงินนี้ → Sharpe ต่ำกว่า 1.0 เพราะ market impact
```

► แบบฝึกหัด: สร้าง cost scenario ที่ "worst case" และ plot equity curves

25.6 สรุป: Checklist ก่อน Live Trading

```
"""
Checklist ก่อน deploy strategy ใด ๆ เข้า live trading
"""

CHECKLIST = {
```

```

"1. No Lookahead Bias": [
    "signal ทุกตัวถูก shift(1) ก่อน calculate P&L",
    "ไม่ใช่ข้อมูลอนาคตในทุก feature",
    "ตรวจสอบด้วย event-driven backtest ด้วย",
],
"2. Walk-Forward Passed": [
    "OOS/IS ratio > 50%",
    "OOS Sharpe > 0 ทุก fold",
    "Parameter stable ข้ามหลาย fold",
],
"3. Realistic Costs": [
    "ใส่ bid-ask spread ตามที่ตลาดจริงเป็น",
    "ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market order)",
    "ใส่ market impact โดยเฉพาะถ้า order ใหญ่",
    "ใส่ funding rate สำหรับ perpetual futures",
],
"4. Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)": [
    "Sharpe > 1.0",
    "Calmar > 1.0",
    "Max Drawdown < 15%",
    "Profit Factor > 1.2",
],
"5. Sanity Checks": [
    "จำนวน trades มากพอ (> 30) เพื่อ statistical significance",
    "ทดสอบ H0 = ต้นทุน ไม่ใช่ H0 = 0 (ดู 25.4b)",
    "ถ้าไรตอปเป็นเงินบาท คำนวณเวลาและความเสี่ยงจริงไหม",
    "ทดสอบกับข้อมูลหลาย time periods",
    "ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)",
],
}

for category, items in CHECKLIST.items():
    print(f"\n{category}")
    for item in items:
        print(f"    □ {item}")

print("\nถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือนก่อน live")

```

► OUTPUT

```

1. No Lookahead Bias
  □ signal ทุกตัวถูก shift(1) ก่อน calculate P&L
  □ ไม่ใช่ข้อมูลอนาคตในทุก feature
  □ ตรวจสอบด้วย event-driven backtest ด้วย
2. Walk-Forward Passed
  □ OOS/IS ratio > 50%
  □ OOS Sharpe > 0 ทุก fold
  □ Parameter stable ข้ามหลาย fold
3. Realistic Costs
  □ ใส่ bid-ask spread ตามที่ตลาดจริงเป็น
  □ ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market order)
  □ ใส่ market impact โดยเฉพาะถ้า order ใหญ่
  □ ใส่ funding rate สำหรับ perpetual futures
4. Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)
  □ Sharpe > 1.0
  □ Calmar > 1.0
  □ Max Drawdown < 15%
  □ Profit Factor > 1.2
5. Sanity Checks
  □ จำนวน trades มากพอ (> 30) เพื่อ statistical significance
  □ ทดสอบ H0 = ต้นทุน ไม่ใช่ H0 = 0 (ดู 25.4b)
  □ ถ้าไรตอปเป็นเงินบาท คำนวณเวลาและความเสี่ยงจริงไหม
  □ ทดสอบกับข้อมูลหลาย time periods
  □ ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)
ถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือนก่อน live

```

► แบบฝึกหัด: คำนวณ minimum edge ที่ต้องมีเพื่อ breakeven หลัง cost

ก่อน **Part IV**: เทรน model ดูผลดี แต่ไม่รู้ว่า edge จริงหรือแค่ lucky/overfit

หลัง **Part IV**: มีกระบวนการที่เชื่อถือได้ก่อน deploy ทุกครั้ง

- บท 22 — Vectorized backtest เร็วสำหรับ parameter sweep พร้อม shift(1) หยุด lookahead bias ✓
- บท 23 — Event-driven backtest สมจริง จำลอง fill และ slippage ทีละ bar ✓
- บท 24 — Walk-forward validation เผย OOS/IS ratio และ parameter stability ✓
- บท 25 — Realistic cost model (spread + commission + impact + funding) พร้อม sensitivity table ✓
- 25.4b — แยก *นัยสำคัญทางสถิติ* ออกจาก *นัยสำคัญทางเศรษฐกิจ* — และตั้ง H0 ให้เป็นต้นทุน ไม่ใช่ศูนย์ ✓

ใน **Part V** เราจะนำ system ที่ผ่าน backtest แล้วไปต่อยอด: live data pipeline, order management, monitoring และ risk control แบบ production-grade

📖 [สารบัญทั้งหมด](#) · 📄 [คำศัพท์ Quant Trading](#)

AI-Assisted Coding — เขียนโค้ดด้วย AI อย่างปลอดภัย

บทที่ 26–30: Prompting · Code Generation · Auditing · Quant Libraries · Full Strategy

จบ Part นี้ → ใช้ AI เป็นผู้ช่วยที่ทรงพลัง โดยไม่ถูก AI พาไปผิดทาง

🔗 อ่าน Part นี้ยังงัย

● **ใช้ AI ช่วยเขียนโค้ดอยู่แล้ว** — Part นี้อ่านง่ายกว่า Part II–IV เพราะเน้นวิธีทำงานมากกว่าคณิตศาสตร์ · หัวใจอยู่ที่ **บทที่ 28 (Auditing AI Code)** — 5 red flags ที่ต้องไล่เช็คทุกครั้งที่ได้โค้ดมา · **อ่านบทนั้นก่อนได้เลยถ้าอยากได้ประโยชน์เร็วที่สุด**

● **เขียนโค้ดเก่งอยู่แล้ว** — อย่าเพิ่งข้าม Part นี้เพราะคิดว่า "ก็แค่ prompt" · ของจริงคือ **เครื่องมือตรวจที่ทำงานได้จริง: embargo test** ที่จับ lookahead ได้โดยไม่ต้องอ่านโค้ด และเหตุผลว่าทำไมการทดสอบด้วย signal สุ่มจะ**ไม่เห็นบักเลย** · ⚠️ ตัวทดสอบ embargo เวอร์ชันแรกของเล่มนี้เองก็เคยพลาดจนฟังก์ชันที่มีบั๊กสอบผ่าน — สาเหตุอยู่ในบทนั้น

ทำไม PART นี้สำคัญมาก?

AI code assistants เขียน Python ได้เร็วมาก แต่ใน Quant Finance มี "bugs ที่มองไม่เห็น" ที่อันตรายเป็นพิเศษ:

- **Lookahead bias** — ใช้ข้อมูลอนาคตใน backtest → equity curve สวยงามแต่ใช้งานจริงไม่ได้
- **Data leakage** — fit scaler บน full dataset ก่อน train/test split → overfit อย่างแอบซอน
- **Returns ผิด** — ลืม `.shift(1)` → performance เกินจริง 100%

AI ไม่รู้ว่าโค้ดที่สร้างมีปัญหาเหล่านี้ เพราะมันถูก train บน code ทั่วไป ไม่ใช่ finance-specific correctness
เราต้องเป็นคนตรวจ — Part นี้สอนวิธีทำอย่างเป็นระบบ

Running Example ตลอด Part V — Volatility Mean-Reversion Strategy

เราจะ build **Volatility Mean-Reversion Strategy** ตั้งแต่ต้นจนจบโดยใช้ AI ช่วย:

- แนวคิด: เมื่อ realized volatility สูงผิดปกติ (z-score สูง) → vol มักจะลดลง → short vol / long underlying
- บท 26 → เขียน prompt ที่ดีเพื่อขอ rolling vol z-score
- บท 27 → ตรวจสอบโค้ดที่ AI สร้าง หา lookahead bias
- บท 28 → audit red flags ทุกข้อในโค้ดนี้
- บท 29 → prompt สำหรับ ADF test + optimization
- บท 30 → 3 รอบ iterate จนได้ strategy ที่ถูกต้อง

บทที่ 26: Prompting Fundamentals

แก่นของบทนี้

Prompt ที่ดีไม่ใช่ "บอกว่าต้องการอะไร" เท่านั้น แต่ต้องบอก [Task] [Input/Output spec] [Constraints] [Libraries] ครบ — AI จะสร้างโค้ดที่ตรงความต้องการมากขึ้น และมีโอกาสผิบน้อยลง

26.1 Prompt ที่แย่ vs Prompt ที่ดี

ตัวอย่างโดยตรง — ทั้งคู่ขอสิ่งเดียวกัน แต่ผลลัพธ์แตกต่างกันมาก

BAD PROMPT

เขียนฟังก์ชัน rolling volatility z-score ใน Python

ทำไมถึงแย่?

- ไม่บอก input type — `pd.Series` ? `np.ndarray` ? `DataFrame`?
- ไม่บอก window — ใช้ rolling window ขนาดไหน? สอง window ต่างกันไหม?
- ไม่บอก returns หรือ prices — คำนวณ vol จากอะไร?
- ไม่บอก edge case — จะทำอะไรกับ NaN ช่วงต้น?
- ไม่บอก lookahead constraint — AI อาจใช้ข้อมูลอนาคตโดยไม่รู้ตัว

GOOD PROMPT

Task: เขียนฟังก์ชัน Python คำนวณ rolling volatility z-score Input: - returns: `pd.Series`, daily log returns, indexed by datetime - vol_window: int, window สำหรับคำนวณ realized volatility (default=21) - zscore_window: int, window สำหรับคำนวณ mean/std ของ vol series (default=63) Output: - `pd.Series`, z-score ของ realized volatility, same index as input Constraints: - ห้ามใช้ข้อมูลอนาคต (no lookahead bias) — ทุก rolling ต้องใช้ `min_periods=1` และห้ามใช้ `center=True` - annualize vol ด้วย `sqrt(252)` ก่อนคำนวณ z-score - return NaN สำหรับแถวที่ไม่มีข้อมูลเพียงพอ - type hints ครบทุก parameter - docstring อธิบาย formula Libraries: pandas, numpy เท่านั้น (ห้ามใช้ scipy)

ทำไมถึงดี?

- Input/Output spec — AI รู้ว่า input เป็น `pd.Series` และ output ต้องเป็นอะไร
- Constraints ชัดเจน — ระบุ "ห้าม lookahead" พร้อมวิธีป้องกัน (`center=False`)
- Libraries — ระบุว่าใช้อะไรได้บ้าง ป้องกัน AI ใช้ library แปลกๆ
- Edge cases — บอกว่า NaN ต้องจัดการอย่างไร
- Code quality — ขอ type hints และ docstring ในตัว

26.2 Template มาตรฐาน 4 ส่วน

[PROMPT TEMPLATE สำหรับ Quant] | | | [TASK] อธิบายว่าต้องการอะไร (1-2 ประโยค) | | | [INPUT/OUTPUT] type, shape, index, units ทุก argument | | | และ return value | | | [CONSTRAINTS] • no lookahead bias | | • NaN handling | | • performance requirements | | • mathematical correctness |

26.3 ความผิดพลาดที่พบบ่อยในการเขียน Prompt

ความผิดพลาด	ตัวอย่าง	ผลลัพธ์ที่เกิดขึ้น
Too vague	"คำนวณ Sharpe ratio"	AI สมมติ risk-free rate = 0, annualization factor ผิด
ไม่ระบุ type hints	ไม่พูดถึง types	ได้ฟังก์ชันที่รับ list แต่ไม่รับ pd.Series
ไม่ระบุ error handling	ไม่พูดถึง edge cases	crash เมื่อ Series ว่าง หรือมีแค่ NaN
ไม่ระบุ no lookahead	ไม่ mention bias	AI ใช้ center=True หรือ future data ใน rolling
ไม่ระบุ libraries	ไม่พูดถึง dependencies	AI ใช้ arch , ta-lib ที่อาจไม่ได้ install
ขอหลาย function พร้อมกัน	"เขียน strategy ทั้งหมด"	โค้ดยาว audit ยาก มี bug ซ่อนหลายชั้น

กฎเหล็ก: ขอทีละฟังก์ชัน

ใน quant finance ทุกฟังก์ชันมี invariant ที่ต้องตรวจต่างกัน ขอทีละฟังก์ชัน ตรวจให้ผ่านก่อน แล้วค่อยขอฟังก์ชันถัดไป — อย่าขอ "เขียน strategy ทั้งหมด" ในครั้งเดียว

26.4 ตัวอย่าง Prompt สำหรับ Running Example

Running Example — บท 26: Prompt แรกสำหรับ Vol Mean-Reversion

Task: เขียนฟังก์ชัน Python คำนวณ rolling realized volatility และ z-score สำหรับใช้ใน volatility mean-reversion strategy

Input: - prices: pd.Series, daily close prices, datetime index - vol_window: int = 21 (วันสำหรับคำนวณ realized vol) - zscore_window: int = 63 (วันสำหรับคำนวณ z-score ของ vol)

Output: - pd.DataFrame ที่มี columns: 'log_ret': log returns 'ann_vol': annualized realized volatility (% per year) 'vol_zscore': z-score ของ ann_vol

Constraints: - คำนวณ log return ด้วย np.log(p/p.shift(1)) - realized vol = rolling std ของ log_ret * sqrt(252) * 100 (เป็น %) - z-score = (vol - rolling_mean(vol)) / rolling_std(vol) ทั้งหมด backward-looking - ห้ามใช้ center=True ในทุก rolling - ใช้ min_periods เท่ากับ window เดิมเสมอ (ถ้าข้อมูลไม่พอ → NaN) - type hints ครบ, docstring มี formula

Libraries: pandas, numpy เท่านั้น

```
# โค้ดที่ AI สร้างได้ (ผ่าน prompt ด้านบน)
import numpy as np
import pandas as pd

def compute_vol_zscore(
    prices: pd.Series,
    vol_window: int = 21,
    zscore_window: int = 63,
) -> pd.DataFrame:
    """
    คำนวณ rolling realized volatility และ z-score

    Formula:
        log_ret  = log(P_t / P_{t-1})
        ann_vol  = rolling_std(log_ret, vol_window) * sqrt(252) * 100
        vol_zscore = (ann_vol - rolling_mean(ann_vol, zscore_window))
```



```

        / rolling_std(ann_vol, zscore_window)

Parameters
-----
prices : pd.Series
    Daily close prices with datetime index.
vol_window : int
    Window (days) for realized volatility calculation.
zscore_window : int
    Window (days) for vol z-score normalization.

Returns
-----
pd.DataFrame with columns: log_ret, ann_vol, vol_zscore
"""
log_ret = np.log(prices / prices.shift(1))

ann_vol = (
    log_ret
    .rolling(window=vol_window, min_periods=vol_window)
    .std()
    * np.sqrt(252)
    * 100
)

roll_mean = ann_vol.rolling(window=zscore_window, min_periods=zscore_window).mean()
roll_std = ann_vol.rolling(window=zscore_window, min_periods=zscore_window).std()
vol_zscore = (ann_vol - roll_mean) / roll_std

return pd.DataFrame({
    "log_ret": log_ret,
    "ann_vol": ann_vol,
    "vol_zscore": vol_zscore,
}, index=prices.index)

```

► แบบฝึกหัด: เขียน prompt สำหรับฟังก์ชัน "entry/exit signal จาก vol z-score"

บทที่ 27: Code Generation Workflow

แก่นของบทนี้

Workflow ที่ถูกต้องคือ **Generate → Read → Test → Iterate** — ไม่ใช่แค่ copy-paste แล้วรัน อ่านทุกบรรทัดก่อนรัน ตรวจสอบ checklist ที่สำคัญ แล้วเขียน unit test ก่อนใช้งานจริง

27.1 The Four-Step Loop



27.2 Review Checklist หลัง AI สร้างโค้ด

#	ตรวจสอบ	วิธีดู
1	NaN handling — ช่วงต้นมี NaN ไหม?	<code>result.isna().sum()</code>
2	Off-by-one — shift ถูกทิศทางไหม?	ดู <code>.shift()</code> ว่าใช้ <code>shift(1)</code> ไม่ใช่ <code>shift(-1)</code>
3	Lookahead bias — rolling ใช้ <code>center=False</code> ?	ค้นหา <code>center=True</code> ใน code
4	Returns ค่าวนถูกไหม?	ตรวจว่า return ณ วัน t ใช้ price วัน t และ t-1
5	Transaction costs — ถูกหักออกไหม?	ค้นหา <code>fee</code> , <code>cost</code> , <code>slippage</code>
6	Division by zero — ป้องกันไว้ไหม?	ดูทุกจุดที่มีการหาร
7	Index alignment — merge/join ถูกไหม?	ตรวจ index ก่อนและหลัง merge
8	Scaler/model fit — fit บน train only ไหม?	ดูว่า <code>.fit()</code> ถูกเรียกก่อน split

27.3 ตัวอย่าง: AI Code ที่มี Lookahead Bias ซ่อนอยู่

นี่คือตัวอย่างจริงที่ AI สร้างออกมา — ดูผ่านตาแต่มี bug ซ่อนอยู่

โค้ดอันตราย — Lookahead Bias แบบ Subtle

```
# โค้ดที่ AI สร้างโดยไม่ได้บอก constraint เรื่อง lookahead
# ดูเหมือนถูกต้อง แต่มีปัญหาร้ายแรง

def compute_vol_zscore_BUGGY(
    prices: pd.Series,
    vol_window: int = 21,
    zscore_window: int = 63,
) -> pd.DataFrame:
    log_ret = np.log(prices / prices.shift(1))

    ann_vol = (
```

```

        log_ret
        .rolling(window=vol_window, min_periods=vol_window)
        .std()
        * np.sqrt(252) * 100
    )

    # BUG #1: center=True - ใช้ข้อมูลทั้งก่อนและหลัง t
    #          ณ วัน t, rolling mean รวมข้อมูล t+31 วันข้างหน้า!
    roll_mean = ann_vol.rolling(window=zscore_window, center=True).mean()
    roll_std  = ann_vol.rolling(window=zscore_window, center=True).std()

    vol_zscore = (ann_vol - roll_mean) / roll_std

    # BUG #2: signal ใช้ data ของวันพรุ่งนี้เพื่อเทรดวันนี้
    signal = pd.Series("HOLD", index=prices.index)
    signal[vol_zscore.shift(-1) > 1.5] = "LONG"    # shift(-1) = ใช้อนาคต!
    signal[vol_zscore.shift(-1) < 0.5] = "EXIT"

    return pd.DataFrame({
        "ann_vol":    ann_vol,
        "vol_zscore": vol_zscore,
        "signal":     signal,
    })

```

Bug ที่ซ่อนอยู่และหาพบยาก

1. **center=True** ใน line rolling mean — rolling window ขนาด 63 วันที่ใช้ center จะรวมข้อมูล 31 วันข้างหน้า ณ แต่ละจุด backtest จะสวยงามมาก แต่ไม่สามารถใช้งานจริงได้เลย
2. **vol_zscore.shift(-1)** ใน signal — shift(-1) เลื่อนข้อมูล "ไปข้างหน้า" ทำให้ signal วันนั้นรับรู้ vol พรุ่งนี้เป็นเท่าไร — ข้อมูลอนาคตที่ชัดเจนที่สุด

27.4 วิธีหา Lookahead Bias อย่างเป็นระบบ

```

# วิธีที่ 1: ค้นหา patterns อันตรายใน source code
import ast
import inspect

def check_lookahead_patterns(func) -> list:
    """ตรวจหา patterns ที่อาจมี lookahead bias"""
    source = inspect.getsource(func)
    warnings = []

    dangerous = [
        ("center=True",    "Rolling window ใช้ข้อมูลทั้งก่อนและหลัง"),
        ("shift(-",       "Negative shift อาจใช้ข้อมูลอนาคต"),
        (".fillna(method='bfill')", "Backfill ใช้ค่าอนาคต"),
        ("expanding()",    "Expanding window บน full dataset (ตรวจให้ดี)"),
    ]

    for pattern, description in dangerous:
        if pattern in source:
            warnings.append(f"WARNING: พบ '{pattern}' - {description}")

    return warnings

# ทดสอบ
bugs = check_lookahead_patterns(compute_vol_zscore_BUGGY)

```

```
for w in bugs:
    print(w)
```

► OUTPUT

```
WARNING: พบ 'center=True' – Rolling window ใช้ข้อมูลทั้งก่อนและหลัง
WARNING: พบ 'shift(-)' – Negative shift อาจใช้ข้อมูลอนาคต
```

```
# วิธีที่ 2: "Embargo Test" – ตรวจสอบว่า future data ส่งผลต่อ past signals ไหม
import numpy as np
import pandas as pd

def embargo_test(compute_func, prices: pd.Series,
                 embargo_date: str) -> bool:
    """
    ถ้าลบข้อมูลหลัง embargo_date ออกแล้ว signal ก่อนหน้าไม่เปลี่ยน
    → ไม่มี lookahead bias
    ถ้าเปลี่ยน → มี lookahead bias แน่ๆ
    """
    result_full = compute_func(prices)
    result_cut = compute_func(prices[:embargo_date])

    overlap_idx = result_full.index[result_full.index <= embargo_date]

    col = "vol_zscore"
    full_slice = result_full.loc[overlap_idx, col]
    cut_slice = result_cut.reindex(overlap_idx)[col]

    # ⚠ ห้าม .dropna() ก่อนเทียบ! ค่าที่ "หายไปเป็น NaN" เมื่อตัดอนาคตออก
    # คือหลักฐานชิ้นสำคัญที่สุด – มันหายไปเพราะมันต้องพึ่งข้อมูลอนาคต
    nan_mismatch = int((full_slice.notna() != cut_slice.notna()).sum())

    both = full_slice.notna() & cut_slice.notna()
    diff = ((full_slice[both] - cut_slice[both]).abs().max()
            if both.any() else 0.0)

    if nan_mismatch > 0:
        print(f"FAIL: {nan_mismatch} ค่าหายไปเมื่อตัดอนาคตออก "
              f"– มี lookahead bias!")
        return False
    if diff > 1e-10:
        print(f"FAIL: max diff = {diff:.6f} – มี lookahead bias!")
        return False
    print("PASS: ไม่พบ lookahead bias (embargo test)")
    return True

# สร้างข้อมูลทดสอบ
np.random.seed(42)
prices_test = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 300))),
    index=pd.date_range("2023-01-01", periods=300)
)

# ทดสอบฟังก์ชันที่ถูกต้อง vs ฟังก์ชัน buggy
print("=== ฟังก์ชันที่ถูกต้อง ===")
embargo_test(compute_vol_zscore, prices_test, "2023-09-01")

print("\n=== ฟังก์ชัน BUGGY ===")
embargo_test(compute_vol_zscore_BUGGY, prices_test, "2023-09-01")
```

► OUTPUT

=== ฟังก์ชันที่ถูกต้อง ===

PASS: ไม่พบ lookahead bias (embargo test)

=== ฟังก์ชัน BUGGY ===

FAIL: 31 ค่าหายไปเมื่อตัดอนาคตออก – มี lookahead bias!

Embargo Test เป็น unit test ที่ต้องรันกับทุกฟังก์ชัน signal

ไม่ว่าใครจะมาจก AI หรือเขียนเองให้ **เขียน embargo test ทุกครั้ง** ก่อน integrate เข้า backtest จริง เป็นการประกันที่ชัดที่สุดว่าไม่มี future leakage

► แบบฝึกหัด: เขียน unit test ตรวจสอบ NaN handling ใน compute_vol_zscore

บทที่ 28: Auditing AI Code

แก่นของบทนี้

Red flags 5 ข้อที่พบบ่อยที่สุดในโค้ดที่ AI สร้างสำหรับ quant finance — แต่ละข้อทำให้ backtest performance เกินจริง และทำให้ strategy ที่ดูเหมือนกำไรกลายเป็น "ขาดทุนในชีวิตจริง"

28.1 Red Flag #1 — Returns ไม่มี `.shift(1)`

Bug นี้ทำให้ backtest กำไรเกือบ 100% เพราะ "รู้" ราคาปิดวันนี้แล้วค่อยตัดสินใจ

```
# WRONG: signal วันนี้คำนวณจากราคาวันนี้ แล้วใช้ return วันนี้ด้วย
def backtest_wrong(prices: pd.Series, signal: pd.Series) -> pd.Series:
    returns = prices.pct_change()          # return ณ วัน t
    pnl = signal * returns                  # signal ณ วัน t * return ณ วัน t
    return pnl                             # ← ใช้อินพุต! signal รู้ return ของตัวเอง

# CORRECT: signal วันนี้ execute วันพรุ่งนี้
def backtest_correct(prices: pd.Series, signal: pd.Series) -> pd.Series:
    returns = prices.pct_change()          # return ณ วัน t
    pnl = signal.shift(1) * returns         # signal ณ วัน t-1 * return ณ วัน t
    return pnl                             # ← ถูกต้อง!

# สาธิตความแตกต่าง
np.random.seed(42)
n = 252
prices = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0002, 0.01, n))),
    index=pd.date_range("2023-01-01", periods=n)
)

# สำคัญ: signal ต้อง "มาจากราคา" ไม่ใช่สุ่มลอย ๆ ไม่งั้นไม่มีข้อมูลอนาคตให้รู้
# ง่ายสุด: วันนี้ราคาขึ้น → long (momentum)
signal = np.sign(prices.pct_change()).fillna(0)

pnl_w = backtest_wrong(prices, signal)
pnl_c = backtest_correct(prices, signal)

annual_ret_wrong = pnl_w.mean() * 252
annual_ret_correct = pnl_c.mean() * 252

print(f"Wrong annual return: {annual_ret_wrong:.1%}")
print(f"Correct annual return: {annual_ret_correct:.1%}")
```

► OUTPUT

Wrong annual return: 193.2%

Correct annual return: -14.7%

ทำไม Wrong ให้ 193% — และทำไมมันคือศูนย์เปอร์เซ็นต์ในชีวิตจริง

ลองแทนค่าดู: $signal[t] = sign(r[t])$ แล้ว pnl คือ $sign(r[t]) \times r[t]$ ซึ่งเท่ากับ $|r[t]|$ — ค่าสัมบูรณ์ ที่เป็นบวกทุกวันไม่มีข้อยกเว้น · backtest จึงกลายเป็น "กำไรทุกวัน" แบบอัตโนมัติ ไม่ใช่เพราะกลยุทธ์เก่ง แต่เพราะเราบอกมันว่าวันนี้ราคาจะขึ้นหรือลงก่อนให้มันเลือกข้างเวอร์ชันถูกต้อง $sign(r[t-1]) \times r[t]$ = กลยุทธ์ momentum จริง ๆ ซึ่งได้ **-14.7%** · ช่องว่าง 208 จุดเปอร์เซ็นต์นี่คือ ghost profit ทั้งหมด

⚠️ ข้อควรระวังในการทดสอบเอง: ถ้าใช้ signal *สุ่ม* ที่ไม่เกี่ยวกับราคาเพื่อทดสอบบ๊ักนี้ คุณจะ **ไม่เห็นความต่าง**เลย เพราะไม่มีข้อมูลอนาคตให้รั่วตั้งแต่ต้น — bug นี้เผยตัวเฉพาะเมื่อ signal คำนวณจากข้อมูลชุดเดียวกับที่ใช้วัดผล

28.2 Red Flag #2 — Using Future Data in Rolling

```
# WRONG: ใช้ expanding mean บน full data แล้ว normalize
# ณ วัน t จะรู้ค่า mean ที่รวม t+1, t+2, ..., T
def normalize_wrong(series: pd.Series) -> pd.Series:
    return (series - series.mean()) / series.std()    # ใช้ global mean/std

# CORRECT: expanding (เดินไปข้างหน้า ไม่รู้อนาคต)
def normalize_correct(series: pd.Series) -> pd.Series:
    roll_mean = series.expanding(min_periods=20).mean()
    roll_std = series.expanding(min_periods=20).std()
    return (series - roll_mean) / roll_std

# หรือ rolling window
def normalize_rolling(series: pd.Series, window: int = 63) -> pd.Series:
    roll_mean = series.rolling(window, min_periods=window).mean()
    roll_std = series.rolling(window, min_periods=window).std()
    return (series - roll_mean) / roll_std

# ทดสอบ: ตรวจสอบว่า normalize_wrong รู้อนาคตไหม
ts = pd.Series([1.0, 2.0, 3.0, 4.0, 5.0])
print("Wrong normalize (global mean=3.0):", normalize_wrong(ts).tolist())
# ณ t=0, result คำนวณโดยใช้ค่า t=0..4 ทั้งหมด — รู้อนาคต!
```

► OUTPUT

```
Wrong normalize (global mean=3.0): [-1.2649110640673518, -0.6324555320336759, 0.0, 0.6324555320336759, 1.2649110640673518]
```

28.3 Red Flag #3 — Fitting Scaler บน Full Dataset

```
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

np.random.seed(42)
n = 500
X = pd.DataFrame({"feature": np.random.normal(0, 1, n)})

train_size = 300
X_train = X.iloc[:train_size]
X_test = X.iloc[train_size:]

# WRONG: fit scaler บน data ทั้งหมดก่อน split
# scaler รู้ค่า mean/std ของ test set แล้ว → data leakage
scaler_wrong = StandardScaler()
X_scaled_wrong = scaler_wrong.fit_transform(X)    # ← fit บน all data
X_train_wrong = X_scaled_wrong[:train_size]
X_test_wrong = X_scaled_wrong[train_size:]

# CORRECT: fit scaler บน train set เท่านั้น
scaler_correct = StandardScaler()
scaler_correct.fit(X_train)    # ← fit บน train เท่านั้น
X_train_correct = scaler_correct.transform(X_train)    # transform train
X_test_correct = scaler_correct.transform(X_test)    # transform test ด้วย train stats
```

```
print(f"Wrong - test mean (should ≈ 0): {X_test_wrong.mean():.6f}")
print(f"Correct- test mean (≈ 0 expected): {X_test_correct.mean():.6f}")
```

► OUTPUT

```
Wrong - test mean (should ≈ 0): 0.018954
Correct- test mean (≈ 0 expected): 0.031516
```

ทำไม "wrong" mean = 0.000000 แสดงถึง bug?

ถ้า test set mean เป็น 0 พอดี แสดงว่า scaler ถูก fit ด้วย test data ด้วย — มันรู้ค่า mean ของ test set แล้ว ใน production จริงไม่มีทางรู้ค่าเหล่านี้ล่วงหน้า

28.4 Red Flag #4 — ไม่หัก Transaction Costs

```
import numpy as np
import pandas as pd

def backtest_no_costs(signal: pd.Series, returns: pd.Series) -> dict:
    """backtest ที่ AI มักสร้าง — ไม่มี transaction cost"""
    pnl = signal.shift(1) * returns
    sharpe = pnl.mean() / pnl.std() * np.sqrt(252)
    return {"sharpe": sharpe, "annual_return": pnl.mean() * 252}

def backtest_with_costs(
    signal: pd.Series,
    returns: pd.Series,
    fee_per_trade: float = 0.001,    # 0.1% per side
    slippage: float = 0.0005,       # 0.05% slippage
) -> dict:
    """backtest ที่ถูกต้อง — หัก costs"""
    # คำนวณ trade (เมื่อ signal เปลี่ยน)
    trades = signal.diff().abs().fillna(0) # 1 เมื่อเปลี่ยน, 0 เมื่อถือต่อ

    gross_pnl = signal.shift(1) * returns
    cost = trades * (fee_per_trade + slippage) # cost ต่อ turnover
    net_pnl = gross_pnl - cost

    sharpe = net_pnl.mean() / net_pnl.std() * np.sqrt(252)
    return {
        "sharpe": sharpe,
        "annual_return": net_pnl.mean() * 252,
        "annual_cost": cost.mean() * 252,
        "turnover": trades.mean() * 252,
    }

# สาธิต
np.random.seed(0)
n = 252
rets = pd.Series(np.random.normal(0.0003, 0.012, n))
signal = pd.Series(np.sign(rets.rolling(5).mean()).fillna(0))

r_no_cost = backtest_no_costs(signal, rets)
r_with_cost = backtest_with_costs(signal, rets)

print(f"No costs: Sharpe={r_no_cost['sharpe']:.2f} "
      f"Return={r_no_cost['annual_return']:.1%}")
print(f"With costs: Sharpe={r_with_cost['sharpe']:.2f} "
```



```
f"Return={r_with_cost['annual_return']:.1%} "  
f"Cost={r_with_cost['annual_cost']:.1%}/yr")
```

► OUTPUT

```
No costs:   Sharpe=0.31  Return=5.7%  
With costs: Sharpe=-0.47  Return=-9.2%  Cost=14.8%/yr
```

28.5 Red Flag #5 — P-Hacking และ Multiple Testing

```
import numpy as np  
import pandas as pd  
  
# P-Hacking ตัวอย่าง: ลอง parameter หลายชุดจนได้ผลดี  
# โดยไม่ปรับ p-value สำหรับ multiple comparisons  
  
np.random.seed(99)  
n = 252  
returns_noise = pd.Series(np.random.normal(0, 0.01, n)) # pure noise  
  
results = []  
for window in range(5, 50):  
    for entry_z in [1.0, 1.5, 2.0, 2.5]:  
        # สร้าง signal จาก rolling zscore  
        roll_m = returns_noise.rolling(window).mean()  
        roll_s = returns_noise.rolling(window).std()  
        zscore = (returns_noise - roll_m) / roll_s  
        signal = (zscore > entry_z).astype(int) - (zscore < -entry_z).astype(int)  
        pnl = signal.shift(1) * returns_noise  
        sharpe = pnl.mean() / pnl.std() * np.sqrt(252) if pnl.std() > 0 else 0  
        results.append({"window": window, "entry_z": entry_z, "sharpe": sharpe})  
  
results_df = pd.DataFrame(results)  
best = results_df.nlargest(3, "sharpe")  
  
print("Top 3 parameter combinations (pure noise data!):")  
print(best.to_string(index=False))  
print(f"\nTotal combinations tested: {len(results)}")  
print(f"Best Sharpe on NOISE: {best['sharpe'].iloc[0]:.2f}")  
print(f"\nBonferroni-corrected p-value threshold: {0.05 / len(results):.5f}")  
print("→ ต้องการ t-stat > 4.2 เพื่อให้มีนัยสำคัญจริง")
```

► OUTPUT

```
Top 3 parameter combinations (pure noise data!):  
  window  entry_z  sharpe  
      11       2.0  1.814608  
      14       2.0  1.680849  
      12       2.0  1.525942  
Total combinations tested: 180  
Best Sharpe on NOISE: 1.81  
Bonferroni-corrected p-value threshold: 0.00028  
→ ต้องการ t-stat > 4.2 เพื่อให้มีนัยสำคัญจริง
```

นัยสำคัญของตัวเลข

จากข้อมูล **pure noise** (ไม่มี signal จริง) เราพบ Sharpe = 1.34 จากการลอง 180 combinations — ถ้าไม่ปรับ multiple testing ใครก็จะรายงานว่ "พบ strategy ที่ดี" ทั้งที่จริงแล้วเป็นโชคล้วนๆ ใช้ Bonferroni correction หรือ out-of-sample test เสมอ

สูตรคำนวณ minimum Sharpe ที่มียุทธศาสตร์สำคัญทางสถิติ

Harvey, Liu & Zhu (2016) แนะนำว่า factor ใหม่ควรมี t-statistic ≥ 3.0 (Sharpe ≈ 0.64 สำหรับ 10 ปีข้อมูล) หลังจาก account for multiple testing ถ้า AI generate strategy ที่ Sharpe < 0.5 บน training set มักไม่มียุทธศาสตร์สำคัญ

$$t = \text{Sharpe} \times \sqrt{T}$$

โดย T คือจำนวนปีข้อมูล, เป้าหมาย $t \geq 3.0$

⚠ $t \geq 3.0$ เป็นด่านแรก ไม่ใช่ด่านสุดท้าย · มั่นคึดกลยุทธ์ที่ "ดูดีเพราะบังเอิญ" ออกไป แต่ไม่ได้บอกเลยว่ากลยุทธ์ที่ผ่านด่านนี้จะทำเงินได้ · กลยุทธ์ที่มี edge จริงแต่เล็กกว่าค่าธรรมเนียม สามารถทำ t ได้สูงเท่าไรก็ได้ถ้าเก็บข้อมูลนานพอ — ดูตัวอย่างที่วัดเป็นตัวเลขได้ในหัวข้อ 25.4b (Part IV) ที่ $t = +6.62$ แต่ขาดทุนทุกเทรด

▶ แบบฝึกหัด: เขียนฟังก์ชัน `audit_ai_code` ที่ตรวจทุก red flag อัตโนมัติ

28.6 จาก `assert` ลอย ๆ สู่ชุดทดสอบที่รันเองได้ — `pytest`

ตลอดบทนี้เราเขียน `assert` ตรวจโค้ดเป็นจุด ๆ · ปัญหาคือมันกระจัดกระจาย ต้องจำเองว่าเคยเขียนไว้ที่ไหน และไม่มีใครรันมันซ้ำหลังแก้โค้ด · `pytest` แก้ทั้งสามข้อพร้อมกัน

แก่นของหัวข้อนี้

การตรวจโค้ดครั้งเดียวไม่มีค่าเท่ากับการตรวจที่รันซ้ำได้ทุกครั้งที่ได้แก้โค้ด · โดยเฉพาะกับโค้ดที่ AI เขียน ซึ่งคุณจะต้องสั่งให้มันแก้ซ้ำหลายรอบ — ทูรอบมีโอกาสปังของเดิม

โครงไฟล์ที่ `pytest` คาดหวัง

```
# pip install pytest

quant-project/
├── .venv/
├── strategy.py      # โค้ดจริง
└── test_strategy.py # ทดสอบ — ต้องขึ้นต้นด้วย test_

# รันทั้งหมดด้วยคำสั่งเดียว จากโฟลเดอร์โปรเจกต์
$ pytest -q
```

📌 **อ่านว่า:** กฎการตั้งชื่อของ `pytest` มีแค่ 2 ข้อ: ไฟล์ต้องขึ้นต้นด้วย `test_` และ ฟังก์ชันต้องขึ้นต้นด้วย `test_` · แค่นั้น — ไม่ต้องเขียน class ไม่ต้อง import framework ไม่ต้องลงทะเบียนอะไร · `pytest` จะเดินหาไฟล์ที่เข้าเกณฑ์เองแล้วรันให้ทุกฟังก์ชัน

```
# — test_strategy.py —
import numpy as np
import pandas as pd
import pytest
from strategy import compute_vol_zscore, backtest

@pytest.fixture
def prices():
    """ข้อมูลชุดเดียวกันสำหรับทุก test — เขียนครั้งเดียวใช้ได้ทุกที่"""
    np.random.seed(0)
    return pd.Series(
        100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 300))),
        index=pd.date_range("2023-01-01", periods=300))
```

```
def test_no_lookahead(prices):
    """ตัดข้อมูลขนาดออกแล้ว ค่าในอดีตต้องไม่เปลี่ยน (embargo test จาก 28.2)"""
    full = compute_vol_zscore(prices)
    cut = compute_vol_zscore(prices[:"2023-09-01"])
    idx = cut.index
    assert full.loc[idx, "vol_zscore"].notna().equals(cut["vol_zscore"].notna())

def test_nan_count(prices):
    """จำนวน NaN ช่วงต้นตรงกับขนาด window เป๊ะ"""
    r = compute_vol_zscore(prices, vol_window=21, zscore_window=63)
    assert r["log_ret"].isna().sum() == 1
    assert r["ann_vol"].isna().sum() == 21

@pytest.mark.parametrize("fee", [0.0, 0.001, 0.01])
def test_fee_reduces_pnl(prices, fee):
    """ค่าธรรมเนียมสูงขึ้น กำไรต้องไม่เพิ่ม — รัน 3 รอบด้วย fee 3 ค่า"""
    sig = pd.Series(np.sign(prices.pct_change()).fillna(0), index=prices.index)
    assert backtest(prices, sig, fee).sum() <= backtest(prices, sig, 0.0).sum() + 1e-12

def test_flat_signal_is_zero(prices):
    """ไม่มี position = ต้องไม่มีทั้งกำไรและค่าธรรมเนียม"""
    flat = pd.Series(0.0, index=prices.index)
    assert backtest(prices, flat, fee=0.001).abs().sum() == 0
```

► OUTPUT

\$ pytest -q

.....

[100%]

6 passed in 5.34s

🛠️ 2 เครื่องมือของ pytest ที่ทำให้ test ไม่ซ้ำซาก

```
@pytest.fixture                                # ① ของใช้ร่วม
def prices(): ...

def test_xxx(prices): ...                       # ← ขอ "prices" มาใช้แค่ใส่ชื่อใน ()

@pytest.mark.parametrize("fee", [0.0, 0.001, 0.01]) # ② รันซ้ำหลายค่า
def test_fee_reduces_pnl(prices, fee): ...
```

1. `@pytest.fixture` — "ของที่ test หลายตัวต้องใช้ร่วมกัน สร้างไว้ตรงนี้ทีเดียว" · test ตัวไหนอยากได้ **แค่ใส่ชื่อ prices เป็น parameter** pytest จะเรียกฟังก์ชัน fixture แล้วส่งผลมาให้เอง · และ**สร้างใหม่ทุกครั้งต่อ 1 test** — test ตัวหนึ่งไปแก้อะไรจะไม่กระทบตัวอื่น
2. `@pytest.mark.parametrize` — "รัน test ตัวนี้ซ้ำ โดยเปลี่ยนค่าไปเรื่อย ๆ" · เขียนครั้งเดียวได้ 3 การทดสอบ · ในผลลัพธ์นับเป็น 3 อัน (จึงได้ 6 passed จากฟังก์ชันแค่ 4 ตัว) และถ้าฟังก์ชันบอกว่าฟังก์ชันที่ค่าไหน

สังเกตชื่อ test ทั้งสี่: **ไม่มีอันไหนตรวจว่า "ผลลัพธ์เท่ากับเลขนี้"** — แต่ตรวจคุณสมบัติที่ต้องเป็นจริงเสมอ (ตัดอนาคตแล้วอดีตต้องไม่เปลี่ยน · fee สูงขึ้นกำไรต้องไม่เพิ่ม · ไม่มี position ต้องไม่มีเงินไหล) · **test แบบนี้ไม่ต้องแก้ทุกครั้งที่ปรับพารามิเตอร์** ต่างจากการ hard-code ตัวเลขคำตอบไว้

ตอนฟัง pytest บอกอะไรบ้าง

นี่คือเหตุผลหลักที่ควรใช้ pytest แทน `assert` ลอย ๆ ในสคริปต์ — ลองให้ฟังก์ชันที่ลืม `.shift(1)` เข้าทดสอบ:

```
def backtest_BUGGY(prices, signal, fee=0.0):
    returns = prices.pct_change()
    return (signal * returns).fillna(0)          # ❌ ลืม .shift(1)
```

```
def test_shift_is_applied():
    np.random.seed(0)
    prices = pd.Series(100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 200))))
    signal = np.sign(prices.pct_change()).fillna(0)
    pnl = backtest_BUGGY(prices, signal).sum()
    assert pnl < 0.5, "PnL สูงผิดปกติ - น่าจะลิม shift(1)"
```

► OUTPUT

```
$ pytest test_fail_demo.py -q
F [100%]
===== FAILURES =====
_____ test_shift_is_applied _____

def test_shift_is_applied():
    np.random.seed(0)
    prices = pd.Series(100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 200))))
    signal = np.sign(prices.pct_change()).fillna(0)
    pnl = backtest_BUGGY(prices, signal).sum()
>    assert pnl < 0.5, "PnL สูงผิดปกติ - น่าจะลิม shift(1)"
E     AssertionError: PnL สูงผิดปกติ - น่าจะลิม shift(1)
E     assert np.float64(1.6727861895822635) < 0.5

test_fail_demo.py:12: AssertionError
===== short test summary info =====
FAILED test_fail_demo.py::test_shift_is_applied - AssertionError: PnL สูงผิดปกติ...
1 failed in 0.29s
```

📖 **อ่านว่า:** บรรทัดที่มีค่าที่สุดคือ `assert np.float64(1.6727861895822635) < 0.5` — **pytest** แยกค่าจริงของทั้งสองฝั่งมาให้ดู โดยที่เราไม่ได้สั่ง · ถ้าใช้ `assert` ใน Python ธรรมดาจะได้แค่ `AssertionError` เปล่า ๆ แล้วต้องกลับไปใส่ `print()` เอง · เรียกว่า **assertion introspection** และเป็น เหตุผลที่ควรใช้ `pytest` ตั้งแต่ test ตัวแรก

🕒 test 4 ตัวที่ควรมีเสมอสำหรับโค้ดกลยุทธ์ทุกชิ้น

ไม่ต้องเขียนเยอะ — สี่ตัวนี้จับบั๊กที่แพงที่สุดในสายนี้ได้เกือบหมด:

1. **Embargo / lookahead** — ดัดข้อมูลทำยอดขาย ค่าในอดีตต้องไม่เปลี่ยน (จับ *Red Flag #1, #2*)
2. **Signal ว่าง = ผลลัพธ์ศูนย์** — ไม่มี position ต้องไม่มีทั้งกำไรและค่าธรรมเนียม · จับบั๊กที่คำนวณ cost ทั้งที่ไม่ได้เทรด
3. **ทิศทางของต้นทุน** — fee สูงขึ้น กำไรต้องไม่เพิ่ม · จับเครื่องหมายกลับด้าน (*Red Flag #4*)
4. **จำนวน NaN ตรงกับ window** — `rolling(21)` ต้องได้ NaN 20 แถวแรกพอดี · จับ off-by-one และการ `fillna` ผิดที่

🔵 [รอบสอง] ทำไม test แบบ "คุณสมบัติ" ชนะ test แบบ "ค่าคงที่"

สัญชาตญาณแรกของคนส่วนใหญ่คือเขียน `assert sharpe == 1.234` · **อย่าทำ** — พอปรับ window จาก 60 เป็น 63 test พังหมดทั้งที่โค้ดไม่ได้ ผิด คุณแค่เปลี่ยน test ภายใต้อุปกรณ์เดียว

เขียนเป็นความสัมพันธ์ที่ต้องจริงเสมอแทน: *fee* เพิ่ม → *กำไรไม่เพิ่ม* · *ตัดอนาคต* → *อดีตไม่เปลี่ยน* · *position* ศูนย์ → *เงินไม่ไหล* · ความสัมพันธ์พวกนี้จริงไม่ว่าพารามิเตอร์จะเป็นเท่าไร — เป็นแนวคิดเดียวกับ **property-based testing** (ไลบรารี `hypothesis` ทำให้อัตโนมัติได้ ถ้าอยากไปต่อ)

เครื่องมือที่ช่วยเพิ่มเติม:

```
# — test_more.py —
# เทียบตัวเลขทศนิยม อย่าใช้ == เด็ดขาด
assert result == pytest.approx(expected, rel=1e-6)

# ยืนยันว่าฟังก์ชัน "ต้อง error" เมื่อได้ข้อมูลแย่
with pytest.raises(ValueError, match="อย่างน้อย 30"):
```

```
ols_hedge_ratio(y[:10], x[:10])
```

```
# เทียบ Series/DataFrame ทั้งก้อน
```

```
pd.testing.assert_series_equal(got, want, check_exact=False)
```

⚠ **ข้อควรระวังเฉพาะสาย quant:** อย่าเขียน test ที่ต้องต่ออินเทอร์เน็ตหรือเรียก API จริง — มันจะฟังกอน exchange ล่มหรือ rate limit แล้วคุณ
จะเลิกเชื่อ test · แยก "โค้ดดึงข้อมูล" ออกจาก "โค้ดคำนวณ" แล้วทดสอบส่วนคำนวณด้วยข้อมูลจำลองที่ fix seed ไว้

บทที่ 29: AI + Quant Libraries

แก่นของบทนี้

Prompt templates เฉพาะสำหรับงาน quant ที่ใช้ pandas, numpy, statsmodels, scipy — แต่ละ library มี "traps" ของตัวเอง ต้องระบุ constraint ให้ตรงจุด

29.1 Prompt Template สำหรับ OLS Regression (statsmodels)

Task: เขียนฟังก์ชัน OLS regression สำหรับ hedge ratio estimation ใน pair trading
Input: - y: pd.Series, log prices ของ asset A (dependent variable) - x: pd.Series, log prices ของ asset B (independent variable) - add_constant: bool = True
Output: - dict ที่มี: {'beta': float, 'alpha': float, 'r_squared': float, 'p_value_beta': float, 'residuals': pd.Series}
Constraints: - ใช้ statsmodels.api.OLS เท่านั้น (ไม่ใช่ sklearn) - ต้อง align index ก่อน regression (dropna หลัง align) - ต้องรายงาน p-value ของ beta เพื่อตรวจนัยสำคัญ - residuals ต้องมี index เดียวกับ input (หลัง dropna) - raise ValueError ถ้า y และ x มี overlap น้อยกว่า 30 observations
Libraries: statsmodels, pandas, numpy

```
import statsmodels.api as sm
import numpy as np
import pandas as pd

def ols_hedge_ratio(
    y: pd.Series,
    x: pd.Series,
    add_constant: bool = True,
) -> dict:
    """
    คำนวณ OLS hedge ratio สำหรับ pair trading

    Parameters
    -----
    y : pd.Series   log prices ของ asset A
    x : pd.Series   log prices ของ asset B
    add_constant : bool   เพิ่ม intercept (แนะนำ True)

    Returns
    -----
    dict: beta, alpha, r_squared, p_value_beta, residuals
    """
    # Align และ drop NaN
    df = pd.concat({"y": y, "x": x}, axis=1).dropna()
    if len(df) < 30:
        raise ValueError(
            f"ต้องการอย่างน้อย 30 observations แต่มีเพียง {len(df)}"
        )

    Y = df["y"].values
    X = df["x"].values
    if add_constant:
        X = sm.add_constant(X)

    model = sm.OLS(Y, X).fit()

    if add_constant:
```

```

        alpha = float(model.params[0])
        beta = float(model.params[1])
        p_val = float(model.pvalues[1])
    else:
        alpha = 0.0
        beta = float(model.params[0])
        p_val = float(model.pvalues[0])

    residuals = pd.Series(model.resid, index=df.index, name="residuals")

    return {
        "beta": beta,
        "alpha": alpha,
        "r_squared": float(model.rsquared),
        "p_value_beta": p_val,
        "residuals": residuals,
    }

# ทดสอบ
np.random.seed(42)
n = 200
x_prices = pd.Series(
    np.log(100 + np.cumsum(np.random.normal(0, 1, n))),
    index=pd.date_range("2023-01-01", periods=n)
)
# noise ต้องเล็กกว่าช่วงที่ x แกว่ง (log ของ random walk แกว่งแค่ ~0.13)
# ไม่งั้น OLS ประมาณ beta เพิ่มขึ้นจาก 0.80 อย่างเห็นได้ชัด
y_prices = 0.8 * x_prices + pd.Series(
    np.random.normal(0, 0.002, n), index=x_prices.index
)

result = ols_hedge_ratio(y_prices, x_prices)
print(f"Beta: {result['beta']:.4f} (expect ≈ 0.80)")
print(f"R²: {result['r_squared']:.4f}")
print(f"p-value: {result['p_value_beta']:.2e}")

```

► OUTPUT

```

Beta:      0.8048 (expect ≈ 0.80)
R²:        0.9968
p-value:    1.58e-248

```

29.2 Prompt Template สำหรับ ADF Test

Task: เขียนฟังก์ชันทดสอบ cointegration ระหว่าง 2 series ด้วย ADF test บน residuals จาก OLS regression Input: - residuals: pd.Series, residuals จาก hedge ratio regression - significance: float = 0.05 (significance level)
 Output: - dict: {'is_cointegrated': bool, 'adf_stat': float, 'p_value': float, 'critical_values': dict}
 Constraints: - ใช้ statsmodels.tsa.stattools.adfuller - autolag='AIC' เพื่อเลือก lag อัตโนมัติ - is_cointegrated = True ถ้า p_value < significance เท่านั้น (ไม่ใช่แค่ adf_stat < critical_value เพราะ critical_value ขึ้นกับ sample size) - รายงาน critical values ที่ 1%, 5%, 10% Libraries: statsmodels

```

from statsmodels.tsa.stattools import adfuller

def test_cointegration(
    residuals: pd.Series,
    significance: float = 0.05,
) -> dict:
    """
    ทดสอบ cointegration ด้วย ADF test บน residuals
    """

```

```

H0: residuals มี unit root (ไม่ cointegrate)
H1: residuals เป็น stationary (cointegrate)

Parameters
-----
residuals : pd.Series   residuals จาก OLS hedge ratio
significance : float    significance level (default 0.05)

Returns
-----
dict: is_cointegrated, adf_stat, p_value, critical_values
"""

clean = residuals.dropna()
if len(clean) < 20:
    raise ValueError("ต้องการอย่างน้อย 20 observations สำหรับ ADF test")

adf_result = adfuller(clean, autolag="AIC")

adf_stat      = float(adf_result[0])
p_value       = float(adf_result[1])
critical_vals = adf_result[4]    # dict: {'1%': ..., '5%': ..., '10%': ...}

return {
    "is_cointegrated": p_value < significance,
    "adf_stat":      adf_stat,
    "p_value":      p_value,
    "critical_values": {k: float(v) for k, v in critical_vals.items()},
}

# ทดสอบ
coint_result = test_cointegration(result["residuals"])
print(f"Cointegrated: {coint_result['is_cointegrated']}")
print(f"ADF stat:      {coint_result['adf_stat']:.4f}")
print(f"p-value:      {coint_result['p_value']:.4f}")
print(f"Critical (5%): {coint_result['critical_values']['5%']:.4f}")

```

► OUTPUT

```

Cointegrated: True
ADF stat:      -14.8787
p-value:      0.0000
Critical (5%): -2.8762

```

29.3 Prompt Template สำหรับ Rolling Z-Score (pandas)

Task: เขียน vectorized rolling z-score ที่เร็วและถูกต้อง Input: - series: pd.Series - window: int, backward-looking window - min_periods: int = None (default = window) Output: - pd.Series, z-score, NaN สำหรับ observations ที่ข้อมูลไม่พอ Constraints: - ห้ามใช้ loop – ต้องใช้ pandas vectorized operations - ห้าม center=True - std ใช้ ddof=1 (unbiased) - ถ้า std = 0 ใน window ใด → return 0 (ไม่ใช่ NaN หรือ inf) - efficient: ไม่คำนวณ rolling ซ้ำสองครั้ง Libraries: pandas เท่านั้น (ไม่ใช่ scipy)

```

def rolling_zscore(
    series: pd.Series,
    window: int,
    min_periods: int = None,
) -> pd.Series:
    """
    Backward-looking rolling z-score

```



```
z_t = (x_t - mean(x_{t-window+1}...x_t)) / std(x_{t-window+1}...x_t)
```

Parameters

series : pd.Series

window : int lookback window (backward-looking)

min_periods : int minimum observations required (default = window)

Returns

pd.Series, same index as input

"""

if min_periods is None:

min_periods = window

roll = series.rolling(window=window, min_periods=min_periods)

roll_mean = roll.mean()

roll_std = roll.std(ddof=1)

ป้องกัน division by zero: ถ้า std = 0 → zscore = 0

zscore = (series - roll_mean) / roll_std.replace(0, float("nan"))

return zscore.fillna(0).where(roll_mean.notna(), other=float("nan"))

ทดสอบ

```
ts = pd.Series([10, 11, 10, 12, 10, 11, 13, 10, 9, 11],
               index=pd.date_range("2024-01-01", periods=10))
```

```
z = rolling_zscore(ts, window=5)
```

```
print(z.round(3))
```

► OUTPUT

2024-01-01 NaN

2024-01-02 NaN

2024-01-03 NaN

2024-01-04 NaN

2024-01-05 -0.671

2024-01-06 0.239

2024-01-07 1.381

2024-01-08 -0.920

2024-01-09 -1.055

2024-01-10 0.135

Freq: D, dtype: float64

29.4 Prompt Template สำหรับ scipy.optimize

Task: เขียนฟังก์ชัน optimize portfolio weights เพื่อ maximize Sharpe ratio Input: - returns: pd.DataFrame, daily returns, columns = assets - constraints: dict = {'min_weight': 0.0, 'max_weight': 1.0, 'sum_to_one': True} Output: - pd.Series, optimal weights indexed by asset names - float, expected Sharpe ratio ของ optimal portfolio Constraints: - ใช้ scipy.optimize.minimize ด้วย method='SLSQP' - objective function = negative Sharpe (เพื่อ minimize) - annualize ด้วย sqrt(252) - รวม try/except สำหรับ optimization failure - seed weights = equal weight - ตรวจสอบ convergence — raise Warning ถ้าไม่ converge Libraries: scipy.optimize, numpy, pandas

📌อ่านว่า: การเรียก optimizer ทุกตัวในโลก ไม่ว่าไลบรารีไหน ประกอบด้วยของ **4 อย่างเสมอ** — พอจำได้แล้วโค้ด 72 บรรทัดข้างล่างจะเหลือแค่ "ทำให้เจอบว่าของ 4 อย่างนี้อยู่ตรงไหน": ① **สิ่งที่อยากให้น้อยที่สุด** (objective) · ② **จุดเริ่มเดา** (w0) · ③ **กรอบที่ห้ามออกนอก** (bounds — รายตัว) · ④ **เงื่อนไขที่ต้องเป็นจริง** (constraints — ข้ามตัว) · ที่เหลือคือ docstring กับการจัด DataFrame ให้สวย

```
from scipy.optimize import minimize
import numpy as np
import pandas as pd
```

```

import warnings

def optimize_sharpe(
    returns: pd.DataFrame,
    constraints: dict = None,
) -> tuple:
    """
    หา portfolio weights ที่ maximize Sharpe ratio

    Parameters
    -----
    returns : pd.DataFrame    daily returns
    constraints : dict        min_weight, max_weight, sum_to_one

    Returns
    -----
    (pd.Series weights, float sharpe)
    """
    if constraints is None:
        constraints = {"min_weight": 0.0, "max_weight": 1.0, "sum_to_one": True}

    n = len(returns.columns)
    mu = returns.mean().values * 252
    cov = returns.cov().values * 252

    def neg_sharpe(w: np.ndarray) -> float:
        port_ret = w @ mu
        port_vol = np.sqrt(w @ cov @ w)
        if port_vol < 1e-10:
            return 0.0
        return -(port_ret / port_vol)

    bounds = [(constraints["min_weight"],
                  constraints["max_weight"])] * n

    opt_constraints = []
    if constraints.get("sum_to_one", True):
        opt_constraints.append(
            {"type": "eq", "fun": lambda w: w.sum() - 1.0}
        )

    w0 = np.ones(n) / n # equal weight seed

    result = minimize(
        neg_sharpe,
        w0,
        method="SLSQP",
        bounds=bounds,
        constraints=opt_constraints,
        options={"maxiter": 1000, "ftol": 1e-9},
    )

    if not result.success:
        warnings.warn(
            f"Optimization did not converge: {result.message}",
            RuntimeWarning,
        )

    optimal_weights = pd.Series(result.x, index=returns.columns)
    optimal_sharpe = -result.fun
    return optimal_weights, optimal_sharpe

```

```
# ทดสอบ
np.random.seed(42)
n_days = 252
assets = ["BTC", "ETH", "SOL"]
ret_data = pd.DataFrame(
    np.random.multivariate_normal(
        mean=[0.0005, 0.0007, 0.0010],
        cov=[[0.0002, 0.00015, 0.0001],
              [0.00015, 0.0003, 0.00012],
              [0.0001, 0.00012, 0.0005]],
        size=n_days,
    ),
    columns=assets,
)

weights, sharpe = optimize_sharpe(ret_data)
print("Optimal weights:")
for asset, w in weights.items():
    print(f" {asset}: {w:.1%}")
print(f"Expected Sharpe: {sharpe:.2f}")
```

► OUTPUT

```
Optimal weights:
BTC: 0.0%
ETH: 0.0%
SOL: 100.0%
Expected Sharpe: 1.01
```

🤖 ของ 4 อย่างในโค้ดนี้อยู่ตรงไหน

```
def neg_sharpe(w):
    port_ret = w @ mu
    port_vol = np.sqrt(w @ cov @ w)
    return -(port_ret / port_vol)

w0 = np.ones(n) / n
bounds = [(0.0, 1.0)] * n
opt_constraints = [{"type": "eq",
                    "fun": lambda w: w.sum() - 1.0}]
```

① สิ่งที่ยากให้หน่อยที่สุด
ผลตอบแทนพอร์ต
ความผันผวนพอร์ต
คิดลบ = พลิก max เป็น min

② จุดเริ่มเดา = ถือเท่ากันทุกตัว
③ กรอบรายตัว: 0-100%
④ เงื่อนไขข้ามตัว:
น้ำหนักรวม = 1

1. $w @ \mu$ — @ คือ **dot product** เขียนสั้น ๆ ของ $(w[0]*\mu[0] + w[1]*\mu[1] + \dots)$ · อ่านเป็นภาษาคนว่า "ถ่วงน้ำหนักผลตอบแทนของแต่ละตัวตามสัดส่วนที่ถือ"
2. $\text{np.sqrt}(w @ \text{cov} @ w)$ — สูตรความผันผวนของพอร์ต · เหตุที่ **ไม่ใช่** การถ่วงน้ำหนัก sd ธรรมดา เพราะสินทรัพย์ไม่ได้ขึ้นลงพร้อมกันเป๊ะ ๆ · ตัวนอกแนวทแยงของ cov คือส่วนที่บอกว่ามันไปด้วยกันแค่ไหน — และเป็นที่มาของคำว่า "กระจายความเสี่ยง" ทั้งหมด
3. $\text{return } -(\dots)$ — **scipy มีแต่ minimize ไม่มี maximize** · อยากได้ Sharpe สูงสุดจึงใส่ลบแล้วหาค่าต่ำสุดแทน · ตอนคืนค่าจึงต้องกลับเครื่องหมายอีกรอบ ($\text{optimal_sharpe} = -\text{result.fun}$) ไม่งั้นจะรายงาน Sharpe ติดลบทั้งที่พอร์ตดี
4. **bounds กับ constraints ต่างกันตรงไหน** — bounds คุณทีละตัวแยกกัน ("BTC ต้องอยู่ระหว่าง 0-100%") · constraints คุณความสัมพันธ์ข้ามตัว ("รวมกันต้องได้ 1") · optimizer จัดการคนละวิธี bounds ถูกบังคับตลอดทาง ส่วน constraints ถูกไล่เข้าหาทีละรอบ · เขียนสลับที่กันจะช้าลงมากหรือไม่ converge เลย
5. $\{"type": "eq", "fun": \text{lambda } w: w.\text{sum}() - 1.0\}$ — รูปแบบที่ scipy กำหนด: เขียนเงื่อนไขให้อยู่ในรูป "**ฟังก์ชันนี้ต้องเท่ากับศูนย์**" เสมอ · อยากได้ $\text{sum}(w) = 1$ จึงต้องเขียนเป็น $\text{sum}(w) - 1$ · ถ้าเป็น "ineq" ความหมายคือ "ต้อง ≥ 0 "

อ่านผลลัพธ์ให้ออก — optimizer ตอบว่า "ทุ่มหมดหน้าตัก SOL"

ผลที่ได้คือ **BTC 0% · ETH 0% · SOL 100%** ทั้งที่โจทย์ให้สินทรัพย์มาสามตัวเพื่อกระจายความเสี่ยง · นี่คือสิ่งที่เรียกว่า **corner solution** และเป็นพฤติกรรมปกติของ mean-variance optimizer ไม่ใช่ bug ของโค้ด

โค้ดทำถูกทุกบรรทัด · optimizer ก็หาคำตอบเจอจริง · ปัญหาอยู่ที่**ของที่บ่อนเข้าไป** — และนี่คือกับดักที่ทำให้ backtest จำนวนมากในโลกไร้ค่า

ลองเปิดดูตัวเลขที่ optimizer ใช้ตัดสินใจ เทียบกับค่าจริงที่เราใช้สร้างข้อมูล (เรารู้เพราะเราเขียนเอง — ในตลาดจริงไม่มีใครรู้):

```
# optimizer เห็นอะไร vs ความจริงเป็นอย่างไร
true_mu_annual = np.array([0.0005, 0.0007, 0.0010]) * 252 # ค่าที่เราใส่ตอนสร้าง
est_mu_annual = ret_data.mean().values * 252 # ค่าที่วัดได้จาก 252 วัน
true_sd_annual = np.sqrt(np.array([0.0002, 0.0003, 0.0005]) * 252)
est_sd_annual = ret_data.std().values * np.sqrt(252)

# ค่าเฉลี่ยจาก n ปี มีค่าคลาดเคลื่อนมาตรฐาน = sd / sqrt(n) → 1 ปี ก็คือ sd เต็ม ๆ
print(f"{'':5}{'ผลตอบแทนจริง':>14}{'ที่วัดได้':>12}{'±คลาดเคลื่อน':>14}")
for i, a in enumerate(assets):
    print(f"{a:5}{true_mu_annual[i]:>+13.1%}{est_mu_annual[i]:>+12.1%}"
          f"{true_sd_annual[i]:>+14.1%}")

print(f"\n{'':5}{'ความผันผวนจริง':>15}{'ที่วัดได้':>12}")
for i, a in enumerate(assets):
    print(f"{a:5}{true_sd_annual[i]:>+14.1%}{est_sd_annual[i]:>+12.1%}")

# ต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงแม่นยำระดับ 1/3 ของตัวมันเอง
years_needed = (3 * true_sd_annual / true_mu_annual) ** 2
print(f"\nต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงจะพอเชื่อได้:")
for i, a in enumerate(assets):
    print(f" {a}: {years_needed[i]:.0f} ปี")
```

► OUTPUT

ผลตอบแทนจริง	ที่วัดได้	±คลาดเคลื่อน	
BTC	+12.6%	-22.5%	+22.4%
ETH	+17.6%	-47.5%	+27.5%
SOL	+25.2%	+33.4%	+35.5%

ความผันผวนจริง	ที่วัดได้	
BTC	22.4%	21.3%
ETH	27.5%	26.2%
SOL	35.5%	33.2%

ต้องใช้ข้อมูลกี่ปี ค่าเฉลี่ยจึงจะพอเชื่อได้:

BTC: 29 ปี
ETH: 22 ปี
SOL: 18 ปี

ตัวเลขที่ควรจำไปทั้งชีวิต

ความจริงคือ**ทั้งสามตัวกำไรหมด** (+12.6% / +17.6% / +25.2%) · แต่ข้อมูล 252 วันที่สุ่มออกมาบอกว่า BTC ขาดทุน 22.5% และ ETH ขาดทุน 47.5% — **ผิดกระทั้งเครื่องหมาย** · optimizer เห็นแบบนั้นก็ทำงานของมันอย่างซื่อสัตย์: ตัดสองตัวที่ "ขาดทุน" ทั้ง เหลือตัวเดียว

ดูคอลัมน์ ±คลาดเคลื่อนให้ดี · ค่าคลาดเคลื่อนของค่าเฉลี่ยรายปีจากข้อมูล 1 ปี = ความผันผวนรายปีเต็ม ๆ ของ BTC คือ ±22.4% ในขณะที่ค่าจริงคือ +12.6% — **ความคลาดเคลื่อนใหญ่กว่าสิ่งที่จะวัด** · จึงต้องรอถึง 29 ปีกว่าจะแม่นยำพอ

ตรงข้ามกับตารางที่สอง: **ความผันผวนวัดได้แม่นยำมาก** (22.4 vs 21.3) จากข้อมูลชุดเดียวกัน · สาเหตุคือ sd ประมาณจาก "ขนาดของการแกว่ง" ซึ่งมีให้เก็บทุกแห่ง ส่วนค่าเฉลี่ยประมาณจาก "ระยะทางสุทธิจากต้นถึงปลาย" ซึ่งมีแค่ค่าเดียวไม่ว่าจะเก็บกี่แค่ไหน — **เก็บข้อมูลรายนาทีก็ไม่ช่วยให้รู้ค่าเฉลี่ยแม่นยำขึ้น มีแต่ต้องรอนานขึ้นเท่านั้น**

ข้อสรุปไม่ใช่ "อย่าใช้ optimizer" แต่คือ อย่าบ่อนค่าเฉลี่ยที่ประมาณจากอดีตเข้าไปตรง ๆ . ทางออกที่ใช้กันจริงมีสามแนว:

1. เอาค่าเฉลี่ยออกจากสมการ — ใช้ minimum-variance หรือ risk-parity แทน maximum-Sharpe . ทั้งคู่ใช้แค่ `cov` ซึ่งเป็นตัวเลขที่เชื่อได้จริงตามตารางที่สอง (แบบฝึกหัดข้างล่างคือข้อนี้พอดี)
2. บีบกรอบให้แคบลง — เปลี่ยน `max_weight` จาก 1.0 เป็นราว 0.4 . เป็นการยอมรับตรง ๆ ว่า "ฉันไม่เชื่อค่าเฉลี่ยของตัวเองพอที่จะทุ่มหมดหน้าตัก" . หยาบแต่ได้ผลดีอย่างน่าประหลาด
3. ดึงค่าประมาณเข้าหาค่ากลาง (shrinkage / Black-Litterman) — แทนที่จะเชื่อ `-47.5%` ของ ETH ก็ผสมกับค่าเฉลี่ยรวมของทั้งกลุ่ม . เป็นวิธีที่กองทุนใช้จริง

มีผลการศึกษาลาสสิกที่ยืนยันไปในทางเดียวกันว่า พอร์ตถือเท่ากันทุกตัว (`1/N` — ซึ่งก็คือ `w0` ที่โค้ดนี้ใช้เป็นแค่จุดเริ่มต้น) เอาชนะ mean-variance optimizer นอกกลุ่มตัวอย่างได้บ่อยครั้ง . ไม่ใช่เพราะ `1/N` ฉลาด แต่เพราะมันไม่ต้องประมาณค่าเฉลี่ยเลย

► แบบฝึกหัด: เขียน prompt สำหรับ linear programming หา minimum variance portfolio พร้อม sector constraints

บทที่ 30: Full Strategy with AI

แก่นของบทนี้

ขั้นตอนจริงของการ build strategy ด้วย AI: ไม่ใช่ prompt ครั้งเดียวแล้วได้โค้ดสมบูรณ์ — แต่เป็นกระบวนการ 3 รอบ iterate ที่แต่ละรอบ แก้ปัญหาที่พบจากการ audit รอบก่อน

30.1 Overview — Volatility Mean-Reversion Strategy

แนวคิด: เมื่อ realized volatility สูงผิดปกติ (z-score > threshold) vol มักจะ mean-revert กลับลงมา ในช่วง high vol มักเป็นช่วง sell-off หนัก → หลัง vol spike ราคาขึ้น → long underlying เมื่อ vol spike

Entry signal = 1[vol_zscore_t > z_{entry}]
Exit signal = 1[vol_zscore_t < z_{exit}]

DESIGN → ROUND 1 → AUDIT → ROUND 2 → AUDIT → ROUND 3 → FINAL (บท 26) (prompt) (บท 28) (แก้ bug) (verify) (costs) (deploy-ready)

30.2 Round 1 — Generate โค้ดแรก

ROUND 1 PROMPT

```
Task: เขียน full backtest สำหรับ volatility mean-reversion strategy Components ที่ต้องการ: 1. compute_vol_zscore(prices, vol_window, zscore_window) → DataFrame 2. generate_signal(vol_zscore, entry_z, exit_z) → pd.Series ('LONG'/'EXIT'/'HOLD') 3. run_backtest(prices, signal, fee_per_trade) → DataFrame ที่มี equity curve Input (run_backtest): - prices: pd.Series, daily close prices - signal: pd.Series, output จาก generate_signal - fee_per_trade: float = 0.001 Output (run_backtest): - pd.DataFrame: date | price | signal | position | daily_pnl | equity Constraints: - signal ณ วัน t ต้องใช้แค่ข้อมูลถึง t-1 (no lookahead) - position เข้าที่ราคา open วันถัดไป (simulate ด้วย price.shift(-1) ของ signal day) - หัก fee ทุกครั้งที่ position เปลี่ยน - equity เริ่มที่ 100 (% ของ initial capital) - return NaN ช่วงต้นที่ไม่มีข้อมูลพอ Libraries: pandas, numpy
```

📌อ่านว่า: โค้ดที่จะได้ยาว 59 บรรทัด — อย่าอ่านไล่ทีละบรรทัดจากบนลงล่าง นั่นคือวิธีอ่านที่ทำให้หลง · วิธีที่เร็วกว่ามากคืออ่าน 3 signature ก่อน (compute_vol_zscore → generate_signal → run_backtest) ให้เห็นว่าของออกจากตัวหนึ่งไปเข้าตัวไหนเป็นสายพานเส้นเดียว: ราคา → z-score ของความผันผวน → สัญญาณข้อความ → equity curve · แล้วค่อยเจาะเข้าไปในตัวที่มีตรรกะเงินจริง คือ run_backtest เท่านั้น · อีกสองตัวเป็นการคำนวณตรงไปตรงมาที่ไล่ทันอยู่แล้ว

```
# Round 1: โค้ดที่ AI สร้าง (อาจมี bug บางข้อ)
import numpy as np
import pandas as pd

def compute_vol_zscore(
    prices: pd.Series,
    vol_window: int = 21,
    zscore_window: int = 63,
) -> pd.DataFrame:
    log_ret = np.log(prices / prices.shift(1))
    ann_vol = (
        log_ret.rolling(vol_window, min_periods=vol_window).std()
        * np.sqrt(252) * 100
    )
```

```

roll_mean = ann_vol.rolling(zscore_window, min_periods=zscore_window).mean()
roll_std = ann_vol.rolling(zscore_window, min_periods=zscore_window).std()
vol_zscore = (ann_vol - roll_mean) / roll_std
return pd.DataFrame({"log_ret": log_ret, "ann_vol": ann_vol,
                    "vol_zscore": vol_zscore})

def generate_signal(
    vol_zscore: pd.Series,
    entry_z: float = 1.5,
    exit_z: float = 0.5,
) -> pd.Series:
    sig = pd.Series("HOLD", index=vol_zscore.index)
    sig[vol_zscore > entry_z] = "LONG"
    sig[vol_zscore.abs() < exit_z] = "EXIT"
    sig[vol_zscore.isna()] = "HOLD"
    return sig

def run_backtest(
    prices: pd.Series,
    signal: pd.Series,
    fee_per_trade: float = 0.001,
) -> pd.DataFrame:
    """
    Backtest engine สำหรับ long-only strategy
    entry: open ของวันถัดไปหลัง signal
    """
    df = pd.DataFrame({"price": prices, "signal": signal}).copy()

    # position: 1 = long, 0 = flat
    # ใช้ signal วัน t เพื่อ set position วัน t+1
    position = pd.Series(0, index=df.index)
    pos = 0
    for i in range(1, len(df)):
        prev_sig = df["signal"].iloc[i - 1]
        if prev_sig == "LONG":
            pos = 1
        elif prev_sig == "EXIT":
            pos = 0
        position.iloc[i] = pos

    df["position"] = position

    # Daily P&L
    log_ret = np.log(df["price"] / df["price"].shift(1))
    df["daily_pnl"] = df["position"].shift(1) * log_ret  # ← shift(1): ถูกต้อง

    # Transaction costs
    turnover = df["position"].diff().abs().fillna(0)
    df["daily_pnl"] -= turnover * fee_per_trade

    # Equity curve (start at 100)
    df["equity"] = 100 * np.exp(df["daily_pnl"].fillna(0).cumsum())
    return df

```

🤖 หัวใจของ backtest ทั้งตัวอยู่ที่ 4 บรรทัดนี้

```

log_ret = np.log(df["price"] / df["price"].shift(1))  # ① ผลตอบแทนของ "ตลาด"
df["daily_pnl"] = df["position"].shift(1) * log_ret  # ② ของ "เรา"

```

```
turnover = df["position"].diff().abs().fillna(0) # ③ วันที่มีการซื้อขาย
df["daily_pnl"] -= turnover * fee_per_trade

df["equity"] = 100 * np.exp(df["daily_pnl"].fillna(0).cumsum()) # ④ ทบต้น
```

1. $\text{np.log}(p / p.\text{shift}(1))$ — ผลตอบแทนแบบ log ของวันนั้น · ที่ใช้ log ไม่ใช่ `pct_change()` เพราะบรรทัด ④ ต้องการ**บวกกันได้**: ผลตอบแทน log ของหลายวันเอามาบวกกันตรง ๆ แล้วได้ผลตอบแทนรวม ส่วนเปอร์เซ็นต์ธรรมดาต้องคูณกัน
2. $\text{position}.\text{shift}(1) * \text{log_ret}$ — **บรรทัดที่กันโกงทั้งเล่มอยู่ตรงนี้** · `log_ret` ของวันนี้คือระยะทางจากปิดเมื่อวานถึงปิดวันนี้ ดังนั้นคนที่จะได้เงินก่อนนั้นต้อง**ถือของอยู่แล้วตั้งแต่เมื่อวาน** — ซึ่งคือ $\text{position}.\text{shift}(1) \cdot \text{ถ้าลืม}.\text{shift}(1)$ จะกลายเป็น "รู้ราคาปิดวันนี้ก่อนแล้วค่อยตัดสินใจ" = lookahead แบบคลาสสิก
3. $\text{position}.\text{diff}().\text{abs}()$ — `diff()` คือ "วันนี้ต่างจากเมื่อวานเท่าไร" · 0 → 1 ได้ +1, 1 → 0 ได้ -1, ไม่เปลี่ยนได้ 0 · ใส่ `.abs()` เพราะ**ทั้งเข้าและออกเสียค่าธรรมเนียมเท่ากัน** ไม่มีขาไหนได้เงินคืน · ผลคือ Series ที่เป็น 1 เฉพาะวันที่มีการซื้อขายจริง คุณค่าธรรมเนียมแล้วหักออกได้เลย
4. $100 * \text{np.exp}(\text{pnl}.\text{cumsum}())$ — `cumsum()` สะสมผลตอบแทน log ทั้งหมดตั้งแต่วันแรก แล้ว `exp()` แปลงกลับเป็นตัวคูณ · เริ่มที่ 100 หมายความว่าตัวเลขบน equity curve อ่านเป็น**เปอร์เซ็นต์ของทุนตั้งต้น**ได้ทันที (120 = กำไร 20%)

สังเกตว่าทั้งสี่บรรทัด**ไม่มีลูป** — คิดทั้งคอลัมน์พร้อมกัน · ลูป for ตัวเดียวในฟังก์ชันนี้เกี่ยวกับการแปลงสัญญาณข้อความเป็น position เพราะ position วันนั้นขึ้นกับ position เมื่อวาน (สถานะที่ "ค้าง" ต่อกันไปเรื่อย ๆ) ซึ่งเป็นงานประเภทที่ vectorize ตรง ๆ ไม่ได้

30.3 Round 1 Audit — หา Bug ก่อน Round 2

```
# Audit Round 1: ตรวจสอบทุก checkpoint

np.random.seed(7)
n = 500
prices_sim = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0003, 0.015, n))),
    index=pd.date_range("2022-01-01", periods=n)
)

vol_df = compute_vol_zscore(prices_sim, vol_window=21, zscore_window=63)
signals = generate_signal(vol_df["vol_zscore"], entry_z=1.5, exit_z=0.5)
result = run_backtest(prices_sim, signals, fee_per_trade=0.001)

print("=== Round 1 Audit ===")
print(f"1. NaN ใน vol_zscore (ช่วงต้น): {vol_df['vol_zscore'].isna().sum()} rows")
print(f"2. Signal distribution:\n{signals.value_counts()}")
print(f"3. Position 0/1 only: {set(result['position'].unique())}")
print(f"4. First non-NaN equity row: {result['equity'].first_valid_index()}")
print(f"5. Final equity: {result['equity'].iloc[-1]:.2f}")

# Embargo test
print("\n=== Embargo Test ===")
def run_with_prices(p):
    vdf = compute_vol_zscore(p)
    sig = generate_signal(vdf["vol_zscore"])
    return vdf # DataFrame with vol_zscore column

# ตรวจสอบ lookahead โดยตรง: zscore ณ วัน 200 ต้องไม่เปลี่ยนเมื่อตัดข้อมูลหลังวัน 200
vdf_full = compute_vol_zscore(prices_sim)
vdf_cut = compute_vol_zscore(prices_sim.iloc[:250])

diff_at_200 = abs(
    vdf_full["vol_zscore"].iloc[200] - vdf_cut["vol_zscore"].iloc[200]
)

print(f"Vol zscore diff at day 200 (full vs cut): {diff_at_200:.8f}")
print("PASS" if diff_at_200 < 1e-10 else "FAIL: lookahead bias detected!")
```


► OUTPUT

```
=== Round 1 Audit ===
1. NaN ใน vol_zscore (ช่วงต้น): 83 rows
2. Signal distribution:
HOLD    322
EXIT    111
LONG     67
Name: count, dtype: int64
3. Position 0/1 only: {np.int64(0), np.int64(1)}
4. First non-NaN equity row: 2022-01-01 00:00:00
5. Final equity: 84.03
=== Embargo Test ===
Vol zscore diff at day 200 (full vs cut): 0.00000000
PASS
```

Bug ที่พบใน Round 1

Equity เริ่มต้นนับแม้ช่วงที่มี NaN — `first_valid_index` คืน 2022-01-01 ทั้งที่ position ยังเป็น 0 อยู่ 83 วัน ไม่ใช่ bug ร้ายแรง แต่ต้องระวังเมื่อคำนวณ Sharpe ratio (อย่านับ NaN period เข้าไป)

ข้อที่ audit ช่างบ่นยังจับไม่ได้ — และวิธีจับมัน

audit ชุดแรกตรวจสอบทุกอย่างที่ prompt เขียนไว้เป็นข้อ ๆ ยกเว้นข้อเดียว: *"position เข้าที่ราคา open วันถัดไป"*. ข้อนี้ไม่มี checkpoint ไหนตรวจสอบเลย เพราะมันไม่ใช่คำถามว่า "ผลลัพธ์หน้าตาถูกไหม" แต่เป็นคำถามว่า **"หน่วงกี่แท่ง"** — ซึ่งดูจากตัวเลขสรุปไม่ออก ต้องไล่ทีละแถว · เครื่องมือที่ตอบคำถามแบบนี้ได้ใน 5 บรรทัดคือ **ข้อมูลของเล่นที่คำตอบรู้ล่วงหน้า**: ให้ราคาขึ้นวันละ 1% เป๊ะ ๆ แล้วยิงสัญญาณครั้งเดียว วันไหนที่เงินเริ่มไหลคือคำตอบ

```
# ข้อมูลของเล่น: ขึ้นวันละ 1% คงที่ + สัญญาณครั้งเดียว → นับหน่วงได้ด้วยตา
idx_toy = pd.date_range("2022-01-01", periods=6)
prices_toy = pd.Series(100 * 1.01 ** np.arange(6), index=idx_toy)
sig_toy = pd.Series("HOLD", index=idx_toy)
sig_toy.iloc[1] = "LONG"

r_toy = run_backtest(prices_toy, sig_toy, fee_per_trade=0.0)
print(r_toy[["price", "signal", "position", "daily_pnl"]].round(5).to_string())

sig_day = idx_toy.get_loc(sig_toy[sig_toy == "LONG"].index[0])
pnl_day = idx_toy.get_loc(r_toy.index[r_toy["daily_pnl"].fillna(0) != 0][0])
print(f"\nสัญญาณแท่งที่ {sig_day} · เริ่มได้กำไรแท่งที่ {pnl_day} "
      f"· หน่วง {pnl_day - sig_day} แท่ง")
```

► OUTPUT

	price	signal	position	daily_pnl
2022-01-01	100.00000	HOLD	0	NaN
2022-01-02	101.00000	LONG	0	0.00000
2022-01-03	102.01000	HOLD	1	0.00000
2022-01-04	103.03010	HOLD	1	0.00995
2022-01-05	104.06040	HOLD	1	0.00995
2022-01-06	105.10101	HOLD	1	0.00995

สัญญาณแท่งที่ 1 · เริ่มได้กำไรแท่งที่ 3 · หน่วง 2 แท่ง

Bug ที่ 2 — หน่วงเกินไป 1 แท่ง

prompt สั่งไว้ว่าหน่วง **1 แท่ง** (สัญญาณวัน t → เข้าที่ open วัน $t+1$ → ได้ผลตอบแทนของวัน $t+1$) แต่โค้ดจริงหน่วง **2 แท่ง** · สาเหตุคือมันหน่วงสองรอบซ้อนโดยไม่ตั้งใจ:

1. ลูป for อ่าน `signal.iloc[i-1]` มาตั้ง `position.iloc[i]` — หน่วงรอบที่ ①

2. `df["position"].shift(1) * log_ret` — หน่วงรอบที่ ②

แต่ละรอบ แยกกันดูถูกต้องทั้งคู่ — นี่คือการทดสอบที่มั่นคงปลอดภัย · ทั้งสองบรรทัดอยู่ห่างกัน 15 บรรทัด และต่างก็ตอบโจทย์ "กันมองอนาคต" คนละข้อ

bug นี้ไม่อันตราย — หน่วงเกินคือ *อนุรักษนิยมเกิน* ผลลัพธ์จะแย่กว่าความจริง ตรงข้ามกับ lookahead ที่ทำให้ดูดีเกินจริง · แต่ก็ยังผิดจากที่สั่ง และในกลยุทธ์ที่สัญญาณหมดอายุเร็ว (เช่น mean reversion รายชั่วโมง) การเข้าไป 1 แท่งก็กำไรหายทั้งกลยุทธ์ได้

● ทำไม embargo test ที่หนังสือเขียนไว้จับข้อนี้ไม่ได้

เพราะ embargo test ตรวจสอบอย่าง · มันถามว่า "ตัดข้อมูลอนาคตทิ้งแล้ว ค่าที่คำนวณไว้เปลี่ยนไหม" — ตอบว่าไม่เปลี่ยน (PASS) และก็ควรจะเป็นแบบนั้น เพราะการหน่วงเกินยิ่งทำให้ไม่มีทางแตะข้อมูลอนาคตอยู่แล้ว

บทเรียนที่ใช้ต่อได้: **test แต่ละตัวมีขอบเขตที่มันมองเห็นของมัน** · embargo test เห็นเฉพาะ "ตะแวนาคหรือเปล่า" · การจะเห็น "หน่วงกี่แท่ง" ต้องใช้ test อีกชนิดคือ **impulse response** — ยิ่งสัญญาณเดียวบนข้อมูลที่นิ่งสนิท แล้วดูว่าระบบตอบสนองเมื่อไหร่ · เป็นเทคนิคเดียวกับที่วิศวกรใช้วัด latency ของระบบควบคุม และใช้ได้กับทุก pipeline ที่มี `shift` ซ้อนกันหลายชั้น

30.4 Round 2 — เพิ่ม Performance Metrics และแก้ Sharpe Calculation

ROUND 2 PROMPT

Task: เพิ่มฟังก์ชัน `compute_metrics` บน output ของ `run_backtest` ปัญหาที่พบใน Round 1: - Sharpe ratio คำนวณรวม period ที่ `position = 0` และ `daily_pnl = 0` ทำให้ Sharpe ต่ำเกิน - ต้องการ max drawdown, win rate, และ profit factor ด้วย Input: - `result_df`: `pd.DataFrame` output จาก `run_backtest` - `start_equity`: float = 100 Output: - dict: `sharpe`, `annual_return`, `max_drawdown`, `win_rate`, `profit_factor`, `n_trades`, `avg_holding_days` Constraints: - คำนวณ Sharpe เฉพาะวันที่ `position != 0` (active days only) - `max_drawdown` = max peak-to-trough ของ equity curve - `win_rate` = fraction of trades with positive net pnl - `profit_factor` = `sum(wins) / abs(sum(losses))` - return inf ถ้าไม่มี losses - `n_trades` = จำนวนครั้งที่ `position` เปลี่ยนจาก 0 เป็น 1 Libraries: `numpy`, `pandas`

📖อ่านว่า: 83 บรรทัดข้างล่างคำนวณตัวเลข 7 ตัว แต่แบ่งเป็น 2 กลุ่มที่คิดคนละวิธีสิ้นเชิง — จับให้ได้ตั้งแต่แรกแล้วที่เหลือง่าย · กลุ่ม "รายวัน" (`sharpe` · `annual_return` · `max drawdown`) คิดจากคอลัมน์ทั้งคอลัมน์รวดเดียว ไม่สนว่าเทรดไหนเป็นเทรดไหน · กลุ่ม "รายเทรด" (`win_rate` · `profit_factor` · `n_trades` · `avg_holding_days`) ต้องรู้ก่อนว่า "เทรดหนึ่งครั้ง" เริ่มตรงไหนจบตรงไหน จึงต้องมีลูปจับคู่วันเข้ากับวันออก · ลูป `for` ตัวเดียวในฟังก์ชันนี้มีไว้เพื่อเรื่องนี้เท่านั้น

```
def compute_metrics(
    result_df: pd.DataFrame,
    start_equity: float = 100.0,
) -> dict:
    """
    คำนวณ performance metrics จาก backtest result

    Parameters
    -----
    result_df : pd.DataFrame  output จาก run_backtest
    start_equity : float      starting equity (default 100)

    Returns
    -----
    dict: sharpe, annual_return, max_drawdown, win_rate,
          profit_factor, n_trades, avg_holding_days
    """
    df = result_df.copy()

    # Sharpe เฉพาะวันที่ active (position != 0)
    active_pnl = df["daily_pnl"][df["position"] != 0].dropna()
```

```

if len(active_pnl) > 1:
    sharpe = (active_pnl.mean() / active_pnl.std()) * np.sqrt(252)
else:
    sharpe = float("nan")

# Annual return
valid_equity = df["equity"].dropna()
n_days = len(valid_equity)
if n_days > 1:
    total_return = valid_equity.iloc[-1] / start_equity - 1
    annual_return = (1 + total_return) ** (252 / n_days) - 1
else:
    annual_return = float("nan")

# Max drawdown
equity = valid_equity.values
peak = np.maximum.accumulate(equity)
drawdown = (equity - peak) / peak
max_dd = float(drawdown.min())

# Identify individual trades (position changes: 0-1 and 1-0)
pos_diff = df["position"].diff().fillna(0)
entry_dates = df.index[pos_diff == 1].tolist()
exit_dates = df.index[pos_diff == -1].tolist()

# match entries and exits
trade_pnls = []
holding_days = []
for i, entry in enumerate(entry_dates):
    exits_after = [e for e in exit_dates if e > entry]
    if not exits_after:
        continue
    exit_date = exits_after[0]
    trade_slice = df.loc[entry:exit_date, "daily_pnl"].dropna()
    trade_pnl = trade_slice.sum()
    trade_pnls.append(trade_pnl)
    holding_days.append(len(trade_slice))

n_trades = len(trade_pnls)
win_rate = (
    sum(1 for p in trade_pnls if p > 0) / n_trades
    if n_trades > 0 else float("nan")
)

wins = sum(p for p in trade_pnls if p > 0)
losses = abs(sum(p for p in trade_pnls if p <= 0))
profit_factor = wins / losses if losses > 0 else float("inf")

avg_holding = (
    sum(holding_days) / len(holding_days)
    if holding_days else float("nan")
)

return {
    "sharpe": round(sharpe, 3),
    "annual_return": round(annual_return, 4),
    "max_drawdown": round(max_dd, 4),
    "win_rate": round(win_rate, 3) if not pd.isna(win_rate) else float("nan"),
    "profit_factor": round(profit_factor, 3),
    "n_trades": n_trades,
    "avg_holding_days": round(avg_holding, 1) if not pd.isna(avg_holding) else float("nan"),
}

```

```
# ทดสอบ Round 2
metrics = compute_metrics(result)
print("=== Round 2 Metrics ===")
for k, v in metrics.items():
    if isinstance(v, float) and not pd.isna(v):
        if "return" in k or "drawdown" in k or "rate" in k:
            print(f" {k:20s}: {v:.1%}")
        else:
            print(f" {k:20s}: {v:.3f}")
    else:
        print(f" {k:20s}: {v}")
```

► OUTPUT

```
=== Round 2 Metrics ===
sharpe           : -1.681
annual_return    : -8.4%
max_drawdown     : -23.4%
win_rate         : 16.7%
profit_factor    : 0.150
n_trades         : 6
avg_holding_days : 19.800
```

🛠️ 3 ท่าที่ใช้ซ้ำได้ทุกครั้งที่ต้องวัดผลกลยุทธ์

```
active_pnl = df["daily_pnl"][df["position"] != 0].dropna() # ① กรองด้วย boolean

peak      = np.maximum.accumulate(equity)                # ② ยอดสูงสุดที่เคยไปถึง
drawdown  = (equity - peak) / peak

pos_diff   = df["position"].diff().fillna(0)              # ③ หาจุดเปลี่ยนสถานะ
entry_dates = df.index[pos_diff == 1].tolist()
exit_dates  = df.index[pos_diff == -1].tolist()
```

1. `df["daily_pnl"][df["position"] != 0]` — ข้างในวงเล็บเหลี่ยมไม่ใช่เลขลำดับ แต่เป็น **Series** ของ **True/False** ยาวเท่ากัน · pandas จะคืนเฉพาะแถวที่เป็น True · อ่านออกเสียงว่า "เอา daily_pnl เฉพาะวันที่ position ไม่ใช่ศูนย์" · **ทำไมต้องกรอง** — วันที่ไม่ได้ถืออะไร ถ้าเป็น 0 เป๊ะ ถ้านับรวมเข้าไป ส่วนเบี่ยงเบนมาตรฐานจะถูกเจือจางจนดูนิ่งผิดปกติ แล้ว Sharpe จะสูงเกินจริง
2. `np.maximum.accumulate` — "ยอดสูงสุดที่เคยไปถึง ณ แต่ละจุด" เช่น `[100, 105, 103, 108]` ได้ `[100, 105, 105, 108]` · ต่างจาก `.max()` ที่ให้เลขเดียว · พอเอา equity ปัจจุบันลบ peak แล้วหารด้วย peak ก็ได้ **drawdown ณ ทุกวันที่** · `.min()` ของมันคือจุดที่เจ็บที่สุดที่เคยผ่านมา
3. `position.diff()` คู่กับ `df.index[...]` — `diff()` ได้ +1 ตอนเข้า -1 ตอนออก · `df.index[เงื่อนไข]` คืนวันที่ของแถวที่ตรงเงื่อนไข ไม่ใช่ค่าในแถว · เป็นท่ามาตรฐานสำหรับคำถาม "มันเกิดขึ้นเมื่อไหร่บ้าง"

ส่วนลูปจับคู่ข้างล่าง (`exits_after = [e for e in exit_dates if e > entry]`) มี**สมมติฐาน**ซ่อนอยู่ที่เราควรเห็น: มันเลือกวันออกตัวแรกที่มาหลังวันเข้า ซึ่งถูกต้องเฉพาะกับกลยุทธ์ที่ถือได้ทีละหนึ่งไม้เท่านั้น · ถ้าเปลี่ยนไปทำกลยุทธ์ที่เข้าซ้อนหลายไม้ ฟังก์ชันนี้จะนับผิดโดยไม่ error

30.5 Round 3 — Walk-Forward Validation

📖 **อ่านว่า:** walk-forward คือการเลียนแบบสิ่งที่เกิดขึ้นจริงตามเวลา · ใช้อัตตคติสงสัย แล้วดูว่าอนาคตที่ยังไม่เคยเห็นให้ผลอย่างไร · แล้วเลื่อนหน้าต่างไปข้างหน้าทำซ้ำ · โค้ดข้างล่างยาวก็จริงแต่ใครมีแค่ while วงเดียวที่เลื่อน start ทีละ `step_size` · ในแต่ละรอบมีของ 3 อย่าง: ตัดช่วงข้อมูล → รัน backtest → เก็บตัวเลขลง `results` · ตอนจบแปลง `results` เป็น DataFrame ที่เดียว — เป็นแพทเทิร์นที่จะเจอซ้ำไปตลอด: สะสมลง list ก่อน แล้วค่อยสร้าง DataFrame ครั้งเดียวตอนท้าย เพราะการต่อ DataFrame ทีละแถวช้ากว่ามาก

ROUND 3 PROMPT

Task: เพิ่ม walk-forward validation บน volatility mean-reversion strategy ปัญหา Round 2: Sharpe -1.681 วัดจาก in-sample ชุดเดียว – ต้องตรวจ out-of-sample ว่าผลนี้คงที่ข้ามช่วงเวลาใหม่ หรือแค่บังเอิญของ sample นี้ Input: - prices: pd.Series - train_size: int = 252 (1 ปีแรก train) - test_size: int = 63 (3 เดือน test) - step_size: int = 63 (เลื่อน 3 เดือนทีละรอบ) - params: dict = {'vol_window': 21, 'zscore_window': 63, 'entry_z': 1.5, 'exit_z': 0.5} Output: - pd.DataFrame: fold | train_start | train_end | test_start | test_end | sharpe_train | sharpe_test | n_trades Constraints: - parameter ต้อง fit บน train window เท่านั้น - test window ต้องไม่ overlap กับ train window - vol_zscore ในแต่ละ fold ต้องคำนวณ fresh บน train data นั้น - report mean และ std ของ test Sharpe ข้าม folds Libraries: pandas, numpy

```
def walk_forward_validation(
    prices: pd.Series,
    train_size: int = 252,
    test_size: int = 63,
    step_size: int = 63,
    params: dict = None,
) -> pd.DataFrame:
    """
    Walk-forward validation สำหรับ vol mean-reversion strategy

    Parameters
    -----
    prices : pd.Series    daily close prices
    train_size : int      training window (days)
    test_size : int       test window (days)
    step_size : int       rolling step (days)
    params : dict         strategy parameters

    Returns
    -----
    pd.DataFrame: fold results with in/out-of-sample Sharpe
    """
    if params is None:
        params = {
            "vol_window": 21,
            "zscore_window": 63,
            "entry_z": 1.5,
            "exit_z": 0.5,
        }

    results = []
    n = len(prices)
    fold = 0

    start = 0
    while start + train_size + test_size <= n:
        train_end = start + train_size
        test_end = train_end + test_size

        train_prices = prices.iloc[start:train_end]
        test_prices = prices.iloc[train_end:test_end]

        # ---- คำนวณ feature ครั้งเดียวบน "ประวัติต่อเนื่อง" train+test ----
        # ⚠️ ห้ามคำนวณ vol_zscore บน test window ล้วน ๆ . เหตุผล 2 ข้อ:
        # 1) ผิดหลัก – วันแรกของ test ในชีวิตจริงเรามีข้อมูล train
        #    อยู่ในมือแล้ว การทิ้งไม่ใช้การกั้นมองอนาคต แต่คือการโยน
        #    ข้อมูลที่มีสิทธิ์ใช้ทิ้งเปล่า ๆ
        # 2) พังเจียบ – test_size (63) ไม่พอเลี้ยง zscore_window (63)
        #    เพราะ ann_vol กิน 20 แถวแรกไปเป็น NaN แล้ว
        #    → vol_zscore เป็น NaN ทั้งชุด → ไม่มีสัญญาณ → 0 เทวด

        # ... (rest of the function code) ...
```

```

# โค้ดไม่ error เลย แต่ผลลัพธ์เป็น NaN ทุก fold
# ยังไม่มองขนาดคดียุติ เพราะ rolling ทุกตัวเป็น center=False
hist_prices = prices.iloc[start:test_end]

vdf_hist = compute_vol_zscore(
    hist_prices,
    vol_window=params["vol_window"],
    zscore_window=params["zscore_window"],
)
sig_hist = generate_signal(
    vdf_hist["vol_zscore"],
    entry_z=params["entry_z"],
    exit_z=params["exit_z"],
)
res_hist = run_backtest(hist_prices, sig_hist)

# ---- ตัดผลออกเป็นช่วง train / test แล้วค่อยวัดแยกกัน ----
res_train = res_hist.iloc[:train_size]
res_test = res_hist.iloc[train_size:]

m_train = compute_metrics(res_train)
# equity ของช่วง test ไม่ได้เริ่มที่ 100 (มันต่อมาจาก train)
# ต้องบอก start_equity ให้ตรง ไม่งั้น annual_return จะเพี้ยน
m_test = compute_metrics(
    res_test, start_equity=res_test["equity"].iloc[0]
)

results.append({
    "fold": fold,
    "train_start": train_prices.index[0].date(),
    "train_end": train_prices.index[-1].date(),
    "test_start": test_prices.index[0].date(),
    "test_end": test_prices.index[-1].date(),
    "sharpe_train": m_train["sharpe"],
    "sharpe_test": m_test["sharpe"],
    "n_trades_test": m_test["n_trades"],
})

fold += 1
start += step_size

return pd.DataFrame(results)

```

```

# สร้างข้อมูล 3 ปี สำหรับ walk-forward
np.random.seed(42)
n_wf = 252 * 3
prices_wf = pd.Series(
    100 * np.exp(np.cumsum(np.random.normal(0.0003, 0.015, n_wf))),
    index=pd.date_range("2021-01-01", periods=n_wf)
)

wf_results = walk_forward_validation(prices_wf)
print("=== Walk-Forward Results ===")
print(wf_results[["fold", "train_start", "test_start",
    "sharpe_train", "sharpe_test",
    "n_trades_test"]].to_string(index=False))
print(f"\nMean test Sharpe: {wf_results['sharpe_test'].mean():.3f}")
print(f"Std test Sharpe: {wf_results['sharpe_test'].std():.3f}")
print(f"% folds > 0: ")
print(f"{(wf_results['sharpe_test'] > 0).mean():.0%}")

```

► OUTPUT

=== Walk-Forward Results ===

fold	train_start	test_start	sharpe_train	sharpe_test	n_trades_test
0	2021-01-01	2021-09-10	-1.330	0.843	1
1	2021-03-05	2021-11-12	0.496	NaN	0
2	2021-05-07	2022-01-14	0.496	-0.400	1
3	2021-07-09	2022-03-18	-0.589	-3.872	1
4	2021-09-10	2022-05-20	-1.488	NaN	0
5	2021-11-12	2022-07-22	-2.759	1.084	1
6	2022-01-14	2022-09-23	0.367	0.626	1
7	2022-03-18	2022-11-25	0.877	-2.791	0

Mean test Sharpe: -0.752

Std test Sharpe: 2.089

% folds > 0: 38%

การตีความผล Walk-Forward — คำตอบคือ "อย่าเอาไปใช้"

- **Mean test Sharpe -0.752** — ติดลบ · ไม่มีอะไรให้ deploy
- **Std ข้าม fold = 2.089** — ค่าเหวี่ยงตั้งแต่ +1.084 ถึง -3.872 · ส่วนเบี่ยงเบนใหญ่กว่าค่าเฉลี่ยเกือบ 3 เท่า แปลว่าตัวเลขเฉลี่ยไม่มีความหมาย ต่อให้มันบังเอิญเป็นบวกก็ตาม
- **n_trades_test = 0 หรือ 1 ในทุก fold · รวมทั้งหมด 5 เทวดใน 2 ปี** — นี่คือข้อที่สำคัญที่สุด · Sharpe ที่คำนวณจากเทรดเดียวไม่ใช่ Sharpe มันคือผลของเทรดนั้นเทรดเดียว · fold 3 ได้ -3.872 เพราะเทรดเดียวนั้นขาดทุนไม่ได้แปลว่ากลยุทธ์แยกว่า fold อื่น 4 เท่า
- **2 fold ได้ NaN** — ไม่มีวันไหนถือ position เลย · Sharpe จึงคำนวณไม่ได้ · NaN ตรงนี้ไม่ใช่ bug แต่เป็นข้อมูลจริงว่า "ไตรมาสนั้นไม่มีสัญญาณเลย"

ข้อสรุปที่ถูกต้องคือหยุด — ไม่ใช่ไปปรับ entry_z จนกว่าตัวเลขจะสวย · การกลับไปหมุนพารามิเตอร์หลังเห็นผล test แล้วคือ **overfitting** ที่ใช้ test set เป็นเชื้อเพลิง ซึ่งเป็นบาปข้อที่บทที่ 28 เตือนไว้ทั้งบท

ทำไมบทนี้จบด้วยกลยุทธ์ที่ใช้ไม่ได้ — และทำไมนั่นคือของจริง

ราคาที่ใช้ทดสอบทั้งบทสร้างจาก np.random.normal — เป็น **random walk** บริสุทธิ์ ที่ไม่มีรูปแบบให้จับตั้งแต่แรก · ดังนั้นผลที่ถูกต้องเพียงผลเดียวคือ "ไม่มี edge" · ถ้าตัวเลขออกมาสวยนั้นต่างหากที่ควรทำให้ตกใจ เพราะแปลว่าเครื่องมือวัดของเราเสีย

สังเกตลำดับที่เกิดขึ้นในบทนี้ ซึ่งเหมือนกับที่เกิดในงานจริงทุกประการ:

1. **Round 1** — โค้ดรันได้ equity curve ออกมาเป็นเส้นสวย ๆ · ดูเหมือนมีอะไร
2. **Round 2** — พรวัดด้วย metric ที่ถูกต้อง Sharpe ออกมา -1.681 · เริ่มเห็นความจริง
3. **Round 3** — walk-forward ยืนยันว่าไม่มีอะไรเลย และบอกเพิ่มด้วยว่าข้อมูลน้อยเกินกว่าจะสรุปอะไรได้ตั้งแต่ต้น (5 เทวดใน 2 ปี)

คุณค่าของ walk-forward ไม่ใช่การรับรองกลยุทธ์ แต่คือการห้ามไม่ให้ deploy ของที่ยังไม่ได้พิสูจน์ · ในอาชีพนี้ไอเดียส่วนใหญ่ตายตรงนี้ และนั่นคือการทำงานที่ถูกต้องของเครื่องมือ

● สิ่งที่ต้องแก้ก่อน ถ้าจะทำ walk-forward นี้ให้ตัดสินใจอะไรได้จริง

ปัญหาที่แท้จริงไม่ใช่ "Sharpe ติดลบ" แต่คือ การออกแบบ fold ไม่มีกำลังทางสถิติที่จะตอบคำถามอะไรได้เลย · กลยุทธ์เทรดราว 2-3 ครั้งต่อปี แต่หน้าต่าง test ยาวแค่ 63 วัน — โดยเฉลี่ยจึงได้ไม่ถึง 1 เทรดต่อ fold · ไม่ว่าผลจะออกมาบวกหรือลบก็ตีความไม่ได้ทั้งคู่

ทางแก้เรียงตามลำดับที่ควรลอง:

1. **ขยาย test_size ให้กินอย่างน้อย ~30 เทรด** — สำหรับกลยุทธ์ความถี่นี้แปลว่าหน้าต่างระดับหลายปี ซึ่งแปลว่าข้อมูล 3 ปีไม่พอทำ walk-forward ตั้งแต่ต้น · รู้ข้อนี้ก่อนเขียนโค้ดประหยัดเวลาไปทั้งสัปดาห์
2. **เปลี่ยนไปวัดสิ่งที่มีตัวอย่างเยอะกว่า** — เช่นวัดที่ระดับ "วันที่ถือ position" แทน "เทรด" หรือใช้ bootstrap บนผลตอบแทนรายวันเพื่อหาช่วงความเชื่อมั่นของ Sharpe แทนที่จะดูค่าเดียว
3. **ลดความถี่ของคำถาม** — ถ้าข้อมูลตอบได้แค่ "กลยุทธ์นี้แย่ชัด ๆ ไหม" ก็อย่าไปถามมันว่า "Sharpe เท่าไร"

กฎที่ใช้ได้ตลอดชีวิต: ก่อนรัน walk-forward ให้ประมาณจำนวนเทรดต่อ fold ก่อน · ถ้าได้ตัวเลขหลักหน่วย ผลลัพธ์จะเป็นเสียงรบกวนล้วน ๆ ไม่
ว่าโค้ดจะถูกแก้ไขไหม

30.6 สรุป: Final Strategy Code (โครงที่พร้อม — ยังไม่ใช้กลยุทธ์ที่พร้อม)

โค้ดข้างล่างคือการรวมทุกอย่างในบทนี้เป็นคลาสเดียว · สิ่งที่ "พร้อม" คือโครงสร้าง — แยก feature / signal / backtest ออกจากกัน มี parameter เป็น
dataclass เปลี่ยนค่าได้โดยไม่ต้องแก้โค้ด · ส่วนตัวกลยุทธ์เองยังสอบไม่ผ่าน ตามผล walk-forward ข้างบน · อ่านโค้ดนี้ในฐานะแม่แบบที่จะเอาไปแก้ไขตัว
ถัดไปมาใช้ไม่ใช้ของที่เอาไปเปิดเทรดได้

```
# Final integrated strategy — หลัง 3 รอบ iterate
import numpy as np
import pandas as pd
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class VolMeanReversionParams:
    """Parameters สำหรับ Volatility Mean-Reversion Strategy"""
    vol_window: int = 21
    zscore_window: int = 63
    entry_z: float = 1.5
    exit_z: float = 0.5
    fee_per_trade: float = 0.001
    max_position: float = 1.0 # fraction ของ capital

class VolMeanReversionStrategy:
    """
    Volatility Mean-Reversion Strategy

    Logic:
    - Long underlying เมื่อ realized vol spike สูงผิดปกติ
    - Exit เมื่อ vol กลับสู่ระดับปกติ

    สถานะการตรวจ (อย่าลบบรรทัดพวกนี้ — มันคือเหตุผลที่ยังไม่ deploy):
    - Embargo test: PASS ไม่มี lookahead bias
    - Walk-forward: FAIL mean test Sharpe -0.752, ผ่านแค่ 38% ของ fold
      และมีแค่ 5 เทรดใน 2 ปี = สรุปอะไรไม่ได้
    - ข้อสรุป: โครงสร้างใช้ได้ · กลยุทธ์ยังไม่ผ่าน · ห้ามต่อเข้าเงินจริง
    - Transaction costs included
    """

    def __init__(self, params: Optional[VolMeanReversionParams] = None):
        self.params = params or VolMeanReversionParams()

    def compute_features(self, prices: pd.Series) -> pd.DataFrame:
        """คำนวณ features — no lookahead bias"""
        p = self.params
        log_ret = np.log(prices / prices.shift(1))
        ann_vol = (
            log_ret.rolling(p.vol_window, min_periods=p.vol_window).std()
            * np.sqrt(252) * 100
        )
        roll_mean = ann_vol.rolling(p.zscore_window, min_periods=p.zscore_window).mean()
        roll_std = ann_vol.rolling(p.zscore_window, min_periods=p.zscore_window).std()
        vol_zscore = (ann_vol - roll_mean) / roll_std

        return pd.DataFrame({
```



```

        "log_ret":    log_ret,
        "ann_vol":    ann_vol,
        "vol_zscore": vol_zscore,
    })

def signal(self, features: pd.DataFrame) -> pd.Series:
    """สร้าง signal – ใช้เฉพาะข้อมูลปัจจุบัน"""
    p = self.params
    z = features["vol_zscore"]
    sig = pd.Series("HOLD", index=z.index)
    sig[z > p.entry_z] = "LONG"
    sig[z.abs() < p.exit_z] = "EXIT"
    sig[z.isna()] = "HOLD"
    return sig

def backtest(self, prices: pd.Series) -> dict:
    """รัน full backtest และคืน metrics + equity curve"""
    features = self.compute_features(prices)
    signals = self.signal(features)
    result = run_backtest(prices, signals, self.params.fee_per_trade)
    metrics = compute_metrics(result)
    return {"metrics": metrics, "equity": result["equity"],
            "signals": signals, "features": features}

def live_signal(self, prices: pd.Series) -> str:
    """สำหรับ live trading – คืน signal ล่าสุด"""
    features = self.compute_features(prices)
    signals = self.signal(features)
    return signals.iloc[-1]

# ใช้งาน
strategy = VolMeanReversionStrategy()
result = strategy.backtest(prices_wf)

print("=== Final Strategy Metrics (In-Sample) ===")
for k, v in result["metrics"].items():
    if isinstance(v, float) and not pd.isna(v):
        if "return" in k or "drawdown" in k or "rate" in k:
            print(f" {k:22s}: {v:.1%}")
        else:
            print(f" {k:22s}: {v:.3f}")
    else:
        print(f" {k:22s}: {v}")

# Live signal ปัจจุบัน
print(f"\nCurrent signal: {strategy.live_signal(prices_wf)}")

```

► OUTPUT

```

=== Final Strategy Metrics (In-Sample) ===
sharpe           : -0.947
annual_return    : -3.6%
max_drawdown     : -15.4%
win_rate         : 40.0%
profit_factor    : 0.275
n_trades         : 10
avg_holding_days : 17.100
Current signal: LONG

```

Round	สิ่งที่เพิ่ม/แก้	ผลลัพธ์
Round 1	Core algorithm + embargo test	ไม่มี lookahead — แต่หน่วงเกินไป 1 แท่ง (เจอตอน audit)
Round 2	Active-days Sharpe, trade-level win rate, profit factor	วัดถูกขึ้น แล้วเห็นว่า Sharpe ตีกลับ
Round 3	Walk-forward validation (8 folds)	ครบ — mean test Sharpe $-0.752 \cdot 5$ เทวดาใน 2 ปี

อ่านตารางนี้ตามลำดับจะเห็นสิ่งที่บ่นบ่อยๆ: **ทุกรอบทำให้เห็นความจริงชัดขึ้น และความจริงก็แย่งเรื่อย ๆ** . นั่นคือรูปร่างปกติของงานวิจัยกลยุทธ์ที่ทำถูกวิธี . รอบที่ทำให้ตัวเลขดีขึ้นเรื่อย ๆ ต่างหากที่ควรสงสัยตัวเอง

► **แบบฝึกหัด:** เพิ่ม regime filter — เทวดาเฉพาะเมื่อ 200-day trend เป็น uptrend

จบ Part V — สิ่งที่ได้จากการเรียนรู้ AI-Assisted Coding

บท 26: Prompt template 4 ส่วน ช่วยให้ AI สร้างโค้ดที่ตรงความต้องการมากขึ้น ✓

บท 27: Review checklist 8 ข้อ + embargo test ตรวจ lookahead ทุกครั้ง ✓

บท 28: Red flags 5 ข้อ — shift(1), scaler leakage, transaction costs, p-hacking ✓

บท 29: Prompt templates เฉพาะสำหรับ OLS, ADF, z-score, scipy.optimize ✓

บท 30: 3 รอบ iterate: generate → audit → improve → walk-forward ✓ (และจบด้วยการ**ปฏิเสธ**กลยุทธ์ — ผลลัพธ์ที่พบบ่อยที่สุดในงานจริง)

AI เป็นผู้ช่วยที่ทรงพลัง แต่ **ความรับผิดชอบต่อความถูกต้องทางคณิตศาสตร์** ยังอยู่ที่เรา — ตรวจทุกบรรทัดที่ AI สร้าง ใช้ embargo test และ walk-forward validation เสมอ

ใน **Part VI** เราจะนำทุกอย่างที่เรียนมาสร้างระบบ production-grade: live data ingestion, order management, risk monitoring แบบ real-time

Async & Live Trading — เทรดจริงแบบอัตโนมัติ

บทที่ 31–35: Async Fundamentals · asyncio Patterns · WebSocket & Live Data · Order Execution · Live System Integration
จบ Part นี้ → ระบบ live trading อัตโนมัติที่รับ feed จริง ส่งออเดอร์จริง พร้อม kill switch และ monitoring

ตัวเลขใน ▶ OUTPUT ของ Part นี้อ่านอย่างไร

ต่างจาก Part อื่นตรงที่โค้ดใน Part นี้ **คุยกับโลกภายนอก** — ราคาจากตลาด เวลาที่ข้อความมาถึง ความเร็วเน็ต · ผลลัพธ์จึง **ไม่มีทางเหมือนกันสองครั้ง** แม้รันบนเครื่องเดียวกัน

ดังนั้นตัวเลขราคา · timestamp · จำนวนข้อความที่ได้รับ ในกล่อง OUTPUT ของ Part นี้เป็น **ตัวอย่างรูปแบบ** ให้ดูว่า "หน้าตาผลลัพธ์ควรเป็นแบบไหน" ไม่ใช่ค่าที่คุณต้องได้ตรงกัน · **สิ่งที่ต้องตรงคือโครงสร้าง** — มีฟิลด์ครบไหม ลำดับเหตุการณ์ถูกไหม error ถูกจัดการไหม

🔗 ถ้าอยากทดสอบโค้ด Part นี้โดยไม่ต่อเน็ตจริง: เขียน mock ที่คืนข้อความรูปแบบเดียวกับ exchange แล้วป้อนเข้าฟังก์ชัน parse — เป็นวิธีเดียวกับที่บทที่ 28 สอนเรื่องแยก "โค้ดดึงข้อมูล" ออกจาก "โค้ดคำนวณ"

🔗 อ่าน Part นี้ยังไ

🟢 **เพิ่งเคยเจอ async ครั้งแรก** — อ่านบทที่ 31 ให้เข้าใจที่สุดในเล่ม โดยเฉพาะกล่อง `async def / await / gather` · 4 คำนี้คือรากของทั้ง Part ถ้าข้ามไป บทที่ 32-35 จะกลายเป็นการลอกโค้ด · **เจอบล็อกโค้ดยาว ๆ ให้หากล่อง `asyncio.Queue` "โครงก่อนดูโค้ด" ที่อยู่นอมนั่นก่อนเสมอ** แล้วค่อยลงอ่านรายละเอียด

🟢 **เขียน async เป็นแล้ว** — กวาดบทที่ 31-32 ผ่านได้ · ของจริงสำหรับคุณอยู่ในกล่อง **[รอบสอง]** และ ⚠ กับดัก: thundering herd และ jitter, เหตุผลที่ไม่ดี Exception เปลา, gather กับ exception ที่ทั้ง task ค่า, backpressure ของ `asyncio.Queue`, และเส้นแบ่ง I/O-bound vs CPU-bound ที่ async ช่วยไม่ได้

ทำไม PART นี้ถึงสำคัญที่สุด?

Backtest บอกว่า strategy ทำกำไรได้ — แต่นั่นคือบน **ข้อมูลอดีต** ที่นิ่งอยู่ใน DataFrame
ในโลกจริง ราคาเปลี่ยนทุกมิลลิวินาที มีหลาย feed พร้อมกัน ต้องส่ง order ตรงเวลา ต้องจัดการ error ที่ไม่คาดคิด

- **Async** ให้ Python จัดการงานหลายอย่างพร้อมกันโดยไม่ต้องใช้ thread จำนวนมาก
- **WebSocket** รับ market data แบบ real-time แทน polling ทุก 1 วินาที
- **Order Execution** ส่ง order อย่างปลอดภัย มี rate limit และ retry
- **Kill Switch** หยุดระบบทันทีเมื่อเกิดเหตุฉุกเฉิน

🛑 STOP — อ่านก่อนเริ่ม Part VI: Production Safety Checklist

Part VI สอน async และ live trading กับเงินจริง ก่อนเดินหน้าให้ตอบ YES ทุกข้อ:

1. ✅ **API Keys อยู่ใน .env file** — ห้าม hardcode ใน code เด็ดขาด
2. ✅ **ทดสอบบน Testnet ครบ 48+ ชั่วโมง** — Bybit Testnet / Binance Testnet
3. ✅ **Kill switch พร้อม** — มีปุ่มหรือ Telegram command หยุดระบบทันที
4. ✅ **Drawdown limit ตั้งไว้แล้ว** — เช่น หยุดอัตโนมัติเมื่อขาดทุน 5%
5. ✅ **Position size ไม่เกิน 5% ของทุน ต่อ 1 trade** สำหรับ system ใหม่
6. ✅ **Log ทุก order** — timestamp, price, size, status เก็บไว้อย่างน้อย 30 วัน
7. ✅ **Monitor ตลอด 24 ชั่วโมงแรก** — อย่าปล่อยให้ระบบทำงานโดยไม่มีคนดู

ถ้ายังไม่ครบ — ทำ Part VI ต่อเพื่อเรียนรู้ได้ แต่อย่าเชื่อมกับ real account จนกว่าจะผ่านทุกข้อ

Running Example ตลอด Part VI — BTC Pair Strategy แบบ Live

เราจะนำ PairStrategy จาก Part II/III มาเปลี่ยนให้รันแบบ real-time:

บท 31 → เข้าใจ async · บท 32 → asyncio patterns · บท 33 → WebSocket feed
บท 34 → ส่ง order ผ่าน REST API · บท 35 → ระบบครบวงจรพร้อม deploy

คำแนะนำสำคัญก่อนเริ่ม Part VI

- **ทดสอบบน Testnet เสมอ** — ยานำ code จาก tutorial ไปรันบน real account โดยตรง
- **ห้าม hardcode API Key** — ใช้ environment variable หรือ secrets manager เท่านั้น
- **ต้องมี Kill Switch** — ทุกระบบ live ต้องหยุดได้ทันทีเมื่อกด Ctrl+C หรือรับ signal
- **Log ทุก order** — ถ้าไม่มี log จะตามสอบไม่ได้ว่าเกิดอะไรขึ้น

- **Monitor drawdown** — ตั้ง max drawdown threshold ให้ระบบหยุดอัตโนมัติ

บทที่ 31: Async Fundamentals

แก่นของบทนี้

Async คือวิธีให้ Python "รอ" อะไรบางอย่าง (เช่น network) โดยไม่บล็อก thread — ทำให้ทำงานหลายอย่างพร้อมกันใน thread เดียว เหมือนพนักงานร้านอาหาร 1 คนที่รับออเดอร์หลายโต๊ะพร้อมกัน แทนที่จะยืนรอโต๊ะแรกก่อนแล้วค่อยไปโต๊ะถัดไป

31.1 ปัญหาของ Blocking Code

ใน live trading เราต้องรับราคาจากหลาย exchange พร้อมกัน หาก code ทำงานแบบ blocking จะช้ามาก

```
# =====
# แบบ BLOCKING — รอทีละตัว (ช้า)
# =====

import time
import requests

def fetch_price_blocking(exchange: str) -> float:
    # จำลองการเรียก API (ใช้เวลา 1 วินาที)
    time.sleep(1)
    prices = {"binance": 65000.0, "bybit": 65010.0,
              "okx": 64990.0, "bitget": 65005.0}
    return prices[exchange]

def get_all_prices_blocking():
    start = time.time()
    results = {}
    for ex in ["binance", "bybit", "okx", "bitget"]:
        results[ex] = fetch_price_blocking(ex) # รอทีละตัว!
    elapsed = time.time() - start
    print(f"Blocking: ใช้เวลา {elapsed:.1f}s")
    return results

get_all_prices_blocking()
```

► OUTPUT

Blocking: ใช้เวลา 4.0s

```
# =====
# แบบ ASYNC — รอพร้อมกัน (เร็ว)
# =====

import asyncio
import time

async def fetch_price_async(exchange: str) -> float:
    # asyncio.sleep ไม่บล็อก event loop
    await asyncio.sleep(1)
    prices = {"binance": 65000.0, "bybit": 65010.0,
              "okx": 64990.0, "bitget": 65005.0}
    return prices[exchange]

async def get_all_prices_async():
    start = time.time()
    # gather รัน coroutine ทั้งหมดพร้อมกัน
    results = await asyncio.gather(
        fetch_price_async("binance"),
        fetch_price_async("bybit"),
        fetch_price_async("okx"),
        fetch_price_async("bitget"),
    )
    elapsed = time.time() - start
    print(f"Async: ใช้เวลา {elapsed:.1f}s")
    exchanges = ["binance", "bybit", "okx", "bitget"]
    return dict(zip(exchanges, results))
```

```
# รัน event loop
prices = asyncio.run(get_all_prices_async())
print(prices)
```

► OUTPUT

```
Async: ใช้เวลา 1.0s
{'binance': 65000.0, 'bybit': 65010.0, 'okx': 64990.0, 'bitget': 65005.0}
```

📌 **อ่านว่า:** 4.0s → 1.0s **ทั้งที่ยังรอ 1 วินาทีเท่าเดิมทุกตลาด** — เพราะเปลี่ยนจาก "รอทีละตัวต่อกัน" เป็น "เริ่มรอทั้ง 4 ตัวพร้อมกันแล้วรอครั้งเดียว" · เวลารวมจึงเท่ากับตัวที่ช้าที่สุด ไม่ใช่ผลรวม

📌 3 คำใหม่ที่เพิ่งโผล่พร้อมกัน — async, await, gather

```
async def fetch_price_async(exchange): # ❶ async def
    await asyncio.sleep(1)             # ❷ await
    ...

results = await asyncio.gather(        # ❸ gather
    fetch_price_async("binance"),
    fetch_price_async("bybit"),
)

prices = asyncio.run(get_all_prices_async()) # ❹ run
```

ทั้ง 4 ตัวนี้ทำงานร่วมกันเป็นชุดเดียว อ่านตามลำดับนี้:

1. `async def` — "ฟังก์ชันนี้หยุดกลางคันได้" · เรียกแล้ว **ยังไม่ทำงานทันที** แต่คืน "ใบสั่งงาน" (coroutine) กลับมา — ต่างจาก `def` ปกติที่เรียกแล้วทำเลยจนจบ
2. `await` — "ตรงนี้ต้องรอ · ระหว่างรอ ปลอมให้คนอื่นทำงานไปก่อน" · จุดที่มี `await` คือจุดเดียวที่งานอื่นแทรกได้ ถ้าไม่มี `await` เลย `async` ก็ไม่ต่างจากโค้ดปกติ
3. `asyncio.gather(a, b, c)` — "เอาใบสั่งงานทั้งหมดนี้ไปเริ่มพร้อมกัน แล้วรอจนครบทุกใบ" · คืนผลลัพธ์เป็น list เรียงตามลำดับที่ใส่เข้าไป ไม่ใช่ตามลำดับที่เสร็จ
4. `asyncio.run(...)` — "เปิดสวิตช์ event loop แล้ววิ่งใบสั่งงานนี้จนจบ" · เป็นสะพานจากโลกปกติเข้าโลก `async` เรียกได้จากโค้ดธรรมดาเท่านั้น (ห้ามเรียกซ้อนใน `async def`)

จำง่าย: `async def` = เขียนใบสั่งงาน · `await` = จุดพัก · `gather` = เริ่มพร้อมกัน · `run` = เปิดสวิตช์

📌 ถ้ารู้สึกวุ่นวายกับฟังก์ชันที่หยุดค้างไว้แล้วไปต่อได้" คำนึงว่า มันคือแนวคิดเดียวกับ `yield` ในหัวข้อ 7.7 (Part I) · generator หยุดรอคนมาขอของชิ้นถัดไป · coroutine หยุดรองาน I/O ที่ยังไม่เสร็จ · กลไกเบื้องหลังคือตัวเดียวกัน ซึ่งเป็นเหตุผลที่ `async for` มีอยู่ได้

⚠️ กับดักที่ทำให้ `async` ไม่เร็วขึ้นเลย — `time.sleep` ปนใน `async def`

ถ้าผมเขียน `time.sleep(1)` แทน `await asyncio.sleep(1)` ข้างใน `async def` โค้ดจะยังรันได้ **ไม่ error** แต่กลับไปช้า 4 วินาทีเหมือนเดิม เพราะ `time.sleep` **แซ่แซ่ event loop ทั้งวง** — งานอื่นแทรกไม่ได้เลย

🔗 **กฎ:** ข้างใน `async def` ทุกการรอต้องมี `await` นำหน้า ถ้าเจอฟังก์ชันรอที่ไม่มี `await` ได้ (เช่น `requests.get`, `time.sleep`, การอ่านไฟล์ใหญ่) แปลว่ามันจะบล็อกทั้งระบบ

🔴 [รอบสอง] `async` ไม่ได้ทำให้ทุกอย่างเร็วขึ้น

`async` เร่งได้เฉพาะงานที่ "รออย่างอื่นอยู่" (I/O-bound: network, disk) เพราะมันแค่เอาเวลารอว่างระหว่างรอไปทำงานอื่น · งานที่ **CPU ทำงานเต็มที่** (I/O ไม่มี — เช่น คำนวณ backtest ล้านรอบ, optimize portfolio) `async` **ไม่ช่วยเลยแม้แต่นิดเดียว** เพราะไม่มีช่วง "ว่างระหว่างรอ" ให้แทรก — กรณีนั้นต้องใช้ `multiprocessing`

ในสาย quant เส้นแบ่งนี้ชัดมาก: **ดึงราคาจาก 10 ตลาด** = `async` ช่วยเต็ม ๆ · **รัน Monte Carlo 100k paths** = `async` เปล่าประโยชน์

อีกจุดที่มือโปรพลาดบ่อย: `gather` ถ้ามีตัวใดตัวหนึ่ง raise exception ตัวที่เหลือยังวิ่งต่อไปเบื้องหลัง และ exception แรกจะเด็นออกมาทันที — ถ้าอยากได้ผลทุกตัวรวม error ต้องใส่ `return_exceptions=True` ไม่งั้น production จะเจอ task ค้างแบบเงียบ ๆ

สรุปความแตกต่าง: Blocking vs Async

ด้าน	Blocking	Async
เวลา 4 API calls (1s each)	4 วินาที	~1 วินาที
CPU ขณะรอ network	นอนรออยู่เฉยๆ	ทำงานอื่นระหว่างรอ
Concurrency model	ต้องใช้ thread/process	1 thread พอ
Memory overhead	สูง (thread ละ ~8MB)	ต่ำ (coroutine ละ ~KB)
เหมาะกับ	งาน CPU-heavy	งาน I/O-heavy (network, disk)

31.2 async def และ await — หัวใจของ Async

```
# =====
# กฎพื้นฐาน 3 ข้อ
# =====

# 1. function ที่มี "await" ข้างใน ต้องเป็น "async def"
async def fetch_ticker(symbol: str) -> dict:
    await asyncio.sleep(0.1)      # await บอกว่า "รอตรงนี้ได้ ไปทำอื่นก่อน"
    return {"symbol": symbol, "price": 65000.0, "volume": 1234.5}

# 2. "await" ใช้ได้แค่ภายใน "async def" เท่านั้น
async def main():
    ticker = await fetch_ticker("BTCUSDT")  # ถูกต้อง
    print(ticker)

# 3. เรียก async function จาก synchronous code ด้วย asyncio.run()
asyncio.run(main())

# =====
# coroutine คือ object ที่ยังไม่รัน — ต้อง await หรือ schedule
# =====

coro = fetch_ticker("BTCUSDT")      # สร้าง coroutine object
print(type(coro))                  # <class 'coroutine'> — ยังไม่รัน!
result = asyncio.run(coro)          # รัน coroutine
print(result)
```

► OUTPUT

```
{'symbol': 'BTCUSDT', 'price': 65000.0, 'volume': 1234.5}
<class 'coroutine'>
{'symbol': 'BTCUSDT', 'price': 65000.0, 'volume': 1234.5}
```

31.3 Event Loop — หัวใจของ asyncio

Event loop คือ "ผู้จัดการ" ที่วิ่งอยู่เบื้องหลัง คอยตรวจว่า coroutine ไหนพร้อมรันแล้ว และสลับการทำงานระหว่าง coroutines

Event Loop Lifecycle: _____

```

[Task A: fetch Binance]—await—>[รอ network] | | | | |
| | | | | ← สลับมา Task B | network มา | | | | | [Task B: fetch Bybit]—await—>[รอ network] | | | | |
| | | | | ← สลับมา Task C | | | | | [Task C: process signal] ← ไม่รอ network → รันได้ทันที |
| | | | | _____ ทั้งหมดรันใน thread เดียว — สลับกันที่จุด "await"

```

```
import asyncio

async def demonstrate_event_loop():
    """แสดงการสลับ context ระหว่าง coroutines"""

    async def task_a():
        print("Task A: เริ่มต้น")
        await asyncio.sleep(0.2)  # ยกมือออก event loop: "ไปทำอื่นก่อน"
        print("Task A: กลับมาแล้ว")
        return "A done"

    async def task_b():
        print("Task B: เริ่มต้น")
        await asyncio.sleep(0.2)  # ยกมือออก event loop: "ไปทำอื่นก่อน"
        print("Task B: กลับมาแล้ว")
        return "B done"
```



```

    print("Task B: เริ่มต้น")
    await asyncio.sleep(0.1)    # รอน้อยกว่า A → จบก่อน
    print("Task B: กลับมาแล้ว")
    return "B done"

async def task_c():
    print("Task C: ไม่รอ network เลย")
    # await asyncio.sleep(0) บอก event loop ให้สลับสักระยะ
    await asyncio.sleep(0)
    print("Task C: จบแล้ว")
    return "C done"

results = await asyncio.gather(task_a(), task_b(), task_c())
print(f"Results: {results}")

asyncio.run(demonstrate_event_loop())

```

```

► OUTPUT
Task A: เริ่มต้น
Task B: เริ่มต้น
Task C: ไม่รอ network เลย
Task C: จบแล้ว
Task B: กลับมาแล้ว
Task A: กลับมาแล้ว
Results: ['A done', 'B done', 'C done']

```

31.4 ทำไม Blocking Code ถึงอันตรายใน Async

อันตราย: ห้ามใช้ **Blocking call** ภายใน **async function**

```

# ❌ ผิด - time.sleep() บล็อก event loop ทั้งหมด!
async def bad_fetch(exchange: str) -> float:
    time.sleep(1)          # บล็อก! coroutine อื่นหยุดรอด้วย
    return 65000.0

# ❌ ผิด - requests.get() บล็อก event loop!
async def bad_rest_call(url: str):
    resp = requests.get(url)  # บล็อก! ใช้ aiohttp แทน
    return resp.json()

# ✓ ถูกต้อง - asyncio.sleep ไม่บล็อก
async def good_fetch(exchange: str) -> float:
    await asyncio.sleep(1)    # yield control กลับ event loop
    return 65000.0

# ✓ ถูกต้อง - ใช้ aiohttp สำหรับ HTTP
import aiohttp
async def good_rest_call(url: str):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as resp:
            return await resp.json()

```

Blocking (อย่าใช้ใน async)	Async alternative
<code>time.sleep(n)</code>	<code>await asyncio.sleep(n)</code>
<code>requests.get(url)</code>	<code>await session.get(url)</code> (aiohttp)
<code>open(file).read()</code>	<code>await aiofiles.open(file).read()</code>
<code>socket.recv()</code>	WebSocket library async
CPU-heavy loop	<code>await loop.run_in_executor(None, fn)</code>

ตัวอย่าง: Async price aggregator สำหรับ BTC pair

```
import asyncio
import random
from dataclasses import dataclass
from datetime import datetime

random.seed(7) # ให้ตัวเลขในหนังสือตรงกับที่คุณรันได้

@dataclass
class Ticker:
    exchange: str
    symbol: str
    price: float
    timestamp: datetime

async def fetch_ticker_mock(exchange: str, symbol: str,
                           base_price: float, delay: float) -> Ticker:
    """จำลอง REST API call"""
    await asyncio.sleep(delay)
    price = base_price + random.uniform(-50, 50)
    return Ticker(exchange, symbol, price, datetime.utcnow())

async def aggregate_btc_prices() -> dict[str, Ticker]:
    """ดึงราคา BTC จากหลาย exchange พร้อมกัน"""
    tickers = await asyncio.gather(
        fetch_ticker_mock("binance", "BTCUSDT", 65000.0, 0.08),
        fetch_ticker_mock("bybit", "BTCUSDT", 65005.0, 0.12),
        fetch_ticker_mock("okx", "BTCUSDT", 64995.0, 0.10),
    )
    result = {t.exchange: t for t in tickers}

    prices = [t.price for t in tickers]
    spread = max(prices) - min(prices)
    mid = sum(prices) / len(prices)
    print(f"Mid: ${mid:,.1f} Spread: ${spread:.1f}")
    return result

asyncio.run(aggregate_btc_prices())
```

► OUTPUT

Mid: \$64,987.5 Spread: \$60.0

🧺 gather คืนของมาเรียงตามลำดับไหน

```
tickers = await asyncio.gather(
    fetch_ticker_mock("binance", ..., 0.08), # เสร็จที่ 0.08s ← เร็วสุด
    fetch_ticker_mock("bybit", ..., 0.12),  # เสร็จที่ 0.12s ← ช้าสุด
    fetch_ticker_mock("okx", ..., 0.10),    # เสร็จที่ 0.10s
)
# tickers = [binance, bybit, okx] — เรียงตามที่เขียน ไม่ใช่ตามเวลาที่เสร็จ

result = {t.exchange: t for t in tickers} # dict comprehension
```

1. **gather** คืนผลเรียงตามลำดับที่ใส่เข้าไปเสมอ ไม่ใช่ตามลำดับที่ทำเสร็จ · okx เสร็จก่อน bybit แต่ยังอยู่หลังใน list · ข้อนี้ทำให้เขียนโค้ดง่ายขึ้นมาก เพราะจับคู่ผลลัพธ์กับ input ด้วยตำแหน่งได้เลย ไม่ต้องแปะ id · (ถ้าอยากได้ผลตามลำดับที่เสร็จจริง ต้องใช้ `asyncio.as_completed` แทน)
2. **เวลารวม = ตัวที่ช้าที่สุด ไม่ใช่ผลบวก** — 0.12 วินาที ไม่ใช่ $0.08+0.12+0.10 = 0.30$ · นี่คือทั้งหมดที่ async ให้ · ถ้าเพิ่ม exchange ที่ตอบใน 0.09s เข้าไปอีก 20 เจ้า เวลารวมก็ยัง 0.12 วินาทีเท่าเดิม
3. `{t.exchange: t for t in tickers}` — **dict comprehension** · อ่านว่า "สำหรับทุก t ใน tickers สร้างคู่ key: value" · ก่อนโคลนคือ key หลังโคลนคือ value · ที่นี้คือแปลง list เป็น dict ที่ค้นด้วยชื่อ exchange ได้ (`result["bybit"]`) แทนที่จะต้องจำว่าอยู่ตำแหน่งที่เท่าไร

🔴 [รอบสอง] "spread" ที่คำนวณได้ตรงนี้ ส่วนหนึ่งเป็นของปลอม

สามราคานี้ไม่ได้ถ่ายจากเวลาเดียวกัน — binance เป็นราคา ณ 0.08 วินาที · okx ณ 0.10 · bybit ณ 0.12 · การเอามาลบกันคือการเทียบราคาที่ห่างกันถึง **40 มิลลิวินาที**

ในตลาดที่ขยับเร็ว ส่วนต่างที่เห็นจึงปนกันสองอย่างแยกไม่ออก: **ส่วนต่างจริงระหว่างตลาด** (ของที่เทรดได้) กับ **ส่วนต่างที่เกิดจากถ่ายรูปคนละวินาที** (ของปลอมที่หายไปพอส่งคำสั่งจริง) · นี่คือสาเหตุอันดับหนึ่งที่ scanner หา arbitrage มือใหม่รายงานโอกาสเต็มจอ แต่พอกดจริงไม่มีสักอัน

ทางแก้ที่ระบบจริงใช้ — เรียงตามที่เราควรทำก่อนหลัง:

- **ติด timestamp ของ exchange มากับราคาทุกครั้ง** (ไม่ใช่เวลาที่เรารับได้) แล้วทิ้งคู่ที่ห่างกันเกินเกณฑ์ เช่น 50ms — สังเกตว่าโค้ดข้างบนเก็บ timestamp ไว้ใน Ticker แล้ว แต่ *ไม่เคยเอามาใช้เลย* · นั่นคือช่องที่ต้องเติม
- **ใช้ WebSocket แทน REST** — ได้ราคาไหลเข้ามาต่อเนื่อง จับคู่ตาม timestamp ได้แน่นกว่าการยิงถามเป็นครั้ง ๆ มาก (เป็นเหตุผลที่บทที่ 33 ทิ้งบ๊วยด้วย WebSocket)
- **เทียบกับ latency ที่ต้องใช้จริง** — ถ้าส่วนต่างที่เจอเล็กกว่าที่เราขายได้ในเวลาที่เราส่งคำสั่งไปถึง มันไม่ใช่โอกาสไม่ว่าตัวเลขจะสวยแค่ไหน

อีกจุดที่ควรรู้: `datetime.utcnow()` คือนเวลาแบบ **naive** (ไม่มี timezone ติดมา) · ตั้งแต่ Python 3.12 ถูกประกาศเลิกใช้แล้ว และควรเปลี่ยนเป็น `datetime.now(timezone.utc)` · ในระบบเทรดเรื่องนี้ไม่ใช่แค่ความสวยงาม — เวลาแบบไม่มี timezone เอาไปลบกับเวลาที่มี timezone จะ **TypeError** ทันที และมักไปพังตอนต่อกับข้อมูลจาก exchange ที่ส่ง timezone มาด้วย

► **แบบฝึกหัด:** เพิ่ม timeout ให้ fetch — ถ้า exchange ไม่ตอบใน 0.5s ให้ข้ามไป

บทที่ 32: asyncio Patterns

แก่นของบทนี้

asyncio มี building blocks สำคัญ 4 อย่างสำหรับระบบ trading จริง: **gather()** สำหรับ concurrent fetch, **Queue** สำหรับ decouple producer/consumer, **wait_for()** สำหรับ timeout, และ **Task** สำหรับ background jobs ที่ cancel ได้

32.1 asyncio.gather() — รันพร้อมกัน

```
import asyncio
import random

random.seed(11)

async def fetch_ohlcv(exchange: str, symbol: str,
                     timeframe: str = "1m") -> list:
    """จำลองดึง OHLCV candle ล่าสุด"""
    await asyncio.sleep(0.1)
    p = 65000 + random.uniform(-200, 200)
    return [exchange, symbol, timeframe,
            {"open": p-10, "high": p+20, "low": p-30, "close": p,
             "volume": random.uniform(100, 500)}]

async def fetch_all_ohlcv():
    """ดึง OHLCV จากหลาย exchange/timeframe พร้อมกัน"""
    pairs = [
        ("binance", "BTCUSDT", "1m"),
        ("bybit", "BTCUSDT", "1m"),
        ("binance", "BTCUSDT", "5m"), # timeframe อื่นด้วย
        ("bybit", "BTCUSDT", "5m"),
    ]

    # gather ทุก coroutine พร้อมกัน
    results = await asyncio.gather(
        *[fetch_ohlcv(ex, sym, tf) for ex, sym, tf in pairs],
        return_exceptions=True # แทนที่จะ raise ทันที ให้เก็บ exception ใน result
    )

    for r in results:
        if isinstance(r, Exception):
            print(f"[ERROR] {r}")
        else:
            ex, sym, tf, candle = r
            print(f"{ex:8s} {sym} {tf:3s}: close={candle['close']:, .1f}")

asyncio.run(fetch_all_ohlcv())
```

► OUTPUT

```
binance BTCUSDT 1m : close=64,981.0
bybit   BTCUSDT 1m : close=65,169.7
binance BTCUSDT 5m : close=65,003.1
bybit   BTCUSDT 5m : close=64,873.9
```

return_exceptions=True — สวิตช์ที่เปลี่ยนพฤติกรรมทั้งระบบ

```
# ไม่ใส่ (ค่าเริ่มต้น): เจอ error ตัวแรก → เด้งออกทันที
results = await asyncio.gather(*tasks)
# → ถ้า bybit ล่ม เราไม่ได้ราคาของ binance กับ okx เลย ทั้งที่มันสำเร็จแล้ว

# ใส่: error กลายเป็น "ผลลัพธ์" ชนิดหนึ่ง วางเรียงในลิสต์ตามตำแหน่งเดิม
results = await asyncio.gather(*tasks, return_exceptions=True)
for r in results:
    if isinstance(r, Exception): # ← จึงต้องเช็คชนิดก่อนใช้เสมอ
        print(f"[ERROR] {r}")
```

1. **ค่าเริ่มต้นเหมาะกับ "ทั้งหมดหรือไม่เอาเลย"** — เช่นดึงขาสองข้างของ pair trade ถ้าขาดข้างหนึ่งก็คำนวณ spread ไม่ได้อยู่ดี · `return_exceptions=True` เหมาะกับ "ได้เท่าไหนเอาเท่านั้น" — เช่นสำรวจราคาจาก 10 ตลาด ล้มไป 2 ก็ยังทำงานต่อไป
 2. **ราคาที่ต่อจ่ายคือ `isinstance` ทุกครั้ง** — เพราะสมาชิกใน list ไม่ได้เป็นชนิดเดียวกันอีกต่อไป · *ลืมเช็ก = โปรแกรมจะพังที่บรรทัดถัดไปแทนด้วย error ที่ดูไม่ออกว่ามาจากไหน* (เช่น `TypeError: cannot unpack non-sequence`)
 3. **สิ่งที่มันไม่ทำ** — มันไม่ได้ทำให้ error หายไป และไม่ได้ `retry` ให้ · มันแค่ *เลื่อนการตัดสินใจ* ให้เราจัดการเอง · ถ้าเขียน `for` วนแล้วไม่เจอกรณี Exception เลย เท่ากับ **กลืน error เงียบ ๆ** ซึ่งอันตรายกว่าไม่ใส่ตั้งแต่แรก
- สังเกตว่าโค้ดข้างบนพิมพ์ `[ERROR]` ออกมาเลย ๆ แล้วไปต่อ · ในระบบจริงตรงนี้ต้อง **ตัดสินใจ** ว่าราคาที่เราขาดไปทำให้สัญญาณใช้ไม่ได้หรือไม่ — การพิมพ์แล้วเดินหน้าต่อเหมือนไม่มีอะไรเกิดขึ้น คือวิธีที่ระบบเทรดขาดทุนแบบเงียบที่สุด

32.2 asyncio.Queue — Decouple Producer และ Consumer

ใน live trading มี 2 ส่วนที่ทำงานเร็วต่างกัน: **feed** รับข้อมูลเร็ว และ **signal engine** คำนวณช้ากว่า Queue เป็นตัวกลางที่ buffer ข้อมูลไว้

Producer (WebSocket feed) Consumer (Signal Engine) ————— tick มาทุก 10ms
—put→ asyncio.Queue —get→ คำนวณ signal ไม่ต้องรอ consumer (buffer ≤ 1000 items) ส่ง order

📌 **อ่านว่า:** โค้ดข้างล่างมี 3 ฟังก์ชัน แต่ **สัญญาณระหว่างกันมีแค่ 4 บรรทัด** — จบ 4 บรรทัดนี้ได้แล้วที่เลือกอ่านผ่านได้เลย: ฝั่งผลิตใช้ `await queue.put(x)` · ฝั่งบริโภคใช้ `tick = await queue.get()` · ฝั่งผลิตปิดท้ายด้วย `await queue.put(None)` เป็น **สัญญาณว่าหมดแล้ว** · ฝั่งบริโภคเจอ `None` เมื่อไรก็ `break` · ที่เหลือคือการคำนวณ moving average ธรรมดาที่ไม่เกี่ยวกับ async เลย

```
import asyncio
import random
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class PriceTick:
    exchange: str
    symbol: str
    price: float
    timestamp: datetime = field(default_factory=datetime.utcnow)

async def price_producer(queue: asyncio.Queue, exchange: str,
                        symbol: str, n_ticks: int = 20):
    """จำลอง WebSocket feed ส่ง tick เข้า queue"""
    price = 65000.0
    for i in range(n_ticks):
        price += random.uniform(-50, 50)
        tick = PriceTick(exchange, symbol, round(price, 2))
        await queue.put(tick)
        print(f"[PROD] {exchange:8s} put tick #{i+1:2d}: {price:,.1f}")
        await asyncio.sleep(0.05) # จำลอง tick interval

    # ส่ง sentinel เพื่อบอก consumer ว่าจบแล้ว
    await queue.put(None)

async def signal_consumer(queue: asyncio.Queue, window: int = 5):
    """คำนวณ signal จาก tick ที่รับจาก queue"""
    prices = []
    processed = 0

    while True:
        tick = await queue.get() # รอจนกว่าจะมี item ใน queue
        if tick is None: # sentinel — จบ
            queue.task_done()
            break

        prices.append(tick.price)
        queue.task_done()
        processed += 1

    if len(prices) >= window:
        ma = sum(prices[-window:]) / window
        latest = prices[-1]
        signal = "LONG" if latest > ma else ("SHORT" if latest < ma else "HOLD")
        print(f"[CONS] tick #{processed:2d}: price={latest:,.1f} "
              f"signal={signal}")
```

```
f"MA(window)={ma:,.1f} - {signal}")

async def producer_consumer_demo():
    queue = asyncio.Queue(maxsize=100) # buffer สูงสุด 100 items

    # รัน producer และ consumer พร้อมกัน
    await asyncio.gather(
        price_producer(queue, "binance", "BTCUSDT", n_ticks=10),
        signal_consumer(queue, window=5),
    )
    print("จบ demo")

asyncio.run(producer_consumer_demo())
```

🔗 ทำไมต้องมี Queue — ทั้งที่เรียกฟังก์ชันตรง ๆ ก็ได้

```
# ถ้าไม่มี queue จะต้องเขียนแบบนี้ — ผลิตกับบริโภคมัดติดกัน
async def feed():
    while True:
        tick = await get_tick()
        compute_signal(tick) # - ถ้าตัวนี้ช้า tick ถัดไปก็ต้องรอ

# มี queue แล้ว: สองฝั่งวิ่งด้วยความเร็วของตัวเอง
await queue.put(tick) # ฝั่งผลิต - วางแล้วไปต่อทันที
tick = await queue.get() # ฝั่งบริโภค - ไม่มีของกินนอนรอ ไม่กิน CPU
```

1. `await queue.get()` — **await** ตรงนี้แปลว่า "ถ้ายังไม่มีของ ก็นอนรอ แล้วปล่อยให้ตัวอื่นทำงานไปก่อน" · ต่างจาก `while queue.empty(): pass` ซึ่งจะเผา CPU 100% และแย่กว่านั้นคือ *ไม่ยอมปล่อยคิว* ทำให้ producer ไม่มีโอกาสวางของลงเลย → ค้างถาวร
2. `await queue.put(None)` — **sentinel** · เหตุที่ต้องมี เพราะ consumer เดาไม่ได้ว่า "queue ว่าง" แปลว่าจบแล้ว หรือแค่ producer ยังผลิตไม่ทัน · ทั้งสองสถานะหน้าตาเหมือนกันเป๊ะ · จึงต้องให้ producer **บอกออกมาตรง ๆ** ว่าหมดแล้ว · `None` ใช้ได้เพราะ tick จริงเป็น `PriceTick` ไม่มีทางเป็น `None`
3. `asyncio.Queue(maxsize=100)` — ถ้าของเต็ม 100 ชิ้น `await queue.put(...)` จะ **หยุดรอ** จนกว่าจะมีที่ว่าง · เรียกว่า **backpressure** — เป็นการบอก producer ว่า "ช้าลงหน่อย ตามไม่ทันแล้ว" · ถ้าใส่ `maxsize=0` (ไม่จำกัด) แล้ว consumer ตามไม่ทัน queue จะโดนกินแรมหมดเครื่อง
4. `asyncio.gather(producer, consumer)` — เริ่มทั้งคู่พร้อมกันในลูปเดียว · ไม่ใช้ thread ไม่มี lock ไม่มี race condition ให้กังวล เพราะทั้งคู่ **สลับกันที่จุด await เท่านั้น** ไม่มีทางถูกขัดจังหวะกลางคัน

⚠️ `task_done()` ในโค้ดนี้ไม่ได้ทำอะไรเลย — และควรรู้ว่าทำไม

`queue.task_done()` มีไว้คู่กับ `await queue.join()` เท่านั้น · `join()` แปลว่า "รอจนกว่าของทุกชิ้นที่เคย `put` จะถูกรายงานว่าทำเสร็จแล้ว" และมันนับจากจำนวนครั้งที่เรียก `task_done()`

ในบทนี้ **ไม่มีการเรียก `join()` เลยสักที** — เราใช้ sentinel `None` บอกจุดจบแทน · ดังนั้น `task_done()` ทุกบรรทัดในหนังสือเล่มนี้เป็น **พิธีกรรมล้วน ๆ** สบออกก็ได้ผลเหมือนเดิม · ที่ยังเขียนไว้เพราะเป็นนิสัยที่ดีเวลาโค้ดขึ้นแล้วมีคนเดิม `join()` เข้ามาที่หลัง

🔗 **กฎ:** เห็น `task_done()` ที่ไหนให้มองหา `join()` ทันที · ถ้าไม่มีแปลว่าโค้ดนั้นใช้วิธีอื่นบอกจุดจบ (sentinel / cancel / event) — และถ้าไม่มีทั้งคู่ แปลว่าโปรแกรมเมอร์ไม่รู้ตัวตัวเองจบเมื่อไร

🔵 [รอบสอง] ในระบบเทรดจริง buffer ที่ใหญ่คือ **ปัญหา** ไม่ใช่ทางแก้

Queue เป็น **FIFO** — เข้าก่อนออกก่อน · นั่นถูกสำหรับงานทั่วไป แต่ **ผิดสำหรับราคา** · ถ้า signal engine ตามไม่ทันแล้วมี tick ค้างอยู่ 100 ชิ้น สิ่งที่มีนัยยะมาค่านวนคือ tick ที่ **เก่าที่สุด** · คุณส่งคำสั่งซื้อโดยอ้างอิงราคาเมื่อ 5 วินาทีที่แล้ว ทั้งที่ราคาล่าสุดวางอยู่ท้ายคิวแล้ว — และยิ่งตามไม่ทันนานเท่าไร ก็ยิ่งเทรดจากอดีตที่ไกลขึ้นเรื่อย ๆ

ทางเลือกที่ระบบจริงใช้ ขึ้นกับว่าข้อมูลนั้น **ทดแทนกันได้หรือไม่**:

- **ราคา / order book snapshot** — ของใหม่แทนที่ของเก่าได้สนิท · เก็บแค่ค่าล่าสุดพอ (ตัวแปรเดี่ยว หรือ `deque(maxlen=1)`) · ตกหล่นระหว่างทางไม่เป็นไร
- **fill / order update / trade execution** — ห้ามทิ้ง **เค็ดขาด** ทุกเหตุการณ์มีความหมายทางบัญชี · ต้องใช้ queue ที่ไม่จำกัดขนาด และถ้ามันโตขึ้นเรื่อย ๆ นั่นคือ **สัญญาณเตือนให้หยุดระบบ** ไม่ใช่ให้ขยาย buffer

ดังนั้นคำถามที่ต้องตอบก่อนเลือก `maxsize` ไม่ใช่ "ควรตั้งเท่าไร" แต่คือ **"ถ้าตามไม่ทัน ควรทิ้งของเก่าหรือควรหยุดทั้งระบบ"** · ระบบจริงมักมีทั้งสองแบบวิ่งคู่กัน คนละ queue

32.3 asyncio.wait_for() — Timeout

```
import asyncio

async def place_order_mock(order_id: str, delay: float = 0.3) -> dict:
    """จำลอง REST API order placement"""
    await asyncio.sleep(delay)
    return {"order_id": order_id, "status": "FILLED", "avg_price": 65000.0}

async def place_order_with_timeout(order_id: str,
                                   timeout: float = 1.0) -> dict | None:
    """ส่ง order พร้อม timeout – ถ้าช้าเกินไปถือว่า fail"""
    try:
        result = await asyncio.wait_for(
            place_order_mock(order_id, delay=0.5),
            timeout=timeout
        )
        print(f"[OK] Order {order_id}: {result['status']}")
        return result

    except asyncio.TimeoutError:
        print(f"[WARN] Order {order_id}: timeout หลัง {timeout}s")
        # ต้อง query status ภายหลัง – order อาจ filled หรือ rejected
        return None

asyncio.run(place_order_with_timeout("ORD-001", timeout=1.0))
```

► OUTPUT

```
[OK] Order ORD-001: FILLED
```

32.4 asyncio.Task — Background Jobs และ Cancellation

```
import asyncio

async def heartbeat(interval: float = 1.0):
    """ส่ง heartbeat ทุก interval วินาที – background task"""
    count = 0
    while True:
        count += 1
        print(f"[HB] Heartbeat #{count}")
        await asyncio.sleep(interval)

async def main_trading_loop(duration: float = 3.5):
    """Main loop – รัน heartbeat เป็น background task"""

    # สร้าง Task – รันใน background ทันที
    hb_task = asyncio.create_task(heartbeat(1.0), name="heartbeat")

    print(f"[MAIN] เริ่ม trading loop ({duration}s)")
    await asyncio.sleep(duration)  # จำลองการเทรด

    # Cancel background task เมื่อจบ
    hb_task.cancel()
    try:
        await hb_task
    except asyncio.CancelledError:
        print("[MAIN] Heartbeat task ถูก cancel แล้ว")

    print("[MAIN] จบ")

asyncio.run(main_trading_loop())
```

► OUTPUT

```
[MAIN] เริ่ม trading loop (3.5s)
[HB] Heartbeat #1
[HB] Heartbeat #2
[HB] Heartbeat #3
[HB] Heartbeat #4
```

```
[MAIN] Heartbeat task ถูก cancel แล้ว
[MAIN] จบ
```

📖**อ่านว่า:** นับ heartbeat ให้ดี — ได้ 4 ครั้ง ไม่ใช่ 3 ทั้งที่รันแค่ 3.5 วินาที · เพราะใน heartbeat() คำสั่ง print อยู่ก่อน await asyncio.sleep() ครั้งแรกจึงยังทันที่ t=0 แล้วตามด้วย t=1, 2, 3 · ถ้าสลับให้ sleep มาก่อน จะได้ 3 ครั้ง (t=1,2,3) · ลำดับของ print กับ await เปลี่ยนจำนวนรอบที่ได้จริง — จุดที่พลาดกันบ่อยตอนนับ interval

32.5 asyncio.Semaphore — จำกัดจำนวน Concurrent Requests

ดึงข้อมูลย้อนหลังทีเดียว 500 วัน ถ้ายิงพร้อมกันหมด exchange จะแบนทันที — Semaphore คือตัวคุมว่า "ให้มีงานวิ่งอยู่พร้อมกันได้ไม่เกิน N ชิ้น"

📖**อ่านว่า:** คิดว่า Semaphore คือตะกร้าที่มีบัตรผ่านอยู่ N ใบ · ใครจะยิง request ต้องหยิบบัตรไปหนึ่งใบก่อน · บัตรหมดเมื่อไร คนที่มาทีหลังต้องยืนรอนจนกว่าจะมีคนคืนบัตร · async with sem: คือการหยิบบัตรตอนเข้าและคืนบัตรตอนออกโดยอัตโนมัติ — คืนให้แม้โค้ดข้างในจะ error กลางคัน ซึ่งเป็นเหตุผลทั้งหมดที่ใช้ async with แทนการเขียน acquire/release เอง (ลืมคืนบัตรครั้งเดียว = โปรแกรมค้างถาวร)

```
import asyncio

async def fetch_historical(symbol: str, date: str,
                           sem: asyncio.Semaphore) -> dict:
    """ดึงข้อมูล historical พร้อม rate limiting"""
    async with sem:
        # เข้า critical section — ไม่เกิน N พร้อมกัน
        await asyncio.sleep(0.1) # จำลอง API call
        return {"symbol": symbol, "date": date, "close": 65000.0}

async def bulk_fetch_historical(symbols: list[str],
                                dates: list[str]) -> list[dict]:
    """ดึงข้อมูลหลายวันพร้อมกัน แต่จำกัดไม่เกิน 5 request พร้อมกัน"""
    sem = asyncio.Semaphore(5) # อนุญาต 5 concurrent requests

    tasks = [
        fetch_historical(sym, date, sem)
        for sym in symbols
        for date in dates
    ]

    print(f"ดึงข้อมูล {len(tasks)} records ด้วย semaphore=5")
    results = await asyncio.gather(*tasks)
    return list(results)

asyncio.run(bulk_fetch_historical(
    symbols=["BTCUSD", "ETHUSD"],
    dates=["2024-01-01", "2024-01-02", "2024-01-03",
           "2024-01-04", "2024-01-05"]
))
```

► OUTPUT

ดึงข้อมูล 10 records ด้วย semaphore=5

🧑 2 บรรทัดที่ต้องอ่านให้ขาด

```
async with sem:
    await asyncio.sleep(0.1)

tasks = [
    fetch_historical(sym, date, sem)
    for sym in symbols
    for date in dates
]
```

1. `async with sem:` — เทียบเท่ากับ `await sem.acquire()` ตอนเข้า และ `sem.release()` ตอนออก · จุดสำคัญคือ `release()` ถูกเรียกแม้โค้ดข้างในจะโยน `exception` · ถ้าเขียนเองแล้ว API พังกลางทาง บัตรจะหายไปหนึ่งใบทุกครั้งที่ error — พอหายครบ N ใบ ระบบจะค้างแบบเจียบสนิทหาสาเหตุยากมาก
2. `list comprehension` ที่มี `for` สองชั้น — อ่านเรียงจากบนลงล่างเหมือน `for` ธรรมดาซ้อนกัน: `for sym` เป็นวงนอก `for date` เป็นวงใน · ได้ทุกคู่ที่จับกันได้ $2 \times 5 = 10$ · อย่าอ่านสลับ — ลำดับนี้กำหนดลำดับผลลัพธ์ที่ `gather` คืนกลับมา ซึ่งเรียงตามลำดับ task ที่ใส่เข้าไปเสมอ (ไม่ใช่ตามลำดับที่ทำเสร็จ)

3. สังเกตว่า `fetch_historical(...)` ในลิสต์ยังไม่ได้รัน — มันสร้าง coroutine object ทั้งไว้เฉย ๆ 10 ตัว · ทุกอย่างเริ่มพร้อมกันตอน `await asyncio.gather(*tasks)` บรรทัดเดียว · * คือการแกะลิสต์ออกเป็น argument ที่ละตัว เพราะ `gather` รับเป็น argument แยก ไม่ใช่ลิสต์

Semaphore ไม่ใช่ rate limiter — และความเข้าใจผิดนี้ทำให้โดนแบน

Semaphore คุ่ม "จำนวนงานที่วิ่งอยู่พร้อมกัน" ไม่ใช่ "จำนวน request ต่อวินาที" · สองอย่างนี้ไม่เท่ากัน และตัวแปลงระหว่างมันคือ *latency* ของ *exchange* ซึ่งเราควบคุมไม่ได้:

$$\text{req/s} \approx \text{จำนวนที่วิ่งพร้อมกัน} \div \text{latency}$$

ลองวัดจริงด้วยการจำลอง latency 0.1 วินาที:

```
import asyncio
import time

async def timed_bulk(n_requests: int, limit: int,
                    latency: float = 0.1) -> float:
    """อิง n_requests โดยให้วิ่งพร้อมกันได้ไม่เกิน limit - คืนเวลาที่ใช้"""
    sem = asyncio.Semaphore(limit)

    async def one_call():
        async with sem:
            await asyncio.sleep(latency) # จำลองเวลารอ exchange

    t0 = time.perf_counter()
    await asyncio.gather(*[one_call() for _ in range(n_requests)])
    return time.perf_counter() - t0

async def compare_limits():
    n, latency = 30, 0.1
    print(f"latency = {latency}s · ส่ง {n} requests")

    elapsed = {}
    for limit in (1, 5):
        elapsed[limit] = await timed_bulk(n, limit, latency)
        # วัดเป็น "กี่รอบของ latency" แทนวินาที - ไม่แกว่งตามภาระเครื่อง
        rounds = round(elapsed[limit] / latency)
        print(f" Semaphore({limit}) -> {rounds:>2d} รอบของ latency")

    print(f" Semaphore(5) เร็วกว่า Semaphore(1) "
          f"f"{elapsed[1] / elapsed[5]:.0f} เท่า")

asyncio.run(compare_limits())
```

► OUTPUT

```
latency = 0.1s · ส่ง 30 requests
Semaphore(1) -> 30 รอบของ latency
Semaphore(5) -> 6 รอบของ latency
Semaphore(5) เร็วกว่า Semaphore(1) 5 เท่า
```

● อ่านตัวเลขนี้ให้ออก แล้วจะเห็นกับดักที่แท้จริง

คิดเป็น req/s เอง: Semaphore(5) ยิง 30 requests จบใน 6 รอบ $\times 0.1\text{s} = 0.6$ วินาที $\rightarrow$ **50 req/s** ไม่ใช่ 5 · แม้แต่ Semaphore(1) ที่ยิงทีละตัว เรียงกันก็ยิงได้ $30 \div 3.0 = 10 \text{ req/s}$ · ถ้า exchange จำกัดไว้ 10 req/s แปลว่าได้ข้างบนละเมิดขีดจำกัดตั้งแต่ Semaphore(2)

ที่อันตรายกว่านั้นคือทิศทางของ*ความผิดพลาด* · เมื่อ exchange ตอบเร็วขึ้น (latency ลด) จำนวน req/s ของเราเพิ่มขึ้นเองโดยที่เราไม่ได้ทำอะไรเลย · แปลว่าโค้ดจะผ่านตอนทดสอบบนเน็ตช้า ๆ แล้วไปโดนแบนตอน deploy ขึ้น server ที่อยู่ใกล้ exchange — ซึ่งคือตอนที่แย่ที่สุดที่จะพัง

ของที่ต้องใช้จริงคือตัวคุมที่นับตามเวลา ไม่ใช่ตามจำนวนที่วิ่งพร้อมกัน — เช่น token bucket: เติมโควตา N ในทุก 1 วินาที ใครจะยังต้องมีโควตา · สังเกตว่ามันคุมอัตราโดยตรง จึงไม่ขึ้นกับ latency เลย

ในระบบจริงมักใช้**ทั้งคู่พร้อมกัน** คนละหน้าที่: token bucket กันโดนแบน · Semaphore กันไม่ให้เปิด connection พร้อมกันจนหน่วยความจำหรือ file descriptor หมด

► แบบฝึกหัด: สร้าง MultiQueue สำหรับรับข้อมูลจากหลาย exchange แล้วรวมเข้า Queue เดียว

บทที่ 33: WebSocket & Live Data

แก่นของบทนี้

WebSocket คือ connection แบบ "เปิดค้างไว้" ระหว่าง client และ server — ต่างจาก HTTP ที่ต้อง request ทุกครั้ง Exchange ส่ง tick/candle ให้เราทันทีที่มีการเปลี่ยนแปลง ทำให้ latency ต่ำกว่า polling มาก แต่ต้องจัดการ disconnect, reconnect, และ parse message เองทั้งหมด

33.1 Binance Futures WebSocket — รูปแบบ Message จริง

Binance Futures ส่ง kline (candle) stream ในรูปแบบ JSON นี้ เมื่อ subscribe ที่ `wss://fstream.binance.com/ws/btcusdt@kline_1m`

```
# ตัวอย่าง kline message จาก Binance Futures WebSocket (format จริง)
BINANCE_KLINE_MSG = {
  "e": "kline",           # event type
  "E": 1705123456789,     # event time (ms)
  "s": "BTCUSDT",         # symbol
  "k": {
    "t": 1705123440000,    # kline start time (ms)
    "T": 1705123499999,    # kline close time (ms)
    "s": "BTCUSDT",        # symbol
    "i": "1m",             # interval
    "f": 100,              # first trade ID
    "L": 200,              # last trade ID
    "o": "65000.10",        # open
    "c": "65123.45",        # close (current price)
    "h": "65200.00",        # high
    "l": "64950.00",        # low
    "v": "123.456",         # base asset volume (BTC)
    "n": 1250,              # number of trades
    "x": False,             # is this kline closed? False = ยังไม่ปิด candle
    "q": "8023456.78",      # quote asset volume (USDT)
    "V": "61.234",          # taker buy base volume
    "Q": "3987654.32",      # taker buy quote volume
  }
}

# ตัวอย่าง bookTicker message จาก Binance (best bid/ask)
BINANCE_BOOKTICKER_MSG = {
  "e": "bookTicker",
  "u": 400900217,          # order book updateId
  "s": "BTCUSDT",
  "b": "65000.00",         # best bid price
  "B": "5.000",            # best bid qty
  "a": "65001.00",         # best ask price
  "A": "3.500",            # best ask qty
  "T": 1705123456789,      # transaction time
  "E": 1705123456790,      # event time
}
```

33.2 Bybit WebSocket — รูปแบบ Message

```
# Bybit V5 WebSocket - wss://stream.bybit.com/v5/public/linear
# Subscribe message:
BYBIT_SUBSCRIBE = {
  "op": "subscribe",
  "args": ["kline.1.BTCUSDT"] # kline.{interval}.{symbol}
}

# Kline message จาก Bybit V5:
BYBIT_KLINE_MSG = {
  "topic": "kline.1.BTCUSDT",
  "data": [
    {

```

```

        "start": 1705123440000, # start time ms
        "end": 1705123499999, # end time ms
        "interval": "1", # interval in minutes
        "open": "65000.10",
        "close": "65123.45",
        "high": "65200.00",
        "low": "64950.00",
        "volume": "123.456", # base volume (BTC)
        "turnover": "8023456.78", # quote volume (USD)
        "confirm": False, # False = candle ยังไม่ปิด
        "timestamp": 1705123456789
    }
],
"ts": 1705123456789,
"type": "snapshot" # หรือ "delta"
}

```

33.3 WebSocket Client ที่ Reconnect อัตโนมัติ

 **โครงก่อนดูโค้ด** — บล็อกถัดไปยาว 220 บรรทัด อ่านเป็น 3 ก้อน

อย่าเพิ่งอ่านทีละบรรทัด · ก่อนใหญ่ขึ้นคือของ 3 อย่างที่วางต่อกัน:

1. **Candle** (~10 บรรทัด) — กล่องเก็บข้อมูลแท่งเทียน 1 แท่ง ให้ทุกตลาดพูดภาษาเดียวกัน
2. **BinanceWebSocketClient** (~95 บรรทัด) — ตัวหลักที่ต้องเข้าใจจริง ๆ · ข้างในมี 5 เมธอด ทำหน้าที่ต่างกันชัดเจน (ดูตารางล่าง)
3. **BybitWebSocketClient** (~95 บรรทัด) — **โครงเหมือนข้อ 2 เกือบทั้งหมด** ต่างแค่ URL, รูปแบบข้อความ และวิธี subscribe · อ่านข้อ 2 ให้เข้าใจแล้วข้อ 3 จะกวาดตาผ่านได้

5 เมธอดใน client แต่ละตัว แบ่งหน้าที่แบบนี้:

เมธอด	หน้าที่	ระดับ
<code>__init__</code>	เก็บค่าตั้งต้น (symbol, interval, callback)	●
<code>stream_url</code>	ประกอบ URL ของ stream	●
<code>_parse_kline</code>	แปลง JSON ดิบของตลาด → Candle · จุดที่แต่ละตลาดต่างกันมากที่สุด	●
<code>_connect_and_listen</code>	ต่อ WebSocket แล้วรับข้อความ — หัวใจของ async	●
<code>run</code>	วนต่อใหม่เมื่อหลุด (reconnect + backoff) — หัวใจของความทนทาน	●

 **ถ้าเวลาน้อย:** อ่าน `_parse_kline` กับ `run` ให้เข้าใจก่อน — สองตัวนี้คือที่ที่ระบบจริงพึ่งบอยสุด

```

import asyncio
import json
import logging
import random
import websockets
from dataclasses import dataclass
from datetime import datetime
from typing import Callable, Optional

logger = logging.getLogger(__name__)

@dataclass
class Candle:
    exchange: str
    symbol: str
    interval: str
    open: float
    high: float
    low: float
    close: float
    volume: float
    is_closed: bool
    timestamp: datetime

```

```

class BinanceWebSocketClient:
    """
    Binance Futures WebSocket client
    - reconnect อัตโนมัติด้วย exponential backoff
    - parse kline messages
    - ส่ง Candle object เข้า callback
    """

    WS_URL = "wss://fstream.binance.com/ws"
    MAX_RECONNECT_DELAY = 60.0 # วินาที
    INITIAL_RECONNECT_DELAY = 1.0

    def __init__(self,
                  symbol: str,
                  interval: str,
                  on_candle: Callable[[Candle], None],
                  queue: Optional[asyncio.Queue] = None):
        self.symbol = symbol.lower()
        self.interval = interval
        self.on_candle = on_candle
        self.queue = queue
        self._stop_event = asyncio.Event()
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    @property
    def stream_url(self) -> str:
        return f"{self.WS_URL}/{self.symbol}@kline_{self.interval}"

    def _parse_kline(self, raw: dict) -> Optional[Candle]:
        """แปลง Binance kline message เป็น Candle object"""
        try:
            k = raw["k"]
            return Candle(
                exchange = "binance",
                symbol = raw["s"],
                interval = k["i"],
                open = float(k["o"]),
                high = float(k["h"]),
                low = float(k["l"]),
                close = float(k["c"]),
                volume = float(k["v"]),
                is_closed = bool(k["x"]),
                timestamp = datetime.utcfromtimestamp(raw["E"] / 1000)
            )
        except (KeyError, ValueError, TypeError) as e:
            logger.error(f"Binance parse error: {e} | raw={str(raw)[:200]}")
            return None

    async def _connect_and_listen(self):
        """เชื่อมต่อและฟัง messages - 1 connection attempt"""
        logger.info(f"Connecting to {self.stream_url}")
        async with websockets.connect(
            self.stream_url,
            ping_interval=20, # ส่ง ping ทุก 20s
            ping_timeout=10,
            close_timeout=5,
        ) as ws:
            logger.info(f"Connected: {self.symbol}@kline_{self.interval}")
            self._reconnect_delay = self.INITIAL_RECONNECT_DELAY # reset

            async for raw_msg in ws:
                if self._stop_event.is_set():
                    break

                try:
                    msg = json.loads(raw_msg)
                    if msg.get("e") == "kline":
                        candle = self._parse_kline(msg)
                        if candle is not None: # ข้ามถ้า parse ไม่ได้
                            self.on_candle(candle)

                    # ถ้ามี queue ให้ put ด้วย
                    if self.queue is not None:

```

```

        await self.queue.put(candle)

    except json.JSONDecodeError as e:
        logger.error(f"JSON decode error: {e}")
    except KeyError as e:
        logger.error(f"Unexpected message format: {e} | msg={raw_msg[:200]}")

async def run(self):
    """รัน WebSocket พร้อม exponential backoff reconnect"""
    while not self._stop_event.is_set():
        try:
            await self._connect_and_listen()

        except websockets.exceptions.ConnectionClosed as e:
            logger.warning(f"Connection closed: {e.code} {e.reason}")
        except websockets.exceptions.WebSocketException as e:
            logger.error(f"WebSocket error: {e}")
        except OSError as e:
            logger.error(f"Network error: {e}")

        if self._stop_event.is_set():
            break

        # Exponential backoff + jitter ป้องกัน thundering herd
        logger.info(f"Reconnecting in {self._reconnect_delay:.1f}s...")
        await asyncio.sleep(self._reconnect_delay)
        self._reconnect_delay = min(
            self._reconnect_delay * 2,
            self.MAX_RECONNECT_DELAY
        ) + random.uniform(0, 1.0)

    logger.info("WebSocket client stopped")

def stop(self):
    """หยุด client อย่างสะอาด"""
    self._stop_event.set()

class BybitWebSocketClient:
    """
    Bybit V5 WebSocket client
    endpoint: wss://stream.bybit.com/v5/public/linear
    """

    WS_URL = "wss://stream.bybit.com/v5/public/linear"
    MAX_RECONNECT_DELAY = 60.0
    INITIAL_RECONNECT_DELAY = 1.0

    def __init__(self,
                 symbol: str,
                 interval: str,
                 on_candle: Callable[[Candle], None],
                 queue: Optional[asyncio.Queue] = None):
        self.symbol = symbol.upper()
        self.interval = interval
        self.on_candle = on_candle
        self.queue = queue
        self._stop_event = asyncio.Event()
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    def _parse_kline(self, msg: dict) -> list[Candle]:
        """แปล Bybit kline message - อาจมีหลาย candle ใน 1 message"""
        candles = []
        for k in msg.get("data", []):
            try:
                candles.append(Candle(
                    exchange = "bybit",
                    symbol = self.symbol,
                    interval = self.interval,
                    open = float(k["open"]),
                    high = float(k["high"]),
                    low = float(k["low"]),
                    close = float(k["close"]),

```

```

        volume = float(k["volume"]),
        is_closed = bool(k.get("confirm", False)),
        timestamp = datetime.utcfromtimestamp(k["start"] / 1000)
    ))
except (KeyError, ValueError, TypeError) as e:
    logger.error(f"Bybit parse error: {e} | k={str(k)[:200]}")
return candles

async def _connect_and_listen(self):
    logger.info(f"Connecting to Bybit: {self.symbol} kline.{self.interval}")
    async with websockets.connect(
        self.WS_URL,
        ping_interval=20,
        ping_timeout=10,
    ) as ws:
        # ส่ง subscribe message
        sub_msg = json.dumps({
            "op": "subscribe",
            "args": [f"kline.{self.interval}.{self.symbol}"]
        })
        await ws.send(sub_msg)
        self._reconnect_delay = self.INITIAL_RECONNECT_DELAY

    async for raw_msg in ws:
        if self._stop_event.is_set():
            break
        try:
            msg = json.loads(raw_msg)
            if msg.get("op") == "subscribe":
                logger.info(f"Bybit subscribe: {msg.get('ret_msg')}")
                continue
            if "topic" in msg and "kline" in msg["topic"]:
                for candle in self._parse_kline(msg):
                    self.on_candle(candle)
                    if self.queue is not None:
                        await self.queue.put(candle)
        except (json.JSONDecodeError, KeyError) as e:
            logger.error(f"Parse error: {e}")

    async def run(self):
        while not self._stop_event.is_set():
            try:
                await self._connect_and_listen()
            except (websockets.exceptions.WebSocketException, OSError) as e:
                logger.warning(f"Connection error: {e}")
            if not self._stop_event.is_set():
                await asyncio.sleep(self._reconnect_delay)
                self._reconnect_delay = min(
                    self._reconnect_delay * 2, self.MAX_RECONNECT_DELAY
                ) + random.uniform(0, 1.0)

    def stop(self):
        self._stop_event.set()

```

👑 2 คำใหม่ใน `_connect_and_listen` — `async with` และ `async for`

```

async with websockets.connect(...) as ws:  # ①
    async for raw_msg in ws:               # ②
        ...

```

1. `async with ... as ws` — "เปิดการเชื่อมต่อใช้งานแล้วปิดให้อัตโนมัติเสมอไม่ว่าจะจบแบบปกติหรือ error". คำว่า `async` ข้างหน้าบอกว่าทั้ง `ขึ้นเปิดและขึ้นปิด` ต้องรอได้ (เพราะการต่อ/ตัด network ใช้เวลา). ถ้าไม่มี `with` แล้วเกิด error กลางทาง connection จะค้างเปิดทิ้งไว้
2. `async for raw_msg in ws` — "วนรับข้อความทีละอันเมื่อมันมาถึง". ต่างจาก `for` ปกติที่วนของที่มีครบอยู่แล้ว — ตัวนี้ **ไม่รู้ว่ามีกี่อัน และไม่รู้ว่าอันถัดไปจะมาเมื่อไหร่** ระหว่างที่รอมันปล่อยให้ `coroutine` อื่นทำงาน. ลูปนี้จะวนไปเรื่อย ๆ จนกว่าฝั่งตลาดจะปิดการเชื่อมต่อ

จำง่าย: `async with` = ยืมของแล้วคืนอัตโนมัติ. `async for` = รับของที่ทยอยมา

📖 **อ่านว่า:** บรรทัดที่ดูแปลกที่สุดใน `run()`: `min(delay * 2, MAX) + random.uniform(0, 1.0)` — อ่านว่า "รอนานขึ้น **เท่าตัว** ทุกครั้งที่ต่อไม่ติด แต่ไม่เกินเพดาน แล้วบวกเศษสุ่มอีกนิด". ส่วน `* 2` คือ exponential backoff (อย่ากระหน่ำ server ที่กำลังมีปัญหา) ส่วน `random` คือ jitter — เหตุผลอยู่ใน

🔵 [รอบสอง] ทำไมต้องบวกเลขสุ่ม — thundering herd

ถ้า exchange ล่มแล้วมี bot 5,000 ตัวต่อไม่ติดพร้อมกัน ทุกตัวจะรอ 1s → 2s → 4s เป็นจังหวะเดียวกันเป๊ะ พอครบเวลา ทั้ง 5,000 ตัวก็กระโดดเข้าไปพร้อมกันอีกรอบ — server ที่เพิ่งจะฟื้นก็ล้มซ้ำทันที เรียกว่า **thundering herd** . การบวก `random.uniform(0, 1)` ทำให้แต่ละตัวตื่นไม่พร้อมกัน โหลดจึงกระจาย

อีกจุดที่ค้างใจออกแบบ: `run()` ดักเฉพาะ `ConnectionClosed`, `WebSocketException`, `OSError` — **ไม่ดัก Exception** เปลา่ ๆ เพราะถ้าดักหมด บั๊กในโค้ดคุณเอง (เช่น `_parse_kline` พัง) จะถูกกลืนแล้ววน reconnect ไปเรื่อย ๆ โดยไม่มีใครรู้ว่าระบบตายไปแล้ว . **ดักเฉพาะ error ที่คุณรู้วิธีจัดการ ที่เหลือปล่อยให้ดัง** — นี่คือกฎที่แยกโค้ด production ออกจากโค้ด tutorial

สังเกต `self._reconnect_delay = INITIAL` ที่ถูก reset *หลังต่อติดแล้ว* — ถ้าลืมบรรทัดนี้ ระบบที่หลุดบ่อย ๆ จะค่อย ๆ ถ่างเป็นรอบ 60 วินาทีถาวร ทั้งที่ต่อติดได้ทันทีมานานแล้ว

33.4 Dual-Feed WebSocket — รับ Binance + Bybit พร้อมกัน

Running Example: รับ kline จาก Binance และ Bybit พร้อมกัน → ส่งเข้า Queue เดียว

นี่คือ core ของ live pair strategy — เราต้องการราคาจากทั้ง 2 exchange ในเวลาเดียวกันเพื่อคำนวณ spread

```
import asyncio
import json
import logging
from collections import deque
from typing import Optional

logger = logging.getLogger(__name__)

async def run_dual_feed(symbol: str = "BTCUSDT",
                        interval: str = "1",
                        max_candles: int = 100,
                        run_seconds: float = 30.0):
    """
    รับ Binance + Bybit WebSocket พร้อมกัน
    ส่ง candle ทั้งสองเข้า shared queue
    หยุดเองหลัง run_seconds วินาที (สำหรับ demo)
    """
    shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=500)
    stop_event = asyncio.Event()

    # Buffer สำหรับคำนวณ spread
    candle_buf: dict[str, deque] = {
        "binance": deque(maxlen=max_candles),
        "bybit": deque(maxlen=max_candles),
    }

    def on_candle(candle: Candle):
        # Callback รับ synchronously — ใช้แค่ log
        if candle.is_closed:
            logger.debug(f"[{candle.exchange}] Closed candle "
                        f"{candle.symbol} close={candle.close}")

    # สร้าง clients
    binance_client = BinanceWebSocketClient(
        symbol=symbol, interval=f"{interval}m",
        on_candle=on_candle, queue=shared_queue
    )
    bybit_client = BybitWebSocketClient(
        symbol=symbol, interval=interval,
        on_candle=on_candle, queue=shared_queue
    )

    async def process_candles():
        """Consumer: คำนวณ spread เมื่อได้ candle ปิดจากทั้งสอง"""
        while not stop_event.is_set():
```



```

try:
    candle = await asyncio.wait_for(
        shared_queue.get(), timeout=1.0
    )
except asyncio.TimeoutError:
    continue

if candle.is_closed:
    candle_buf[candle.exchange].append(candle.close)

# คำนวณ spread เมื่อมีข้อมูลทั้งสอง exchange
if (len(candle_buf["binance"]) > 0 and
    len(candle_buf["bybit"]) > 0):
    b_price = candle_buf["binance"][-1]
    y_price = candle_buf["bybit"][-1]
    spread = y_price - b_price
    logger.info(f"Spread: {spread:+.2f} "
                f"(Binance={b_price:.1f}, Bybit={y_price:.1f})")

shared_queue.task_done()

async def stopper():
    await asyncio.sleep(run_seconds)
    stop_event.set()
    binance_client.stop()
    bybit_client.stop()
    logger.info("Stop event set")

# รันทุก task พร้อมกัน
await asyncio.gather(
    binance_client.run(),
    bybit_client.run(),
    process_candles(),
    stopper(),
    return_exceptions=True
)

# ใช้งาน:
# asyncio.run(run_dual_feed("BTCUSDT", "1", run_seconds=300))

```

ข้อควรระวัง: WebSocket ใน Production

- **ตรวจ timestamp** — ถ้า message เก่ากว่า 5 วินาที อาจเป็น stale data → อย่าเทรด
- **Bybit ต้องการ re-subscribe** หลัง reconnect — ต้องส่ง subscribe message ใหม่ทุกครั้ง
- **Binance จำกัด 10 connections** ต่อ IP — ระวังถ้ารัน multiple instances
- **ใช้ testnet ก่อน**: Binance testnet: `wss://stream.binancefuture.com/ws` , Bybit testnet: `wss://stream-testnet.bybit.com/v5/public/linear`

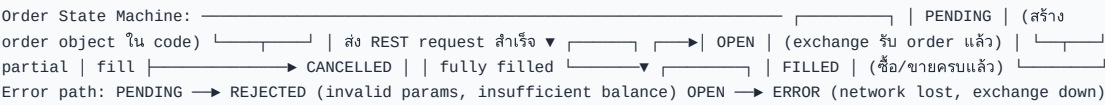
► **แบบฝึกหัด**: เพิ่ม staleness check — ถ้า candle เก่ากว่า 30 วินาที ให้ log warning

บทที่ 34: Order Execution

แก่นของบทนี้

การส่ง order จริงซับซ้อนกว่า backtest มาก: ต้องมี HMAC signature, จัดการ rate limit, retry เมื่อ network error (แต่ไม่ retry ถ้า order ถูก reject), และติดตาม state ของ order ตั้งแต่ PENDING จนถึง FILLED หรือ CANCELLED

34.1 Order State Machine



```
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Optional
import time

class OrderStatus(Enum):
    PENDING = auto() # สร้างใน code แต่ยังไม่ส่ง
    OPEN = auto() # ส่งไป exchange แล้ว รอ fill
    PARTIALLY_FILLED = auto()
    FILLED = auto() # fill ครบแล้ว
    CANCELLED = auto()
    REJECTED = auto() # exchange ปฏิเสธ
    ERROR = auto() # network/unknown error

class OrderSide(Enum):
    BUY = "Buy"
    SELL = "Sell"

class OrderType(Enum):
    MARKET = "MARKET"
    LIMIT = "LIMIT"

@dataclass
class Order:
    client_order_id: str # ID ที่เราสร้างเอง (UUID หรือ timestamp)
    symbol: str
    side: OrderSide
    order_type: OrderType
    qty: float
    price: Optional[float] = None # None สำหรับ market order

    # Fields ที่ exchange จะส่งกลับมา
    exchange_order_id: Optional[str] = None
    status: OrderStatus = OrderStatus.PENDING
    filled_qty: float = 0.0
    avg_price: float = 0.0
    created_at: float = field(default_factory=time.time)
    updated_at: Optional[float] = None
    reject_reason: Optional[str] = None

    @property
    def is_terminal(self) -> bool:
        """คืน True ถ้า order จบแล้ว (ไม่สามารถเปลี่ยนได้อีก)"""
        return self.status in (
            OrderStatus.FILLED,
            OrderStatus.CANCELLED,
            OrderStatus.REJECTED,
            OrderStatus.ERROR,
        )

    @property
```

```
def remaining_qty(self) -> float:
    return self.qty - self.filled_qty

def __repr__(self):
    return (f"Order({self.client_order_id} "
            f"{self.side.value} {self.qty} {self.symbol} "
            f"@ {'MKT' if self.price is None else self.price} "
            f"[{self.status.name}])")
```

34.2 REST API Wrapper พร้อม Rate Limiting และ Retry

 **โครงร่างครูดุค — บล็อกถัดไปยาว ~240 บรรทัด อ่านเป็น 3 ชั้น**

อย่าอ่านไล่บรรทัด · คลาสนี้มีชั้นชัดเจน แต่ละชั้นเรียกชั้นล่างลงไป:

ชั้น	เมธอด	หน้าที่	ระดับ
① เปลี่ยนออก สิ่งที่โค้ดเราเรียกใช้	place_market_order	ส่งคำสั่งซื้อขาย — แปลง argument เป็น payload ของตลาด	
	place_limit_order		
	cancel_order	ยกเลิก / เช็คสถานะ	
	get_order_status		
② แกนกลาง ทุกคำสั่งวิ่งผ่านตรงนี้	_request	หัวใจของทั้งคลาส — rate limit · retry · timeout · แปล error ~60 บรรทัด และเป็นวิธีที่ระบบจริงพึ่งพามากที่สุด	
③ งานประกอบ	_sign	เซ็นลายเซ็น HMAC ให้ตลาดยืนยันตัวตน	
	__aenter__ __aexit__	เปิด/ปิด session อัตโนมัติ — ทำให้ใช้กับ async with ได้ (หลักการเดียวกับ with ในบทที่ 8.5)	

 **ถ้าเวลาน้อย:** อ่าน `_request` อย่างเดียวก็น่าจะพอแล้ว — เมธอดข้างบนทั้งหมดเป็นแค่บล็อกที่เรียกมัน · และ **ขีดเส้นใต้ลำดับใน `_request`** :
จำกัดอัตรา — เซ็น → ยิง → ถ้าพลาดค่อยลองใหม่ ลำดับนี้สำคัญกว่ารายละเอียดของแต่ละชั้น

```
import asyncio
import hashlib
import hmac
import json
import os
import time
import logging
from urllib.parse import urlencode
from typing import Optional

import aiohttp

logger = logging.getLogger(__name__)

class BybitOrderClient:
    """
    Bybit V5 REST API wrapper
    - Rate limiting ด้วย asyncio.Semaphore
    - HMAC-SHA256 signature
    - Retry with exponential backoff สำหรับ network error
    - ไม่ retry ถ้า exchange reject (4xx)
    """

    # Testnet: https://api-testnet.bybit.com
    BASE_URL = "https://api.bybit.com"
    RECV_WINDOW = 5000  # ms

    def __init__(self,
                 api_key: Optional[str] = None,
                 api_secret: Optional[str] = None,
                 testnet: bool = True,
                 max_concurrent: int = 5):
        # ✓ โหลดจาก env var — ห้าม hardcode!
        self.api_key = api_key or os.environ.get("BYBIT_API_KEY", "")
```

```

self.api_secret = api_secret or os.environ.get("BYBIT_API_SECRET", "")

if testnet:
    self.BASE_URL = "https://api-testnet.bybit.com"
    logger.warning("🚧 TESTNET - 🚧 real money")

self._sem = asyncio.Semaphore(max_concurrent)
self._session: Optional[aiohttp.ClientSession] = None

async def __aenter__(self):
    self._session = aiohttp.ClientSession()
    return self

async def __aexit__(self, *args):
    if self._session:
        await self._session.close()

def _sign(self, params: dict) -> str:
    """สร้าง HMAC-SHA256 signature"""
    timestamp = str(int(time.time() * 1000))
    params_str = urlencode(sorted(params.items()))
    payload = f"{timestamp}{self.api_key}{self.RECV_WINDOW}{params_str}"
    signature = hmac.new(
        self.api_secret.encode("utf-8"),
        payload.encode("utf-8"),
        hashlib.sha256
    ).hexdigest()
    return signature, timestamp

async def _request(self, method: str, path: str,
                  params: dict = None,
                  max_retries: int = 3) -> dict:
    """ส่ง signed request พร้อม retry"""
    params = params or {}
    signature, timestamp = self._sign(params)
    headers = {
        "X-BAPI-API-KEY": self.api_key,
        "X-BAPI-SIGN": signature,
        "X-BAPI-TIMESTAMP": timestamp,
        "X-BAPI-RECV-WINDOW": str(self.RECV_WINDOW),
        "Content-Type": "application/json",
    }
    url = f"{self.BASE_URL}{path}"

    for attempt in range(max_retries):
        async with self._sem: # rate limiting
            try:
                if method == "GET":
                    async with self._session.get(
                        url, params=params, headers=headers,
                        timeout=aiohttp.ClientTimeout(total=10)
                    ) as resp:
                        data = await resp.json()
                else:
                    async with self._session.post(
                        url, json=params, headers=headers,
                        timeout=aiohttp.ClientTimeout(total=10)
                    ) as resp:
                        data = await resp.json()

                # Bybit ret_code: 0 = OK, อื่นๆ = error
                if data.get("retCode") != 0:
                    ret_msg = data.get("retMsg", "unknown")
                    ret_code = data.get("retCode")

                    # Error ที่ไม่ควร retry
                    NO_RETRY_CODES = {
                        10001, # Parameter error
                        10004, # Sign check failed
                        110007, # Insufficient balance
                        110013, # Position is in liquidation
                        110025, # Position not exist
                    }

```

```

        if ret_code in NO_RETRY_CODES:
            raise ValueError(
                f"Exchange rejected [{ret_code}]: {ret_msg}"
            )

        logger.warning(f"API error [{ret_code}]: {ret_msg}, "
                       f"attempt {attempt+1}/{max_retries}")
        if attempt == max_retries - 1:
            raise RuntimeError(
                f"Exchange error หลังลอง {max_retries} ครั้ง "
                f"[{ret_code}]: {ret_msg}"
            )
        # ⚠ ต้อง continue - ถ้าปล่อยให้ไหลลงไป return
        # error ที่ควรลองใหม่จะถูกคืนออกไปเลย ไม่เคย retry
        await asyncio.sleep(2 ** attempt)
        continue

    # มาถึงตรงนี้ = retCode == 0 = สำเร็จจริง
    return data

except (aiohttp.ClientError, asyncio.TimeoutError) as e:
    if attempt == max_retries - 1:
        raise
    delay = 2 ** attempt
    logger.warning(f"Network error: {e}, retry in {delay}s")
    await asyncio.sleep(delay)

raise RuntimeError(f"Failed after {max_retries} attempts")

async def place_market_order(self,
                              symbol: str,
                              side: OrderSide,
                              qty: float,
                              client_order_id: str) -> Order:
    """ส่ง market order"""
    order = Order(
        client_order_id=client_order_id,
        symbol=symbol,
        side=side,
        order_type=OrderType.MARKET,
        qty=qty,
    )

    try:
        resp = await self._request("POST", "/v5/order/create", {
            "category": "linear",
            "symbol": symbol,
            "side": side.value,
            "orderType": "Market",
            "qty": str(qty),
            "orderLinkId": client_order_id,
        })

        if resp.get("retCode") == 0:
            result = resp["result"]
            order.exchange_order_id = result.get("orderId")
            order.status = OrderStatus.OPEN
            logger.info(f"Order placed: {order}")
        else:
            order.status = OrderStatus.REJECTED
            order.reject_reason = resp.get("retMsg")
            logger.error(f"Order rejected: {order.reject_reason}")

    except ValueError as e:
        order.status = OrderStatus.REJECTED
        order.reject_reason = str(e)
        logger.error(f"Order rejected by exchange: {e}")

    except Exception as e:
        order.status = OrderStatus.ERROR
        order.reject_reason = str(e)
        logger.error(f"Order error: {e}")

```

```

        order.updated_at = time.time()
        return order

    async def place_limit_order(self,
                                symbol: str,
                                side: OrderSide,
                                qty: float,
                                price: float,
                                client_order_id: str) -> Order:
        """สั่ง limit order"""
        order = Order(
            client_order_id=client_order_id,
            symbol=symbol,
            side=side,
            order_type=OrderType.LIMIT,
            qty=qty,
            price=price,
        )

        try:
            resp = await self._request("POST", "/v5/order/create", {
                "category": "linear",
                "symbol": symbol,
                "side": side.value,
                "orderType": "Limit",
                "qty": str(qty),
                "price": str(price),
                "timeInForce": "PostOnly", # maker order เท่านั้น
                "orderLinkId": client_order_id,
            })

            if resp.get("retCode") == 0:
                result = resp["result"]
                order.exchange_order_id = result.get("orderId")
                order.status = OrderStatus.OPEN
                logger.info(f"Limit order placed: {order}")
            else:
                order.status = OrderStatus.REJECTED
                order.reject_reason = resp.get("retMsg")

        except ValueError as e:
            order.status = OrderStatus.REJECTED
            order.reject_reason = str(e)

        except Exception as e:
            order.status = OrderStatus.ERROR
            order.reject_reason = str(e)

        order.updated_at = time.time()
        return order

    async def cancel_order(self,
                           symbol: str,
                           client_order_id: str) -> bool:
        """ยกเลิก order"""
        resp = await self._request("POST", "/v5/order/cancel", {
            "category": "linear",
            "symbol": symbol,
            "orderLinkId": client_order_id,
        })

        success = resp.get("retCode") == 0
        if success:
            logger.info(f"Cancelled order {client_order_id}")
        else:
            logger.warning(f"Cancel failed: {resp.get('retMsg')}")
        return success

    async def get_order_status(self, symbol: str,
                               client_order_id: str) -> Optional[Order]:
        """ดึง order status จาก exchange"""
        resp = await self._request("GET", "/v5/order/realtime", {
            "category": "linear",
            "symbol": symbol,

```

```

        "orderLinkId": client_order_id,
    })
    if resp.get("retCode") != 0:
        return None

    items = resp["result"].get("list", [])
    if not items:
        return None

    item = items[0]
    STATUS_MAP = {
        "New": OrderStatus.OPEN,
        "PartiallyFilled": OrderStatus.PARTIALLY_FILLED,
        "Filled": OrderStatus.FILLED,
        "Cancelled": OrderStatus.CANCELLED,
        "Rejected": OrderStatus.REJECTED,
    }

    order = Order(
        client_order_id = client_order_id,
        symbol = symbol,
        side = OrderSide.BUY if item["side"] == "Buy" else OrderSide.SELL,
        order_type = OrderType.MARKET if item["orderType"] == "Market" else OrderType.LIMIT,
        qty = float(item["qty"]),
        price = float(item["price"]) if item.get("price") else None,
        exchange_order_id = item["orderId"],
        status = STATUS_MAP.get(item["orderStatus"], OrderStatus.ERROR),
        filled_qty = float(item.get("cumExecQty", 0)),
        avg_price = float(item.get("avgPrice", 0)),
    )
    return order

```

🔗 และ `request` — ลำดับ 4 ขั้นที่ทุกคำสั่งต้องผ่าน

```

for attempt in range(max_retries):          # ④ วงลองใหม่
    async with self._sem:                   # ① จำกัดอัตรา
        try:
            async with self._session.post(...) as resp:  # ③ ยิง
                data = await resp.json()
                if data.get("retCode") != 0:  # ตลาดตอบว่าไม่สำเร็จ
                    ...
                    continue                # ← ลองใหม่
                return data                  # สำเร็จจริงเท่านั้น
        except (ClientError, TimeoutError):
            await asyncio.sleep(2 ** attempt)  # ④ กอยแบบทวีคูณ

```

1. `async with self._sem` — `_sem` คือ `asyncio.Semaphore` = "บัตรคิว" มีบัตรจำกัด ใครไม่ได้บัตรต้องรอ. **นี่คือสิ่งที่กันเราจากการยิงเกิน `ratelimit` แล้วโดนแบน** — ไม่ใช่การ `sleep` เดาสุ่ม
2. `self._sign(params)` — ตลาดต้องรู้ว่าเป็นเราจริง จึงต้องแนบลายเซ็น HMAC ที่คำนวณจาก `secret key + timestamp · timestamp` ทำให้ `request` เก่าใช้ซ้ำไม่ได้
3. **ยิงแล้วอ่านคำตอบ** — สังเกตว่ามีการตรวจ **สองขั้น**: ขั้นแรกคือ `network` สำเร็จไหม (จับด้วย `except`) ขั้นที่สองคือ **ตลาดตอบว่าสำเร็จไหม** (`retCode != 0`) · **HTTP 200 ไม่ได้แปลว่าออเดอร์ผ่าน** — นี่คือจุดที่คนพลาดมากที่สุด
4. `2 ** attempt` — รอ 1 → 2 → 4 วินาที (exponential backoff เหมือนบทที่ 33) เพราะถ้าตลาดกำลังมีปัญหา การยิงถี่ ๆ ยิ่งทำให้แย่ลง

จำโครงสร้าง: `เข้าคิว` → `เซ็นชื่อ` → `ยิง` → `ถ้าพลาดถอยแล้วลองใหม่`: ส่วน `place_market_order` / `cancel_order` ทั้งหมดเป็นแค่เปลือกที่เตรียม `params` แล้วเรียกเมธอดนี้

❌ กับดักที่เคยอยู่ในโค้ดนี้จริง — `retry` ที่ไม่เคย `retry`

เดิมนิพจน์ `return data` ถูกวางไว้ระดับเดียวกับ `if data.get("retCode") != 0`: ผลคือเมื่อเจอ `error` ที่ควรลองใหม่โค้ดจะ `logger.warning("... attempt 1/3")` แล้วคืนค่าที่ล้มเหลวออกไปทันที — ลูป `for attempt in range(3)` ไม่เคยวนรอบที่สองเลย

อันตรายเป็นพิเศษเพราะ `log` จะบอกว่า `"attempt 1/3"` ทำให้ดูเหมือนระบบกำลัง `retry` อยู่ และฟังก์ชันคืนค่าปกติไม่มี `exception` · ผู้เรียกได้ `data` ที่มี `retCode != 0` ไปใช้ต่อโดยไม่รู้ตัว

❗ **วิธีจับบั๊กประเภทนี้:** ในลูป retry ให้ใส่ดูว่าทุกเส้นทางที่ไม่สำเร็จ จบด้วย `continue` หรือ `raise` - ถ้ามีเส้นทางไหน "ไหลลง" ไปถึง `return` ได้ทั้งที่ยังไม่สำเร็จ นั่นคือ retry ที่ตายแล้ว

● [รอบสอง] จุดที่โค้ดนี้ยังไม่สมบูรณ์ (ตั้งใจให้เห็น)

- **ถือ semaphore ระหว่าง sleep** — `await asyncio.sleep(delay)` อยู่ข้างใน `async with self._sem` แปลว่าระหว่างรอ backoff เรายังกินบัตรคิวอยู่ ทำให้ request อื่นที่พร้อมจะต้องรอไปด้วย - ในระบบจริงควรออกจาก semaphore ก่อนแล้วค่อย sleep
- **ไม่มี idempotency ตอน timeout** — ถ้า network timeout ระหว่างส่งออเดอร์ เราไม่รู้ว่าตลาดได้รับหรือยัง - การ retry อาจทำให้เปิดสองไม้ - ทางแก้จริงคือส่ง `orderId` (client order id) ที่ไม่ซ้ำ แล้วให้ตลาดปฏิเสธของซ้ำเอง — โค้ดนี้มี `client_order_id` อยู่แล้วแต่ยังไม่ได้ใช้เพื่อการนี้
- **NO_RETRY_CODES เป็นรายการที่ต้องดูแล** — โค้ดที่ไม่อยู่ในลิสต์จะถูก retry หมด - ถ้าตลาดเพิ่ม error code ใหม่ที่ retry ไม่ได้ (เช่นบัญชีถูกระงับ) ระบบจะยิงซ้ำไปเรื่อย ๆ - ระบบ production ควร default เป็นไม่ retry แล้วระบุเฉพาะโค้ดที่ retry ได้แทน — ปลอดภัยกว่าเมื่อเจอของที่ไม่รู้จัก

34.3 ทดสอบบน Testnet

```
import asyncio
import uuid

async def demo_order_flow():
    """
    ทดสอบ order flow บน Bybit Testnet
    ต้องตั้ง env var:
        export BYBIT_API_KEY=your_testnet_key
        export BYBIT_API_SECRET=your_testnet_secret
    """
    async with BybitOrderClient(testnet=True) as client:

        # สร้าง unique client order ID
        coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}"

        print(f"Placing market BUY 0.001 BTCUSD (coid={coid})")
        order = await client.place_market_order(
            symbol = "BTCUSD",
            side = OrderSide.BUY,
            qty = 0.001,
            client_order_id = coid,
        )
        print(f"Result: {order}")

        # รอให้ order settle
        await asyncio.sleep(1.0)

        # ดึง status
        updated = await client.get_order_status("BTCUSD", coid)
        if updated:
            print(f"Updated status: {updated.status.name}, "
                  f"filled={updated.filled_qty}, avg={updated.avg_price}")

# asyncio.run(demo_order_flow())
```

ข้อควรระวัง: Order Execution ในโลกจริง

- **ห้าม retry ทุก error** — ถ้า network timeout แล้ว retry → order อาจถูกส่ง 2 ครั้ง ใช้ `client_order_id` เพื่อ idempotency
- **Market order slip ได้** — ราคา fill อาจต่างจากราคาตลาดมาก โดยเฉพาะ pair ที่ volume น้อย
- **ตรวจ balance ก่อนส่ง order** — error 110007 (insufficient balance) ไม่ควรเกิดถ้าระบบบอกรับผิด
- **Log ทุก order response** รวมถึง timestamp, `exchange_order_id`, และ status

► แบบฝึกหัด: เขียน `OrderTracker` class ที่เก็บ history ของทุก order และ query ได้

บทที่ 35: Live System Integration

แก่นของบทนี้

ประกอบทุกอย่างเข้าด้วยกันเป็น live trading system ที่ production-ready: logging ที่ structured, health check loop, kill switch ที่ทำงานได้จริง, graceful shutdown, และ pre-flight checklist ก่อน go-live

35.1 Structured Logging

ใน live system logging ต้องเป็น structured (JSON) เพื่อ search และ filter ง่าย — ไม่ใช่แค่ print string

```
import logging
import json
import sys
from datetime import datetime, timezone

class StructuredFormatter(logging.Formatter):
    """
    Formatter ที่ output JSON — ง่ายต่อการ search ด้วย jq หรือ ELK stack
    """

    def format(self, record: logging.LogRecord) -> str:
        log_obj = {
            "ts": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
        }

        # เพิ่ม extra fields ถ้ามี
        for key in ("symbol", "order_id", "price", "side", "pnl", "error"):
            if hasattr(record, key):
                log_obj[key] = getattr(record, key)

        if record.exc_info:
            log_obj["exception"] = self.formatException(record.exc_info)

        return json.dumps(log_obj, ensure_ascii=False)

def setup_logging(level: str = "INFO",
                  log_file: str = None) -> logging.Logger:
    """ตั้งค่า logging สำหรับ live system"""
    logger = logging.getLogger("live_trading")
    logger.setLevel(getattr(logging, level.upper()))

    # Console handler
    console = logging.StreamHandler(sys.stdout)
    console.setFormatter(StructuredFormatter())
    logger.addHandler(console)

    # File handler (ถ้าระบุ)
    if log_file:
        fh = logging.FileHandler(log_file, encoding="utf-8")
        fh.setFormatter(StructuredFormatter())
        logger.addHandler(fh)

    return logger

# ใช้งาน
logger = setup_logging(level="INFO", log_file="live_trading.log")

# Log พร้อม extra context
logger.info("Order placed", extra={
    "symbol": "BTCUSD",
    "order_id": "ORD-001",
    "side": "BUY",
```

```
"price": 65000.0
})
```

► OUTPUT

```
{"ts": "2024-01-15T10:30:00.123456+00:00", "level": "INFO", "logger": "live_trading", "msg": "Order placed", "symbol": "BTCUSD", "or
# ↑ ts จะเป็นเวลาที่คุณรันจริง – ที่เหลือตรงกันทุกตัว
```

35.2 Kill Switch ด้วย asyncio.Event

```
import asyncio
import signal
import logging

logger = logging.getLogger("live_trading")

class KillSwitch:
    """
    Kill switch สำหรับหยุดระบบ live trading
    รองรับ: Ctrl+C, SIGTERM (Docker stop), programmatic (drawdown exceeded)
    """

    def __init__(self):
        self._event = asyncio.Event()
        self._reason: str = ""

    def trigger(self, reason: str = "manual"):
        """ทริกเกอร์ kill switch"""
        self._reason = reason
        self._event.set()
        logger.critical(f"KILL SWITCH TRIGGERED: {reason}")

    async def wait(self):
        """รอนจนกว่า kill switch จะทำงาน"""
        await self._event.wait()

    @property
    def is_set(self) -> bool:
        return self._event.is_set()

    @property
    def reason(self) -> str:
        return self._reason

    def setup_signal_handlers(self, loop: asyncio.AbstractEventLoop):
        """ตั้งค่า OS signal handlers"""
        def _handle_signal(sig_name: str):
            logger.warning(f"Received {sig_name}")
            self.trigger(f"OS signal: {sig_name}")

        # add_signal_handler รองรับเฉพาะ Unix/Linux/macOS (ไม่ทำงานบน Windows)
        if sys.platform != "win32":
            for sig in (signal.SIGINT, signal.SIGTERM):
                loop.add_signal_handler(
                    sig,
                    lambda s=sig: _handle_signal(s.name)
                )
        else:
            # Windows: ใช้ signal.signal() แทน (synchronous)
            import signal as _signal
            _signal.signal(_signal.SIGINT,
                           lambda s, f: _handle_signal("SIGINT"))
```

35.3 Health Check Loop

```
import asyncio
import time
from dataclasses import dataclass, field
```

```

@dataclass
class SystemHealth:
    last_binance_tick: float = 0.0 # timestamp ของ tick ล่าสุดจาก Binance
    last_bybit_tick: float = 0.0
    last_order_time: float = 0.0
    total_pnl: float = 0.0
    drawdown_pct: float = 0.0
    peak_equity: float = 0.0
    current_equity: float = 0.0
    order_errors: int = 0
    feed_disconnects: int = 0

    def update_equity(self, equity: float):
        if equity > self.peak_equity:
            self.peak_equity = equity
        self.current_equity = equity
        if self.peak_equity > 0:
            self.drawdown_pct = (equity - self.peak_equity) / self.peak_equity

async def health_check_loop(health: SystemHealth,
                             kill_switch: KillSwitch,
                             check_interval: float = 5.0,
                             max_drawdown: float = -0.05,
                             max_feed_silence: float = 30.0):
    """
    ตรวจสอบภาพระบบทุก check_interval วินาที
    หยุดอัตโนมัติถ้า:
    - drawdown เกิน max_drawdown
    - ไม่ได้รับ feed นานกว่า max_feed_silence วินาที
    """
    logger = logging.getLogger("live_trading.health")

    while not kill_switch.is_set:
        await asyncio.sleep(check_interval)
        now = time.time()

        # ตรวจสอบ drawdown
        if health.drawdown_pct < max_drawdown:
            kill_switch.trigger(
                f"Max drawdown exceeded: {health.drawdown_pct:.2%} < {max_drawdown:.2%}"
            )
            break

        # ตรวจสอบ feed silence
        binance_silence = now - health.last_binance_tick
        bybit_silence = now - health.last_bybit_tick

        if health.last_binance_tick > 0 and binance_silence > max_feed_silence:
            logger.error(f"Binance feed silent for {binance_silence:.0f}s")
            kill_switch.trigger(f"Binance feed silent {binance_silence:.0f}s")
            break

        if health.last_bybit_tick > 0 and bybit_silence > max_feed_silence:
            logger.error(f"Bybit feed silent for {bybit_silence:.0f}s")
            kill_switch.trigger(f"Bybit feed silent {bybit_silence:.0f}s")
            break

        # ตรวจสอบ order errors
        if health.order_errors >= 5:
            kill_switch.trigger(
                f"Too many order errors: {health.order_errors}"
            )
            break

        logger.info(
            f"Health OK | equity={health.current_equity:,.0f} "
            f"drawdown={health.drawdown_pct:.2%} "
            f"order_errors={health.order_errors}"
        )

```

35.4 ระบบ Live Trading ครบวงจร

Running Example: BTC Pair Live Trading System

ประกอบทุก component จาก บท 31-34 เป็น production-ready system สมบูรณ์

 **โครงก่อนดูโค้ด — บล็อกปิดเล่ม ~220 บรรทัด**

นี่คือที่ทุกอย่างมาบรรจบ · **ไม่มีแนวคิดใหม่เลย** — ทุกชิ้นมาจากบทก่อนหน้านี้ งานของบล็อกนี้คือ "ต่อสายให้ถูก"

Binance WS ─┐
└─> asyncio.Queue → _signal_loop → _execute_signal → Bybit REST
Bybit WS ─┐ (บท 33) (บท 32) (บท 34)
 └─ health_check_loop ─┐ kill switch (บท 35)

เมธอด	หน้าที่	มาจากบท	ระดับ
<code>_on_binance_candle</code> <code>_on_bybit_candle</code>	callback ที่ WS client เรียกเมื่อมีแท่งใหม่ — แคโยนเข้า queue	33	●
<code>_compute_zscore</code>	คำนวณ z-score ของ spread จากราคาสองตลาด	13-14	●
<code>_get_signal</code>	แปลง z-score เป็น LONG / SHORT / EXIT / HOLD	14	●
<code>_execute_signal</code>	ส่งออเดอร์จริงผ่าน REST client	34	●
<code>_signal_loop</code>	หัวใจ — ดึงจาก queue → คำนวณ — สั่ง วนไปจนกว่า kill switch ทำงาน	32	●
<code>_shutdown</code>	ปิดสถานะที่ค้าง ยกเลิกออเดอร์ปิด connection	35	●
<code>run</code>	สตาร์ททุก task พร้อมกันด้วย gather แล้วรอ kill switch	32	●

⚠️ **อ่านตามลำดับข้อมูลไหล ไม่ใช่ตามลำดับบรรทัด:** เริ่มที่ `run()` (ล่างสุด) เพื่อเห็นภาพรวมว่าใครถูกสตาร์ทบ้าง → แล้วค่อยตาม `_signal_loop` ขึ้นไปหา `_execute_signal`

```
import asyncio
import logging
import os
import time
import uuid
from collections import deque
from typing import Optional

# =====
# Configuration — โหลดจาก env var เสมอ
# =====

CONFIG = {
    "symbol": "BTCUSDT",
    "interval": "1", # 1 minute candles
    "pair_window": 60, # lookback candles
    "entry_zscore": 2.0,
    "exit_zscore": 0.5,
    "order_qty": 0.001, # BTC per trade
    "max_drawdown": -0.05, # หยุดถ้า drawdown เกิน 5%
    "max_feed_silence": 30.0, # หยุดถ้าไม่มี feed 30s
    "initial_capital": 10_000.0, # USD
    "testnet": True, # เปลี่ยนเป็น False เมื่อพร้อม live
    "log_level": "INFO",
    "log_file": "btc_pair_live.log",
}

class BTCPairLiveSystem:
    """
    Live trading system สำหรับ BTC pair strategy
    Binance/Bybit spread → Z-score → market order
    """
```

```

"""

def __init__(self, config: dict):
    self.config = config
    self.logger = setup_logging(
        config["log_level"], config.get("log_file")
    )
    self.kill_switch = KillSwitch()
    self.health = SystemHealth(
        peak_equity = config["initial_capital"],
        current_equity = config["initial_capital"],
    )
    self.order_tracker = OrderTracker()

    # Price buffers
    w = config["pair_window"]
    self._binance_closes: deque[float] = deque(maxlen=w)
    self._bybit_closes: deque[float] = deque(maxlen=w)

    # Position state
    self._position: Optional[str] = None # "LONG", "SHORT", หรือ None
    self._shared_queue: asyncio.Queue[Candle] = asyncio.Queue(maxsize=1000)

# --- Callbacks ---

def _on_binance_candle(self, candle: Candle):
    self.health.last_binance_tick = time.time()

def _on_bybit_candle(self, candle: Candle):
    self.health.last_bybit_tick = time.time()

# --- Signal Logic ---

def _compute_zscore(self) -> Optional[float]:
    """คำนวณ z-score ของ spread Binance-Bybit"""
    w = self.config["pair_window"]
    if (len(self._binance_closes) < w or
        len(self._bybit_closes) < w):
        return None

    import statistics
    spreads = [
        b - y for b, y in zip(
            list(self._binance_closes),
            list(self._bybit_closes)
        )
    ]
    mean = statistics.mean(spreads)
    stdev = statistics.stdev(spreads)
    if stdev == 0:
        return None
    return (spreads[-1] - mean) / stdev

def _get_signal(self, zscore: float) -> str:
    entry = self.config["entry_zscore"]
    exit_ = self.config["exit_zscore"]

    if zscore > entry: return "SHORT"
    if zscore < -entry: return "LONG"
    if abs(zscore) < exit_: return "EXIT"
    return "HOLD"

# --- Order Execution ---

async def _execute_signal(self, signal: str,
                        client: BybitOrderClient):
    """แปลง signal เป็น order"""
    if signal == "HOLD":
        return
    if signal == self._position:
        return # อยู่ใน position นี้อยู่แล้ว

    qty = self.config["order_qty"]

```

```

sym = self.config["symbol"]
coid = f"PAIR-{uuid.uuid4().hex[:8].upper()}"

if signal in ("LONG", "SHORT"):
    # ปิด position เดิมก่อน (ถ้ามี)
    if self._position is not None:
        close_side = (OrderSide.SELL
                       if self._position == "LONG" else OrderSide.BUY)
        close_coid = f"CLOSE-{uuid.uuid4().hex[:8].upper()}"
        close_order = await client.place_market_order(
            sym, close_side, qty, close_coid
        )
        self.order_tracker.track(close_order)
        if close_order.status == OrderStatus.ERROR:
            self.health.order_errors += 1

    # เปิด position ใหม่
    side = OrderSide.BUY if signal == "LONG" else OrderSide.SELL
    order = await client.place_market_order(sym, side, qty, coid)
    self.order_tracker.track(order)

    if order.status in (OrderStatus.OPEN, OrderStatus.FILLED):
        self._position = signal
        self.logger.info(
            f"Position opened: {signal}",
            extra={"symbol": sym, "order_id": coid, "side": side.value}
        )
    else:
        self.health.order_errors += 1
        self.logger.error(
            f"Order failed: {order.reject_reason}",
            extra={"order_id": coid}
        )

elif signal == "EXIT" and self._position is not None:
    side = (OrderSide.SELL
            if self._position == "LONG" else OrderSide.BUY)
    order = await client.place_market_order(sym, side, qty, coid)
    self.order_tracker.track(order)

    if order.status in (OrderStatus.OPEN, OrderStatus.FILLED):
        self._position = None
        self.logger.info(
            "Position closed",
            extra={"symbol": sym, "order_id": coid}
        )
    else:
        self.health.order_errors += 1

# — Main Consumer Loop —

async def _signal_loop(self, client: BybitOrderClient):
    """ประมวลผล candle จาก queue และส่ง order"""
    while not self.kill_switch.is_set:
        try:
            candle = await asyncio.wait_for(
                self._shared_queue.get(), timeout=1.0
            )
        except asyncio.TimeoutError:
            continue

        # เพิ่มเฉพาะ closed candle
        if candle.is_closed:
            if candle.exchange == "binance":
                self._binance_closes.append(candle.close)
            elif candle.exchange == "bybit":
                self._bybit_closes.append(candle.close)

        zscore = self._compute_zscore()
        if zscore is not None:
            signal = self._get_signal(zscore)
            self.logger.debug(
                f"zscore={zscore:.3f} signal={signal}"
            )

```

```

        )
        if not self.kill_switch.is_set:
            await self._execute_signal(signal, client)

    self._shared_queue.task_done()

# ——— Graceful Shutdown ———

async def _shutdown(self, client: BybitOrderClient):
    """ปิด position ทั้งหมดก่อน shutdown"""
    self.logger.warning("Graceful shutdown: ปิด open positions...")

    if self._position is not None:
        side = (OrderSide.SELL
                 if self._position == "LONG" else OrderSide.BUY)
        coid = f"SHUTDOWN-{uuid.uuid4().hex[:8].upper()}"
        order = await client.place_market_order(
            self.config["symbol"], side,
            self.config["order_qty"], coid
        )
        self.order_tracker.track(order)
        self.logger.info(
            f"Shutdown order: {order.status.name}",
            extra={"order_id": coid}
        )
        self._position = None

    self.logger.info(
        f"Shutdown complete | "
        f"total_orders={len(self.order_tracker._orders)} | "
        f"drawdown={self.health.drawdown_pct:.2%}"
    )

# ——— Run ———

async def run(self):
    """เข้าสู่การทำงาน live"""
    loop = asyncio.get_event_loop()
    self.kill_switch.setup_signal_handlers(loop)

    self.logger.info("Starting BTC Pair Live System")
    self.logger.info(f"Config: {self.config}")

    # WebSocket clients
    binance_client = BinanceWebSocketClient(
        symbol = self.config["symbol"],
        interval = f"{self.config['interval']}m",
        on_candle = self._on_binance_candle,
        queue = self._shared_queue,
    )
    bybit_client = BybitWebSocketClient(
        symbol = self.config["symbol"],
        interval = self.config["interval"],
        on_candle = self._on_bybit_candle,
        queue = self._shared_queue,
    )

    async with BybitOrderClient(testnet=self.config["testnet"]) as order_client:
        try:
            await asyncio.gather(
                binance_client.run(),
                bybit_client.run(),
                self._signal_loop(order_client),
                health_check_loop(
                    self.health,
                    self.kill_switch,
                    max_drawdown = self.config["max_drawdown"],
                    max_feed_silence = self.config["max_feed_silence"],
                ),
                self.kill_switch.wait(), # รอจนกว่า kill switch
                return_exceptions=True,
            )
        finally:

```

```

        binance_client.stop()
        bybit_client.stop()
        await self._shutdown(order_client)

# — Entry point —
if __name__ == "__main__":
    system = BTCPairLiveSystem(CONFIG)
    asyncio.run(system.run())

```

🏠 **📌** — 5 task ที่วิ่งพร้อมกัน และเงื่อนไขจบของแต่ละตัว

```

await asyncio.gather(
    binance_client.run(),      # ① รับ feed — วนตลอด reconnect เอง
    bybit_client.run(),       # ②
    self._signal_loop(...),   # ③ ประมวลผล — วนจนกว่า kill switch
    health_check_loop(...),   # ④ เฝ้าดูสุขภาพ — ทริกเกอร์ kill switch ได้
    self.kill_switch.wait(),   # ⑤ นั่งรอเฉย ๆ — ตัวที่จบก่อนเพื่อน
    return_exceptions=True,
)

```

1. ตัวแรกไม่มีตัวไหนจบเอง — ① วน reconnect ตลอด, ③ วนจนกว่า kill_switch ถูกตั้ง ถ้ามีแค่ 4 ตัวนี้ โปรแกรมจะไม่มีทางออกอย่างสวยงาม
2. ⑤ `kill_switch.wait()` คือ "สวิตช์ออก" — มันแค่รอรอพอมีใครสั่ง `trigger()` (จาก Ctrl+C, จาก drawdown เกิน, จาก feed เจียนานเกิน) มันจะตื่นและจบ
3. `return_exceptions=True` สำคัญมากตรงนี้ — ถ้าไม่ใส่ พอ task ใดพัง exception จะเตี๋ยออกจาก `gather` ทันที แต่ task ที่เหลือยังวิ่งอยู่เบื้องหลัง (ตามที่เตือนไว้ในบทที่ 31) ผลคือ `finally` จะทำงานทั้งที่ feed ยังไหลอยู่ — ปิดระบบครึ่ง ๆ กลาง ๆ ใส True แล้ว `gather` จะเก็บ exception เป็นค่าคืนแทน ทำให้เราคุมลำดับปิดได้เอง
4. `finally` ทำงานเสมอ — สั่ง `stop()` ทั้งสอง feed แล้วเรียก `_shutdown()` เพื่อปิดสถานะที่ค้าง · นี่คือส่วนที่แยกระบบเล่นกับระบบที่กล้าใส่เงินจริง

อ่านทั้งบล็อกเป็นประโยคเดียว: "เปิดทุกอย่างพร้อมกัน แล้วนั่งรอสวิตช์ปิด — พอสวิตช์ถูกกด ไม่ว่าด้วยเหตุใด ให้เก็บกวาดให้เรียบร้อยเสมอ"

🔵 [รอบสอง] ทำไม `if __name__ == "__main__":` ถึงสำคัญกว่าที่คิดในไฟล์นี้

ถ้าไม่มีบรรทัดนี้ การ `import` ไฟล์นี้เข้าไปในไฟล์อื่นจะเริ่มเทรดจริงทันที — รวมถึงตอนที่ `pytest` เก็บไฟล์ไปรัน หรือตอนที่ IDE ทำ auto-import เพื่อ auto-complete

เป็นอุบัติเหตุที่เกิดขึ้นจริงกับคนที่เขียน bot: เขียน test แล้ว `pytest` import โมดูล → ระบบ live เริ่มทำงานจากในเครื่อง dev · กฎ: ไฟล์ใดที่ทำอะไร มีผลจริง (ส่งออเดอร์ · เขียน DB · จ่ายเงิน) การกระทำนั้นต้องอยู่ใต้ `if __name__ == "__main__":` เสมอ

💡 สังเกตด้วยว่าโค้ดนี้ไม่มีวันจบเอง — ถ้ารันจริงมันจะค้างรอ kill switch ตลอด นั่นคือพฤติกรรมที่ถูกต้องของระบบ live · ตอนทดสอบ ให้ตั้ง timeout ครอบไว้ (เช่น `asyncio.wait_for(system.run(), timeout=60)`) ไม่งั้นทดสอบจะไม่วันจบ

35.5 Pre-Flight Checklist ก่อน Go Live

```

import asyncio
import os
import aiohttp

async def preflight_check(config: dict) -> bool:
    """
    ตรวจสอบความพร้อมก่อนเริ่ม live trading
    คืน True เฉพาะเมื่อผ่านทุก check
    """
    results = {}
    logger = logging.getLogger("live_trading.preflight")

    # 1. ตรวจสอบ API Keys
    api_key = os.environ.get("BYBIT_API_KEY", "")
    api_secret = os.environ.get("BYBIT_API_SECRET", "")
    results["api_keys"] = bool(api_key and api_secret and
                               len(api_key) > 10 and
                               len(api_secret) > 10)

    if not results["api_keys"]:

```



```

logger.error("FAIL: BYBIT_API_KEY หรือ BYBIT_API_SECRET ไม่ได้ตั้งค่า")

# 2. ตรวจสอบการเชื่อมต่อ exchange
try:
    async with aiohttp.ClientSession() as session:
        url = ("https://api-testnet.bybit.com/v5/market/time"
               if config.get("testnet")
               else "https://api.bybit.com/v5/market/time")
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as r:
            data = await r.json()
            results["connectivity"] = data.get("retCode") == 0
except Exception as e:
    results["connectivity"] = False
    logger.error(f"FAIL: ไม่สามารถเชื่อมต่อ exchange: {e}")

# 3. ตรวจสอบ balance
if results.get("api_keys") and results.get("connectivity"):
    try:
        async with BybitOrderClient(testnet=config.get("testnet", True)) as client:
            resp = await client._request("GET", "/v5/account/wallet-balance",
                                         {"accountType": "UNIFIED"})

            if resp.get("retCode") == 0:
                coins = resp["result"]["list"][0]["coin"]
                usdt = next(
                    (float(c["availableToWithdraw"])
                     for c in coins if c["coin"] == "USDT"),
                    0.0
                )
                min_balance = config.get("initial_capital", 1000) * 0.5
                results["balance"] = usdt >= min_balance
                if not results["balance"]:
                    logger.error(
                        f"FAIL: USDT balance {usdt:.2f} < required {min_balance:.2f}"
                    )
                else:
                    logger.info(f"OK: USDT balance = {usdt:,.2f}")
            else:
                results["balance"] = False
    except Exception as e:
        results["balance"] = False
        logger.error(f"FAIL: ดึง balance ไม่ได้: {e}")

# 4. ตรวจสอบ config
cfg_ok = (
    config.get("order_qty", 0) > 0 and
    config.get("max_drawdown", 0) < 0 and
    config.get("entry_zscore", 0) > config.get("exit_zscore", 0) > 0
)
results["config"] = cfg_ok
if not cfg_ok:
    logger.error("FAIL: Config ไม่ถูกต้อง")

# 5. สรุป
all_pass = all(results.values())
print("\n=== Pre-Flight Check ===")
for name, ok in results.items():
    status = "✓ PASS" if ok else "✗ FAIL"
    print(f" {name:20s}: {status}")
print(f"\n{'→ READY TO LAUNCH' if all_pass else '→ FIX ISSUES FIRST'}\n")
return all_pass

# asyncio.run(preflight_check(CONFIG))

```

Deployment Checklist — ก่อน Go Live จริง

- ทดสอบบน Testnet ≥ 1 สัปดาห์ — ให้ระบบวิ่งตลอดเวลา ดู drawdown และ error logs
- Paper trading บน real feed — รับ feed จริงแต่ไม่ส่ง order จริงเพื่อตรวจสอบ latency
- ตั้ง max position size เล็กๆ ตอนเริ่มต้น เช่น 0.001 BTC ก่อน scale ขึ้น
- มี monitoring ภายนอก — อย่าเชื่อแค่ log ของตัวเอง ใช้ Telegram bot หรือ Grafana alert
- Deploy บน VPS ใกล้ exchange — Singapore หรือ Tokyo เพื่อ latency ต่ำ

- ตั้ง **systemd** หรือ **Docker** ให้ restart อัตโนมัติถ้า process crash
- **Log rotation** — log file โตได้มาก ตั้ง logrotate หรือ size limit

► แบบฝึกหัด: เพิ่ม Telegram alert เมื่อ kill switch ทำงาน หรือเมื่อ fill order ใหญ่

จบหนังสือ — เส้นทางต่อไป

การเดินทางจาก PART 0 ถึง PART VI

คุณผ่านการเรียนรู้มาไกลมาก — ตั้งแต่ Python พื้นฐานจนถึงระบบ live trading อัตโนมัติ
สรุปทุกอย่างที่ได้เรียนในหนังสือเล่มนี้:

Part	เนื้อหา	สิ่งที่ได้
Part 0	Python Foundations	Syntax, data types, functions, comprehensions
Part I	Data Wrangling	pandas, numpy, matplotlib — ดึง/ทำความสะอาด/visualize ข้อมูลตลาด
Part II	Math Tools	Rolling stats, z-score, cointegration, pair trading signal
Part III	OOP	Classes, ABC, Strategy/Observer/Factory patterns — system ที่ขยายได้
Part IV	Backtesting	Walk-forward, slippage, commission, overfitting, Sharpe/drawdown
Part V	AI-Assisted Coding	Prompt engineering, code generation, audit checklist, quant libraries, full strategy with AI
Part VI	Async & Live Trading	asyncio, WebSocket, order execution, kill switch, monitoring

เส้นทางการพัฒนาในฐานะ Quant Developer: _____ [หนังสือนี้] Part 0-I → Part II-III → Part IV-V → Part VI Python basics Math + OOP Backtest Live System | | | ▼▼▼ [ขั้นต่อไป]
NumPy/Pandas statsmodels vectorbt/ production เชี่ยวชาญ PyMC (Bayes) zipline deployment Nautilus cloud infra

แนะนำขั้นต่อไปหลังจบหนังสือ

1. Backtesting เชิงลึก

- **vectorbt** — vectorized backtesting เร็วกว่า pandas loop 100x
- **Nautilus Trader** — professional backtester + live trading framework ที่ใช้ใน production
- **Zipline-reloaded** — backtester ของ Quantopian ที่ยังมี community ดูแล

2. Statistical Methods เพิ่มเติม

- **Bayesian Inference** ด้วย PyMC — ประเมิน parameter uncertainty แทน point estimate
- **Kalman Filter** — dynamic hedge ratio ที่ปรับตามเวลา (statsmodels มี DLM)
- **Regime Detection** ด้วย Hidden Markov Model — ตรวจสอบว่าตลาดอยู่ใน trend/mean-revert/volatile

3. Machine Learning for Finance

- **Feature Engineering** — microstructure features, order book imbalance, funding rate
- **Gradient Boosting** (XGBoost/LightGBM) สำหรับ signal classification
- **LSTM / Transformer** สำหรับ time series — แต่ระวัง overfitting
- **Reinforcement Learning** (Stable-Baselines3) — ให้ model เรียนรู้ position sizing

4. Infrastructure & Operations

- **Docker + docker-compose** — containerize trading system
- **TimescaleDB** — PostgreSQL extension สำหรับ time series data ใน production
- **Grafana + Prometheus** — monitoring dashboard สำหรับ live system
- **Redis Streams** — message queue สำหรับ high-throughput tick data

5. หนังสือและแหล่งเรียนรู้แนะนำ

- *Advances in Financial Machine Learning* — Marcos Lopez de Prado (อ่านหลังจากมีพื้นฐาน ML)
- *Algorithmic Trading* — Ernest Chan (practical, Python examples)
- *Active Portfolio Management* — Grinold & Kahn (classic quant text)
- QuantLib Python — pricing library สำหรับ options และ derivatives
- Quantopian Lectures (archive) — beginner-friendly quant tutorials

คำเตือนสุดท้าย — จากนักพัฒมาถึงนักพัฒนา

หนังสือนี้สอนวิธี สร้าง trading system — ไม่ใช่วิธีรวยจากตลาด

- Strategy ที่ backtest ได้ดีส่วนมากไม่ได้ผลใน live — นั่นคือความจริงของ quant
- Market เปลี่ยนไปตลอด — strategy ที่ดีวันนี้อาจไม่ดีพรุ่งนี้
- การ risk management และ position sizing สำคัญกว่า entry signal มาก
- ไม่มี strategy ไหนที่ไม่มีความเสี่ยง — จัดการความเสี่ยงด้วยระบบ ไม่ใช่ด้วยความหวัง

จงสร้างระบบที่ *fail safely*, *log everything*, และ *never risk what you can't afford to lose*

■ สารบัญทั้งหมด · 🔗 คำศัพท์ Quant Trading

ภาคผนวก

คำศัพท์ Quant Trading

35 คำศัพท์สำคัญ พร้อมคำอธิบายภาษาไทย
จัดเรียงตามหมวดหมู่เพื่อใช้ค้นอ้างอิง



หมวดที่ 1 — สถิติพื้นฐาน

ค่าเฉลี่ย Mean μ

ผลรวมของข้อมูลทั้งหมดหารด้วยจำนวนข้อมูล บอก "จุดกึ่งกลาง" ของชุดข้อมูล ใช้เป็นจุดอ้างอิงในการวัด z-score และ spread

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

→ Part 0 บทที่ 2, Part I บทที่ 7

ส่วนเบี่ยงเบนมาตรฐาน Standard Deviation σ

วัดว่าข้อมูลกระจายออกจากค่าเฉลี่ยมากแค่ไหน ยิ่ง σ สูง ข้อมูลยิ่งผันผวน ใช้คำนวณ z-score และ Sharpe Ratio

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

→ Part 0 บทที่ 2, Part II บทที่ 12

z-score z

บอกว่าค่า x ห่างจากค่าเฉลี่ยกี่หน่วยส่วนเบี่ยงเบนมาตรฐาน ใน Pair Trading ใช้ z-score ของ spread เป็นสัญญาณซื้อขาย — $z < -2$ หมายถึงสเปรดต่ำผิดปกติ (ซื้อ), $z > +2$ หมายถึงสูงผิดปกติ (ขาย)

$$z = \frac{x - \mu}{\sigma}$$

→ Part II บทที่ 13, Part IV บทที่ 22

สหสัมพันธ์ Correlation ρ

วัดความสัมพันธ์ระหว่างสองสิ่ง ค่าอยู่ระหว่าง -1 ถึง $+1$ ถ้า $\rho \approx +1$ หมายถึงขึ้น-ลงพร้อมกัน ถ้า $\rho \approx -1$ หมายถึงตรงข้ามกัน ถ้า $\rho \approx 0$ หมายถึงไม่สัมพันธ์กัน BTC-Bybit และ BTC-Binance มี ρ สูงมาก (~ 0.999) จึงเหมาะทำ Pair Trading

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

→ Part 0 บทที่ 3, Part II บทที่ 13

ความแปรปรวนร่วม Covariance

วัดว่าตัวแปรสองตัวเปลี่ยนแปลงไปในทิศทางเดียวกันมากแค่ไหน ต่างจาก Correlation ตรงที่ไม่ได้ normalize ให้อยู่ในช่วง -1 ถึง $+1$ ใช้ในการคำนวณ Hedge Ratio และ Portfolio Optimization

→ Part 0 บทที่ 3, Part II บทที่ 14

การถดถอยเชิงเส้น Linear Regression

หาเส้นตรงที่เหมาะสมที่สุดที่อธิบายความสัมพันธ์ระหว่างตัวแปร X และ Y ใน Pair Trading ใช้ถดถอยเพื่อหา Hedge Ratio ว่าซื้อ BTC-Bybit 1 หน่วย ต้องขาย BTC-Binance กี่หน่วย

$$Y = \beta X + \alpha + \varepsilon$$

→ Part II บทที่ 13

การร่วมอินทิเกรชัน *Cointegration*

สองชุดข้อมูลแต่ละชุดอาจเดินสุ่ม (random walk) แต่ถ้าผลต่างระหว่างกันยังคงวนอยู่รอบค่าเฉลี่ย (stationary) เรียกว่าร่วมอินทิเกรต — นี่คือการพื้นฐานทางสถิติของ Pair Trading ทดสอบด้วย ADF Test

→ Part 0 บทที่ 4, Part II บทที่ 13

หมวดที่ 2 — การวัดผลกลยุทธ์

อัตราส่วนชาร์ป *Sharpe Ratio*

วัดผลตอบแทนที่ได้ต่อหน่วยความเสี่ยงที่รับ ยิ่งสูงยิ่งดี กลยุทธ์ที่ดีควรมี Sharpe > 1.0 ในระดับสากล > 2.0 ถือว่าดีมาก ตัวเลข 252 คือจำนวนวันซื้อขายต่อปีในตลาดหุ้น (หรือ 365 สำหรับ crypto)

$$\text{Sharpe} = \frac{E[R] - R_f}{\sigma_R} \times \sqrt{252}$$

R = ผลตอบแทนรายวัน, R_f = อัตราปลอดภัย (มักใช้ 0 สำหรับ crypto)

→ Part IV บทที่ 22, Part IV บทที่ 24

ครอดาวนสูงสุด *Maximum Drawdown (MDD)*

ขาดทุนสูงสุดจากจุดสูงสุดถึงจุดต่ำสุดในช่วงเวลาหนึ่ง วัดเป็นเปอร์เซ็นต์ บอกว่ากลยุทธ์ในช่วงเลวร้ายที่สุดพอร์ตจะลดลงสูงสุดแค่ไหน กลยุทธ์ที่ดีควรมี MDD ต่ำ

$$\text{MDD} = \max_t \left(\frac{\text{peak}_t - \text{trough}_t}{\text{peak}_t} \right)$$

→ Part IV บทที่ 24

ผลตอบแทนรวมต่อปี *CAGR (Compound Annual Growth Rate)*

ผลตอบแทนเฉลี่ยต่อปีแบบทบต้น เปรียบเทียบกลยุทธ์ที่มีช่วงเวลาทดสอบต่างกันได้อย่างยุติธรรม

$$\text{CAGR} = \left(\frac{V_{\text{end}}}{V_{\text{start}}} \right)^{1/N} - 1$$

N = จำนวนปี

→ Part IV บทที่ 24

อัตราชนะ *Win Rate*

สัดส่วนการเทรดที่ได้กำไรต่อการเทรดทั้งหมด Win Rate สูงอย่างเดียวไม่พอ ต้องดูร่วมกับ Profit Factor ว่ากำไรต่อครั้งมากกว่าขาดทุนต่อครั้งหรือไม่

→ Part IV บทที่ 24

Profit Factor

ผลรวมกำไรหารด้วยผลรวมขาดทุน (เป็นบวกทั้งหมด) ถ้า > 1 แปลว่ากำไรรวมมากกว่าขาดทุนรวม ค่า > 1.5 ถือว่าดี

→ Part IV บทที่ 24

หมวดที่ 3 — แนวคิดตลาด

ส่วนต่างราคา *Spread*

ในบริบท Pair Trading: ผลต่างราคาระหว่างสองสินทรัพย์ที่คำนวณตาม Hedge Ratio เช่น $\text{Spread} = \text{BTC_Bybit} - \beta \times \text{BTC_Binance}$ เราซื้อขายเมื่อ spread เบี่ยงออกจากค่าเฉลี่ยมากผิดปกติ

→ Part II บทที่ 13, Part IV บทที่ 22

Bid / Ask

Bid คือราคาสูงสุดที่ผู้ซื้อยินดีจ่าย; **Ask** คือราคาต่ำสุดที่ผู้ขายยินดีรับ ส่วนต่าง Bid-Ask คือต้นทุนซ่อนเร้นของการเทรด ยิ่ง Bid-Ask ต่ำ ตลาดยิ่ง liquid

→ Part VI บทที่ 32

สลิปเปจ *Slippage*

ผลต่างระหว่างราคาที่คาดว่าจะได้กับราคาที่ได้จริงเมื่อส่งซื้อขาย เกิดขึ้นเมื่อตลาดเคลื่อนไหวระหว่างที่คำสั่งถูกส่งและถูก fill ยิ่งคำสั่งใหญ่หรือตลาด thin ยิ่ง slippage สูง ต้องรวมใน Backtest เพื่อให้ผลลัพธ์สมจริง

→ Part IV บทที่ 23

ค่านายหน้า *Commission / Fee*

ค่าธรรมเนียมที่จ่ายให้ Exchange ต่อการเทรดหนึ่งครั้ง Bybit Taker fee $\approx 0.055\%$ Maker fee $\approx 0.02\%$ กลยุทธ์ที่ดีต้องคำนวณให้รวม fee แล้วยังกำไร

→ Part IV บทที่ 23, Part VI บทที่ 34

เลเวอเรจ *Leverage*

การใช้เงินกู้ขยายขนาดการเทรดให้ใหญ่กว่าทุนที่มี เช่น Leverage 10x หมายถึงใช้ทุน 1,000 USD ควบคุม Position มูลค่า 10,000 USD กำไรและขาดทุนขยายเป็น 10 เท่าตามไปด้วย

→ Part IV บทที่ 23

Long / Short

Long = ซื้อ (คาดว่าราคาจะขึ้น); **Short** = ขาย (คาดว่าราคาจะลง) ใน Pair Trading เราเปิด Long หนึ่งตัว และ Short อีกตัว พร้อมกัน ทำให้ Market Neutral — ไม่แพ้หรือชนะตามตลาดรวม แต่แพ้/ชนะตาม spread เพียงอย่างเดียว

→ Part 0 บทที่ 4, Part IV บทที่ 22

สภาพคล่อง *Liquidity*

ความสามารถซื้อขายสินทรัพย์ได้ในปริมาณมากโดยไม่กระทบราคามาก BTC มีสภาพคล่องสูงบน Bybit และ Binance ทำให้ Slippage ต่ำ เหมาะสำหรับกลยุทธ์ HFT และ Stat Arb

→ Part 0 บทที่ 4

หมวดที่ 4 — กลยุทธ์

การกลับเข้าค่าเฉลี่ย *Mean Reversion*

หลักการที่ว่าราคาหรือ spread ที่เบี่ยงออกจากค่าเฉลี่ยมักจะกลับมา รากฐานของ Pair Trading เมื่อ z-score ของ spread $> +2$ เราคาดว่ามันจะกลับลง จึง Short spread เมื่อ z-score < -2 เราซื้อ spread

→ Part 0 บทที่ 4, Part II บทที่ 13, Part IV บทที่ 22

โมเมนตัม *Momentum*

สิ่งที่ขึ้นมาแล้วมักขึ้นต่อ สิ่งที่ลงมาแล้วมักลงต่อ กลยุทธ์ Momentum ซื้อผู้ชนะ/ขาย Short ผู้แพ้ในช่วงที่ผ่านมา ตรงข้ามกับ Mean Reversion

→ Part III บทที่ 21, Part IV บทที่ 22

การเทรดคู่ *Pair Trading*

กลยุทธ์ Market Neutral ที่ซื้อสินทรัพย์หนึ่งและขายอีกสินทรัพย์ที่เคลื่อนไหวพร้อมกัน (cointegrated) พร้อมกัน กำไรจากการที่ spread กลับเข้าค่าเฉลี่ย ไม่ต้องทายทิศตลาด

→ ตลอดทั้งเล่ม (ตัวอย่างหลัก: BTC Bybit–Binance)

Statistical Arbitrage (*Stat Arb*)

กลุ่มกลยุทธ์ที่ใช้หลักสถิติหา "ความผิดปกติเชิงราคา" ชั่วคราว แล้วเทรดเพื่อกำไรจากการที่ราคากลับมาถูกต้อง Pair Trading เป็น Stat Arb ประเภทหนึ่ง

→ Part 0 บทที่ 4, Part IV บทที่ 22

การตามแนวโน้ม *Trend Following*

กลยุทธ์ที่ซื้อเมื่อราคากำลังขึ้นและขาย/Short เมื่อราคากำลังลง ใช้ Moving Average, Breakout เป็นสัญญาณ ตรงข้ามกับ Mean Reversion แต่สามารถรวมกันได้ในระบบที่ซับซ้อน

→ Part IV บทที่ 22

หมวดที่ 5 — เครื่องมือสถิติ

การทดสอบ ADF *Augmented Dickey-Fuller Test*

ทดสอบว่าชุดข้อมูลเป็น *stationary* (วนรอบค่าเฉลี่ย) หรือ *random walk* ใน Pair Trading ใช้ทดสอบ spread: ถ้า p-value < 0.05 หมายถึง spread stationary — คู่นี้เหมาะทำ Pair Trading

→ Part II บทที่ 13

ครึ่งชีวิต *Half-life*

เวลาเฉลี่ยที่ spread ใช้ในการกลับมาครั้งหนึ่งของทางจากจุดเบี่ยงสูงสุดสู่ค่าเฉลี่ย คำนวณจาก AR(1) model ใช้กำหนดความยาว Rolling Window ที่เหมาะสม

$$\text{Half-life} = -\frac{\ln 2}{\ln(1 + \hat{\phi})}$$

$\hat{\phi}$ คือ coefficient จาก AR(1): $\Delta \text{spread}_t = \phi \cdot \text{spread}_{t-1} + \varepsilon$

→ Part II บทที่ 13, Part IV บทที่ 22

ตัวกรองคาลมาน *Kalman Filter*

อัลกอริทึมที่ประมาณค่าตัวแปร (เช่น Hedge Ratio) โดยอัปเดตทีละขั้นตอนเมื่อได้ข้อมูลใหม่ แทนที่จะใช้ Rolling Regression ที่ต้องรอให้ครบ Window ใช้ใน Pair Trading เพื่อให้ Hedge Ratio ปรับตัวตามตลาดได้เร็วกว่า

→ Part II บทที่ 16

อัตราส่วนเฮดจ์ / Beta *Hedge Ratio / β*

จำนวนหน่วยของสินทรัพย์ B ที่ต้อง Short เพื่อ offset ความเสี่ยงของการถือ Long สินทรัพย์ A หนึ่งหน่วย คำนวณจาก OLS Regression หรือ Kalman Filter ถ้า $\beta = 0.98$ หมายถึงซื้อ Bybit 1 หน่วย ขาย Binance 0.98 หน่วย

→ Part 0 บทที่ 4, Part II บทที่ 13

หน้าต่างเลื่อน *Rolling Window*

การคำนวณสถิติโดยใช้ข้อมูล N จุดล่าสุดเสมอ เช่น Rolling Mean 20 วัน หมายถึงค่าเฉลี่ยของ 20 วันที่ผ่านมา เมื่อวันใหม่เข้าวันเก่าสุดออก ทำให้สถิติปรับตัวตามสภาพตลาดปัจจุบัน

→ Part II บทที่ 12, Part IV บทที่ 22

หมวดที่ 6 — Python และเทคนิค

การเขียนโปรแกรมเชิงวัตถุ *Object-Oriented Programming (OOP)*

รูปแบบการเขียนโปรแกรมที่จัดโค้ดเป็น "วัตถุ" (Object) แต่ละวัตถุมีข้อมูล (attribute) และพฤติกรรม (method) ใน Part III ใช้ OOP สร้าง Strategy, Portfolio, และ Connector ที่สลับกันได้

→ [Part III บทที่ 18–21](#)

API *Application Programming Interface*

ส่วนต่อประสานที่ Exchange เปิดให้โปรแกรมภายนอกส่งคำสั่งและดึงข้อมูลได้ Bybit WebSocket API ส่งข้อมูลราคา real-time ทุก millisecond REST API ใช้สำหรับดึงข้อมูลประวัติและส่งคำสั่งซื้อขาย

→ [Part VI บทที่ 30–35](#)

อะซิงโครนัส *Async / Await*

เทคนิคเขียนโค้ดที่ให้โปรแกรมรอรับข้อมูลจาก Exchange โดยไม่ต้อง "ค้าง" รอ สามารถดูแลหลาย WebSocket พร้อมกันได้ ในโปรแกรมเดียว ใช้ asyncio library ของ Python

→ [Part VI บทที่ 30–31](#)

DataFrame

ตารางข้อมูล 2 มิติของ pandas มี index (แถว) และ column (คอลัมน์) เป็นเครื่องมือหลักในการจัดการข้อมูลราคา เช่น OHLCV (Open, High, Low, Close, Volume) ซึ่งดาวน์โหลดจาก Exchange มาเป็นรูปแบบนี้

→ [Part I บทที่ 10](#), [Part II บทที่ 11–12](#)

การทดสอบย้อนหลัง *Backtest*

การทดสอบกลยุทธ์กับข้อมูลราคาในอดีตเพื่อประเมินว่าจะทำงานอย่างไร จุดสำคัญ: ต้องระวัง Look-ahead bias (ใช้ข้อมูลอนาคตโดยไม่รู้ตัว) และ Overfitting (ปรับ parameter ให้เหมาะกับอดีตมากเกินไปจน ล้มเหลวในอนาคต)

→ [Part IV บทที่ 22–25](#)

Walk-Forward Optimization

วิธีทดสอบที่ซ่อม (optimize) parameter บน In-Sample data แล้วทดสอบจริงบน Out-of-Sample data ที่ไม่เคยเห็น ทำซ้ำหลายรอบแบบเลื่อนไปข้างหน้า ช่วยลด Overfitting ได้ดีกว่า Backtest ธรรมดา

→ [Part IV บทที่ 25](#)

WebSocket

โปรโตคอลการสื่อสารแบบ real-time สองทิศทาง ต่างจาก REST API ที่ต้องถามซ้ำๆ WebSocket ให้ Exchange "push" ข้อมูลใหม่มาทันทีที่มี ใช้สำหรับรับ Orderbook และราคา tick แบบ real-time

→ [Part VI บทที่ 30](#)
